TECHNICAL INTEGRATION GUIDE

Graine in-app agents: integrating apps and websites

Everything your engineers need to put a Graine agent inside your mobile app and on your website, from the dashboard to the store submission: install, permissions, CallKit and Android call notifications, identity and memory, chat, voice, components, screen context and actions, WhatsApp-style calls in both directions, scheduled calls, several agents in one app, the complete SDK reference, testing and go-live.

Date 14 October 2026 **For** mobile and web engineers, QA and release managers

Server needed none — a publishable key and the SDKs

Platform	Package	Version	Where it comes from
iOS (Swift)	`GraineInApp` — Swift package; CocoaPods pod `GraineInApp`	**0.7.0**	`https://github.com/GraineAI/inapp-ios`
Android (Kotlin, Java)	`ai.graine:inapp-android`	**0.7.0**	Maven Central
Flutter	`graine_inapp`	**0.7.0**	pub.dev
Kotlin Multiplatform	`ai.graine:inapp-kmp`, `ai.graine:inapp-kmp-compose`	**0.7.0**	Maven Central
React Native	`@graineai/inapp-react-native`	**0.41.0**	npm
React Native — native calls	`@graineai/inapp-react-native-calls`	**0.2.0**	npm
Website	The loader, `https://www.graine.ai/embed/v1.js` (`GraineAgent.version`)	**1.2.0**	Served by Graine

Every version above is read from the package files themselves: `Version.swift` and the podspec (iOS), `GRAINE_VERSION` (Android, Kotlin Multiplatform), `pubspec.yaml` (Flutter), `package.json` (React Native) and `GraineAgent.version` in the loader.

Contents

1. What you get

- The agent in your app
- The website widget
- WhatsApp-style calls, in both directions
- Scheduled calls — booked by the agent, your app or your team
- Memory, and conversations that continue
- The customer's identifier in Call History
- Several agents in one app
- No server of yours
- What each platform supports

2. How it works end to end
The parts and who builds them
A session starts, with identity
A chat, with memory and continuation
The customer calls the agent
A booked call rings the customer's phone
Three ways to book

3. Agent types: prompt agents and flow agents
How a flow agent is built
Flow agents in your app and on your website

4. Dashboard setup, step by step
Embed & Widgets → Install: the key and Allowed domains
Embed & Widgets → Appearance: the Garage and Setup
Actions, On screen and Friction
Agent → Memory: People and Settings
Agent → Calls & scheduling
Workspace settings → In-app calling (push credentials)
The prompt: who_called and call_reason

5. Install and set up, per platform
SDK sizes (measured)
React Native
iOS (Swift)
Android (Kotlin, Java)
Flutter
Kotlin Multiplatform
Website
Store review

6. Identity and memory
identify(...) after sign-in, identify(null) on sign-out
Installs and the device limit
Reading and erasing memory: memories(), setMemory(), forgetMemory()
What Graine stores

7. Conversations continue
Who gets it
What is kept, and for how long
What the customer sees, and what the agent knows
Forgetting, opting out, and a new conversation

8. Chat, voice, components, screen context and actions
Voice in the assistant bar
The chat window
Components
Screen context
Actions

9. WhatsApp-style calls
The customer calls the agent: startCall
Graine rings the customer: hosted ringing
The call screen: six moments
Outcomes, and their webhooks
Locked screens, other apps, weak networks
10. Scheduled calls
The rules every booking follows
scheduleCall, scheduledCalls, cancelScheduledCall
Checks made on the device first
Booking by the agent
Booking by your team
Example: a "book a consultation call" sheet with the native pickers
The chat window's upcoming calls
11. Several agents in one app
What is per agent, and what is shared
Worked example: a voice assistant and a separate support chat
Calls with several agents
The multi-agent API
12. SDK reference
Data structures, and what to expect
React Native — @graineai/inapp-react-native 0.41.0
React Native — @graineai/inapp-react-native-calls 0.2.0
iOS — GraineInApp 0.7.0
Android — ai.graine:inapp-android 0.7.0
Flutter — graine_inapp 0.7.0
Kotlin Multiplatform — ai.graine:inapp-kmp 0.7.0
Website — the loader 1.2.0
13. Errors, limits and webhooks
Error codes and their fixes
Rate limits and quotas
Webhooks for in-app conversations
14. Data, privacy and security
What leaves the phone or the page
What a ring carries
What Graine stores, and for how long
The publishable key, and what protects it
Erasure
15. Testing end to end
Before you start
The QA script
What each step should look like
The demo apps
16. Go-live, troubleshooting and support
Go-live checklist
Troubleshooting
Support

1. What you get

One agent, built and designed in the Graine dashboard, running in your app and on your website — talking, chatting, seeing the screen the customer is on, acting in your app, drawing components, remembering each customer, and calling them the way WhatsApp does.

1.1 The agent in your app

- **The assistant bar** — a bar drawn by the SDK in your brand's design (chosen in the dashboard, never in code): the agent's avatar, its name and a live status line, the microphone, the hang-up, and a caption card with the line being said. The customer can drag it up or down; it rises above the keyboard.
- **The chat window** — typed conversation with the same agent, opened from the bar, with the same screen context, actions and components as voice. The dashboard decides what customers get: a call and a chat, voice only, or chat only.
- **Components drawn on screen** — product lists, plan comparisons, info cards, calendar slots, quick actions, lead forms and your own kinds, drawn natively in your design. A tap is the customer's next turn, and your app can be told to open a screen.
- **Screen awareness** — your app reports which screen the customer is on, its fields and their state, taps on labelled controls and product events. Personal data (card, account, PAN and Aadhaar numbers, phone numbers, email addresses, plus your own patterns) is masked on the device before anything leaves it.
- **Actions** — the agent asks your app to do something you declared (fill a field, open a screen, change a setting) and waits up to six seconds for your handler's answer.

1.2 The website widget

The same agent on your website from two script tags: a launcher and a panel with voice and chat, components, and actions on the page — in the same design as your app. It runs in its own frame, so your page's styles cannot reach it and its code cannot reach your page. The page can identify the visitor, describe what they see, and book calls that ring the customer's app.

1.3 WhatsApp-style calls, in both directions

- **The customer calls the agent** from a button in your app (`startCall`): the system's call screen on iPhone (CallKit), the SDK's call screen on Android, and the conversation carries on with the screen locked, over Bluetooth, while they use other apps.
- **Graine rings the customer** at a booked time — on the lock screen, with the system's incoming-call screen on iPhone and a full-screen call notification on Android. Answered, the call connects to your agent, which knows why it is calling.

Both run on a **native call engine** inside your app (the platform's audio and echo cancelling). Every outcome — answered, declined, missed, ended, failed — reaches your app; every conversation lands in Call History; and for a call Graine rang, the outcome also updates the booking and reaches your webhooks.

1.4 Scheduled calls — booked by the agent, your app or your team

A call can be booked three ways: by **the agent** during a conversation ("call me back at six"), by **your app or website** with `scheduleCall`, or by **your team** in the dashboard (*Schedule a call*, or *Ring now* for a test). Every booking follows the same rules — the agent's Calling Window in the customer's own time zone, quiet hours, how far ahead, how much notice, how many may be waiting — and rings the customer's app.

1.5 Memory, and conversations that continue

- **Memory** — the agent remembers facts about each customer across conversations, devices and channels (app, website, and phone when the identifier is the phone number), under a one-way key, for the retention you choose. Customers can see what is remembered, turn it off, and erase it from your app.
- **Conversations continue** — a returning customer sees the earlier conversation in the chat window and the agent picks up where they left off, like a messaging app: the last 50 messages, kept for 7 days after the last one.

1.6 The customer's identifier in Call History

Every in-app and website conversation in Call History shows who it was with — the field your agent identifies people by and its value, for example **User ID · u_8812**, **Email · asha@acme.example** or **Customer ID · C-20931**. The memory service never receives that value: only a one-way key derived from it.

1.7 Several agents in one app

One app can run several agents — support in Help, sales on Plans, a coach on Home — each with its own publishable key, design, actions, chat, voice bar and scheduling. They share who the customer is, the device's install, the memory choice and the current screen, and one conversation runs at a time across all of them. Rings are answered by the agent that booked them.

1.8 No server of yours

Your app and site integrate with a **publishable key** (`pk_live_…`) and the SDKs only. Graine holds the conversation, memory, the bookings, the ringing (with the push credentials you upload once) and the record. The key is public by design: an app request is accepted only when the agent's **Allowed domains** include `app://` , a website request only from a listed host, and every route is rate-limited.

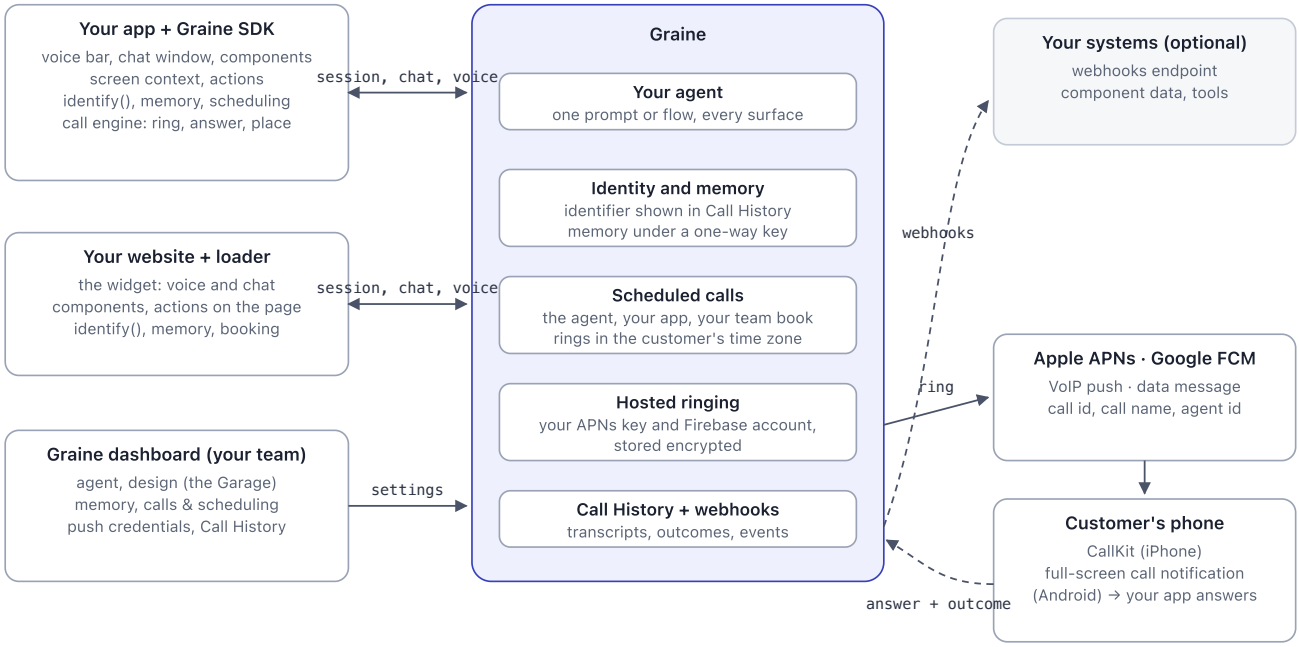
1.9 What each platform supports

Capability	React Native	iOS	Android	Flutter	Kotlin MP	Website
Assistant bar (voice)	Yes	Yes	Yes	Yes	Yes (Compose)	Widget
Chat window	Yes	Yes	Yes	Yes	Yes (Compose)	Widget
Components on screen	Yes	Yes	Yes	Yes	Yes	Yes
Screen context and actions	Yes	Yes	Yes	Yes	Yes	Yes
Customer calls the agent (native call)	With the calls package	Yes	Yes	Yes	Yes	—
Graine rings the customer	With the calls package	Yes	Yes	Yes	Yes	— (books; rings the app)
Identity and memory API	Yes	Yes	Yes	Yes	Yes	Yes
Scheduling API	Yes	Yes	Yes	Yes	Yes	Yes
Several agents	One provider per agent	Named agents	Named agents	Named agents	Named agents	One at a time, switched by `load()`

"Kotlin MP" is the Kotlin Multiplatform SDK: one API in shared code over the iOS and Android SDKs.

2. How it works end to end

2.1 The parts and who builds them



Your app and your website talk to the same agent at Graine. Graine keeps one key per customer, remembers under it, books calls for it, and rings the customer's phone through Apple's and Google's push services with the credentials you upload once.

Part	What it does	What you write
The SDK, in your app	Draws the assistant bar, the chat window, components and the call screen in your design; masks personal data before anything leaves the phone; keeps the install ID; runs the voice engine and the native call engine; registers the device for rings and reports every call outcome.	A few lines at launch, `identify()` at sign-in, one screen report per screen, one handler per action, and — for calls — two lines that hand the SDK its push tokens.
The loader, on your website	Draws the launcher and the panel (an isolated frame) in the same design; voice, chat, components and actions on the page.	Two script tags; optionally `identify()`, `setScreen()` and `defineAction()`.
Graine	Runs the agent, keeps memory and the recent conversation under a one-way key, holds the bookings and rings the customer with your credentials, records every conversation in Call History, sends your webhooks.	Nothing — settings in the dashboard.
Apple and Google push	Deliver the ring (an APNs VoIP push; a high-priority FCM data message).	Upload an APNs auth key and a Firebase service account once (Workspace settings → In-app calling).
Your systems (optional)	Receive webhooks; serve live data for components; run the agent's tools.	Only if you want them.

Two engines, one agent

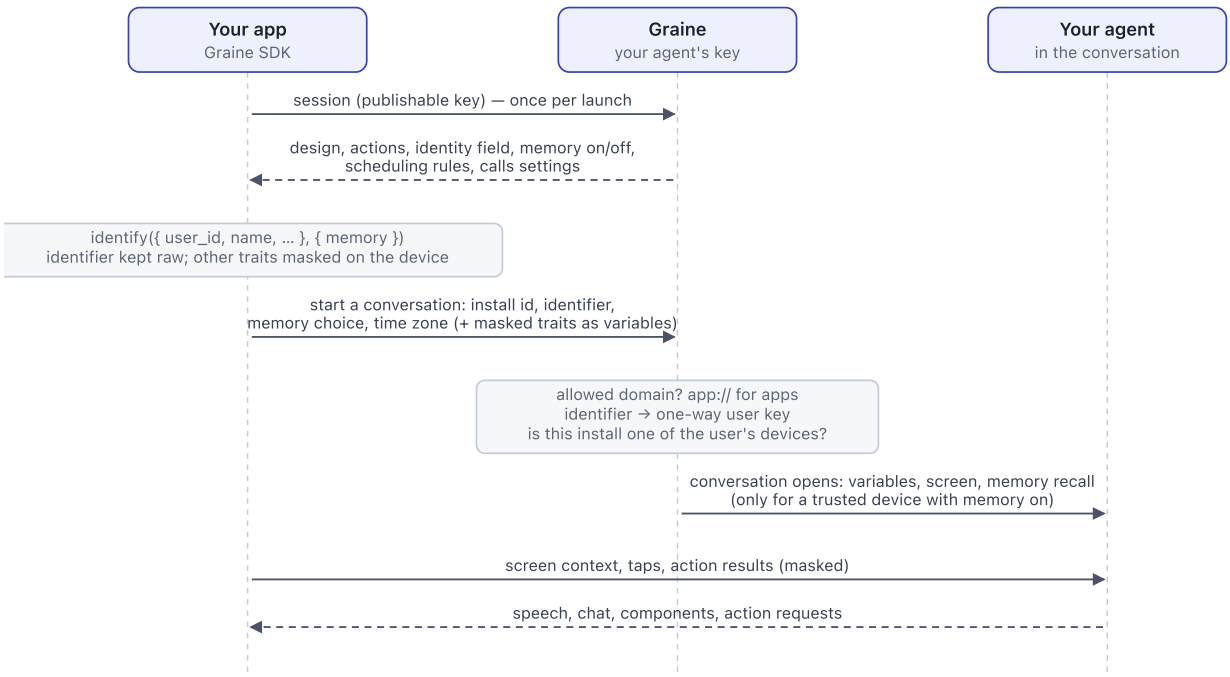
- **The voice engine** carries the assistant bar's voice. Every SDK hosts the same engine, loaded from Graine into a small invisible web view the SDK owns, so an audio fix reaches your app without a release. Its audio goes over the high-

quality path (WebRTC) when the agent's *Call audio* is set to it, falling back to the standard path when it cannot connect. The chat has its own connection.

- **The native call engine** carries WhatsApp-style calls. Its audio runs in your app's own process with the platform's echo canceller, which is what lets a call ring through CallKit and keep going with the screen off. It improves when you update the SDK.
- **One conversation at a time.** Starting a native call ends a conversation in the bar; the bar does not start while a call is live; and with several agents, starting a conversation on one ends a live one on another.

Your app passes nothing about how the agent looks. The SDK reads the design from the agent's settings once per launch, so a design saved in the dashboard reaches a website on its next page load and an app the next time it starts. The last design is kept on the device, so a ring that wakes a closed app is drawn in your brand.

2.2 A session starts, with identity

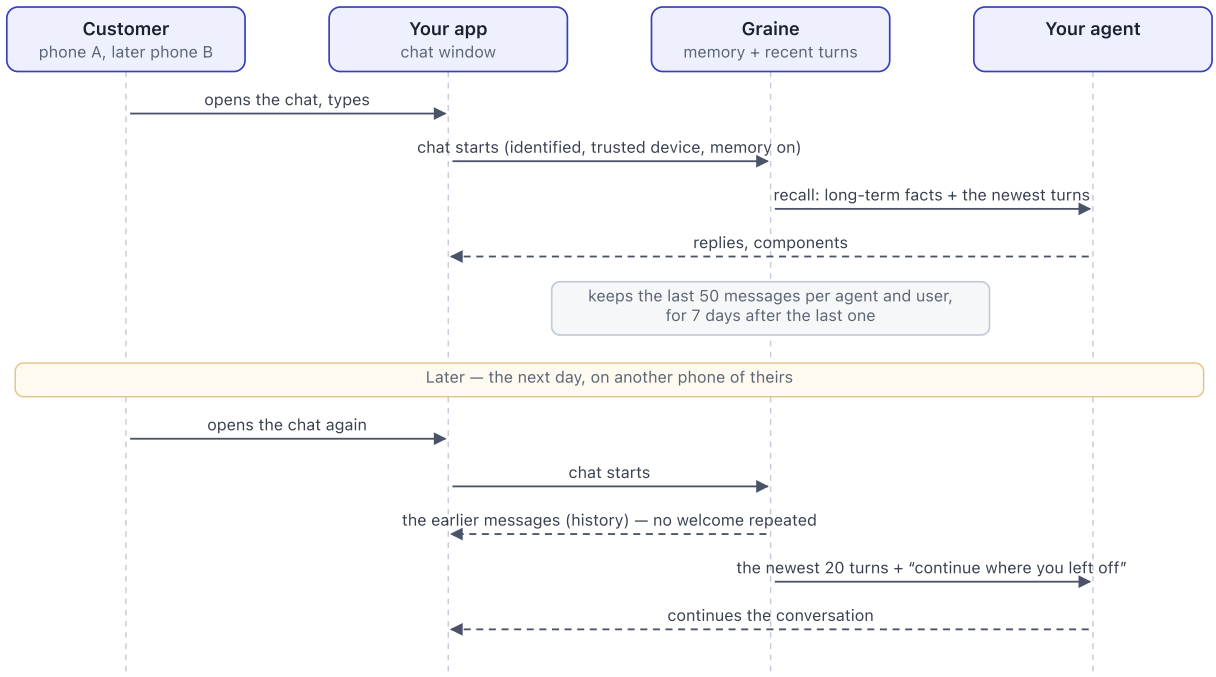


From launch to an identified conversation.

1. **Session.** Once per launch the SDK fetches the agent's settings with the publishable key: the design, the declared actions, which trait identifies a customer (*Who is the user?*), whether memory is on, the scheduling rules and the call settings. The last one is kept on the device for a cold start.
2. **Identify.** After sign-in your app calls `identify(traits, memory)`. The trait the agent's *Who is the user?* setting names (`user_id` unless you chose another) is the **identifier**: it is kept as it is and never becomes a prompt variable. Every other trait is masked on the device and becomes a prompt variable (`{name}`, `{plan}`). Calling it before the session has arrived is fine.
3. **A conversation starts** (a chat, the bar's voice, or a native call). The SDK asks Graine for a single-use connection ticket that is valid for two minutes, sending the install ID, the identifier, the customer's memory choice, the device's time zone and, for a call, its call ID.
4. **Graine checks** the key and the agent's Allowed domains (an app needs `app://`), turns the identifier into a one-way user key, and checks that this install is one of the customer's allowed devices (*Devices per user*).
5. **The conversation opens** on the agent, with the customer's prompt variables. Memory is recalled only when the agent remembers, the customer has not opted out, and the device is one of theirs; recall is given 1.5 seconds and the conversation goes ahead without it rather than wait.
6. **While it runs**, the SDK sends the screen as it changes (masked, numbered, the first change at once and later ones at most every 400 ms), taps on labelled controls, and the answers to the agent's action requests.

Identity is fixed for a conversation when it starts: a sign-in during a conversation applies to the next one.

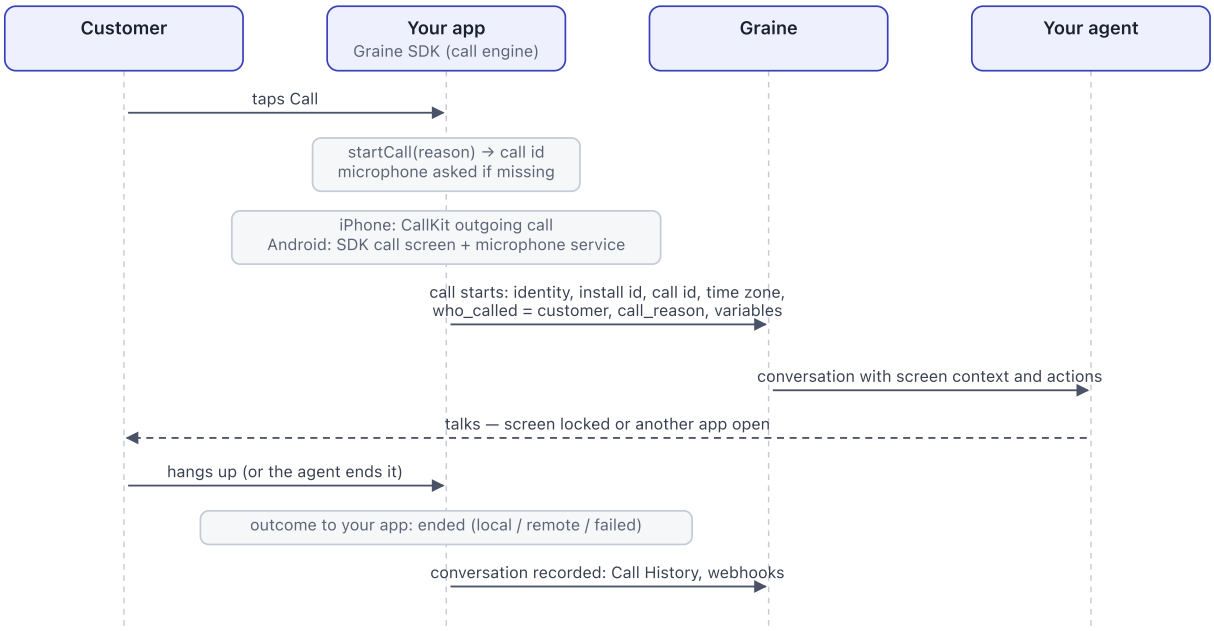
2.3 A chat, with memory and continuation



The same customer today on one phone, tomorrow on another of theirs.

- **Memory** holds long-term facts ("prefers a call after 6 pm"). The agent recalls them once when a conversation starts — every kind of agent, every step of a flow — and new facts are stored in the background as the conversation goes and when it ends, so allow a short while before a new fact can be recalled.
- **Continuation** holds the recent conversation itself: the last 50 messages per agent and customer, for 7 days after the last one. A new chat starts from them — the chat window shows the earlier messages and the agent continues; no welcome message is repeated. Conversations continue has the details.
- Both apply only to an identified customer, on a device that is one of theirs, on an agent with memory on, who has not opted out. Anyone else gets nothing kept and nothing restored — never a shared bucket.

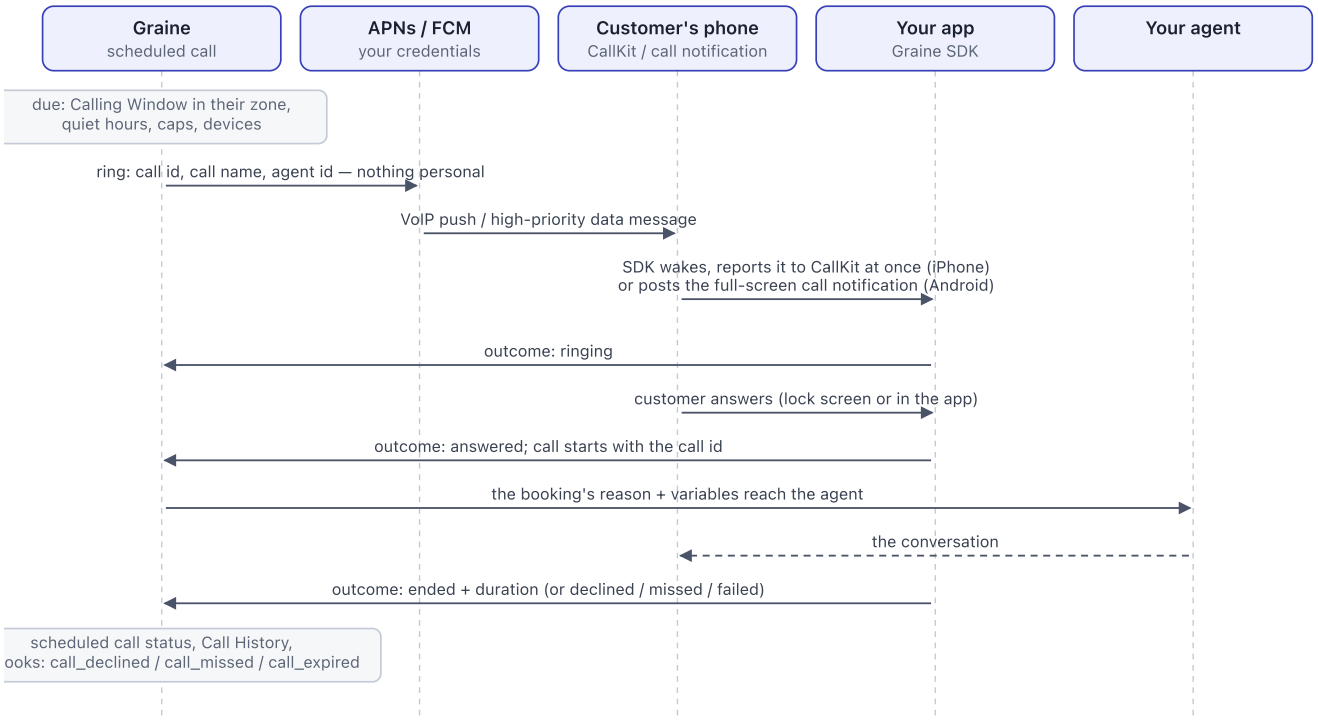
2.4 The customer calls the agent



A call your customer places from your app.

1. Your call button calls `startCall(reason)`. It returns the call's ID (`call_` and a lowercase UUID), or nothing while another call is live.
2. The SDK asks for the microphone if the app does not have it, places the call — a CallKit call on iPhone, the SDK's call screen on Android — and connects to the agent with `who_called = "customer"`, `call_reason` set to your reason, your call variables, the customer's identity and the current screen. A conversation in the bar ends first.
3. The call goes on with the screen locked, in another app, or on a Bluetooth headset. The agent can still run your actions.
4. When it ends, your app hears the outcome — `ended` with `local` (the customer hung up), `remote` (the agent did) or `failed` (the line broke; calls do not reconnect) — and the conversation is in Call History.

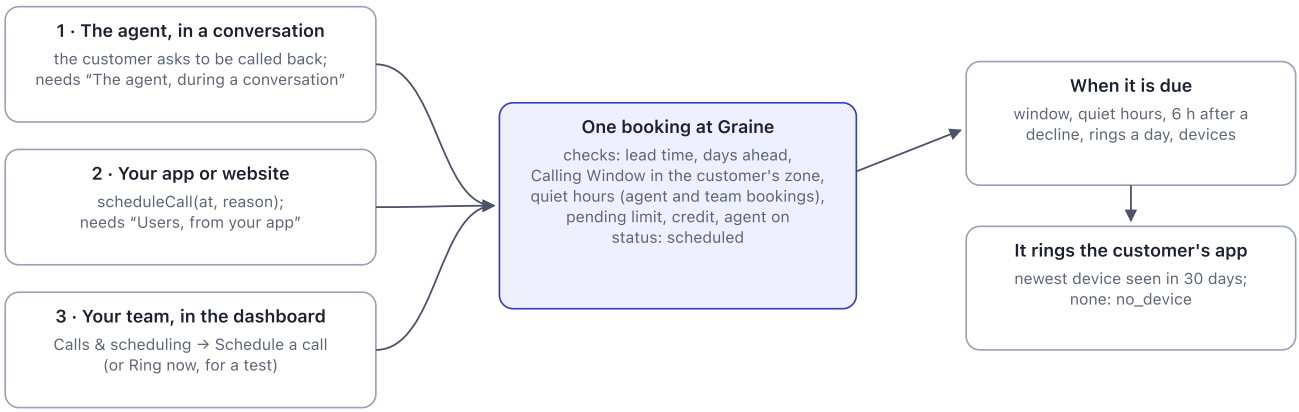
2.5 A booked call rings the customer's phone



Hosted ringing: Graine sends the ring with your credentials; the SDK rings, connects and reports.

1. **When the time comes**, Graine checks that the call may ring now: credit and the agent being on; the agent's Calling Window in the customer's own time zone; quiet hours for calls the agent or your team booked; no ring within 6 hours of a decline; the daily ring limit; and a device to ring (seen in the last 30 days).
2. **The ring** goes to the customer's newest device through APNs (a VoIP push) or FCM (a high-priority data message). It carries the call's ID, the call name and which agent is calling — no reason, no variables, nothing about the customer.
3. **The SDK rings.** On iPhone it reports the call to CallKit at once — iOS requires every VoIP push to be reported, and the SDK reports every one. On Android it posts a call notification that fills a locked screen. It tells Graine the phone is ringing.
4. **The customer answers** on the lock screen or in your app. The SDK reports `answered` and opens the conversation with the call's ID; the agent receives the booking's reason (as `call_reason`) and variables, and the customer's memory.
5. **The outcome** — `ended` with its duration, or `declined`, `missed` or `failed` — goes to your app and to Graine, which updates the scheduled call, files the conversation in Call History and sends your webhooks (`call_declined`, `call_missed`, `call_expired`). A missed ring can ring once more after the delay you set; a declined one never rings again.

2.6 Three ways to book



The agent, your app or website, and your team all create the same kind of booking.

Booked by	How	Needs (Calls & scheduling)	Quiet hours apply?
The agent	During a conversation, when the customer asks to be called back	*The agent, during a conversation*	Yes
Your app or website	`scheduleCall(at, reason)`	*Users, from your app*	No — a time the customer picked is theirs to keep
Your team	Agent → Calls & scheduling → *Schedule a call*; *Ring now* for a test	Push credentials uploaded	Yes

Every booking must fall inside the agent's Calling Window in the customer's time zone, and is refused at booking time if it does not. On a phone call, the agent's booking is an ordinary call back to the caller's number.

3. Agent types: prompt agents and flow agents

Both kinds of agent use the same voices, languages, knowledge base, webhooks and widgets, and both go into your app and onto your website the same way — the SDKs do not change. They differ in how the conversation is written.

	Prompt agent	Flow agent
Made of	One prompt that covers the whole conversation	A briefing every step shares, plus **steps** that each carry their own instructions, joined by **paths**
Each turn, the model reads	The whole prompt	The briefing and the current step only, so a long script stays focused
Choose it for	Open conversations where the order does not matter: support questions, FAQs, discovery	A conversation with a shape it must keep: confirm who you are speaking to, check something, agree a date, say goodbye
Created from	Sidebar *Build* → *Agents* → *New agent*	Sidebar *Build* → *Flow agents* → *New flow agent*, or *Convert to a flow agent* in a prompt agent's editor (it drafts steps and creates a new agent; the original is unchanged)

3.1 How a flow agent is built

- **Steps.** *AI reply* — the AI talks, following the step's instructions on top of the briefing; it can read the knowledge base and use tools, and it is the only kind of step that can end the conversation. *Fixed line* — says the same line every time, word for word, in the caller's language. *Branch* — says nothing; it checks its paths and moves on within the same turn.
- **Paths.** *AI decides*, *Rule*, *Always* (the fallback) and *Event*. Rules are checked first, then the model's decision, then *Always*; otherwise the conversation stays on its step. The customer always hears the step they have just been moved to.
- **Test and publish.** The builder's *Test* area runs a chat, a web call or a phone call against the agent **as saved**. A new flow agent is a draft until *Go live*.

3.2 Flow agents in your app and on your website

	In your app and on your website	On a phone call
Which agent answers	The agent's web-call variant, kept in step with every save	The agent itself
Components	Offered on every *AI reply* step (say in the step's instructions when to show one); a choice in a component arrives as the customer's next turn, so a path can branch on it. A step set to *Uses a tool* cannot be a component	None — write such steps so they also work by voice
Screen context and in-app actions	Every *AI reply* step knows which screen the customer is on and which component is open, and can call your actions	None
Memory	Recalled once when the conversation starts, and reaching every step	The same, keyed on the caller's number
Fixed lines	Said word for word and captioned; in a chat they arrive as text	Said word for word
Your systems moving the conversation (events)	Not available	Event paths, through the API

For a flow agent, *Embed & Widgets* sits in the flow builder's header and shows *Install*, *Appearance*, *Actions*, *On screen* and *Friction*; its *Users & memory* and *Calls* settings are reached through *All settings*, which opens the agent editor. Embed settings save on their own, not with the flow's *Save*.

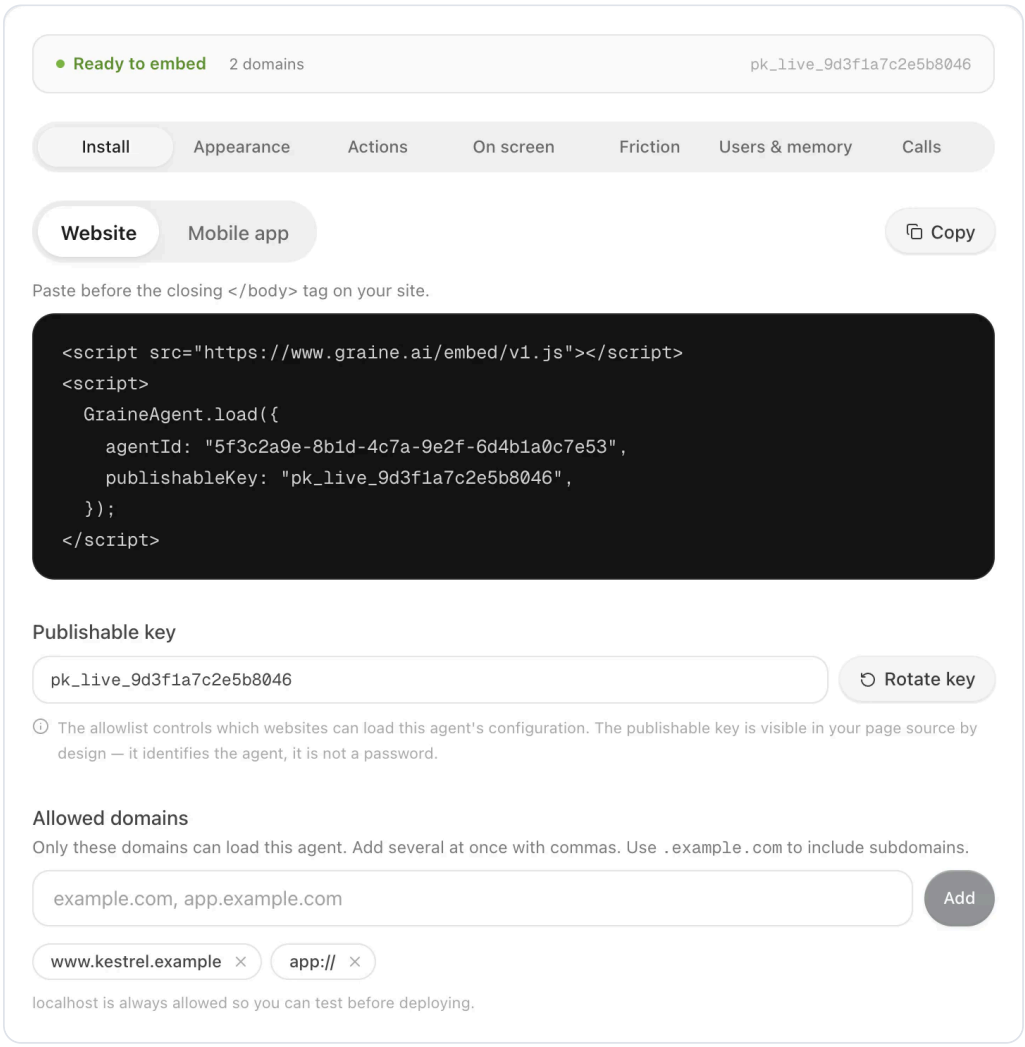
One agent per surface.
Actions are declared on the agent, and the model is offered every one of them on every surface that agent serves. Give your app its own agent — clone it from your existing one (the agent editor's *More actions* menu → *Clone this agent*) so the prompt and knowledge base come across — and change only what differs. A clone starts without the original's embed and widgets.

4. Dashboard setup, step by step

Everything below is in the Graine dashboard at `https://www.graine.ai`. Labels are written exactly as the dashboard shows them. An agent's settings are under *Build* → *Agents* → the agent → the *Configuration* panel, whose sections include *Embed & Widgets*, *Memory*, *Calling Window*, *Calls & scheduling*, *Custom Variables* and *Integrations & Tools*. Workspace-wide settings are under *Account* → *Workspace settings*.

#	Where	What you set	Who
1	*Build* → *Agents*	An agent for the app (clone your existing agent) and one for the website	Agent editors
2	Agent → *Embed & Widgets* → *Install*	*Set up embedding*; copy the publishable key; **Allowed domains**: `app://` for the app, your hosts for the website	A role that manages integrations
3	*Embed & Widgets* → *Appearance*	The design in the Garage (*Assistant bar*, *Chat window*, *Call screen*) and the *Setup* card	Agent editors
4	*Actions*, *On screen*, *Friction*	What the agent may do in your app, the components it may draw, when it speaks up first	Agent editors
5	Agent → *Memory* (also *Embed & Widgets* → *Users & memory*)	Memory on, *Who is the user?*, *Devices per user*, what is remembered and for how long	Agent editors
6	Agent → *Calls & scheduling* (also *Embed & Widgets* → *Calls*)	Who may book, the call name, ringing rules, quiet hours; booked calls	Agent editors; placing a call needs permission to place calls
7	*Account* → *Workspace settings* → *In-app calling*	APNs key and Firebase service account, until each shows *Verified*	An Admin (manages integrations)
8	The agent's prompt	Use `{who_called}` and `{call_reason}`; when to offer a call back	Agent editors

4.1 Embed & Widgets → Install: the key and Allowed domains

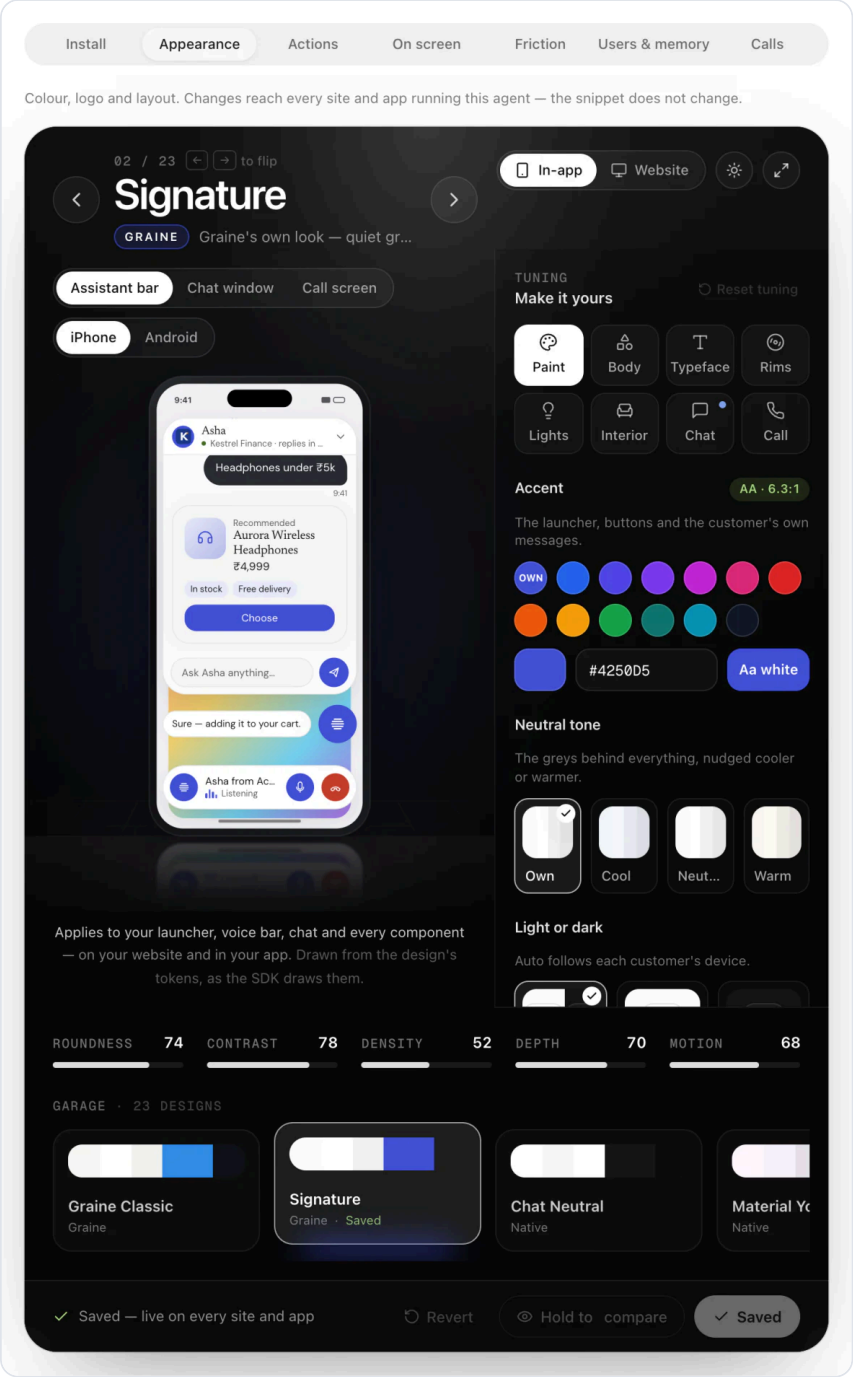


Embed & Widgets → Install. The tabs across the top are *Install*, *Appearance*, *Actions*, *On screen*, *Friction*, *Users & memory* and *Calls*.

1. Open the agent → *Embed & Widgets*. If it shows *Not set up for the web yet*, press **Set up embedding**: it creates the agent's **publishable key** (`pk_live_…`). The status strip then reads *Ready to embed* (or *No domains yet*).
2. **Publishable key** — copy it into your app's configuration and your website snippet. It identifies the agent and is public by design. **Rotate key** issues a new one; every page and build still using the old key stops working immediately, with no grace period — ship the new key first.
3. **Allowed domains** — *Only these domains can load this agent. Add several at once with commas. Use .example.com to include subdomains, and app:// for your mobile app.*
   - **For your app, add** `app://` (typing `app` or `app://*` saves the same entry). A native app sends no browser origin; Graine refuses such requests unless `app://` is listed, with the error *This agent does not accept requests from a mobile app yet. Add "app://" to its Allowed domains under Agent → Embed & Widgets.* The entry lets in any app that holds the key: treat the key as public; rate limits and single-use connection tickets bound it.
   - **For your website, add every host exactly**, staging included: `example.com` does not cover `www.example.com`. An empty list refuses every site. A page on `localhost` is refused like any other site: add `localhost` to the list while you test from your machine, and remove it before you go live.
4. The *Website* | *Mobile app* switch shows a ready snippet with your agent ID and key filled in (*Mobile app* shows React Native; the other platforms are in Install and set up, per platform).

4.2 Embed & Widgets → Appearance: the Garage and Setup

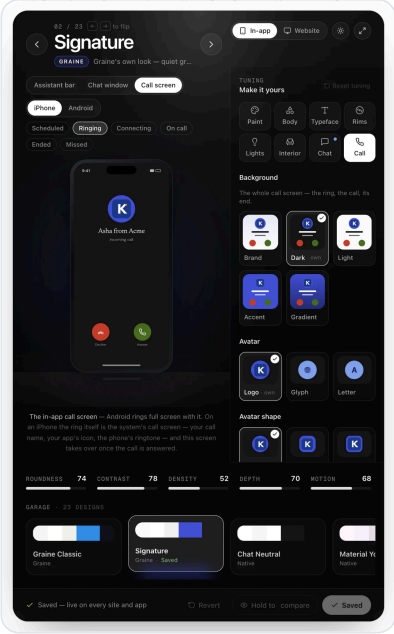
Colour, logo and layout. Changes reach every site and app running this agent — the snippet does not change. The Garage is a showroom of designs (*Garage · 23 designs*); pick one, tune it under *Make it yours* (*Paint*, *Body*, *Typeface*, *Rims*, *Lights*, *Interior*, *Chat*, *Call*), preview it as *In-app* or *Website* on an *iPhone* or *Android*, and **Save design**. The footer then reads *Saved — live on every site and app*.



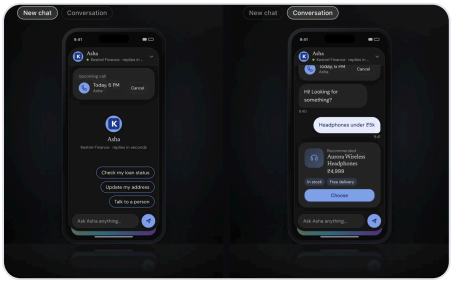
The Garage. The three tabs over the preview are *Assistant bar*, *Chat window* and *Call screen* (in-app only).

Garage tab	Controls (Tuning → the matching category)
Assistant bar	The bar's look comes from the design and its tuning: colours (*Accent*, *Neutral tone*, *Light or dark*), typeface, corners, depth, motion.

Garage tab	Controls (Tuning → the matching category)
Chat window (moments *New chat*, *Conversation*, *Returning customer*)	*Title and subtitle* (title up to 40 characters, subtitle up to 60, default *Online*); *Avatar*: *Logo* / *Glyph* / *None*; *Header*: *Surface* / *Accent*; *Background*: *Plain* / *Tint* / *Pattern*; *Starters* — up to 4, each up to 40 characters, sent exactly as written when tapped; *Composer* → *Placeholder* (default *Message…*); *Timestamps*: *Hidden* / *Shown*. *Window size*: *Compact* · *Large* · *Full screen* — *How big the chat opens on a phone. On a tablet or computer, Large and Full screen open a panel.* *New conversation button*: *Shown* / *Hidden* — *The compose button in the header that starts a fresh conversation. It shows while earlier messages are on screen.* Hidden, there is no control; `newConversation()` still works from a button of yours. *Booked calls*: *Shown* / *Hidden*, and *Card title* (empty: "Upcoming call" / "Upcoming calls") — the card of the customer's upcoming calls at the top of the chat, each with Cancel. The iOS, Android, Flutter and Kotlin Multiplatform SDKs draw it; React Native's chat and the website have no booked-calls card, so this control changes nothing there. *Earlier messages*: *Shown* / *Hidden* — *A customer who comes back sees the conversation from before. Hidden, the chat opens empty, like a new one, and your assistant still remembers it.* These save in the design as `chat.showUpcomingCalls`, `chat.upcomingTitle`, `chat.showHistory` and `chat.newConversation`; the SDKs read them into their chat tokens (iOS `GraineChatTokens`).
Call screen (moments *Scheduled*, *Ringing*, *Connecting*, *On call*, *Ended*, *Missed*)	*Background*: *Brand* / *Dark* / *Light* / *Accent* / *Gradient*; *Avatar*: *Logo* / *Glyph* / *Letter*; *Avatar shape*: *Circle* / *Squircle* / *Rounded*; *End button*: *Round* / *Pill*; *Buttons on a call*: *Mute*, *Speaker*, *Chat*, *End* (which and in what order; End is always there); *Captions*: *Shown* / *Hidden*; *Timer*: *Shown* / *Hidden*; *Ringtone*: *Phone's own* / *Graine* / *Soft*; *Words* for each moment: *Scheduled call ringing* (default "Scheduled call"), *Other call ringing* ("Incoming call"), *Connecting* ("Connecting…"), *Ended* ("Call ended"), *Missed* ("Missed call"), up to 40 characters each.



Call screen tab, *Ringing*. On an iPhone the ring itself is the system's call screen; this screen takes over once the call is answered in the app.



Chat window tab in dark: *New chat* (upcoming call, starters) and *Conversation* (a component).

Setup (under the design, saved on its own with **Save setup**): *What visitors get — Call and chat* / *Voice only* / *Chat only*; *Opens on — Call* / *Chat*; *Corner* (websites only); *Heading* (default "Talk to our AI assistant"); *Launcher tooltip* (websites only, default "Chat with us"); *Call audio — Standard* / *High quality (WebRTC)*; *Logo*; *Background behind the logo*; *Show "Powered by Graine AI"*; and *Loading* (the loader and its words while connecting, reconnecting, thinking and loading a component). *Native apps use What visitors get, Opens on, Heading, Logo, Loading and Call audio from here, and the whole design above.*

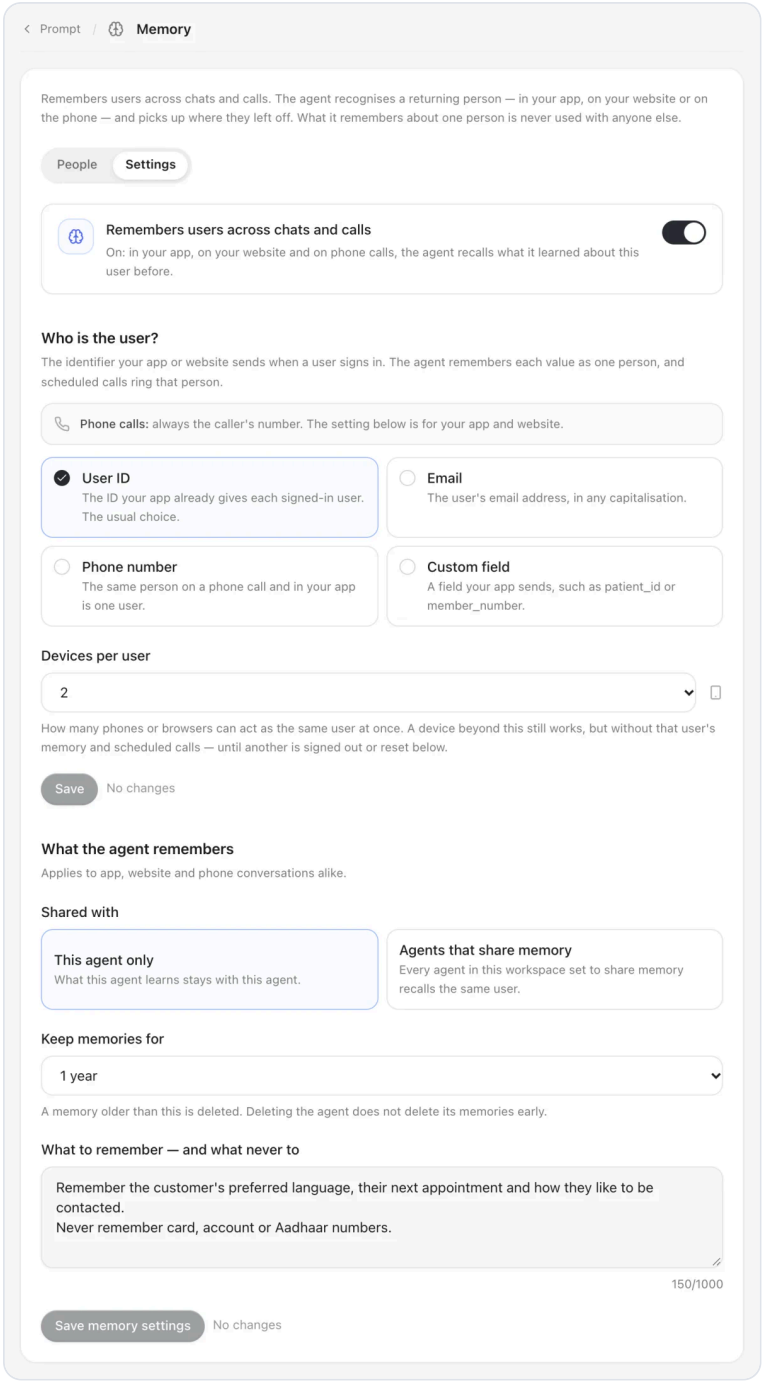
The call screen names the call with Calls & scheduling → *Name on the incoming call* — on the phone and in the Garage's preview alike (the Appearance *Heading* when it is not set).

4.3 Actions, On screen and Friction

- **Actions** — *What this agent may do in your app*. Each action has a *Tool name*, *When should the agent use this?* and *Arguments*; ready-made ones are *Navigate*, *Fill a field*, *Highlight*, *Retry* and *Blank*. **Save and sync to agent** pushes them to the agent at once. *Your app must implement a handler with the same name — declaring it here does not create the behaviour, it creates the contract.*
- **On screen** — *What this agent can put on screen*: tick the components (built in the dashboard's components editor) and **Save selection**.
- **Friction** — *When the agent should notice someone is stuck*: *Friction detection* on or off; rules *Idle on a step*, *Same field rejecting them*, *Going back and forth*, *Rage taps*, *Moving on with something missing*, *Error banner*; per-screen overrides; *How often it may speak up*; *Open the assistant when nobody is talking to it*.

4.4 Agent → Memory: People and Settings

Remembers users across chats and calls. The agent recognises a returning person — in your app, on your website or on the phone — and picks up where they left off. What it remembers about one person is never used with anyone else. The identifier your app sends is shown in Call History beside each conversation. The same screen is the *Users & memory* tab of Embed & Widgets. It has two tabs, **People** and **Settings**.



Agent → Memory → Settings.

Settings

Setting	Choices	What it does
Remembers users across chats and calls	On / off	On: in your app, on your website and on phone calls, the agent recalls what it learned about the user before. Off: every conversation starts fresh.
Who is the user?	*User ID* (default) · *Email* · *Phone number* · *Custom field* (with *Field name*, for example `patient_id`, `member_number` or `customer_id`)	The trait your app's `identify()` must carry: `user_id`, `email`, `phone` or your field's name. The agent remembers each value as one person, scheduled calls ring that person, and Call History shows it. *Phone calls: always the caller's number. The setting below is for your app and website.* Changing it starts every app user fresh.
Devices per user	1–5 (default 2)	*How many phones or browsers can act as the same user at once. A device beyond this still works, but without that user's memory and scheduled calls — until another is signed out or reset below.*

Setting	Choices	What it does
Shared with	*This agent only · Agents that share memory*	Whether what this agent learns is recalled by other agents in the workspace that also share memory.
Keep memories for	30 days, 90 days, 180 days, 1 year (default), 2 years, or another number of days	*A memory older than this is deleted. Deleting the agent does not delete its memories early.*
What to remember — and what never to	Up to 1,000 characters	Instructions for what is kept and what never is (for example: never card numbers).

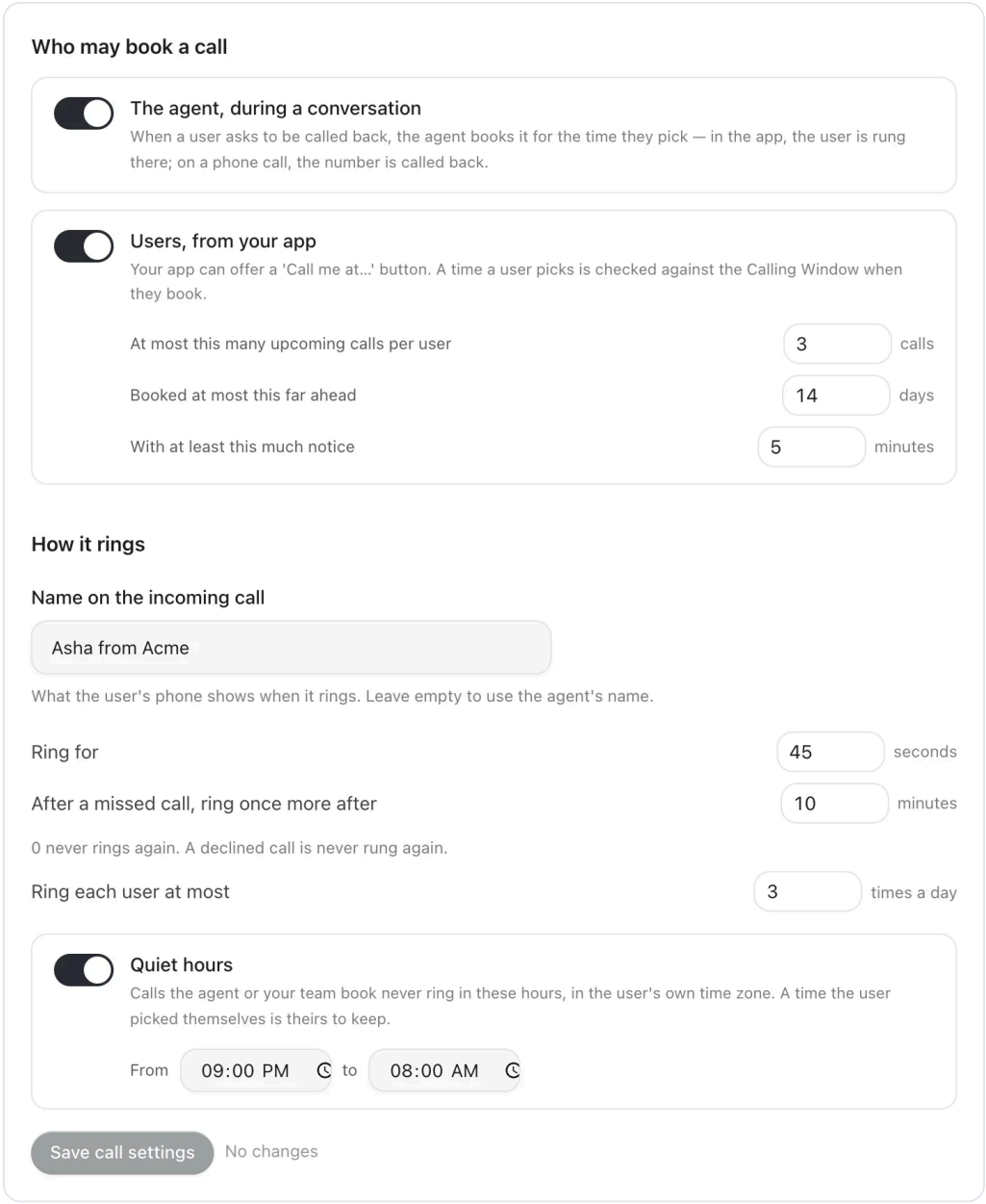
Two buttons save these: **Save** (the identity settings) and **Save memory settings**.

People, and a person's panel

- **People** lists *People … has talked to* (the newest 1,000 conversations), searchable *by name or identifier*. Each row shows **<field label> · <value>** (for example *User ID · u_8812*), the name, the number of conversations and the last one, and channel chips *Chat*, *Voice*, *In-app call*, *Phone*. *Find a person* looks someone up by identifier (*App user*) or by number (*Caller*).
- **The person's panel** shows *What … remembers about this person*, each memory with *Learned/Updated* and *Delete this memory*; their recent conversations; and two buttons: **Forget this person** (*Everything this agent remembers about them is deleted, now. Their next conversation starts fresh. This can't be undone.*) and **Reset devices** (*Every phone and browser signed in as them stops acting as them until they sign in again. Use this when someone changed phones and hit the device limit.*). The panel does not list the devices themselves.

4.5 Agent → Calls & scheduling

Graine can ring this agent's users inside your app — full screen, like a phone call — at a time the agent, the user or your team booked. Calls keep to this agent's Calling Window in the user's own time zone. The same screen is the *Calls* tab of Embed & Widgets. Set the agent's hours under *Calling Window* first.

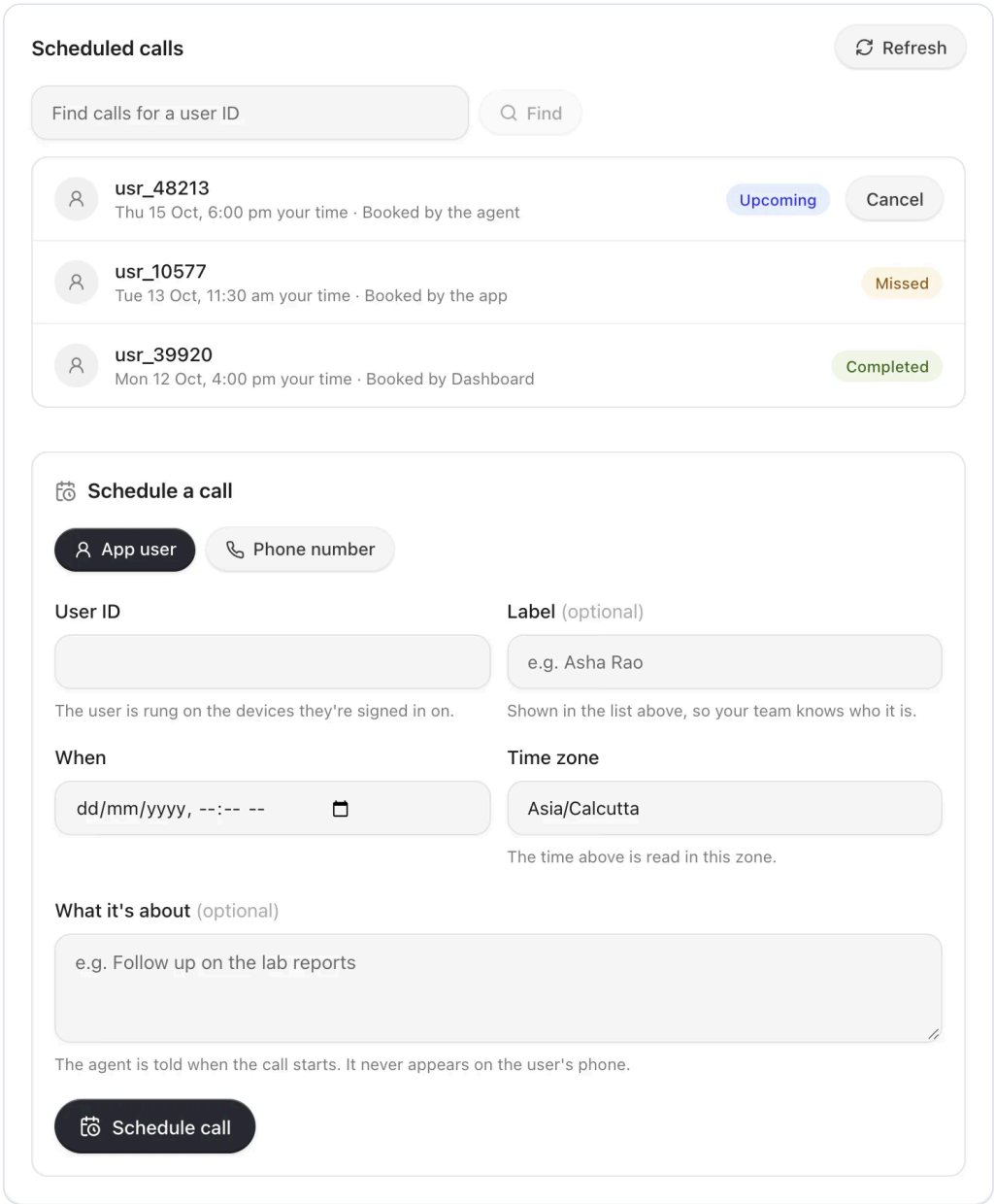


Who may book a call, and How it rings.

Setting	Default	Range	What it does
Who may book a call			
The agent, during a conversation	Off	On / off	When a user asks to be called back, the agent books it for the time they pick — in the app, the user is rung there; on a phone call, the number is called back.
Users, from your app	Off	On / off	Your app (or website) may call `scheduleCall`. A time a user picks is checked against the Calling Window when they book.
At most this many upcoming calls per user	3 calls	1–10	Pending bookings per customer (`maxPendingPerUser`).
Booked at most this far ahead	14 days	1–30	The furthest ahead (`maxDaysAhead`).
With at least this much notice	5 minutes	2–1440	The soonest (`minLeadMinutes`).
How it rings			

Setting	Default	Range	What it does
Name on the incoming call	The agent's name	Up to 40 characters	What the user's phone shows when it rings.
Ring for	45 seconds	20–60	How long a ring lasts before it is missed.
After a missed call, ring once more after	10 minutes	0–120	One re-ring after a miss; 0 never rings again. A declined call is never rung again.
Ring each user at most	3 times a day	1–10	Rings per customer per day, in their own day.
Quiet hours	On, 21:00 to 08:00	From / to	Calls the agent or your team book never ring in these hours, in the user's own time zone. A time the user picked themselves is theirs to keep.

Save with **Save call settings**. Above the settings, *The call screen* previews the ring, the call and a missed call from the saved design, and the card *Your app's push credentials* shows the workspace's credentials (below).



Scheduled calls and Schedule a call.

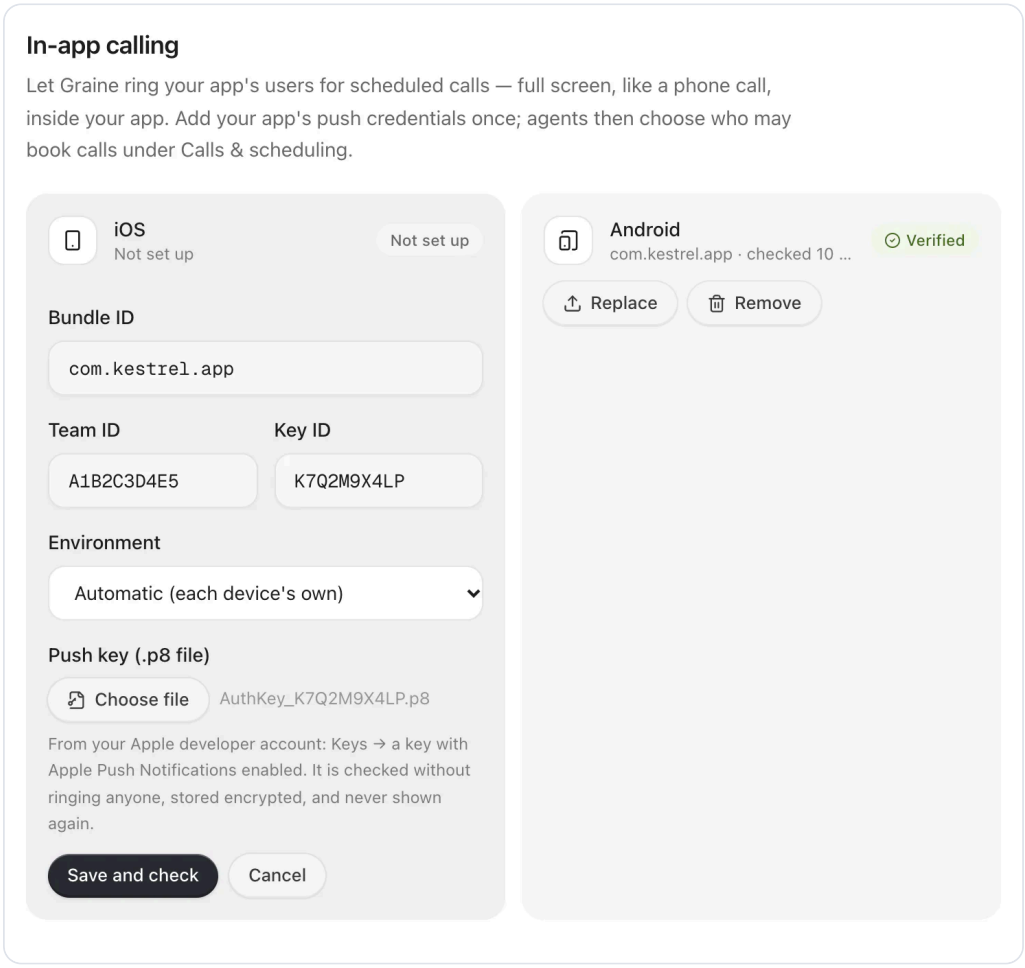
- **Schedule a call** (a form on the page): *App user* or *Phone number*; the identifier (labelled with your *Who is the user?* field, for example *User ID — The user is rung on the devices they're signed in on.*); *Label (optional)*; *When*; *Time zone*

(*The time above is read in this zone*); *What it's about (optional) — The agent is told when the call starts. It never appears on the user's phone.* Press **Schedule call**.

- **Ring a user now → Ring now**: *A test: rings this user's phone right away with the screen above, and shows what happened.* It follows every booking rule except the notice period (a test outside the Calling Window or quiet hours is refused), counts toward that user's rings for the day, and appears in the list as *Test ring*.
- **Scheduled calls** lists bookings (*Find calls for a …* by identifier, *Show all*, *Refresh*): who, "*… their time (zone) · … your time*", who booked it (*Booked by the agent*, *Booked by the app*, *Booked by Dashboard*, *Booked by API*) and a status chip: *Upcoming*, *Ringing*, *Answered*, *Completed*, *Declined*, *Missed*, *Failed*, *Cancelled*, *Too late to ring* (expired), *No device to ring*. **Cancel** appears while a call is *Upcoming* (*Cancel this call? → Cancel call*).

4.6 Workspace settings → In-app calling (push credentials)

Let Graine ring your app's users for scheduled calls — full screen, like a phone call, inside your app. Add your app's push credentials once; agents then choose who may book calls under Calls & scheduling. One card per platform; each shows *Not set up*, *Verified*, *Not working* (with the last error) or *Not checked yet*.



Account → Workspace settings → In-app calling.

Card	Fields	Where to get them
iOS	*Bundle ID*; *Team ID* and *Key ID* (10 letters and digits each); *Environment*: *Automatic (each device's own)*, *Production (App Store and TestFlight)*, *Sandbox (development builds)*; *Push key (.p8 file)*	Apple Developer account → Keys → a key with Apple Push Notifications enabled (download the `.p8` once; note its Key ID). Your Team ID is on your membership page.
Android	*Package name*; *Service-account key (.json file)*	Firebase console → Project settings → Service accounts → Generate new private key (a service account allowed to send messages for that project).

Press **Set up** (or **Replace**), choose the file, and **Save and check**. Graine checks without ringing anyone: on iOS it sends a push on your app's VoIP topic to a device token that does not exist — Apple answers "bad device token" when the key and topic are good, and refuses the key when not; on Android it sends a validate-only message. The credentials are stored encrypted and never shown again. **Remove** stops all ringing for that platform at once. If Apple or Google later refuses the credentials when a ring is sent, the card turns *Not working* and your admins get one notification in the dashboard.

4.7 The prompt: who_called and call_reason

For every native call the agent receives two prompt variables (they are not shown in the dashboard; use them in the prompt):

Variable	Values
`who_called`	`customer` — the customer placed the call from your app; `agent` — a ring your own push server sent; `scheduled` — a call Graine rang (a booked call), on every platform, and a phone call-back the agent booked. Treat `agent` and `scheduled` alike: your side rang the customer.
`call_reason`	The reason: what your app passed to `startCall`, or the booking's *What it's about* / `reason`. Left out when there is none.

A variable nothing sets is left out of the prompt, so write the prompt to read well without it:

Prompt (example)

```
If {who_called} is "agent" or "scheduled", you are calling the customer: greet them by name, say you are
calling from Acme, and ask whether now is a good time. The reason for the call, if there is one: {call_reason}.
If {who_called} is "customer", they called you: ask how you can help.
```

Other variables come from `identify()` traits (masked; never the identifier), `setVariables()`, the call's own variables and the agent's *Custom Variables*. A conversation carries up to 12 variables, about 1 KB in all.

5. Install and set up, per platform

Each platform below has the same parts: requirements, install, size, permissions and capabilities (word for word), what the SDK asks for at run time, store declarations, and the code that starts it. One rule holds everywhere: **the SDK asks for the microphone and nothing else**, and only when a conversation or a call starts without it. Everything else is declared by your app at build time; the notification permission Android needs for rings is yours to ask for.

5.1 SDK sizes (measured)

Measured on 14 October 2026 from the release sources, each in a clean temporary copy. Sizes are bytes as reported by the tool named.

SDK	What was measured	Size	How
React Native `@graineai/inapp-react-native` 0.41.0	The npm package (JavaScript and type definitions; no native code)	**144,420 B** download (≈141 KiB); 578,820 B unpacked; 62 files	`npm pack --dry-run` on a fresh build. Metro bundles only what your app imports.
React Native calls `@graineai/inapp-react-native-calls` 0.2.0	The npm package with the native call engine's sources, as published	**553,662 B** download (≈541 KiB); 1,986,348 B unpacked; 123 files	`npm pack --dry-run` after the publish step adds the engine. Its compiled size in your app is the iOS and Android rows.
iOS `GraineInApp` 0.7.0	The module's compiled object code, arm64, Release (`-O` , whole module)	**2.47 MB** of code and data: `__TEXT` 2,315,455 B (of which machine code 1,990,396 B) and `__DATA` 152,368 B. The object file is 8,053,768 B including debug information, symbols and relocations the linker drops.	`xcodebuild -scheme GraineInApp -destination generic/platform=iOS -configuration Release ARCHS=arm64` , Xcode 27 / Swift 6.4. The App Store download grows by less: the linker removes unused code, and the store compresses and thins the binary.
Android `ai.graine:inapp-android` 0.7.0	The release AAR	**1,198,357 B** AAR (≈1.14 MiB); `classes.jar` 1,245,300 B uncompressed, 468 classes	`./gradlew :graine-inapp:assembleRelease` . Runtime dependency (from the POM): `org.jetbrains.kotlin:kotlin-stdlib` 2.1.20 only. R8 in your release build shrinks it further.
Flutter `graine_inapp` 0.7.0	The pub.dev archive, including the native call engine's iOS and Android sources (vendored into the package)	**888 KB** compressed	`flutter pub publish --dry-run` on the package as it is exported for publishing (0 warnings). Its native part compiles to the iOS and Android rows.
Kotlin Multiplatform 0.7.0	The Android AARs of the shared library	`inapp-kmp` **237,741 B** (classes.jar 252,692 B, 125 classes); `inapp-kmp-compose` **12,952 B**	`assembleRelease` of the Android side. Adds `ai.graine:inapp-android` (row above), `kotlinx-coroutines-core` 1.10.2 and `kotlinx-datetime` 0.6.2; Compose for the compose module. The iOS side links the iOS SDK (row above).
Website loader 1.2.0	`https://www.graine.ai/embed/v1.js`	**64,667 B**; **20,560 B** gzipped (level 9)	`wc -c` ; `gzip -9`
Engine page (loaded by the SDKs and the widget)	What a first load downloads	About 5 kB of HTML and 362 kB of compressed scripts over 20 requests, cached as immutable	Graine's own measurement against the live site, recorded with its engine-weight check

5.2 React Native

Packages	`@graineai/inapp-react-native` 0.41.0 (the agent) · `@graineai/inapp-react-native-calls` 0.2.0 (native calls, optional)
Requires	The SDK: React Native 0.68+ and React 17+ (its peer dependencies), plus `react-native-webview` (the voice engine runs in it). The calls package: React Native 0.71+, React 18+, a native build (not Expo Go), iOS 15+ (its pod) and Android 7 / API 24+ (its Gradle module, Java 17). Node 18+ for development.
Recommended	`expo-font` (passed as `loadFont`: the SDK then loads the design's typeface) and `react-native-svg` (passed as `svg`: icons in the design's style). Without them: the system font and plain glyphs.
Expo	A development build (SDK 50+). Expo Go cannot load `react-native-webview` or the calls package. Neither package ships an Expo config plugin: put the settings in `app.json` (below).

Install

```
npm install @graineai/inapp-react-native react-native-webview
npm install @graineai/inapp-react-native-calls      # native calls (optional)
npm install expo-font react-native-svg              # the design's typeface and icons (recommended)
cd ios && pod install
```

Permissions and capabilities

- **The SDK package carries no manifest.** Declare the microphone yourself: Android `RECORD_AUDIO` and `MODIFY_AUDIO_SETTINGS`; iOS `NSMicrophoneUsageDescription`.
- **The calls package's Android manifest merges everything a call needs** — the same list as the Android SDK (microphone, notifications, full-screen ring, microphone foreground service, the call screen, its service and receiver). On iOS, add the capabilities and the usage string from the iOS section: Push Notifications, Background Modes → Audio and Voice over IP.
- **At run time:** on Android the SDK asks for the microphone when a call starts; on iOS the system asks the first time the microphone is used. The calls package asks for the microphone before it places or answers a call. **Notifications on Android 13+ are yours to ask for** (`POST_NOTIFICATIONS`) — without them a ring cannot show while your app is closed.
- **iPhone ringer switch:** the SDK leaves the app's audio session to you for the bar's voice — while a conversation is live, set your audio library's equivalents of `playsInSilentModeIOS: true` and `allowsRecordingIOS: true`, or the agent is silent when the ringer is off. (Native calls set their own session.)

JSON — `app.json (Expo)`

```
{
  "expo": {
    "ios": {
      "infoPlist": {
        "NSMicrophoneUsageDescription": "Lets you talk to the assistant in the app and on calls.",
        "UIBackgroundModes": ["audio", "voip"]
      },
      "entitlements": { "aps-environment": "production" }
    },
    "android": {
      "permissions": ["android.permission.RECORD_AUDIO", "android.permission.MODIFY_AUDIO_SETTINGS"]
    }
  }
}
```

`aps-environment` is the Push Notifications capability: `development` for development builds, `production` for TestFlight and App Store builds. With the calls package installed, its manifest adds the Android entries at build time anyway.

Initialise

React Native · TypeScript — App.tsx

```
import { useEffect, useState } from "react";
import { WebView } from "react-native-webview";
import * as Font from "expo-font";
import Svg, { Path } from "react-native-svg";
import {
  GraineProvider, GraineVoiceLauncher, GraineVoiceBar, GraineLauncherButton, useGraineClient,
} from "@graineai/inapp-react-native";
import { GraineCalls } from "@graineai/inapp-react-native-calls";

export default function App() {
  const [open, setOpen] = useState(false);
  return (
    <GraineProvider
      baseUrl="https://www.graine.ai"
      publishableKey="pk_live_…"            // Agent → Embed & Widgets → Install
      navigationRef={navigationRef}         // the same ref as your NavigationContainer: screens are reported
for you
      loadFont={Font.loadAsync}
      svg={{ Svg, Path }}
    >
      <NavigationContainer ref={navigationRef}><RootNavigator /></NavigationContainer>
      <GraineVoiceLauncher webView={WebView} active={open} autoStart>
        {(api) => (open ? <GraineVoiceBar api={api} onClose={() => setOpen(false)} /> : null)}
      </GraineVoiceLauncher>
      {!open && <GraineLauncherButton onPress={() => setOpen(true)} style={{ position: "absolute", right: 16,
bottom: 24 }} />}
      <CallsBridge />
    </GraineProvider>
  );
}

// Native calls: configure once and hand the call what the app tells the agent.
function CallsBridge() {
  const client = useGraineClient();
  useEffect(() => {
    GraineCalls.configure({ publishableKey: "pk_live_…", displayName: "Asha from Acme" });
    return GraineCalls.attach(client);      // identity, variables, the screen and your actions reach the call
  }, [client]);
  return null;
}
```

`GraineVoiceLauncher` owns the engine and the call; mounted early with `active={false}` it loads the engine before the tap. `GraineVoiceBar` draws the bar, the chat window (where the agent offers chat) and components, in the dashboard's design.

5.3 iOS (Swift)

Package	`GraineInApp` 0.7.0 — a Swift package with no dependencies
Requires	iOS 15+ (Mac Catalyst 15+ also builds); Swift tools 5.9 (Xcode 15 or later). It links Apple's AVFoundation, CallKit, Combine, CoreGraphics, CoreText, PushKit, SwiftUI, UIKit and WebKit.
Calls need	A real iPhone (CallKit and VoIP pushes do not run in the simulator), an Apple Developer account and an APNs auth key for hosted ringing

Install — Swift Package Manager (primary)

Xcode → **File → Add Package Dependencies…** → `https://github.com/GraineAI/inapp-ios`, version **0.7.0** or later, and add the `GraineInApp` library to your app target. In a `Package.swift`:

Package.swift

```
dependencies: [
    .package(url: "https://github.com/GraineAI/inapp-ios", from: "0.7.0"),
],
targets: [
    .target(name: "AcmeApp", dependencies: [.product(name: "GraineInApp", package: "inapp-ios")]),
]
```

Install — CocoaPods

CocoaPods — Podfile

```
platform :ios, '15.0'
target 'AcmeApp' do
  use_frameworks!
  pod 'GraineInApp', '~> 0.7'
  # or straight from the tag:
  # pod 'GraineInApp', :git => 'https://github.com/GraineAI/inapp-ios.git', :tag => '0.7.0'
end
```

The pod builds exactly what the Swift package builds (the same sources, iOS 15, Swift 5.9) and carries the privacy manifest in its own resource bundle, `GraineInApp_Privacy`.

Permissions and capabilities

Add (Xcode → the app target)	For	Why
`NSMicrophoneUsageDescription` in `Info.plist`	Voice and calls	Required: iOS stops an app that asks for the microphone without it. Suggested text: *Lets you talk to the assistant in the app and on calls.*
Signing & Capabilities → Push Notifications	Calls that ring	VoIP pushes reach the app (the `aps-environment` entitlement)
Background Modes → Audio, AirPlay, and Picture in Picture (`audio`)	Calls	The call's audio keeps going with the screen off
Background Modes → Voice over IP (`voip`)	Calls that ring	iOS wakes a closed app for a VoIP push

XML — Info.plist

```
<key>NSMicrophoneUsageDescription</key>
<string>Lets you talk to the assistant in the app and on calls.</string>
<key>UIBackgroundModes</key>
<array>
    <string>audio</string>
    <string>voip</string>
</array>
```

Nothing else: no notification permission (VoIP pushes and CallKit need none), no Bluetooth usage description, no remote-notification background mode. The SDK ships a **privacy manifest** declaring the required-reason APIs it calls — system boot time (reason `35F9.1`, for timing) and `UserDefaults` (reason `CA92.1`) — and no tracking and no tracking domains; Xcode adds it to your app's privacy report. What your app collects through the agent, the customer's voice included, goes in your App Privacy details.

What the SDK configures in CallKit

CallKit setting	Value
Call name (caller on the system screen, Recents)	For a ring Graine sends: the agent's *Name on the incoming call* (else the agent's name). For a call the customer places, and a ring from your own server without a name: `GraineCallsConfig.displayName`; with several agents, each agent's call name first; empty, your app's display name.
Icon template (the app button on the system call screen)	`GraineCallsConfig.iconTemplateImageData` (a 40 × 40 pt transparent PNG); otherwise the brand's launcher glyph from the design, drawn as a template image
Ringtone	`GraineCallsConfig.ringtoneSound` (a sound file in your bundle, 30 seconds or less); otherwise the Garage's *Ringtone — Graine* or *Soft* written as a sound file for CallKit, or the phone's own ringtone for *Phone's own*
Calls in the Phone app's Recents	`GraineCallsConfig.includesCallsInRecents`, off by default
Video, hold, grouping, keypad	Off: voice only, one call at a time (one call group, one call per group)
Ring timeout	A ring Graine sends rings for the agent's *Ring for*; any other for `GraineCallsConfig.ringTimeout` (30 seconds by default)
Audio session	Play-and-record in voice-chat mode, Bluetooth hands-free allowed, the earpiece by default; activated by CallKit, and the app's own session restored afterwards

- **Every VoIP push is reported to CallKit at once.** iOS requires it and stops delivering VoIP pushes to an app that does not. The SDK reports every one — even one it cannot use, or one that arrives during a live call (that one is ended at once and reported `missed`; the live call is untouched). Send VoIP pushes only for calls. An app with its own `PKPushRegistry` hands the payload to `reportIncomingCall(payload:completion:)` instead.
- **On the lock screen** the ring is the system's incoming-call screen (a banner while the phone is in use), with your call name, icon and ringtone. Answered there, the call connects; the system call screen stays on the lock screen for the call, and your app shows `GraineCallView` when it is opened. Do Not Disturb, a Focus or the block list can stop a ring; it is then reported `missed`.
- **The microphone:** for a call the customer places, the SDK asks before placing it. For a ring, the customer answers first; the SDK reports `answered`, iOS asks while the call connects, and a refusal — or no answer within 60 seconds — ends it `failed`. Ask during onboarding so a call answered on the lock screen already has it.
- Apple does not allow CallKit in apps on the App Store in China: leave calls off there.

Initialise

iOS · Swift — AcmeApp.swift

```
import SwiftUI
import GraineInApp

@main
struct AcmeApp: App {
    @UIApplicationDelegateAdaptor(AppDelegate.self) private var appDelegate
    var body: some Scene { WindowGroup { RootView() } }
}

final class AppDelegate: NSObject, UIApplicationDelegate {
    func application(_ application: UIApplication,
                     didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]? = nil)
-> Bool {
        let graine = GraineInApp.configure(GraineConfig(
            baseUrl: "https://www.graine.ai",
            publishableKey: "pk_live_…",          // Agent → Embed & Widgets → Install
            appVersion: "2.4.0"
        ))
        graine.calls.configure(GraineCallsConfig(displayName: "Asha from Acme"))
```

```
        graine.calls.enableVoIPPushes()          // every launch: iOS hands over a VoIP push only once this has run
        return true
    }
}

struct RootView: View {
    @ObservedObject private var calls = GraineInApp.shared.calls
    var body: some View {
        AppContent()
            .overlay(GraineAgentBar())           // the bar: voice, chat and components, in your design
            .fullScreenCover(isPresented: .constant(calls.showsCallScreen)) {
                GraineCallView()                 // the call while your app is open
            }
    }
}
```

The bar mounts the voice engine in the key window the first time it appears (`autoAttach`), so a call carries on as the customer moves through your app. UIKit apps host the SwiftUI views in a `UIHostingController`, or use `GraineChatViewController`; without the bar, mount the engine with `GraineInApp.shared.voice.attach()`.

5.4 Android (Kotlin, Java)

Artifact	`ai.graine:inapp-android:0.7.0` on Maven Central
Requires	Android 7.0 (API 24) and up; Java 17 bytecode (Android Gradle Plugin 8 or later); built against compileSdk 36 with Kotlin 2.1.20. No AndroidX, OkHttp or Compose dependency; the only runtime dependency is `kotlin-stdlib`. Its R8/ProGuard rules ship inside it.
Calls need	Firebase Cloud Messaging in your app (`google-services.json` and the messaging library) for rings

Gradle (Kotlin DSL) `app/build.gradle.kts`

```
dependencies {
    implementation("ai.graine:inapp-android:0.7.0")
    // For calls that ring (hosted ringing): Firebase Cloud Messaging, as Firebase describes
    implementation(platform("com.google.firebase:firebase-bom:<latest>"))
    implementation("com.google.firebase:firebase-messaging")
}
```

What the library's manifest merges into your app

Entry	Why
`android.permission.INTERNET`	The voice engine and the conversation's connection
`android.permission.RECORD_AUDIO`	The agent hears the customer (voice and calls). A runtime permission: the SDK asks
`android.permission.MODIFY_AUDIO_SETTINGS`	A call routes its audio — earpiece, loudspeaker or a headset, Bluetooth included (no Bluetooth permission is needed)
`android.permission.POST_NOTIFICATIONS`	The ring, the ongoing call and a missed call are notifications. A runtime permission on Android 13+: **your app asks**
`android.permission.USE_FULL_SCREEN_INTENT`	A ring that fills a locked screen. On Android 14+ Google Play grants it by default only to apps whose main purpose is calling or alarms; without it the ring is a heads-up notification, still ringing
`android.permission.FOREGROUND_SERVICE`	The call runs in a foreground service

Entry	Why
`android.permission.FOREGROUND_SERVICE_MICROPHONE`	That service keeps the microphone with the screen off (type `microphone`)
`ai.graine.inapp.GraineCallActivity`	The call screen: its own task (so ending a call returns the customer to where they were), shown over the lock screen, not exported
`ai.graine.inapp.GraineChatActivity`	The chat on a screen of its own (`GraineChatActivity.open`), above the keyboard; not exported
`ai.graine.inapp.GraineCallService`	The call's foreground service, `foregroundServiceType="microphone"`, not exported
`ai.graine.inapp.GraineCallReceiver`	The notifications' Decline and Hang up, not exported

What the SDK asks at run time: only `RECORD_AUDIO` — at `voice.start()` for the bar (forward your activity's `onRequestPermissionsResult` to `voice.onRequestPermissionsResult`), and from the call screen when a call is placed or after Answer (a refusal ends the call `failed`; no answer within 60 seconds too). **Your app asks for notifications** on Android 13+; without them, a ring with your app closed or in the background cannot show and ends `missed`. **Your app offers the full-screen setting** on Android 14+:

Android · Kotlin — onboarding

```
private val ask = registerForActivityResult(ActivityResultContracts.RequestMultiplePermissions()) { }

fun askForAgentPermissions() {
    val wanted = mutableListOf(Manifest.permission.RECORD_AUDIO)
    if (Build.VERSION.SDK_INT >= 33) wanted += Manifest.permission.POST_NOTIFICATIONS   // rings
    ask.launch(wanted.toTypedArray())
}

fun offerFullScreenRings() {
    val nm = getSystemService(NotificationManager::class.java)
    if (Build.VERSION.SDK_INT >= 34 && !nm.canUseFullScreenIntent()) {
        startActivity(Intent(Settings.ACTION_MANAGE_APP_USE_FULL_SCREEN_INTENT,
Uri.parse("package:$packageName")))
    }
}
```

Notification channels the SDK creates in `calls.configure` (visible in the phone's settings): *Incoming calls* (high importance, the ringtone, vibration, shown on the lock screen), *Incoming calls (app ringtone)* (used while the SDK plays the brand's *Graine* or *Soft* ring), *Missed calls* and *Ongoing calls*.

The ring is a call-style notification (Android 12+; *Decline* and *Answer* actions before that) that keeps ringing until answered, declined or timed out, with a full-screen intent to the call screen; when your app is in front, the call screen opens straight away. **The call** runs in the microphone foreground service with an ongoing notification (*On a call* / *Connecting…*, a timer, *Hang up*). A ring nobody answered leaves a *Missed calls* notification with **Call back**.

Not using calls? On Android 14+, Google Play asks apps that declare `USE_FULL_SCREEN_INTENT` or `FOREGROUND_SERVICE_MICROPHONE` to justify them. An app that never places or receives calls removes them in its own manifest:

XML — AndroidManifest.xml (an app without calls)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.USE_FULL_SCREEN_INTENT" tools:node="remove" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_MICROPHONE" tools:node="remove" />
    <application>
        <service android:name="ai.graine.inapp.GraineCallService" tools:node="remove" />
```

```
    </application>
</manifest>
```

Initialise

Android · Kotlin — AcmeApp.kt

```kotlin
class AcmeApp : Application() {
    override fun onCreate() {
        super.onCreate()
        val graine = GraineInApp.init(this, GraineConfig(
            baseUrl = "https://www.graine.ai",
            publishableKey = "pk_live_…",               // Agent → Embed & Widgets → Install
            appVersion = BuildConfig.VERSION_NAME,
            debug = BuildConfig.DEBUG,
        ))
        graine.calls.configure(GraineCallsConfig(
            displayName = "Asha from Acme",
            smallIcon = R.drawable.ic_stat_call,        // monochrome; 0 = the platform's call icon
        ))
    }
}

class MainActivity : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main)
        GraineInApp.get().voice.attach(this)            // the engine, idle: once; it follows the customer
between activities
        findViewById<FrameLayout>(R.id.root).addView(GraineAgentBarView(this),
            FrameLayout.LayoutParams(MATCH_PARENT, WRAP_CONTENT, Gravity.BOTTOM))
    }

    override fun onRequestPermissionsResult(requestCode: Int, permissions: Array<out String>, grantResults:
IntArray) {
        if (!GraineInApp.get().voice.onRequestPermissionsResult(requestCode, grantResults))
            super.onRequestPermissionsResult(requestCode, permissions, grantResults)
    }
}
```

Jetpack Compose hosts the views with `AndroidView(factory = { GraineAgentBarView(it) })`. From Java, every entry point works with only the arguments you need (`new GraineConfig(baseUrl, publishableKey)`); listeners have default methods.

5.5 Flutter

Package	`graine_inapp` 0.7.0 on pub.dev (Android and iOS)
Requires	Flutter 3.19+ and Dart 3.3+; iOS 15+ (its pod, Swift 5.9); Android 7 / API 24+ (Java 17, compileSdk 36). Depends on `http` and `webview_flutter` (4.8+) only.
Native code	The package carries the native call engine (the iOS and Android SDKs' call code), so adding it or updating it needs a new store build

YAML — pubspec.yaml

```yaml
dependencies:
  graine_inapp: ^0.7.0
  permission_handler: ^11.0.0     # or whatever your app already uses to ask for permissions
  firebase_core: any              # rings on Android (hosted ringing)
  firebase_messaging: any
```

- **Android:** the package's manifest merges every entry listed in the Android section; set `minSdk` 24 or higher. Ask for notifications on Android 13+ yourself (for example `Permission.notification.request()`).
- **iOS:** on the Runner target add `NSMicrophoneUsageDescription`, Push Notifications, and Background Modes → Audio and Voice over IP (the iOS section).
- **Microphone:** for the bar's voice, your app asks, in `GraineConfig.requestMicrophone`, which the SDK calls before every call (left out, the SDK assumes you asked). For native calls, the SDK asks.

Flutter · Dart `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:graine_inapp/graine_inapp.dart';
import 'package:permission_handler/permission_handler.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  GraineInApp.init(GraineConfig(
    baseUrl: 'https://www.graine.ai',
    publishableKey: 'pk_live_…',                       // Agent → Embed & Widgets → Install
    requestMicrophone: () async => (await Permission.microphone.request()).isGranted,
  ));
  await GraineCalls.instance.configure(const GraineCallsOptions(
    publishableKey: 'pk_live_…',
    displayName: 'Asha from Acme',
  ));
  GraineCalls.instance.attach();                       // identity, variables, the screen and actions reach the
call
  runApp(MaterialApp(
    // Once, above the navigator: a call outlives every route.
    builder: (context, child) => GraineEngine(child: Stack(children: [child!, const GraineAgentBar()])),
    home: const HomeScreen(),
  ));
}
```

5.6 Kotlin Multiplatform

Artifacts	`ai.graine:inapp-kmp:0.7.0` (the shared API) and `ai.graine:inapp-kmp-compose:0.7.0` (the chat, call screen and voice bar for Compose Multiplatform), on Maven Central
Requires	Kotlin 2.1.20+; targets `androidTarget`, `iosArm64`, `iosSimulatorArm64`, `iosX64`; Android 7 / API 24+; iOS 15+. Depends on `kotlinx-coroutines` and `kotlinx-datetime` (and Compose for the compose artifact)
How it works	One API in shared code over the native SDKs: the Android SDK comes with it; on iOS your app links the iOS SDK (`GraineInApp` 0.7.0), so it is never in the app twice

Gradle (Kotlin DSL) `shared/build.gradle.kts`

```
kotlin {
    androidTarget()
    iosArm64(); iosSimulatorArm64(); iosX64()
    sourceSets {
        commonMain.dependencies {
            implementation("ai.graine:inapp-kmp:0.7.0")
            implementation("ai.graine:inapp-kmp-compose:0.7.0")
        }
    }
}
```

- **Android:** the Android SDK's manifest (above) comes with it; the library finds your application itself, so `Graine.configure(config)` works from shared code. Ask for `POST_NOTIFICATIONS` on Android 13+. An activity that shows the voice bar or places calls forwards permission answers to

`Graine.onRequestPermissionsResult(requestCode, grantResults)`. For rings, your `FirebaseMessagingService` calls `Graine.calls.onNewToken(token)` and `Graine.calls.handlePush(message.data)`.

- **iOS:** add the iOS SDK to the app — Swift Package Manager (`https://github.com/GraineAI/inapp-ios`, from 0.7.0) with your shared framework built static (`isStatic = true`, the Compose Multiplatform default), or CocoaPods (`pod 'GraineInApp'`; with Kotlin's CocoaPods plugin and a dynamic framework, `pod("GraineInApp") { version = "0.7.0"; linkOnly = true }`). Add the `Info.plist` keys and capabilities of the iOS section, and call `Graine.calls.enableVoIPPushes()` at every launch.

Kotlin Multiplatform · Kotlin — commonMain

```
Graine.configure(GraineConfig(
    baseUrl = "https://www.graine.ai",
    publishableKey = "pk_live_…",          // Agent → Embed & Widgets → Install
    appVersion = "2.4.0",
))
Graine.calls.configure(GraineCallsConfig(displayName = "Asha from Acme", ringTimeoutSeconds = 30))

// Compose Multiplatform UI
Box(Modifier.fillMaxSize()) {
    AppContent()
    GraineAgentBar(Modifier.align(Alignment.BottomCenter).fillMaxWidth().height(180.dp))
}
```

Every method may be called from any thread; suspending functions never block the main thread; states are flows that always hold the latest value.

5.7 Website

Loader	`https://www.graine.ai/embed/v1.js`, version 1.2.0 — plain JavaScript, no build step, no dependencies
Requires	The page's host in the agent's Allowed domains. A site with a Content-Security-Policy allows `https://www.graine.ai` in `script-src`, `connect-src` and `frame-src` (the loader fetches from and frames that origin). The panel's frame asks for the microphone (`allow="microphone"`), so a Permissions-Policy must not deny it to that frame.

HTML — Before </body>, on every page the agent belongs on

```
<script src="https://www.graine.ai/embed/v1.js"></script>
<script>
  GraineAgent.load({
    agentId: "your-website-agent-id",      // Embed & Widgets → Install fills these in
    publishableKey: "pk_live_…",
  });
</script>
```

The widget draws the Garage's design, loads its typeface inside its own frame, and keeps the browser's install ID in `localStorage`. Everything the page can call — `identify`, `setScreen`, `defineAction`, `scheduleCall` — is safe before `load()`: it is held until the widget exists.

5.8 Store review

Store	What to declare
App Store	**Review notes:** say the app offers voice calls with your assistant; give the reviewer a test account and where the call button is; explain that agent calls ring through VoIP pushes your backend (or Graine) sends, which the reviewer cannot trigger. **VoIP:** pushes are used only for calls, and every one is reported to CallKit. **Privacy:** the microphone purpose string; App Privacy details for the audio and data your app sends through the agent; the SDK's privacy manifest covers its required-reason APIs. **China:** no CallKit on the China storefront.

Store	What to declare
Google Play	**Foreground service permissions** (App content): the call's service is type `microphone` — describe two-way voice calls with your assistant that keep going when the screen locks, with a short video. **Full-screen intent:** declare it (granted by default only to calling and alarm apps) or go without; rings are then heads-up notifications. **Data safety:** the audio and data your app sends for the conversation.

6. Identity and memory

Memory, conversations that continue and calls that ring a customer all need to know **who** the customer is. On a phone call it is always the phone number. In your app and on your website it is the trait the agent's **Who is the user?** setting names (Agent → Memory → Settings): `user_id` by default, or `email`, `phone`, or a field of your own such as `customer_id`. Your app passes it with `identify()`.

6.1 identify(...) after sign-in, identify(null) on sign-out

React Native · TypeScript

```
const identify = useGraineIdentify();
useEffect(() => {
  identify(user ? { user_id: user.id, name: user.firstName, plan: user.plan } : null, { memory:
user?.allowsMemory ?? true });
}, [user]);
// outside React: client.identify({ user_id: user.id, name: user.firstName }); client.identify(null);
```

iOS · Swift

```
GraineInApp.shared.identify(["user_id": user.id, "name": user.firstName, "plan": user.plan], memory:
user.allowsMemory)
// on sign-out, and before another customer signs in
GraineInApp.shared.identify(nil)
```

Android · Kotlin

```
GraineInApp.get().identify(mapOf("user_id" to user.id, "name" to user.firstName, "plan" to user.plan), memory =
user.allowsMemory)
// on sign-out
GraineInApp.get().identify(null)
```

Flutter · Dart

```
GraineInApp.instance.identify({'user_id': user.id, 'name': user.firstName, 'plan': user.plan}, memory:
user.allowsMemory);
// on sign-out
GraineInApp.instance.identify(null);
```

Kotlin Multiplatform · Kotlin

```
Graine.identify(mapOf("user_id" to user.id, "name" to user.firstName), memory = user.allowsMemory)
Graine.identify(null)                                    // sign out
```

Website · JavaScript

```
GraineAgent.identify({ user_id: user.id, name: user.firstName }, { memory: user.allowsMemory });
GraineAgent.identify(null);                              // on sign-out: every agent on the page
```

Trait	Masked on the device	A prompt variable	Identifies the customer
The one *Who is the user?* names (here `user_id`)	No — kept exactly	**Never**	Yes
Every other trait (`name`, `plan`, …)	Yes	Yes — `{name}`, `{plan}`	No

- **How values are matched.** *User ID* and custom fields: exactly, case-sensitive, up to 256 characters. *Email*: lower-cased; it must look like an address (up to 254 characters). *Phone number*: its digits, 8 to 15 of them, and the last ten count — so `+91 98765 43210` and `098765 43210` are one person, and a caller with that number is the same person as the app user. Send the identifier as a string (an integer is read as its digits).
- **What is no identifier at all:** empty, `null`, a boolean, an object, a decimal; a placeholder in any case — `none`, `nil`, `null`, `undefined`, `nan`, `unknown`, `anonymous`, `anon`, `guest`, `test`, `0`, `-`, `n/a`, `na`; or an already-masked value such as `[PHONE]` or `[EMAIL]`. Such a conversation has no memory and cannot be the target of a call.
- **When it applies.** Identity is fixed when a conversation starts; a sign-in during a conversation applies to the next one. You can call `identify()` before the SDK has loaded the agent's settings.
- **`identify(null)` signs out:** the identity is forgotten on the device, the device is released from the customer and unlinked from their calls, and later conversations are anonymous. Identifying a **different** customer signs the previous one out first, and the chat on screen is cleared. With several agents, one `identify()` applies to all of them, and `identify(null)` signs out of all of them.
- **The memory choice** (`memory: false`, or `setMemory(false)`) is kept for that customer on this install; nothing of theirs is recalled or stored (and, from 0.7.0, their conversation is not kept) until they turn it on again.

6.2 Installs and the device limit

Each install gets an **install ID** the first time it runs — a random UUID, never shown to the agent, kept across launches: in the iOS keychain (this device only; it survives a reinstall), in Android app storage excluded from backups, through the native side in React Native (or the `storage` you give the client) and Flutter, and in the browser's `localStorage` on a website. Read it as `installId`.

Devices per user (1–5, default 2) caps how many installs can act as one customer. The first installs to identify as a customer are **bound** to them. Once the limit is reached, another install that claims the same identifier still works, but for that customer it gets no memory, cannot be rung for their calls, and is answered `device_limit` when it asks for or forgets their memory. A device is released by `identify(null)` on it, after 30 days unseen, or by **Reset devices** on the person's panel in Agent → Memory → People.

6.3 Reading and erasing memory: memories(), setMemory(), forgetMemory()

Platform	What is remembered	Forget it	Turn it off / on
React Native	`client.memories(): Promise<GraineMemory[]>` · `useGraineMemories()`	`client.forgetMemory(): Promise<number>`	`client.setMemory(enabled)` · `client.memoryEnabled`
iOS	`memories() async throws -> [GraineMemory]`	`forgetMemory() async throws -> Int`	`setMemory(_:)` · `memoryChoice`
Android	`memories(done: GraineCallback<List<GraineMemory>>)`	`forgetMemory(done: GraineCallback<Int>? = null)`	`setMemory(enabled)` · `memoryEnabled`
Flutter	`Future<List<GraineMemory>> memories()`	`Future<int> forgetMemory()`	`setMemory(bool)` · `Future<bool> memoryEnabled()`
Kotlin MP	`suspend fun memories(): List<GraineMemory>`	`suspend fun forgetMemory(): Int`	`setMemory(enabled)` · `memoryEnabled`
Website	`GraineAgent.memories(): Promise<Array>`	`GraineAgent.forgetMemory(): Promise<number>`	`GraineAgent.setMemory(enabled)`

`GraineMemory` is `{ id, text, createdAt, updatedAt }` (dates may be null), newest first, at most 100, already redacted. `forgetMemory()` returns how many were deleted and erases what is remembered for this agent's memory scope (for every agent sharing it when *Shared with* is *Agents that share memory*); with conversations that continue (0.7.0) it also clears the customer's kept conversation with this agent. Both work only for an identified customer on one of their devices (`identity_required`, `device_limit` otherwise).

iOS · Swift — a privacy screen

```
@MainActor final class PrivacyModel: ObservableObject {
    @Published var remembered: [GraineMemory] = []
    @Published var problem: String?

    func load() async {
        do { remembered = try await GraineInApp.shared.memories() }
        catch let error as GraineServiceError { problem = error.message }
        catch { problem = "Couldn't load what the assistant remembers." }
    }

    func forgetAll() async {
        do {
            let deleted = try await GraineInApp.shared.forgetMemory()
            remembered = []
            problem = "Deleted \(deleted) memories."
        } catch let error as GraineServiceError where error.code == .deviceLimit {
            problem = "Sign out on another device first."
        } catch { problem = "Couldn't delete them just now." }
    }

    func setRemembering(_ on: Bool) { GraineInApp.shared.setMemory(on) }
}
```

Android · Kotlin — a privacy screen

```
GraineInApp.get().memories { memories, error ->
    if (error != null) showProblem(error.message) else render(memories.orEmpty())   // main thread; exactly one is set
}
GraineInApp.get().forgetMemory { deleted, error ->
    if (error?.code == GraineErrorCode.DEVICE_LIMIT) showProblem("Sign out on another device first.")
    else if (error == null) showDone("Deleted $deleted memories.")
}
GraineInApp.get().setMemory(false)
```

React Native — a privacy screen

```
const client = useGraineClient();
const { memories, loading, error, refresh } = useGraineMemories();   // loads on mount and after forgetMemory
async function forgetAll() {
  try { const deleted = await client.forgetMemory(); show(`Deleted ${deleted} memories.`); }
  catch (e) { if (e instanceof GraineError) show(e.message); }
}
```

Flutter · Dart — a privacy screen

```
try {
  final remembered = await GraineInApp.instance.memories();
  final deleted = await GraineInApp.instance.forgetMemory();
} on GraineException catch (e) {
  if (e.code == GraineErrorCode.deviceLimit) showProblem('Sign out on another device first.');
}
```

6.4 What Graine stores

Graine stores the identifier your app sends so your team can see who each conversation was with; the memory service only ever receives a one-way key derived from it.

- **The identifier** is kept on each conversation's record with its field, in the form it is compared in (trimmed, an email lower-cased, a phone number as its last ten digits, at most 256 characters). Call History shows it as **<field label> ·**

<value> — *User ID · u_8812*, *Email · asha@acme.example*, *Customer ID · C-20931* (a custom field's name written out) — and so do Agent → Memory → People and the person's panel. A call your team booked in the dashboard shows the identifier they typed from the moment it rings. It never becomes a prompt variable.

- **The user key** — a one-way fingerprint of your workspace, the *Who is the user?* field and the value — is what memory, the kept conversation, devices and bookings are filed under. The same value in two workspaces gives two unrelated keys. Keys never change, so memory survives anything short of changing the identity setting. The install ID is kept only as a one-way key too.
- **Memory** holds short facts, redacted before they are stored, for the agent's *Keep memories for*. Deleting an agent does not delete its memories early; they expire.

> **How far to trust an identifier**
>
> With a publishable key and no server of yours in the path, the identifier is what your app says. What bounds it: the device limit and the customer's own sign-out; rings that carry nothing personal; rate limits per key, per install and per address. Treat identity as recognising a returning customer, not as proof: before the agent discusses an account, confirm the customer through one of your actions.

7. Conversations continue

A customer who comes back — later today, tomorrow, on their other phone — sees the earlier messages in the chat window, and the agent picks up where they left off. Memory keeps long-term facts; this keeps the recent conversation itself.

7.1 Who gets it

Exactly the customers memory applies to: an in-app or website conversation (a chat, the bar's voice, a native call) with an identified customer, on an agent whose memory is on, who has not opted out (`memory: false`), on a device that is one of theirs. Anyone else — anonymous, opted out, a device past the limit, a dashboard preview, a phone call — gets nothing kept and nothing restored. There is never a shared bucket.

7.2 What is kept, and for how long

- The most recent **50 messages** per agent and customer — each `{ id, role, text, at, channel }`, where `channel` is `chat` or `voice` — for **7 days after the last one** (each new message renews the 7 days).
- Per **agent**, even when memory is shared across agents: your support agent never shows your sales agent's chat. One customer's chat, bar calls and native calls with the same agent are one conversation.
- Messages are kept as said — the same text Call History keeps — up to 2,000 characters each. Nothing is kept until the customer has said something in the conversation.

7.3 What the customer sees, and what the agent knows

- **No repeated welcome.** A new chat with earlier messages starts from them: the chat window receives the earlier conversation before any reply, and the scripted welcome message is not sent — the customer types next. The window opens on the conversation, not on the empty "New chat" state with its starters.
- **One row per voice call.** Earlier chat messages are bubbles; a run of messages said on a call is one compact row, *Voice call · {time}* (the transcript stays in Call History). Date separators (*Today*, *Yesterday*, a date) mark where the local day changes.
- **The agent** receives the newest 20 turns (up to 8,000 characters) and is told it is continuing an earlier conversation and should not introduce itself again unless asked. A voice call still opens with its spoken greeting — a call is a new call — but the agent knows the history.
- **Restored turns** are delivered (not live), carry their original times, and are marked `restored` on the chat turn: `restored?: true` on React Native's chat turns, `restored: Bool` on iOS's `GraineChatTurn`, `restored: Boolean` on Android's, `restored` (a `bool`) on Flutter's. A reconnect within the same app run receives the same history again, so nothing is duplicated; the SDK's own messages still sending or failed stay after the restored ones.

React Native — drawing your own transcript

```
import { useGraineAgent, chatRows } from "@graineai/inapp-react-native";

function Transcript() {
  const { messages } = useGraineAgent();
  // chatRows groups the turns the way the built-in window does
  const rows = chatRows(messages, (turn) => ({ at: turn.at, voice: turn.channel === "voice" }));
  return rows.map((row) =>
    row.kind === "day" ? <DaySeparator key={row.key} label={row.label} />          // Today, Yesterday or a
date
    : row.kind === "voice" ? <VoiceCallRow key={row.key} label={row.label} />      // one row per call: "Voice
call · 6:05 PM"
    : <Bubble key={row.key} turn={row.item} faded={row.item.restored === true} />);
}
```

iOS · Swift — drawing your own transcript

```
// A bubble per chat turn; one row per run of voice turns (one call)
let turns = GraineInApp.shared.chat.turns
ForEach(Array(turns.enumerated()), id: \.element.id) { index, turn in
    if turn.channel == .voice {
        if index == 0 || turns[index - 1].channel != .voice { VoiceCallRow(at: turn.at) }
    } else {
        Bubble(turn: turn, earlier: turn.restored)
    }
}
```

7.4 Forgetting, opting out, and a new conversation

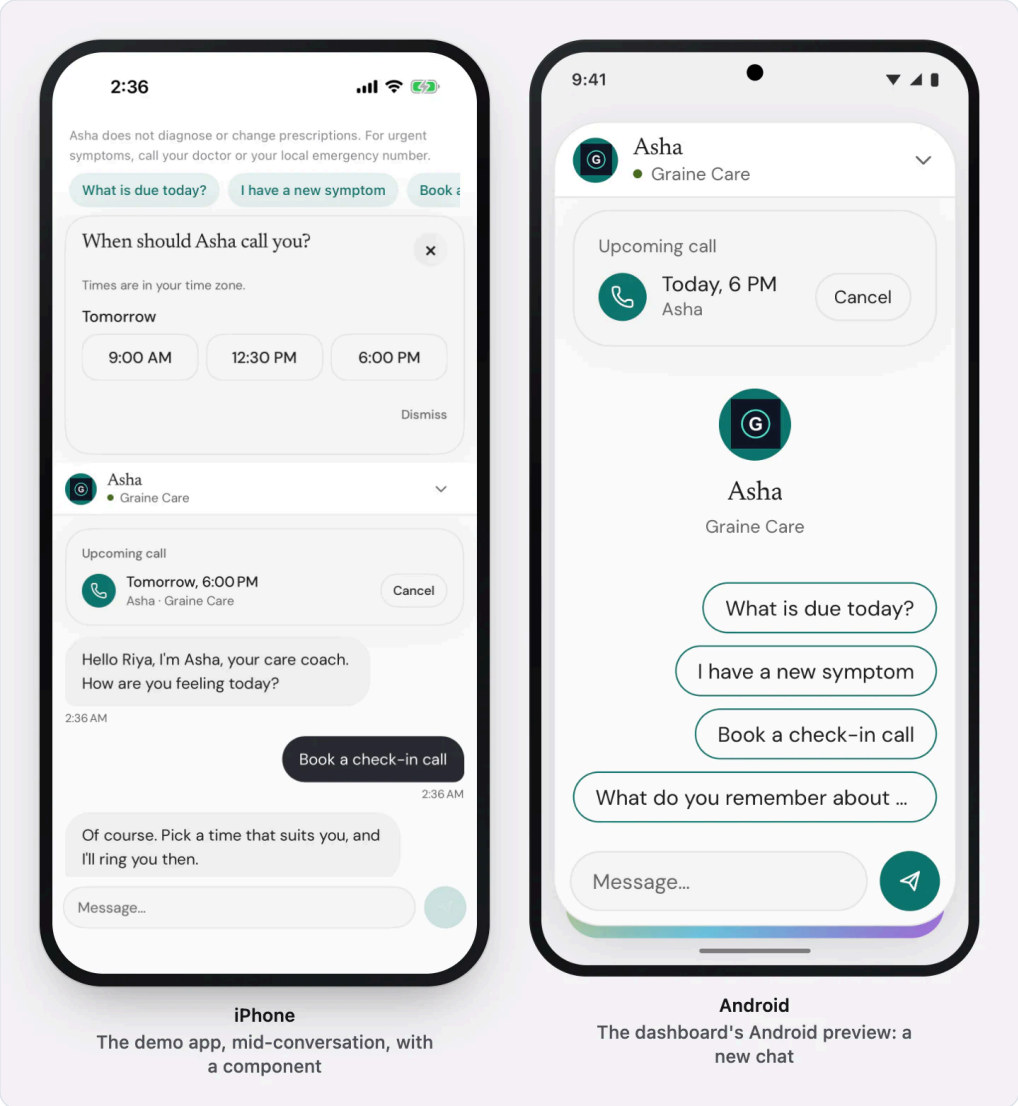
- **Forget clears it.** Everything that deletes a customer's memory deletes the kept conversation too: `forgetMemory()` from your app (which, once it succeeds, also clears the chat on screen — a refusal leaves both as they were), **Forget this person** in the dashboard, and an agent tool that forgets.
- **Opting out** (`setMemory(false)`) stops keeping and restoring it, but deletes nothing.
- **Sign-out** (`identify(null)`) or another customer signing in clears the chat on screen.
- **New conversation.** For a support-style chat that should start a fresh thread, the SDKs add `newConversation()` : it ends this agent's chat connection, asks Graine to drop the kept conversation for this agent and customer, clears the window, and opens a fresh chat — the scripted welcome and the starters appear again. **Memory facts are untouched**, and Call History keeps every earlier conversation. For an anonymous customer it clears the window only. The built-in chat window shows a **New conversation** control in its header (a compose icon beside close) while the window holds earlier, restored messages; it needs no confirmation. The agent's design can hide that control (Garage → *Chat window* → *New conversation button*); `newConversation()` still works from a button of yours.
- **Earlier messages on screen.** The design can also hide them (Garage → *Chat window* → *Earlier messages*): the window then opens like a new chat, while the agent still receives the history and continues. The Garage previews it as the *Returning customer* moment.

Platform	Start a new conversation
React Native	`client.newConversation(): Promise<void>`
iOS	`GraineInApp.shared.chat.newConversation() async throws`
Android	`GraineInApp.get().chat.newConversation(done: GraineCallback<Unit>? = null)`
Flutter	`GraineInApp.instance.chat.newConversation()` (on `GraineChatSession`)
Website	`GraineAgent.newConversation()`
Kotlin MP	`Graine.chat.newConversation()` (a `suspend` function)

Older SDKs ignore the earlier conversation and show an empty chat as before; nothing breaks.

8. Chat, voice, components, screen context and actions

What a customer can use is the dashboard's *What visitors get* (*Call and chat*, *Voice only*, *Chat only*) and *Opens on*. An app can choose among those, never add one. One conversation runs at a time: opening the chat ends a voice call in the bar, and a native call ends both.



iPhone
The demo app, mid-conversation, with a component

Android
The dashboard's Android preview: a new chat

The chat window: on an iPhone mid-conversation, with a component and an upcoming call; and the dashboard's Android preview of a new chat with its starters.

8.1 Voice in the assistant bar

The bar places the call, asks for the microphone if it must, shows the connection in words (*Weak connection*, *No connection*, *Reconnecting*…), draws captions and components, and ends the call. Each SDK exposes the same state for an app that draws its own bar: connected, connecting, ready, muted, a one-word phase (`idle` , `connecting` , `reconnecting` , `thinking` , `speaking` , `listening`), the line quality, captions, and whether the microphone was refused.

Platform	Start, end, mute	State
React Native	The `GraineVoiceLauncher` render prop's `api.start()` , `api.end()` , `api.setMuted(m)` , `api.retry()`	The same `api` (`connected` , `phase` , `captions` , `quality` , `micDenied` , …)
iOS	`GraineInApp.shared.voice.start()` , `end()` , `setMuted(_:)` , `retry()`	`GraineVoice` is an `ObservableObject`

Platform	Start, end, mute	State
Android	`GraineInApp.get().voice.start()`, `end()`, `setMuted(muted)`, `retry()`	Properties on `GraineVoice`; `GraineListener` callbacks
Flutter	`GraineInApp.instance.voice.start()`, `end()`, `setMuted(…)`, `retry()`	`GraineVoice` is a `ChangeNotifier`
Kotlin MP	`Graine.voice.start()`, `end()`, `setMuted(…)`	`Graine.voice.state` (a `StateFlow`)

By default a call ends two seconds after the app leaves the foreground (`endOnBackground`); a native call (section 9) is the way to keep talking with the screen off.

8.2 The chat window

The bar opens the chat where the agent offers it. Typed messages sent while offline wait (*Waiting for connection…*) and go in order when the chat reconnects, after drops, with a backoff of 1, 2, 4, 8, 15 and then 30 seconds; a message that cannot be sent is marked failed. The typing indicator shows from a send until the agent's first words, a component, or 12 seconds. The chat window also lists the customer's upcoming calls with **Cancel**.

React Native · TypeScript

```
const { messages, send, typing, connected } = useGraineAgent();   // the provider's chat
send("Where is my order?");
```

iOS · Swift

```
// The bar carries the chat. To place the chat yourself — a whole screen, say:
.sheet(isPresented: $showChat) { GraineChatView(onClose: { showChat = false }) }
// UIKit: present(GraineChatViewController(), animated: true)
// Drive it: let chat = GraineInApp.shared.chat; chat.open(); chat.send("Where is my order?"); chat.close()
```

Android · Kotlin

```
GraineChatActivity.open(this)                                    // the chat, full screen, in the brand's design
// or place GraineChatView in your layout; it draws GraineInApp.get().chat
val chat = GraineInApp.get().chat
chat.addChangeListener { render(chat.messages) }
chat.send("Where is my order?")
```

Flutter · Dart

```
Navigator.of(context).push(MaterialPageRoute(builder: (_) => Scaffold(
  body: SafeArea(child: GraineChat(onClose: () => Navigator.of(context).pop())),
)));
// Drive it: final chat = GraineInApp.instance.chat; chat.open(); chat.send('Where is my order?');
```

Kotlin Multiplatform · Kotlin

```
GraineChat(Modifier.fillMaxSize(), onClose = { navigator.pop() })   // the SDK's chat screen (inapp-kmp-compose)
Graine.chat.open(); Graine.chat.send("Where is my order?")
```

Chat window size

The Garage's *Window size* sets how big the chat opens: **Compact** (the window above the bar), **Large** (on a phone, a sheet from the bottom about 92% of the screen tall with a grab handle; on a tablet or computer, a 420-wide panel up to

760 tall) or **Full screen** (the whole screen inside the safe areas; on a computer the large panel, and the whole page below 640 px wide). An app can override it: the open call's argument wins over the app's configuration, which wins over the design.

Platform	App default	For one open
React Native	`chatSize` on `GraineProvider` / the client, a `ChatSize` (`"compact" | "large" | "full"`)	`client.openChat({ size })`; `GraineChatScreen` renders the chat full screen for a route of your own
iOS	`GraineConfig(…, chatSize: GraineChatSize?)` — `.compact`, `.large`, `.full`	`GraineInApp.shared.chat.open(size: .full)`
Android	`GraineConfig(…, chatSize = GraineChatSize.FULL)`	`chat.open(size)`
Flutter	`GraineConfig(chatSize: GraineChatSize.full)`	`GraineChat.open(context, app: …, size: GraineChatSize.full)`
Kotlin MP	`GraineConfig(…, chatSize = GraineChatSize.FULL)`	`Graine.chat.open(size)`
Website	`GraineAgent.load({ …, chatSize: "large" })` — plain strings	`GraineAgent.open({ mode: "chat", size: "full" })`

8.3 Components

Build components in the dashboard's components editor, tick them under *On screen*, and name them in the prompt (on a flow agent, in the step that should show them). The SDKs draw them in your design: the bar draws them by default; place a component view elsewhere if you prefer — iOS and Android `GraineWidgetView`, Flutter `GraineWidgetView` (with `showComponents: false` on the bar), React Native `GraineWidget`. A tap answers the agent as the customer's next turn (a sentence for the agent and structured data for your webhooks), and a component can open a screen in your app through your navigation action. A multi-select reply reads "*label*: *a* and *b*" — the button's label, then the names, for example "Taken: Morning and Evening" (no label: the names alone) — and in a text chat it is also the customer's own bubble; React Native exports the rule as `manySummary(titles, label = "Compare")`. A component's live data, when it comes from your API, has 8 seconds; after that the authored content is shown.

8.4 Screen context

Tell the agent what the customer can see whenever the screen or a field changes. Values are masked on the device; an unchanged screen is not sent again; the first change goes at once and later ones at most every 400 ms; the newest screen is sent again when the connection comes back and when the app returns to the foreground. Give every field a `status` (`empty`, `filled`, `invalid`, `pending`, `locked`): it is what turns "need a hand?" into naming the field that is wrong. Taps on labelled controls are reported by themselves (`captureTaps`, on by default on iOS, Android, Flutter and Kotlin MP).

React Native · TypeScript

```
useGraineScreen({
  screen: "checkout_address", title: "Delivery address",
  journey: { name: "Checkout", step: 2, of: 3, next: "Payment" },
  fields: [{ name: "postal_code", label: "Postal code", value: postalCode, status: "invalid", error: "We don't
deliver there yet" }],
});
// With navigationRef on the provider, every route is reported by name automatically.
```

iOS · Swift

```
GraineInApp.shared.setScreen(GraineScreen(
    screen: "checkout_address", title: "Delivery address",
    journey: .init(name: "Checkout", step: 2, of: 3, next: "Payment"),
    fields: [GraineField(name: "postal_code", label: "Postal code", value: postalCode, status: .invalid, error:
"We don't deliver there yet")]
))
```

```
GraineInApp.shared.updateField(GraineField(name: "postal_code", value: postalCode, status: .filled))
GraineInApp.shared.track("coupon_applied", data: ["code_length": 8])
```

Android · Kotlin

```
GraineInApp.get().setScreen(GraineScreen(
    screen = "checkout_address", title = "Delivery address",
    journey = GraineScreen.Journey("Checkout", step = 2, of = 3, next = "Payment"),
    fields = listOf(GraineField(name = "postal_code", label = "Postal code", value = postalCode,
        status = GraineField.Status.INVALID, error = "We don't deliver there yet")),
))
GraineInApp.get().track("coupon_applied", mapOf("code_length" to 8))
```

Flutter · Dart

```
GraineInApp.instance.setScreen(GraineScreen('checkout_address',
    title: 'Delivery address',
    journey: const GraineJourney('Checkout', step: 2, of: 3, next: 'Payment'),
    fields: [GraineField('postal_code', label: 'Postal code', value: postalCode, status: FieldStatus.invalid,
error: "We don't deliver there yet")]));
```

Kotlin Multiplatform · Kotlin

```
Graine.setScreen(GraineScreen(
    screen = "checkout_address", title = "Delivery address",
    journey = GraineScreen.Journey("Checkout", step = 2, of = 3),
    fields = listOf(GraineField("postal_code", label = "Postal code", value = postalCode, status =
GraineFieldStatus.INVALID)),
))
```

Website · JavaScript

```
GraineAgent.setScreen({
  screen: "pricing", title: "Plans and pricing",
  fields: [{ name: "plan", label: "Plan", value: selectedPlan, status: "filled" }],
});
```

Your own sensitive formats can be added to the masking once at start-up: React Native `addMaskRule(pattern, token)`, iOS `GraineMasking.addRule(pattern:token:)`, Android and Flutter `Masking.addRule(…)`, website `GraineAgent.addMaskRule(pattern, token)`. Built in: PAN, Aadhaar, card and account numbers, phone numbers, email addresses.

8.5 Actions

Declare each action on the agent (*Embed & Widgets → Actions*) and implement a handler with the same name. The handler runs on the main thread (iOS, Android), gets the agent's arguments, and answers `ok`, `refused` or `error` with a message the agent uses to say what happened. Answer within about five seconds: the SDKs tell the agent an action did not finish after 5.5–6 seconds (the website loader after 5.5). A throw is answered as `error`, and a declared action this build does not implement as `refused`.

React Native · TypeScript

```
useGraineAction("set_language", ({ language }) => {
  setLanguage(String(language));
  return { status: "ok", message: `The app is now in ${language}.` };
});
```

iOS · Swift

```
let unregister = GraineInApp.shared.registerAction("set_language") { args in
    guard let language = args["language"] as? String else { return .refused("No language was given.") }
    settings.setLanguage(language)
    return .ok("The app is now in \(language).")
}
// A handler that takes time answers through the deferred form:
GraineInApp.shared.registerAction("upload_document", deferred: { args, done in
    upload(args) { ok in done(ok ? .ok("Uploaded.") : .error("The upload failed.")) }
})
```

Android · Kotlin

```
val unregister = GraineInApp.get().registerAction("set_language") { args ->
    val language = args.optString("language")
    if (language.isEmpty()) ActionResult.refused("No language was given.")
    else { settings.setLanguage(language); ActionResult.ok("The app is now in $language.") }
}
```

Flutter · Dart

```
final unregister = GraineInApp.instance.registerAction('set_language', (args) async {
  final language = args['language'];
  if (language is! String) return const ActionResult.refused('No language was given.');
  await settings.setLanguage(language);
  return ActionResult.ok('The app is now in $language.');
});
```

Kotlin Multiplatform · Kotlin

```
val unregister = Graine.registerAction("set_language") { args ->
    val language = args["language"] as? String ?: return@registerAction GraineActionResult.refused("No language
was given.")
    settings.setLanguage(language)
    GraineActionResult.ok("The app is now in $language.")
}
```

Website · JavaScript

```
GraineAgent.defineAction("scroll_to_section", ({ section }) => {
  const el = document.getElementById(section);
  if (!el) return { status: "refused", message: "That section is not on this page." };
  el.scrollIntoView({ behavior: "smooth" });
  return { status: "ok", message: `Scrolled to ${section}.` };
});
```

`checkActions()` (iOS, Android, React Native) compares what the build registered with what the agent declares — usable, undeclared, unhandled — for a test or a debug screen. On the website, navigate with `window.location`, not `window.open` (popup blockers stop a window opened from the widget's callback).

9. WhatsApp-style calls

Two directions, one native call engine in your app: the customer calls the agent from a button, or Graine rings the customer at a booked time. Calls are voice only (components are not drawn during a call), one at a time, and they keep going with the screen locked, in another app, or on a Bluetooth headset.

9.1 The customer calls the agent: startCall

`startCall` returns the call's ID — `call_` and a lowercase UUID, carried by every outcome — or nothing while another call is live. It asks for the microphone if it is missing, ends a conversation in the bar, opens the call (CallKit on iPhone, the SDK's call screen on Android), and connects with `who_called = "customer"`, `call_reason` = your reason (send an identifier such as `order_help`) and your call variables (masked; up to 12 variables in all with those two, about 1 KB).

React Native · calls package

```
const id = await GraineCalls.startCall({ reason: "order_help", variables: { order: "A-1043" } });   // null while another call is live
```

iOS · Swift

```
let id = GraineInApp.shared.calls.startCall(reason: "order_help", variables: ["order": "A-1043"])   // nil while another call is live
```

Android · Kotlin

```
// Main thread, from an activity in the foreground (the microphone service may only start from there)
val id = GraineInApp.get().calls.startCall(this, reason = "order_help", variables = mapOf("order" to "A-1043"))
```

Flutter · Dart

```
final id = await GraineCalls.instance.startCall(reason: 'order_help', variables: {'order': 'A-1043'});
```

Kotlin Multiplatform · Kotlin

```
val id: String? = Graine.calls.startCall(reason = "order_help", variables = mapOf("order" to "A-1043"))
```

On iOS present `GraineCallView` while `calls.showsCallScreen` is true (section 5.3); Android opens `GraineCallActivity` itself; React Native and Flutter show the SDK's screen with `callScreen: "sdk"` (the default) or let you draw your own with `"app"`.

9.2 Graine rings the customer: hosted ringing

1. **Upload your push credentials once** — Workspace settings → In-app calling (section 4.6) — until each platform shows *Verified*.
2. **Hand the SDK its push tokens.** The SDK registers the device with Graine — with the install, the time zone, the app version and, when the customer is signed in on one of their devices, the customer — on every new token, when its details change, and otherwise at most daily; with several agents, under each agent's key. Sign-out unlinks the device from the customer.
3. **Book a call** (section 10). When it is due, Graine rings the customer's newest device.

iOS · Swift

```swift
// didFinishLaunching, every launch — iOS hands over a VoIP push only after this has run
GraineInApp.shared.calls.enableVoIPPushes()     // with hosted ringing on, the SDK registers the VoIP token itself
```

Android · Kotlin — AcmeMessagingService.kt

```kotlin
class AcmeMessagingService : FirebaseMessagingService() {
    override fun onNewToken(token: String) = GraineCalls.onNewToken(this, token)   // registers this device with Graine

    override fun onMessageReceived(message: RemoteMessage) {
        if (GraineCalls.handlePush(this, message.data)) return                     // a call: it rings, even with the app closed
        // …your app's other messages
    }
}
```

XML — AndroidManifest.xml, inside <application>

```xml
<service android:name=".AcmeMessagingService" android:exported="false">
    <intent-filter><action android:name="com.google.firebase.MESSAGING_EVENT" /></intent-filter>
</service>
```

React Native · calls package

```javascript
// index.js, before AppRegistry.registerComponent — rings even when the app was closed
messaging().setBackgroundMessageHandler(async (m) => { await GraineCalls.handlePush(m.data ?? {}); });

// at launch
GraineCalls.enableVoIPPushes(() => {});                                          // iOS: the token is registered by the SDK
messaging().getToken().then((token) => GraineCalls.registerPushToken({ token }));  // Android
messaging().onTokenRefresh((token) => GraineCalls.registerPushToken({ token }));
messaging().onMessage((m) => GraineCalls.handlePush(m.data ?? {}));              // Android, app open
```

Flutter · Dart

```dart
@pragma('vm:entry-point')
Future<void> onBackgroundPush(RemoteMessage message) async {
  await GraineCalls.instance.handlePush(message.data);           // rings, even when the app was closed
}

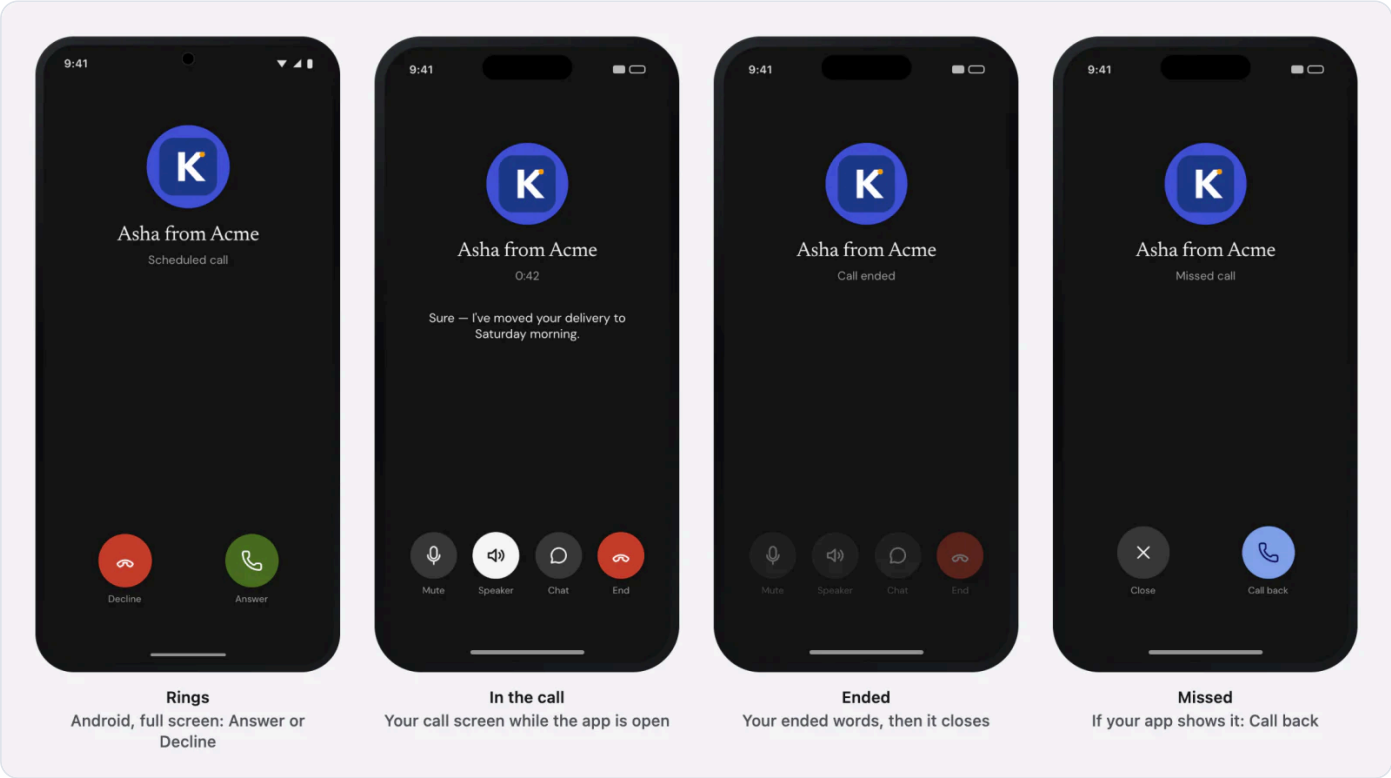
// in main(), after Firebase.initializeApp()
FirebaseMessaging.onBackgroundMessage(onBackgroundPush);
FirebaseMessaging.onMessage.listen((m) => GraineCalls.instance.handlePush(m.data));
GraineCalls.instance.enableVoIPPushes((_) {});                   // iOS
FirebaseMessaging.instance.getToken().then((t) { if (t != null) GraineCalls.instance.registerPushToken(t); });
// Android
FirebaseMessaging.instance.onTokenRefresh.listen(GraineCalls.instance.registerPushToken);
```

Kotlin Multiplatform

```kotlin
// iOS, every launch
Graine.calls.enableVoIPPushes()
// Android, in your FirebaseMessagingService
override fun onNewToken(token: String) = Graine.calls.onNewToken(token)
override fun onMessageReceived(message: RemoteMessage) { if (Graine.calls.handlePush(message.data)) return }
```

What a ring carries: an APNs VoIP push (`apns-push-type: voip` , topic `<bundle ID>.voip` , priority 10, expiration 0 — now or never) or a data-only FCM message at high priority that lives 30 seconds, with exactly `type: "agent_call"` , `call_id` , `caller_name` (the agent's call name), `hosted: "1"` and `agent_id` (the booking agent's ID, so an app with several agents answers with the right one). No reason, no variables, nothing about the customer: a push can wait at Apple or Google until the phone is reachable, so it carries only what the ring shows. The agent learns why it is calling when the call connects. A push your own server sends (with `call_id` , `reason` , `caller_name` , `variables`) still rings exactly as before, side by side.

9.3 The call screen: six moments



One scheduled call, moment by moment (from the dashboard's previews): the ring, the call, its end, and a missed ring.

The SDKs draw the call screen from the Garage's *Call screen* tab: background, avatar (logo, glyph or the call name's initial) in the chosen shape, the call name in the design's heading face, the status in your words, and the controls in your order. The last design is kept on the device, so a ring that wakes a closed app is drawn the same way.

Moment	Status line	Buttons	Notes
Scheduled call ringing	"Scheduled call" (your *Scheduled call ringing* words)	Decline · Answer	A call Graine rang at a booked time. A ring pulses from the avatar. On a locked iPhone, the system's incoming-call screen; on Android, the full-screen ring
Other call ringing	"Incoming call" (*Other call ringing*)	Decline · Answer	A ring your own server sent
Connecting	"Connecting…"	Mute · Speaker · End	Answered, or placed by the customer; the line is opening (it must open within 15 seconds)
On the call	The timer (`m:ss`), with " · Muted" while muted — or, with the timer off, "Listening", "Speaking", "Muted"	Mute (Unmute) · Speaker · End, in your order; End round or pill	Captions show the line being said when on. The avatar moves while the agent speaks (not with reduced motion). The chat button is not drawn on a native call
Ended	"Call ended" (your words); "Call failed" or "Declined" for those endings	Dimmed	Shows for about a second, then the screen closes

Moment	Status line	Buttons	Notes
Missed	"Missed call"	Close · Call back	A ring nobody answered. iOS keeps it on the call screen for 30 seconds (`missedCall` , `callBack()` , `dismissMissedCall()`); Android leaves a notification with **Call back** and shows the missed screen when it is opened. Call back calls the same agent with the missed call's reason

Android's call screen shows over the lock screen and turns the screen on; closing it never ends the call (the ongoing notification brings it back). An Android app can name its own call screen (`GraineCallsConfig(callActivity = …)`); it then shows over the lock screen, forwards `onRequestPermissionsResult` to `calls` , and handles the intent actions `GraineCalls.ACTION_SHOW` , `ACTION_ANSWER` , `ACTION_MISSED` and `ACTION_CALL_BACK` .

9.4 Outcomes, and their webhooks

Outcome	When	Booked call's status	Webhook
`ringing`	The phone is ringing (reported to Graine for its rings; React Native, Flutter and Kotlin MP also hand it to your listener)	`ringing`	—
`answered`	The customer picked up an incoming call; an `ended` or `failed` follows	`answered` , then `completed`	The conversation's own events
`declined`	The customer declined it. Nothing rings for that customer for 6 hours, and that call never rings again	`declined`	`call_declined`
`missed`	Nobody answered within the ring timeout; another call was live (Android reports it as busy); or, on iOS, Do Not Disturb, a Focus or the block list stopped it. A booked call can ring once more after your re-ring delay	`missed`	`call_missed`
`ended`	A call that connected is over: `local` (the customer hung up), `remote` (the agent did) or `failed` (the line broke; calls do not reconnect), with its duration	`completed`	The conversation's own events
`failed`	It never connected: the microphone was refused (or not granted within 60 seconds), the line did not open within 15 seconds, or the agent could not be reached	`failed`	—
(never rang)	Its time passed before it could ring (calling hours, credit, the agent switched off, a recent decline, the daily limit)	`expired`	`call_expired`
(never rang)	No device seen in the last 30 days, or no working credentials	`no_device`	—

Your app hears every outcome through the calls listener — React Native `GraineCalls.onOutcome(fn)` , iOS `GraineCallsListener.onCallOutcome(_:callId:)` , Android `GraineCallListener.onCallOutcome(callId, outcome)` , Flutter `GraineCalls.instance.outcomes` , Kotlin MP `Graine.calls.outcomes` . React Native and Flutter keep the last 20 outcomes and rings and hand them to the first listener, so an app launched by a push still hears them. The webhook events `call_declined` , `call_missed` and `call_expired` are opt-in: tick them in Agent → *Integrations & Tools* → *Custom Webhook* → *Send these events*, under *In-app calls*. On Android, Flutter, React Native and Kotlin MP an outcome also carries `hosted` (Graine rang it) and, for a call that connected, `durationSeconds` .

9.5 Locked screens, other apps, weak networks

What happens	iPhone	Android
The phone is locked when a ring arrives	The system incoming-call screen, with your call name, icon and ringtone	The full-screen ring (with the full-screen permission; otherwise a heads-up call notification), Answer and Decline
The screen locks during a call	The call goes on (CallKit and the audio background mode)	The call goes on (the microphone foreground service)

What happens	iPhone	Android
The customer opens another app	The call goes on; the system shows it is live	The call goes on; the ongoing notification has Hang up
A phone call comes in during the agent's call	CallKit handles it as for any call app	The agent's call is held while the phone has the audio, and ends if the phone call is answered
Airplane mode, or the network drops, mid-call	The call ends: outcome `ended`, reason `failed`. Calls do not reconnect; offer a call back	
The phone is offline when a ring is sent	The push expires; the booking is marked missed when its ring time runs out	
A battery saver stops background work	—	Some phones stop a call after the screen locks: ask the customer to allow your app in battery settings

10. Scheduled calls

A scheduled call is a booking for a customer that Graine rings on their phone when it is due — booked by the agent, by your app or website, or by your team.

10.1 The rules every booking follows

The SDK reads the agent's rules with its settings: `schedulingRules` (a property; no request). Before the settings load, and from an older server, it holds the defaults with booking off.

Field (GraineSchedulingRules)	Default	Dashboard setting
`usersCanBook`	`false`	*Users, from your app*
`minLeadMinutes`	5	*With at least this much notice* — the soonest
`maxDaysAhead`	14	*Booked at most this far ahead* — the furthest ahead
`maxPendingPerUser`	3	*At most this many upcoming calls per user*
`ringTimeoutSeconds`	45	*Ring for*
`callName`	none	*Name on the incoming call*
`hostedRinging`, `platforms`	`false`, empty	Whether Graine rings this workspace's apps, and on which platforms (Workspace settings → In-app calling)
`earliest(now)`	`now` plus `minLeadMinutes` — the first time a picker should offer	
`latest(now)`	`now` plus `maxDaysAhead` days — the last	

- **The calling window.** Every booking must fall inside the agent's *Calling Window*, read in the **customer's time zone** (the device's IANA zone, which every booking and device registration carries). A booking outside it is refused with `outside_window` and a sentence naming a time that would work.
- **Quiet hours** apply to calls the agent or your team book (refused at booking if inside them), never to a time the customer picked themselves.
- **When it is due,** Graine checks again: a call more than 10 minutes late is not rung (`expired`); a call outside the window or quiet hours moves to the next opening (within the days-ahead limit, else `expired`); no ring within 6 hours of a decline; at most *Ring each user at most* rings a day; then the customer's newest device seen in the last 30 days (among the devices bound to them), trying the next one on a dead push token. Credit and a switched-off agent hold a ring for up to 10 minutes.
- **Who it rings.** A booking made for an identified customer rings their app, on the devices they are signed in on. A booking from an app with no identified customer rings that install only. A booking from a device that is not one of the customer's is refused (`device_limit`).

10.2 scheduleCall, scheduledCalls, cancelScheduledCall

Platform	Book	List (upcoming first)	Cancel
React Native	`client.scheduleCall(at, reason?)` or `({ at, reason })` → `Promise<GraineScheduledCall>` ; `useGraineScheduling()`	`client.scheduledCalls()` → `Promise<GraineScheduledCall[]>`	`client.cancelScheduledCall(id)` → `Promise<GraineCallStatus>`
iOS	`scheduleCall(at: Date, reason: String? = nil) async throws -> GraineScheduledCall`	`scheduledCalls() async throws -> [GraineScheduledCall]`	`cancelScheduledCall(id:) async throws -> GraineCallStatus`

Platform	Book	List (upcoming first)	Cancel
Android	`scheduleCall(at: Date, reason: String? = null, done: GraineCallback<GraineScheduledCall>? = null)`	`scheduledCalls(done: GraineCallback<List<GraineScheduledCall>>)`	`cancelScheduledCall(id, done: GraineCallback<GraineCallStatus>? = null)`
Flutter	`Future<GraineScheduledCall> scheduleCall({required DateTime at, String? reason})`	`Future<List<GraineScheduledCall>> scheduledCalls()`	`Future<GraineCallStatus> cancelScheduledCall(String id)`
Kotlin MP	`suspend fun scheduleCall(at: Instant, reason: String? = null): GraineScheduledCall`	`suspend fun scheduledCalls(): List<GraineScheduledCall>`	`suspend fun cancelScheduledCall(id: String): GraineCallStatus`
Website	`GraineAgent.scheduleCall(at, reason)` or `({ at, reason })` → Promise	`GraineAgent.scheduledCalls()`	`GraineAgent.cancelScheduledCall(id)` → `"cancelled"`

`reason` (up to 120 characters) reaches the agent as `call_reason` when the call connects and never travels in the ring. A **GraineScheduledCall** is `{ id, at, status, source, reason, isUpcoming, canCancel }`: `at` is the platform's date type (UTC; null only if Graine's time could not be read), `isUpcoming` is true while `scheduled` or `ringing`, and `canCancel` while `scheduled`. Only a call whose `canCancel` is true can be cancelled.

Status (wire)	Swift / Dart / TS	Android	Kotlin MP	Means
`scheduled`	`scheduled`	`SCHEDULED`	`Scheduled`	Booked, waiting — or held for credit or a switched-off agent
`ringing`	`ringing`	`RINGING`	`Ringing`	Ringing on the customer's device
`answered`	`answered`	`ANSWERED`	`Answered`	Picked up; the conversation is on
`completed`	`completed`	`COMPLETED`	`Completed`	The conversation ended
`declined`	`declined`	`DECLINED`	`Declined`	The customer declined it
`missed`	`missed`	`MISSED`	`Missed`	Nobody answered within the ring time
`failed`	`failed`	`FAILED`	`Failed`	It could not be rung or connected
`cancelled`	`cancelled`	`CANCELLED`	`Cancelled`	Called off by the customer, your app or your team
`expired`	`expired`	`EXPIRED`	`Expired`	Its time passed before it could ring
`no_device`	`noDevice`	`NO_DEVICE`	`NoDevice`	No device to ring
anything newer	`unknown` (carries the word)	`Unknown(raw)`	`Unknown(raw)`	A status newer than the SDK

Sources (who booked it): `agent`, `app` (wire `sdk`), `dashboard`, `api`.

10.3 Checks made on the device first

`scheduleCall` refuses on the device, before any request, from `schedulingRules`: `scheduling_off` when `usersCanBook` is false; `too_soon` before `earliest(now)` (React Native and the website allow 30 seconds' grace); `too_far` after `latest(now)`; Flutter also refuses a reason over 120 characters (`invalid_request`; the website trims it). Graine then checks the rest: `outside_window`, `too_many_pending`, `identity_required`, `device_limit`, `invalid_request` and `rate_limited` (10 bookings an hour from one install). The scheduling refusals come with a sentence written for the customer — show it as it is.

10.4 Booking by the agent

With *The agent, during a conversation* on, the agent can book a call back when the customer asks ("call me tomorrow at six"). It works out the time in the customer's zone and tells them the confirmed time, or the reason it could not book, in

plain words. In your app or on your website the booking is for the conversation's identified customer (else its install) and rings their app; on a phone call it is an ordinary call back to the caller's number. Mention in the prompt when to offer a call back.

10.5 Booking by your team

Agent → Calls & scheduling → **Schedule a call**: *App user* (by the identifier your *Who is the user?* setting names) or *Phone number*, *When*, *Time zone*, and *What it's about*. **Ring now** rings a user straight away as a test. Both follow the same rules, quiet hours included, and show in the *Scheduled calls* list with their status.

> **A website cannot ring.**
> A browser cannot receive a WhatsApp-style call. A call booked on your website — by the agent or with `scheduleCall` — is for the identified customer and rings **their app** on their phone. A customer with no app device seen in the last 30 days is not rung: the booking ends `no_device`. So does a booking from a website visitor who is not identified: it is made for the browser's install, which has nothing to ring. Identify the visitor before offering a booking on the web.

10.6 Example: a "book a consultation call" sheet with the native pickers

This is the pattern the demo apps use: the system's own date and time pickers, in 5-minute steps, bounded by `earliest(now)` and `latest(now)`, and the SDK's message shown as it is when a booking is refused. The calling hours are not encoded in the picker; Graine answers `outside_window` with a time that would work.

iOS · Swift — BookConsultationSheet.swift

```
import SwiftUI
import GraineInApp

struct BookConsultationSheet: View {
    @Environment(\.dismiss) private var dismiss
    @State private var when = Date()
    @State private var problem: String?
    @State private var sending = false

    var body: some View {
        let rules = GraineInApp.shared.schedulingRules
        let range = rules.earliest()...rules.latest()
        NavigationStack {
            Form {
                Section {
                    DatePicker("Date", selection: $when, in: range, displayedComponents:
.date).datePickerStyle(.graphical)
                    DatePicker("Time", selection: $when, in: range, displayedComponents:
.hourAndMinute).datePickerStyle(.wheel)
                } footer: {
                    Text("Between \(rules.minLeadMinutes) minutes and \(rules.maxDaysAhead) days from now,
within our calling hours.")
                }
                if let problem { Text(problem).foregroundColor(.orange) }
            }
            .navigationTitle("Book a consultation call")
            .toolbar {
                ToolbarItem(placement: .confirmationAction) {
                    Button(sending ? "Booking…" : "Book") {
                        sending = true
                        Task {
                            do {
                                _ = try await GraineInApp.shared.scheduleCall(at: when, reason: "Consultation")
                                dismiss()
                            } catch let error as GraineServiceError {
                                problem = error.message          // written for the customer
                            } catch {
```

```
                                problem = "Couldn't book the call. Try again."
                            }
                            sending = false
                        }
                    }
                    .disabled(sending || !rules.usersCanBook)
                }
            }
            .onAppear {
                UIDatePicker.appearance().minuteInterval = 5
                when = rules.earliest().addingTimeInterval(15 * 60)
            }
        }
    }
}
```

Android · Kotlin — Jetpack Compose (Material 3)

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun BookConsultationSheet(onDone: () -> Unit) {
    val rules = GraineInApp.get().schedulingRules
    val earliest = rules.earliest(Date()).time
    val latest = rules.latest(Date()).time
    var problem by remember { mutableStateOf<String?>(null) }
    val dateState = rememberDatePickerState(
        initialSelectedDateMillis = earliest,
        selectableDates = object : SelectableDates {
            override fun isSelectableDate(utcTimeMillis: Long) =
                utcTimeMillis >= earliest - 86_400_000 && utcTimeMillis <= latest
        },
    )
    val timeState = rememberTimePickerState()
    Column(Modifier.padding(16.dp)) {
        Text("Book a consultation call", style = MaterialTheme.typography.titleLarge)
        DatePicker(state = dateState)
        TimePicker(state = timeState)
        problem?.let { Text(it, color = MaterialTheme.colorScheme.error) }
        Button(enabled = rules.usersCanBook, onClick = {
            val at = combineInDeviceZone(dateState.selectedDateMillis ?: earliest, timeState.hour,
timeState.minute / 5 * 5)
            GraineInApp.get().scheduleCall(at, "Consultation") { call, error ->
                if (error != null) problem = error.message else onDone()      // main thread; the message is
for the customer
            }
        }) { Text("Book") }
    }
}
```

Flutter · Dart — Material pickers

```
Future<void> bookConsultation(BuildContext context) async {
  final rules = GraineInApp.instance.schedulingRules;
  final now = DateTime.now();
  final first = rules.earliest(now), last = rules.latest(now);
  final day = await showDatePicker(context: context, initialDate: first, firstDate: first, lastDate: last);
  if (day == null || !context.mounted) return;
  final time = await showTimePicker(context: context, initialTime: TimeOfDay.fromDateTime(first));
  if (time == null) return;
  final at = DateTime(day.year, day.month, day.day, time.hour, time.minute ~/ 5 * 5);
  try {
    await GraineInApp.instance.scheduleCall(at: at, reason: 'Consultation');
  } on GraineException catch (e) {
    showMessage(e.message);                                  // written for the customer
  }
}
```

React Native — @react-native-community/datetimepicker

```
const { rules, schedule } = useGraineScheduling();
const [when, setWhen] = useState(() => new Date(rules.earliest().getTime() + 15 * 60_000));
const book = async () => {
  try { await schedule(when, "Consultation"); close(); }
  catch (e) { setProblem(e instanceof GraineError ? e.message : "Couldn't book the call."); }
};
// iOS: <DateTimePicker value={when} mode="datetime" display="inline" minuteInterval={5}
//        minimumDate={rules.earliest()} maximumDate={rules.latest()} onChange={(_, d) => d && setWhen(d)} />
```

Website — the browser's own picker

```
const rules = GraineAgent.schedulingRules;
const input = document.querySelector("#when");            // <input type="datetime-local" step="300">
input.min = toLocalInputValue(rules.earliest(new Date()));
input.max = toLocalInputValue(rules.latest(new Date()));
try {
  await GraineAgent.scheduleCall({ at: new Date(input.value), reason: "Consultation" });
} catch (e) {
  showMessage(e.message);                                  // e.code: too_soon, outside_window,
too_many_pending…
}
```

`combineInDeviceZone`, `showMessage`, `toLocalInputValue`, `setProblem` and `close` stand for your app's own helpers. The demo apps' full sheets also offer quick-time chips and, with two agents, which agent should call.

10.7 The chat window's upcoming calls

The chat window of the iOS, Android, Flutter and Kotlin Multiplatform SDKs lists the customer's upcoming calls with **Cancel** — the *Booked calls* card, which the design can hide or retitle (Garage → *Chat window*). React Native's chat and the website have no such card. The same list is yours to read: iOS `chat.upcomingCalls`, `refreshUpcomingCalls()`, `cancelUpcomingCall(id:)`; Android `chat.upcomingCalls`, `refreshUpcomingCalls()`, `cancelUpcomingCall(id)`; Flutter `chat.upcomingCalls`, `refreshUpcoming()`, `cancelUpcoming(id)`; React Native `useGraineScheduling()`.

11. Several agents in one app

One app can run several agents — each with its own publishable key — and choose one per screen: a voice-first assistant on Home, a support chat in Help, a sales agent on Plans. **Each agent is a separate session**: its own design, chat, kept conversation, voice bar, actions and bookings. They never mix.

11.1 What is per agent, and what is shared

Per agent (never mixed)	Shared by every agent in the app
• Its session and its **Garage design** (colours, chat title, starters, call screen, *Window size*) • Its **chat** and its **kept conversation** (continuation is per agent, even when memory is shared) • Its voice bar, its actions (each agent is offered only its own), its components • Its scheduling and memory requests, which go to that agent's key; its call screen design and call name	• **Who the customer is**: `identify()` on any agent identifies them for all; `identify(null)` signs out everywhere (each agent reads its own *Who is the user?* field) • The **install ID** and the customer's **memory choice** • The **current screen** (each agent receives it while it has a live conversation) • The **call engine**, and the rule of **one conversation at a time** across all agents: starting a call or chat on one ends a live one on another

Memory facts are shared between agents only when both agents' *Shared with* is *Agents that share memory*; otherwise each agent remembers on its own.

11.2 Worked example: a voice assistant and a separate support chat

The default agent is a voice-first assistant shown in the bar on every screen. A second agent named `support` is a support-style chat, opened full screen from your Help screen. In the dashboard they are two agents, each with its own key and its own Garage design:

Dashboard setting	Default agent (the assistant)	`support` agent
Setup → *What visitors get*	*Voice only* (or *Call and chat*, *Opens on Call*)	*Chat only*
Garage → *Chat window* (*Make it yours* → *Chat*)	—	*Window size Full screen*; its own *Title* ("Support"); its own *Starters* ("Track my order", "Talk to a person")
Garage → *Make it yours* → *Paint*	Your brand's accent	Its own accent and tone
Embed & Widgets → *Actions*	App actions (navigation, settings)	Support actions (open a ticket, find an order)

iOS · Swift — Two agents — iOS

```
// At launch
GraineInApp.configure(GraineConfig(baseUrl: "https://www.graine.ai", publishableKey: "pk_live_…assistant"))
// the default agent
GraineInApp.configure(GraineConfig(baseUrl: "https://www.graine.ai", publishableKey: "pk_live_…support"), name:
"support")

// Every screen: the assistant's bar
RootView().overlay(GraineAgentBar())

// Help screen: the support agent's chat, full screen, in its own design
struct HelpView: View {
    @State private var showSupport = false
    var body: some View {
        Button("Chat with support") { showSupport = true }
            .fullScreenCover(isPresented: $showSupport) {
```

```
                GraineChatView(graine: GraineInApp.named("support"), onClose: { showSupport = false })
            }
        }
    }
// UIKit: present(GraineChatViewController(graine: GraineInApp.named("support")), animated: true)
// A call with the support agent: GraineInApp.shared.calls.startCall(reason: "support_help", agent: "support")
```

Android · Kotlin — Two agents — Android

```
// Application.onCreate
GraineInApp.init(this, GraineConfig(baseUrl = "https://www.graine.ai", publishableKey = "pk_live_…assistant"))
GraineInApp.init(this, GraineConfig(baseUrl = "https://www.graine.ai", publishableKey = "pk_live_…support"),
name = "support")

// Every activity: the assistant's bar (the default agent)
root.addView(GraineAgentBarView(this), FrameLayout.LayoutParams(MATCH_PARENT, WRAP_CONTENT, Gravity.BOTTOM))

// Help screen: the support agent's chat on a screen of its own, in its design
helpButton.setOnClickListener { GraineChatActivity.open(this, "support") }

// A call with the support agent
GraineInApp.get().calls.startCall(this, reason = "support_help", agent = "support")
```

Flutter · Dart — Two agents — Flutter

```
GraineInApp.init(GraineConfig(baseUrl: 'https://www.graine.ai', publishableKey: 'pk_live_…assistant'));
GraineInApp.init(GraineConfig(baseUrl: 'https://www.graine.ai', publishableKey: 'pk_live_…support'), name:
'support');

// Help screen: the support agent's chat as a full-screen route
Navigator.of(context).push(MaterialPageRoute(builder: (_) => Scaffold(
  body: SafeArea(child: GraineChat(app: GraineInApp.named('support'), onClose: () =>
Navigator.of(context).pop())),
)));

// A call with the support agent
await GraineCalls.instance.startCall(reason: 'support_help', agent: 'support');
```

Flutter · Dart — or open it at a size, on the navigator or in that agent's bar

```
await GraineChat.open(context, app: GraineInApp.named('support'), size: GraineChatSize.full);
```

React Native · TypeScript — Two agents — React Native

```
// The app's root: the assistant (default agent)
<GraineProvider baseUrl="https://www.graine.ai" publishableKey="pk_live_…assistant">
  <AppNavigator />
</GraineProvider>

// The Help route: a second provider is that subtree's agent — its own client, chat and design
function HelpScreen() {
  return (
    <GraineProvider baseUrl="https://www.graine.ai" publishableKey="pk_live_…support">
      <GraineChatScreen />
    </GraineProvider>
  );
}

// Calls: one engine, both agents
await GraineCalls.configure({
  publishableKey: "pk_live_…assistant", displayName: "Asha from Acme",
  agents: { support: { publishableKey: "pk_live_…support" } },
});
```

```
await GraineCalls.startCall({ reason: "support_help", agent: "support" });
// Inside the Help provider: GraineCalls.attach(useGraineClient(), { agent: "support" })
```

Kotlin Multiplatform · Kotlin — Two agents — Kotlin Multiplatform

```
Graine.configure(GraineConfig(baseUrl = "https://www.graine.ai", publishableKey = "pk_live_…assistant"))
Graine.configure(GraineConfig(baseUrl = "https://www.graine.ai", publishableKey = "pk_live_…support"), name =
"support")

// Help screen (Compose Multiplatform)
GraineChat(Modifier.fillMaxSize(), onClose = { navigator.pop() }, agent = "support")
Graine.calls.startCall(reason = "support_help", agent = "support")
```

Website · JavaScript — Website — one agent at a time, switched per route

```
const agent = location.pathname.startsWith("/help")
  ? { agentId: "support-agent-id", publishableKey: "pk_live_…support" }
  : { agentId: "assistant-agent-id", publishableKey: "pk_live_…assistant" };
if (GraineAgent.switchesAgents) {
  await GraineAgent.load(agent);        // takes the one on the page off first; true once the new one is on it
} else {
  location.reload();                    // a loader older than 1.2.0
}
```

`GraineChat.open` (Flutter) opens the chat at the size you pass, else the agent's *Window size*: in that agent's `GraineAgentBar` when one is on screen, otherwise on the navigator (`large` as a sheet, `full` as a screen of its own). `GraineChatScreen` (React Native) fills the route it is placed in, drawn in the support agent's design.

11.3 Calls with several agents

- **Placing:** `startCall(…, agent: name)` places the call with that agent (the default when omitted; an unknown name places nothing and logs an error, or rejects in React Native).
- **Rings are routed by agent.** Graine's ring carries `agent_id` ; the SDK answers it with the configured agent whose agent (or web-call variant) it names — that agent's call name, ring time, design and outcome reporting. A ring naming none of your agents rings with the default agent (and says so in the log).
- **Devices register once per agent key**, so every agent's workspace can ring the phone. A missed call is called back on the agent that rang.

11.4 The multi-agent API

Platform	Configure	Look up	Views take the agent
iOS	`GraineInApp.configure(_:name:)` (returns the agent)	`GraineInApp.named(_:)` , `.all` , `.shared` (default)	`GraineChatView(graine:)` , `GraineChatViewController(graine:)` , `GraineAgentBar(graine:)`
Android	`GraineInApp.init(application, config, name)`	`GraineInApp.get(name)` , `GraineInApp.find(name)` , `GraineInApp.all`	`GraineChatActivity.open(context, agent)` , `GraineChatView.source` , `GraineAgentBarView.graine`
Flutter	`GraineInApp.init(config, name: …)`	`GraineInApp.named(name)` , `.all` , `.instance`	`GraineChat(app:)` , `GraineChat.open(context, app:)` , `GraineAgentBar(app:)`
Kotlin MP	`Graine.configure(config, name)`	`Graine.agent(name)` , `Graine.agents`	`GraineChat(agent =)` , `GraineAgentBar(agent =)`
React Native	One `GraineProvider` per agent (a nested provider is that screen's agent)	`useGraineClient()` in that subtree	The nearest provider's (`GraineChatScreen` , `GraineVoiceBar` , …)

Platform	Configure	Look up	Views take the agent
Website	`GraineAgent.load({ agentId, publishableKey })` again with another agent	`GraineAgent.switchesAgents`	One agent on the page at a time

A name is trimmed; an empty name is the default agent; configuring a name again returns that agent unchanged. The default agent is the one configured without a name, else the first one configured.

12. SDK reference

Every public declaration of each SDK, read from its source for 0.7.0 (React Native 0.41.0, calls 0.2.0, loader 1.2.0): the signature, what it returns, the errors it can raise, and what it does. The data structures, across the platforms and with what to expect from each, come first. Within a platform, the agent's entry point comes first, then chat, voice, views, calls, scheduling, memory, errors, screen context and actions, and finally the lower-level types the SDK's own components are built from (public, but not needed for an integration).

> **How errors arrive**
> React Native and the website: promises reject with `GraineError` (`code` , `message` , `status` , `retryAfterSeconds`). iOS: `async throws` a `GraineServiceError` (`code: GraineErrorCode` , `message` , `retryAfterSeconds` , `status`). Android: the callback's `error` , a `GraineError` (`code` , `message` , `retryAfterSeconds`), on the main thread — never thrown. Flutter: `GraineException` (`message` , `code` , `codeName` , `statusCode` , `retryAfterSeconds`). Kotlin MP: `GraineException` (`code` , `message` , `retryAfterSeconds`). Calling before start-up is a programming error: iOS stops with a message, Android throws `IllegalStateException` , Flutter throws `StateError` .

12.1 Data structures, and what to expect

Every type the SDKs hand your app, across the platforms: its exact name on each, every field with its type and default, and what to expect — when it appears or changes, what can be empty, times and time zones, and the edge cases. The per-platform tables after this part list every member of every type. Types are written **Swift** (iOS) · **Android** (Kotlin) · **KMP** (Kotlin Multiplatform) · **Dart** (Flutter) · **TS** (TypeScript: React Native, and the website where it applies).

> **Three rules that hold everywhere**
> - **Names.** A type has the same name on every platform. Enum cases follow each language: Swift `.noDevice` , Android `NO_DEVICE` , KMP `NoDevice` , Dart `noDevice` , TS `"noDevice"` — the word Graine sends is `no_device` . The status, source and error-code types keep that word in `wire` (Swift, Kotlin, Dart).
> - **Times are moments, not wall-clock readings.** Swift `Date` , Android `java.util.Date` , KMP `Instant` , Dart `DateTime` in UTC (call `.toLocal()` to show it), TS `Date` for bookings and memories, and a number of milliseconds since the epoch for chat turns and call timers. Show them in the device's time zone. A booking you send carries the device's offset (for example `2026-10-14T18:00:00+05:30`) and its time zone, and Graine applies the agent's calling hours in the customer's own time zone.
> - **Values from a newer Graine never crash an older SDK.** A status, source or error code the SDK does not know arrives as Swift `.unknown(String)` , Kotlin `Unknown(raw)` (its `wire` holds the word), Dart `unknown` (the word in `statusName` , `sourceName` or `codeName`) and TS the word itself (the unions end in `string & {}`). Give every `switch` a default case.

GraineChatTurn — one message in the chat

iOS `GraineChatTurn` in `GraineInApp.shared.chat.turns` · Android `GraineChatTurn` in `chat.messages` · Flutter `GraineChatTurn` in `GraineInApp.instance.chat.turns` · React Native `Turn` in `useGraineAgent().messages` (the client's own chat state uses `ChatTurn` , with the same fields) · KMP `GraineChatTurn` in `Graine.chat.turns` (a `StateFlow`), with Android's fields. The website's widget draws the chat itself and hands no turns to the page. Oldest first, always.

Field	Type	Meaning, and what to expect
`id`	Swift, Kotlin, Dart `String` · TS `id?: string`	Swift, Kotlin, Dart: `t1` , `t2` , …, stable for the life of the transcript. React Native: ties a waiting message to its sending. A restored turn keeps Graine's id.

Field	Type	Meaning, and what to expect
`role`	Swift `GraineRole` (`.agent` , `.customer`) · Android `Role` (`AGENT` , `CUSTOMER`) · KMP `GraineRole` (`USER` , `AGENT`) · Dart `GraineChatRole` (`user` , `agent`) · TS `"agent" | "customer"`	Who said it.
`text`	`String` · TS `string`	The words. Grows while `live` . Can be empty for an agent turn that only drew a `component` .
`at`	Swift `Date` · Android `java.util.Date` · KMP `Instant` · Dart `DateTime` (UTC) · TS `at?: number` (ms since the epoch)	When the turn began (an agent's: its first words). A restored turn keeps the time it was said.
`live`	`Bool` · TS `live?: boolean`	The agent's words are still arriving: draw them as provisional. The same turn (same `id`) updates until `live` is false. Restored turns are never live.
`delivery`	Swift `GraineChatTurn.Delivery` (`.sending` , `.sent` , `.failed`) · Android `GraineChatTurn.Delivery` (`SENDING` , `SENT` , `FAILED` ; default `SENT`) · KMP `GraineDelivery?` · Dart `GraineChatDelivery?` · TS `pending?: boolean`	A customer's message only (an agent's turn is `sent` , or null in KMP and Dart). `sending` : typed while the chat was not connected; it goes as soon as it is. `failed` : every reconnect was spent — the next message, or the chat's retry, sends it again. React Native marks only `pending` .
`proactive`	`Bool` · TS `proactive?: boolean`	The agent spoke without being asked: its welcome, or something it noticed.
`component`	Android `GraineComponent?` · Dart `GraineComponent?` · KMP `GraineComponent?` (on Android)	What the agent drew with this turn. iOS draws components in its component views and React Native in `widgets` , so neither puts them on the turn.
`restored`	Swift `Bool` · Android, KMP `Boolean` (default false) · Dart `bool` (default false) · TS `restored?: true`	From an earlier conversation, restored when the chat opened: delivered, with its own time and id, not said just now.
`channel`	Swift `GraineChatChannel` (`.chat` , `.voice`) · Android `GraineChatTurn.Channel` (`CHAT` , `VOICE`) · KMP `GraineChatChannel` (`CHAT` , `VOICE`) · Dart `GraineChatChannel` (`chat` , `voice`) · TS `channel?: "voice"` (absent for the chat)	Where it was said. Only a restored turn can be `voice` .

- **Restored turns arrive once, when the chat opens**, before any reply — only for an identified customer, on an agent with memory on, who has not opted out, on one of their devices. At most 50, oldest first, delivered and never live; the welcome message is not sent then. A reconnect in the same app run receives them again and the SDK does not duplicate them.
- **A run of consecutive `voice` turns is one row**, "Voice call · {time}" — the built-in window never shows what was said on a call (Call History keeps it). React Native's `chatRows` and Kotlin Multiplatform's `GraineChatRows` group them for a transcript of your own.
- **The list empties** on sign-out (`identify(null)`), when another customer signs in, after a successful `forgetMemory()` , and on `newConversation()` .

Chat state — is the chat connected?

Platform	Where	What you get
iOS	`GraineInApp.shared.chat.status` (published)	`GraineChatStatus` : `.idle` (not in the chat: never opened, left, or the app is in the background) · `.connecting` · `.online` · `.reconnecting(attempt:)` (an attempt out of six) · `.offline` (it could not reconnect, or the conversation was ended; a message or `retry()` connects again). Beside it: `online` , `status.isConnecting` , `typing` , `isOpen` , `upcomingCalls` .
Android	`GraineInApp.get().chat`	`connected` , `connecting` , `failed` (every retry spent), `online` , `typing` , `messages` , `upcomingCalls` (soonest first, at most three).
Flutter	`GraineInApp.instance.chat` (a `ChangeNotifier`)	`connected` , `connecting` , `agentTyping` , `failed` , `error` (why the last connect failed, for a log; null once connected), `isOpen` , `turns` , `upcomingCalls` .
React Native	`useGraineAgent()` ; the client	`connected` , `connecting` , `reconnecting` , `typing` , `surface` (`"call"` or `"chat"` : which one the customer is in). The client's `conversation` is `"call"` , `"chat"` or `null` when nothing is live.

Platform	Where	What you get
Kotlin MP	`Graine.chat.status` (`StateFlow<GraineChatStatus>`)	`IDLE` · `CONNECTING` · `ONLINE` · `RECONNECTING` · `OFFLINE`; beside it `typing` and `turns`.
Website	—	The widget shows its own state ("Online", reconnecting). The page opens and closes it; no chat events reach the page.

Reconnecting. When the connection drops, the native SDKs try six times — 1, 2, 4, 8, 15 and 15 seconds apart — and then go offline until the customer sends something or the app retries. React Native waits 1, 2, 4, 8 and 15 seconds, then keeps trying every 30 seconds for as long as the chat is open. One conversation at a time on a device: opening the chat ends a call the bar holds, and a native call holds the chat until it is over.

GraineChatSize — how big the chat window opens

iOS `GraineChatSize` (`.compact`, `.large`, `.full`) · Android `GraineChatSize` (`COMPACT`, `LARGE`, `FULL`) · Flutter `GraineChatSize` (`compact`, `large`, `full`) · React Native `ChatSize` (`"compact" | "large" | "full"`) · KMP `GraineChatSize` (`COMPACT`, `LARGE`, `FULL`) · the website takes the same three words as plain strings.

- **Which size wins:** the size passed to the open call (`chat.open(size:)`, `chat.open(size)`, `GraineChat.open(context, size:)`, `openChat({ size })`, `Graine.chat.open(size)`, `GraineAgent.open({ size })`), then the app's `chatSize`, then the agent's design (*Window size* in the Garage), else compact.
- An open's own size lasts until the chat closes; moving to the call and back keeps it.
- On a phone, large is a sheet over about 92% of the screen and full the whole screen inside the safe areas; on a tablet or computer, large and full open a panel 420 wide and up to 760 tall; on a web page narrower than 640 px, large is a sheet and full the whole page.
- A size word the SDK does not know (from a newer design) is passed over: the next rule decides.

newConversation() — a fresh thread

Platform	Call	Fails with
iOS	`GraineInApp.shared.chat.newConversation() async throws`; Objective-C `newConversationWithCompletion:`	`GraineServiceError` (`deviceLimit`, `network`, …)
Android	`GraineInApp.get().chat.newConversation(done: GraineCallback<Unit>? = null)`	the callback's `GraineError`
Flutter	`Future<void> newConversation()` on `GraineInApp.instance.chat`	`GraineException`
React Native	`client.newConversation(): Promise<void>`, also on `useGraineAgent()`	rejects with `GraineError`
Website	`GraineAgent.newConversation()` → `Promise`	rejects with a code
Kotlin MP	`suspend fun newConversation()` on `Graine.chat`	`GraineException`

- It returns nothing. On success the chat hangs up, Graine drops the conversation it kept for this agent and customer, the window clears and — when the customer was in the chat — a fresh chat opens with the welcome and the starters.
- Memory facts are untouched, and Call History keeps the earlier conversation. With nobody identified nothing was kept, so only the screen clears.
- A refusal (for example `device_limit`) leaves the chat as it was, connected again. A call in progress is left alone.
- The window's own **New conversation** control does the same. The agent's design can hide that control; the call still works from a button of yours.

identify() — who the customer is

Platform	Call
iOS	`GraineInApp.shared.identify(_ traits: [String: Any?]?, memory: Bool? = nil)`
Android	`GraineInApp.get().identify(traits: Map<String, Any?>?, memory: Boolean? = null)`
Flutter	`GraineInApp.instance.identify(Map<String, Object?>? traits, {bool? memory})`
React Native	`client.identify(user: Record<string, unknown> | null, { memory?: boolean })`; `useGraineIdentify()`
Website	`GraineAgent.identify(traits, { memory })`

Platform	Call
Kotlin MP	`Graine.identify(traits: Map<String, Any?>?, memory: Boolean? = null)`

- **What is sent.** The value of the trait the agent's *Who is the user?* field names (`user_id` unless the dashboard chose email, phone or a custom field) goes as it is, only with the requests that start a conversation — never into prompt variables or the screen context. Every other trait becomes a prompt variable, masked on the device.
- **What is kept.** The identifier is stored with the conversation, so your team can find the person in the dashboard; memory is filed under a one-way key made from it, not the value.
- A placeholder (`"guest"` , `"null"` , a masked `[EMAIL]`), null, a boolean or a fraction identifies nobody: that conversation is anonymous, with no memory.
- `memory` : `false` — this customer's conversations are neither remembered nor recalled, kept on the device for them; left out — their last choice stands (on until they make one).
- Identity is fixed per conversation: a change reaches the next one. One customer for the whole app: every configured agent hears it.
- `identify(null)` signs out: the device stops being that customer's and a chat open for them ends with its transcript. Someone else signing in without a sign-out first signs the last customer out the same way.

GraineMemory — one thing the agent remembers

Field	Type	Meaning, and what to expect
`id`	`String`	The memory's id.
`text`	`String`	What is remembered, in a sentence, already redacted.
`createdAt`	Swift `Date?` · Android `java.util.Date?` · KMP `Instant?` · Dart `DateTime?` (UTC) · TS `Date | null`	When it was first remembered; null when Graine did not say.
`updatedAt`	as `createdAt`	When it last changed; null when Graine did not say.

- **`memories()`** returns a list — Swift `[GraineMemory]` , Android `List<GraineMemory>` in the callback, Dart `List<GraineMemory>` , TS `GraineMemory[]` (React Native also has `useGraineMemories()`), the website plain `{ id, text, createdAt, updatedAt }` objects, KMP `List<GraineMemory>` — newest first, at most 100. Only for the identified customer on one of their devices; otherwise `identity_required` or `device_limit` .
- **`forgetMemory()`** returns how many memories were deleted — Swift `Int` , Android `Int` in the callback, Dart `int` , TS `number` , KMP `Int` — and deletes the kept conversation as well. When it succeeds the chat on screen clears; a failure, `unavailable` included, leaves both as they were.

Session settings the app can read

The agent's settings load once per launch. Until then the session is empty and the booking rules are the defaults, with booking off — on iOS and Android, the last rules kept on the device. A setting an older server does not send means its default: `user_id` , 2 devices per user.

Platform	Where	Identity, memory and calls
iOS	`GraineInApp.shared.session: GraineSessionConfig?` ; `GraineListener.onSessionReady(_:)`	`identity: GraineIdentitySettings?` (`field` , `devicesPerUser`) · `memoryEnabled: Bool?` · `calls: GraineHostedCallsSettings?` (`hosted` , `platforms` , `ringTimeoutSeconds` , `callName`) · rules: `GraineInApp.shared.schedulingRules`
Android	`GraineInApp.get().session: SessionConfig?` ; `GraineListener.onSessionReady(session)`	`identity: SessionIdentity?` (`field` , `devicesPerUser`) · `memoryEnabled: Boolean?` · `schedulingRules` (also `GraineInApp.get().schedulingRules`)
Flutter	`GraineInApp.instance.session: SessionConfig?` ; `onSessionReady`	`identity: IdentitySetting?` (`field` , `devicesPerUser`) · `memory: MemorySetting?` (`enabled`) · `calls: CallsSetting?` (`hosted` , `platforms` , `ringTimeoutSeconds` , `callName` , `scheduling`) · rules: `schedulingRules`
React Native	`await client.session(): SessionConfig`	`identity?: { field?, devicesPerUser? }` · `memory?: { enabled? }` · `calls?: { hosted?, platforms?, ringTimeoutSeconds?, callName?, scheduling? }` · rules: `client.schedulingRules`
Website	`GraineAgent.schedulingRules`	The booking rules only.

Platform	Where	Identity, memory and calls
Kotlin MP	`Graine.session: StateFlow<GraineSession?>`	`identityField` (`"user_id"`) · `devicesPerUser` (2) · `memoryEnabled: Boolean?` · `schedulingRules` (also `Graine.schedulingRules`)

GraineScheduledCall — a booked call

Returned by `scheduleCall` and, as a list, by `scheduledCalls()`, on every platform under this name (the website returns plain objects of the same shape). Upcoming first.

Field	Type	Meaning, and what to expect
`id`	`String`	Graine's id for the booking, a UUID: what `cancelScheduledCall` takes.
`at`	Swift `Date?` · Android `java.util.Date?` · KMP `Instant?` · Dart `DateTime?` (UTC) · TS `Date | null`	When it rings. Null only when Graine's time could not be read.
`status`	`GraineCallStatus` (below)	Where it is. Dart adds `statusName` and `statusWire`, the word as Graine sent it.
`source`	`GraineCallSource?` (TS `| null`)	Who booked it; null when Graine did not say. Dart adds `sourceName`.
`reason`	`String?` (TS `| null`)	What it is about, as booked: at most 120 characters. Null when none was given.
`isUpcoming`	`Bool` (computed)	Still to come: `scheduled`, or `ringing` now.
`canCancel`	`Bool` (computed)	It can still be called off: `scheduled` (it has not started ringing).

Return types: `scheduleCall` → `GraineScheduledCall` (status `scheduled`); `scheduledCalls()` → a list of them; `cancelScheduledCall(id)` → the `GraineCallStatus` afterwards, `cancelled` when Graine does not say (the website resolves the word, `"cancelled"`). Signatures per platform: section 10.

GraineCallStatus — where a booked call is

Status	What it means
`scheduled`	Booked, waiting for its time. The only status that can be cancelled.
`ringing`	Graine is ringing the customer's device now.
`answered`	The customer answered; the conversation is on.
`completed`	The call took place and ended.
`declined`	The customer declined the ring. Nothing rings them again for 6 hours.
`missed`	It rang out with no answer.
`failed`	It could not be rung or connected.
`cancelled`	Called off before it rang.
`expired`	Its time passed while it could not ring: more than 10 minutes late, or no opening left in the calling hours within the days-ahead limit.
`no_device`	Graine had no device of this customer's to ring: none of theirs seen in the last 30 days (a website visitor's browser cannot ring).
`unknown(raw)`	A status newer than this SDK, carrying its word.

Names: Swift `.scheduled` … `.noDevice`, `.unknown(String)` · Android `GraineCallStatus.SCHEDULED` … `NO_DEVICE`, `Unknown(raw)` · KMP `GraineCallStatus.Scheduled` … `NoDevice`, `Unknown` · Dart `scheduled` … `noDevice`, `unknown` · TS `"scheduled"` … `"noDevice"`, or the new word. Read it from Graine with Swift `GraineCallStatus(wire:)`, Kotlin `GraineCallStatus.from(wire)`, Dart `GraineCallStatus.fromWire`.

GraineCallSource — who booked it

`agent` (the agent, in a conversation) · `app` (your app, through the SDK; the word Graine sends is `sdk`) · `dashboard` (your team) · `api` · `unknown(raw)`. Swift `.agent` …, Android `AGENT`, `APP`, `DASHBOARD`, `API`, KMP `Agent`, `App`, `Dashboard`, `Api`, Dart `agent`, `app`, `dashboard`, `api`, TS `"agent" | "app" | "dashboard" | "api"`.

GraineSchedulingRules — what may be booked

iOS `GraineInApp.shared.schedulingRules` · Android `GraineInApp.get().schedulingRules` · Flutter `GraineInApp.instance.schedulingRules` · React Native `client.schedulingRules` (and `useGraineScheduling().rules`) · website `GraineAgent.schedulingRules` · KMP `Graine.schedulingRules`. A property — no request.

Field	Type, default	Meaning, and what to expect
`usersCanBook`	`Bool`, false	Customers may book from the app (*Users, from your app*). False when the settings say nothing.
`minLeadMinutes`	`Int`, 5	The least notice a booking may give.
`maxDaysAhead`	`Int`, 14	How far ahead a booking may be.
`maxPendingPerUser`	`Int`, 3	The most calls one customer may have waiting.
`ringTimeoutSeconds`	`Int`, 45	How long a ring Graine sends lasts before it is missed: 20–60 seconds, as the dashboard allows; every SDK holds a value outside that range to it.
`callName`	`String?`, null	The name the phone shows when Graine rings; null — the agent's name.
`hostedRinging`	`Bool`, false	Graine rings this workspace's apps (push credentials uploaded).
`platforms`	list, empty	Where Graine rings: `ios`, `android` (Swift `[GraineRingPlatform]`, KMP `List<GrainePlatform>`, the others strings). Android adds `ringsAndroid`.
`earliest(now)`	→ a time	`now` plus `minLeadMinutes`: the first time a picker should offer.
`latest(now)`	→ a time	`now` plus `maxDaysAhead` days: the last.

Checked on the device first. `scheduleCall` refuses before sending anything when booking is off (`scheduling_off`), the time is before `earliest(now)` or not in the future (`too_soon`; the website allows 30 seconds' grace), or after `latest(now)` (`too_far`) — these errors have status 0. Graine checks again, and checks what only it knows: the calling hours, the calls already waiting.

GraineCallOutcome — what became of a call

Reported once per call — iOS `GraineCallsListener.onCallOutcome(_:callId:)`, Android `GraineCallListener.onCallOutcome(callId, outcome)`, Flutter `GraineCalls.instance.outcomes` (a stream), React Native `GraineCalls.onOutcome(fn)`, KMP `Graine.calls.outcomes` (a flow that does not replay: collect it from app start). Send it to your server with the call id. For a call Graine rang (`hosted`), the SDK has already told Graine.

Field	Type	Meaning, and what to expect
`callId`	`String` (iOS: the listener's argument)	Your server's `call_id`, Graine's for a hosted ring, or the id `startCall` returned.
`outcome`	Swift `GraineCallOutcome` (`.answered`, `.declined`, `.missed`, `.ended(GraineCallEndReason)`, `.failed(String)`) · Android `kind: GraineCallOutcome.Kind` (`ANSWERED`, `DECLINED`, `MISSED`, `ENDED`, `FAILED`) · KMP `outcome: GraineCallOutcome.Kind` · Dart `outcome: GraineCallOutcomeKind` · TS `outcome` string	`answered`, `declined`, `missed`, `ended`, `failed`; React Native, Flutter and KMP also report `ringing` when a ring starts. One word for your server: `wire`.
`reason`	Dart, KMP `reason: String?` · TS `reason` (with ended) or `error` (with failed) · Swift `endReason`, `error` · Android `endReason`, `error`, `reason`	With `ended`: `local` (this app hung up), `remote` (the agent did) or `failed` (the line broke). With `failed`: why it never connected, for your logs.
`durationSeconds`	`Int?` · TS `number | null`	With `ended`: how long the conversation lasted, in whole seconds. Android, Flutter, React Native, KMP.
`hosted`	`Bool`, false	Graine rang it (a booked call), rather than your server's push. Android, Flutter, React Native, KMP.
`busy`	`Boolean`, false (Android)	A `missed` ring refused because another call was live.

Field	Type	Meaning, and what to expect
`agent`	`String?` (Dart, TS, KMP)	The configured agent the call was with — `"default"`, or the name it was configured under. Null when the platform cannot say (on Android, a ring refused while another call was live).

Do not rely on the order of an outcome relative to the call's state changes.

The call itself: incoming, missed, live state, startCall

Platform	Incoming and missed	Live state
iOS	`GraineIncomingCall`: `callId`, `reason?`, `callerName?`, `variables`, `hosted`, `agentId?` · `GraineMissedCall`: `callId`, `callerName`, `reason?`, `hosted`, `at`, `agent?`	`GraineCallState`: `.idle`, `.ringing`, `.connecting`, `.active`, `.ended(GraineCallEndReason)` (then `.idle` once reported)
Android	`GraineCall` (incoming and outgoing): `callId`, `direction` (`INCOMING`, `OUTGOING`), `callerName?`, `reason?`, `variables`, `hosted`. A missed ring becomes the SDK's *Missed call* notification and a `MISSED` outcome; `calls.callBack(activity, missedCallId, reason)` returns it.	`GraineCallState`: `IDLE`, `RINGING`, `CONNECTING`, `ACTIVE`, `ENDED`
Flutter	`GraineIncomingCall` (`GraineCalls.instance.incomingCalls`): `callId`, `reason?`, `callerName?`, `hosted`, `agent?`	`GraineCalls.instance.value`: `GraineCallSnapshot` — `status` (`GraineNativeCallStatus`: idle, ringing, connecting, active, ended), `callId?`, `direction?`, `reason?`, `callerName?`, `muted`, `speakerOn`, `agentSpeaking`, `caption?`, `connectedAt?`, `error?`, `agent?`
React Native	`GraineIncomingCall`: `callId`, `reason`, `callerName`, `agent?`	`useGraineCall()`: `GraineCallSnapshot` — `state` (`"idle" | "ringing" | "connecting" | "active" | "ended"`), `callId`, `direction`, `reason`, `callerName`, `muted`, `speakerOn`, `agentSpeaking`, `caption`, `connectedAt` (ms), `error`, `agent`
Kotlin MP	`GraineIncomingCall` (`Graine.calls.incoming`): `callId`, `callerName?`, `reason?`, `hosted`, `agent?` · `GraineMissedCall`: `callId`, `callerName?`, `reason?`, `hosted`, `at: Instant`	`Graine.calls.state`: `GraineCallInfo` — `state`, `callId?`, `callerName?`, `incoming`, `hosted`, `muted`, `speakerOn`, `agentSpeaking`, `caption?`, `missedCall?`, `agent?`

- **`startCall(…)` returns the call's id**, or nothing when no call was placed: the calls are not configured, a call is already live, or no configured agent has that name (React Native rejects that last one). Swift `String?`, Kotlin `String?`, Dart `Future<String?>`, TS `Promise<string | null>`. The website cannot place native calls.
- **A missed ring** stays news for 30 seconds on the call screen (Call back · Close); `at` is when it rang.
- **The push you hand to `handlePush`** (Android; iOS's VoIP push reaches the SDK directly) carries `type` (`agent_call`), `call_id`, `caller_name`, `hosted` (`"1"` when Graine rang) and `agent_id` — nothing personal. `handlePush` returns false for a push that is not a call, so your own pushes pass through.

Errors — GraineError, GraineServiceError, GraineException

Platform	Type	Fields
iOS	`GraineServiceError` (thrown)	`code: GraineErrorCode` · `message: String` · `retryAfterSeconds: Int?` · `status: Int` (HTTP; 0 when no answer came back or the device refused first)
Android	`GraineError` (in the callback, never thrown)	`code: GraineErrorCode` · `message: String` · `retryAfterSeconds: Int?`
Flutter	`GraineException` (thrown)	`message` · `code: GraineErrorCode` · `codeName: String?` (the word as Graine said it) · `statusCode: int?` · `retryAfterSeconds: int?`
React Native	`GraineError` (rejects)	`code: GraineErrorCode` · `message` · `status: number` · `retryAfterSeconds: number | null`
Website	`Error` (rejects)	`code` · `message` · `status` and `retryAfterSeconds` when Graine answered
Kotlin MP	`GraineException` (thrown)	`code: GraineErrorCode` · `message` · `retryAfterSeconds: Int?`

Codes: `invalid_key` · `origin_refused` · `rate_limited` · `invalid_request` · `identity_required` · `device_limit` · `scheduling_off` · `too_soon` · `too_far` · `outside_window` · `too_many_pending` · `not_found` · `unavailable` · `network` · `unknown(raw)` — Swift and Dart `invalidKey` …, Android `INVALID_KEY` …, KMP `InvalidKey` …, TS the word. When

each happens, which methods raise it and what to do (retry, wait `retryAfterSeconds`, or fix the set-up): section 13. Branch on the code; the message is a sentence fit to show the customer and never carries the identifier, traits or a push token.

Several agents

iOS `GraineInApp.configure(_:name:)`, `GraineInApp.named(_:)`, `GraineInApp.all`, `GraineInApp.shared` · Android `GraineInApp.init(application, config, name)`, `GraineInApp.get(name)`, `GraineInApp.find(name)`, `GraineInApp.all`, `GraineInApp.DEFAULT` · Flutter `GraineInApp.init(config, name:)`, `GraineInApp.named(name)`, `GraineInApp.all`, `GraineInApp.instance` · React Native one `GraineProvider` per agent; calls `GraineCalls.configure({ agents })`, `DEFAULT_AGENT` · KMP `Graine.configure(config, name)`, `Graine.agent(name)`, `Graine.agents` · website one agent on the page at a time, switched with `GraineAgent.load()`. The default agent's name is `"default"` everywhere. What is per agent and what is shared: section 11.

What the SDKs handle for you (for the curious)

- **The `history` frame** — the one frame a continued chat opens with, before any reply: `{ "type": "history", "turns": [{ "id", "role", "text", "at", "channel" }], "continued": true }`. Roles are `user` / `assistant`, `at` is ISO-8601 UTC, `channel` is `chat` or `voice`, at most 50 turns, each text at most 2,000 characters. The SDKs turn it into restored `GraineChatTurn`s; a turn they cannot read is skipped.
- **The website's messages to its widget** — `graine:identity` `{ value, installId, memory, timezone }` (who the visitor is, the browser's install ID, the memory choice and the IANA time zone), `graine:window` `{ size, layout }`, `graine:new-conversation` and `graine:memory-forgotten`. They travel only between your page and Graine's frame.

12.2 React Native — @graineai/inapp-react-native 0.41.0

TypeScript. Everything below is exported from `@graineai/inapp-react-native` (subpaths `/mask` and `/client` re-export the same names). Hooks need a `GraineProvider` above them, unless `setDefaultClient(client)` was called. Requests to Graine reject with `GraineError`.

GraineProvider

`function GraineProvider(props: GraineProviderProps): JSX.Element` — One provider per agent, at the root of your app (a nested provider is that subtree's agent). It builds the agent's client from the props below, or uses the client you pass.

Field / prop	Type		Meaning
`children`	`React.ReactNode`	required	
`client`	`GraineInAppClient`	optional	Use THIS client rather than building one.
`autoConnect`	`boolean`	optional	Connect as soon as the app starts, rather than when the customer opens the agent.
`surface`	`AgentSurface`	optional	Open on the call or on the chat, when the dashboard allows both.
`onSurfaceChange`	`(surface: AgentSurface) => void`	optional	The customer moved between the call and the chat.
`onProactive`	`(text: string) => void`	optional	Called when the agent speaks first, so you can draw a nudge in your own UI.
`endOnBackground`	`boolean`	optional	End the session when the app leaves the foreground, and open a new one when it comes back.
`navigationRef`	`{ current: any } | null`	optional	The ref you already pass to NavigationContainer.
`appVersion`	`string`	optional	Your app's version, for analytics.
`friction`	`FrictionConfig`	optional	This app's friction rules, layered over the dashboard's.

Field / prop	Type		Meaning
fab	`Omit<import("./fab.js").GraineFabProps, "webView"> & { webView: React.ComponentType<any> }`	optional	Mount the floating button for you.
captureTaps	`boolean`	optional	Best-effort tap tracking on every touchable, with no per-button wrapping: short still touches are tracked as tap:screen while a screen has been reported and the visibility rules allow.
includeScreens	`string[]`	optional	Route names where the launcher may appear.
launcherDelayMs	`number`	optional	Wait this long after landing on a screen before showing the launcher.
visibility	`LauncherVisibility`	optional	Per-flow rules: groups, continuity, delays, insets.
fonts	`Record<string, string>`	optional	The fonts this app has registered, by the family name the dashboard uses: { Inter: "Inter-Regular" } maps the design's "Inter" to the fontFamily your app loaded (expo-font, react-native.config.js, Info.plist).
loadFont	`FontLoader`	optional	Lets the SDK fetch the design's typefaces itself.
svg	`SvgKit`	optional	Lets the SDK draw its icons — microphone, hang up, send, chat — as vectors in the design's icon style, exactly as the dashboard's preview shows them.

GraineInAppOptions

The client's options (and the provider's, which accepts them too).

Field / prop	Type		Meaning
baseUrl	`string`	required	Where Graine is deployed — "https://www.graine.ai" for production.
publishableKey	`string`	required	
variables	`Record<string, string>`	optional	Prompt {placeholders} — the customer's name, their plan, whatever the agent needs.
stallAfterMs	`number`	optional	How long a customer may sit on one screen, untouched, before the app tells the agent they look stuck.
friction	`FrictionConfig`	optional	Friction detection — when the app should tell the agent the customer looks stuck.
voice	`boolean`	optional	Open the socket for a VOICE conversation rather than a text chat.
storage	`GraineStorage | null`	optional	Where this install's id and the customer's memory choice are kept between launches.
installIdSource	`() => string | null | undefined | Promise<string | null | undefined>`	optional	An install id the platform already keeps, asked first.
timezone	`string`	optional	The device's IANA time zone.
chatSize	`ChatSize`	optional	How big the chat window opens: "compact" (the window above the bar), "large" (a sheet over most of a phone, a panel on a tablet) or "full" (the whole screen).
reconnectDelaysMs	`number[]`	optional	How long to wait before each try to reopen a chat that dropped.
WebSocketImpl	`any`	optional	Swap in for tests.
fetchImpl	`typeof fetch`	optional	

GraineInAppClient

`class GraineInAppClient` — One agent's client: useGraineClient() inside a provider, or new GraineInAppClient(options). Methods are safe to call at any time; identity, memory choice, screen and actions are shared by every client on the device.

Member	Returns	Errors	What it does
`readonly opts: GraineInAppOptions`	`GraineInAppOptions`	—	Used by the SDK's own components; apps do not need it.
`config: SessionConfig | null`	`SessionConfig | null`	—	Used by the SDK's own components; apps do not need it.
`allVariables(): Record<string, string>`	`Record<string, string>`	—	Every variable the agent should have, in precedence order. Used by the SDK's own components; apps do not need it.
`getRawTraits(): Record<string, unknown>`	`Record<string, unknown>`	—	The traits exactly as identify() was given them — raw, unmasked.
`getVariables(): Record<string, string>`	`Record<string, string>`	—	What identify() and setVariables() have set so far, for hosts that carry it themselves.
`setVariables(vars: Record<string, unknown> | null): void`	`void`	—	Set prompt variables while the app is running.
`get connectionQuality(): ConnectionQuality`	`ConnectionQuality`	—	The connection's quality, as last measured.
`get networkState(): "online" | "offline" | "unknown"`	`"online" | "offline" | "unknown"`	—	Whether the device is online, as the engine last reported.
`reportQuality(sample: QualitySample | null): ConnectionQuality`	`ConnectionQuality`	—	Record a quality sample. Used by the SDK's own components; apps do not need it.
`reportNetwork(online: boolean): void`	`void`	—	The device went offline or came back. Used by the SDK's own components; apps do not need it.
`constructor(opts: GraineInAppOptions)`	–	—	Builds a client for one agent. GraineProvider builds one for you; build your own when code outside React needs it, and pass it as the provider's client.
`joinDevice(): void`	`void`	—	Share the device again (made, or a provider mounting it). Used by the SDK's own components; apps do not need it.
`leaveDevice(): void`	`void`	—	Stop sharing the device — a provider unmounting a client it made. Used by the SDK's own components; apps do not need it.
`dispose(): void`	`void`	—	Releases the client for good: closes its connections and leaves the device.
`get conversation(): "call" | "chat" | null`	`"call" | "chat" | null`	—	Which conversation this client holds now: "chat", "call" or null. One conversation at a time across every client on the device.
`yieldLine(): void`	`void`	—	Another agent took the line: the call the launcher holds ends as a hang-up does, and this client's socket closes. Used by the SDK's own components; apps do not need it.
`configureFriction(override: FrictionConfig | null | undefined): void`	`void`	—	Replace the app's layer of friction rules.
`get friction()`	–	—	The rules in force right now, every layer applied.
`on(event: string, fn: Listener): () => void`	`() => void`	—	Adds a listener for one of the client's events; returns a function that removes it.
`get pendingWidget(): any`	`any`	—	The component still waiting to be drawn or answered, if any.

Member	Returns	Errors	What it does
`clearPendingWidget(): void`	`void`	—	Forget the retained component — it has been answered, dismissed, or the call ended.
`get hasVoiceHost(): boolean`	`boolean`	—	Used by the SDK's own components; apps do not need it.
`setVoiceHost(on: boolean): void`	`void`	—	Called by GraineVoiceLauncher on mount and unmount. Used by the SDK's own components; apps do not need it.
`get nativeCallLive(): boolean`	`boolean`	—	Used by the SDK's own components; apps do not need it.
`setNativeCallLive(live: boolean): void`	`void`	—	Called by the calls package's attach() as its call goes live and ends — every agent's, on a shared device. Used by the SDK's own components; apps do not need it.
`get surface(): AgentSurface`	`AgentSurface`	—	
`openChat(options?: { size?: ChatSize }): void`	`void`	—	Open the chat, at size for this open — else the app's chatSize, else the design's.
`takeChatOpen(): { size: ChatSize | null } | null`	`{ size: ChatSize | null } | null`	—	For GraineProvider: the open to answer, once.
`setSurface(next: AgentSurface): void`	`void`	—	Set by GraineProvider from the dashboard's policy.
`emit(event: string, payload?: any): void`	`void`	—	Used by the SDK's own components; apps do not need it.
`async session(): Promise<SessionConfig>`	`Promise<SessionConfig>`	—	The agent's settings, fetched once and cached.
`checkActions(): ActionDiagnosis`	`ActionDiagnosis`	—	Which of this app's actions the agent can actually call.
`async connect(options: { automatic?: boolean } = {}): Promise<void>`	`Promise<void>`	—	Opens the chat connection. GraineProvider does this when autoConnect is on.
`async executeAction(name: string, args: Record<string, unknown> = {}): Promise<AppActionResult>`	`Promise<AppActionResult>`	—	Run a registered action and return what it said — WITHOUT replying.
`get isConnected(): boolean`	`boolean`	—	Whether THIS client is holding a live socket.
`get inTextChat(): boolean`	`boolean`	—	The conversation is a typed chat: the customer is on the chat surface, or this client's own socket is open as text.
`getScreen(): ScreenContext | null`	`ScreenContext | null`	—	
`getIdentity(): Record<string, string>`	`Record<string, string>`	—	What identify() last set, and what the next frame will carry.
`getRecentEvents(): Array<{ name: string; at: number; data?: Record<string, unknown> }>`	`Array<{ name: string; at: number; data?: Record<string, unknown> }>`	—	The recent events, oldest first.
`getAvailableActions(): string[]`	`string[]`	—	
`registerHighlightable(name: string): () => void`	`() => void`	—	Things on this screen the agent may point AT, as opposed to act on.

Member	Returns	Errors	What it does
`getHighlightables(): string[]`	`string[]`	—	What the agent can point at right now.
`highlight(name: string | null, ttlMs = 6000): void`	`void`	—	Point at something.
`registerAction(name: string, handler: ActionHandler): () => void`	`() => void`	—	Teach the agent one thing it can do in your app.
`get availableActions(): string[]`	`string[]`	—	Which actions this build can currently perform. Used by the SDK's own components; apps do not need it.
`updateField(field: ScreenField): void`	`void`	—	One field changed — report just that.
`removeField(name: string): void`	`void`	—	A field left the screen.
`clearScreen(screenId: string | null | undefined): void`	`void`	—	
`setScreen(context: ScreenContext | null): void`	`void`	—	
`reportEvent(event: AppEvent): void`	`void`	—	Tell the agent something happened that it may want to speak up about.
`track(name: string, data?: Record<string, unknown>, opts?: { coalesce?: boolean }): void`	`void`	—	Record something that happened in the app, WITHOUT making the agent speak.
`setAppVersion(version: string | null | undefined): void`	`void`	—	Which build of the host app this conversation happened in.
`buildContextFrame():`	``	—	Used by the SDK's own components; apps do not need it.
`identify(user: Record<string, unknown> | null, options: { memory?: boolean } = {}): void`	`void`	—	Tell the agent who this is.
`setMemory(enabled: boolean): void`	`void`	—	identify(currentTraits, { memory: enabled }): remember this customer's conversations, or not.
`get memoryEnabled(): boolean`	`boolean`	—	Whether the current customer's conversations may be remembered, as the identity channel will say it.
`async forgetMemory(): Promise<number>`	`Promise<number>`	Rejects with GraineError: identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	Forget everything this agent remembers about the current customer.
`async memories(): Promise<GraineMemory[]>`	`Promise<GraineMemory[]>`	Rejects with GraineError: identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	What the agent remembers about the current customer, so they can see it: newest first, already redacted, at most a hundred.
`get installId(): string | null`	`string | null`	—	This install's id: a UUID made on first launch and kept, so the agent can tell this device from the customer's others.
`async getInstallId(): Promise<string | null>`	`Promise<string | null>`	—	
`async identityChannel(): Promise<IdentityChannel>`	`Promise<IdentityChannel>`	—	The identity channel as it stands: who (the raw identity value, or null), this install, the memory choice, the time zone.
`async newConversation(): Promise<void>`	`Promise<void>`	—	START A NEW CONVERSATION — a fresh thread, as a support chat offers one.

Member	Returns	Errors	What it does
`get schedulingRules(): GraineSchedulingRules`	`GraineSchedulingRules`	—	What this customer may book, from the session: whether they may at all, how soon and how far ahead, how many at once, and how Graine rings.
`async scheduleCall(at: Date | string | number, reason?: string): Promise<GraineScheduledCall>`	`Promise<GraineScheduledCall>`	Rejects with GraineError: on the device: scheduling_off, too_soon, too_far; from Graine: outside_window, too_many_pending, identity_required, device_limit, invalid_request, rate_limited, network, unavailable	Book a call from the agent at at — a Date, an ISO-8601 time, or a timestamp.
`async scheduleCall(request: { at: Date | string | number; reason?: string }): Promise<GraineScheduledCall>`	`Promise<GraineScheduledCall>`	Rejects with GraineError: on the device: scheduling_off, too_soon, too_far; from Graine: outside_window, too_many_pending, identity_required, device_limit, invalid_request, rate_limited, network, unavailable	
`async scheduledCalls(): Promise<GraineScheduledCall[]>`	`Promise<GraineScheduledCall[]>`	Rejects with GraineError: network, rate_limited, unavailable, invalid_key, origin_refused	This customer's calls (or this install's, for someone not identified), upcoming first.
`async cancelScheduledCall(id: string): Promise<GraineCallStatus>`	`Promise<GraineCallStatus>`	Rejects with GraineError: not_found, network, rate_limited, unavailable	Cancel one of this customer's calls.
`noteInteraction(): void`	`void`	—	
`say(text: string): boolean`	`boolean`	—	Sends text into a live conversation. Used by the SDK's own components.
`sendText(text: string): { id: string; queued: boolean }`	`{ id: string; queued: boolean }`	—	A MESSAGE THE CUSTOMER TYPED — never dropped in silence.
`get queuedTexts(): Array<{ id: string; text: string }>`	`Array<{ id: string; text: string }>`	—	Messages still waiting for a connection, oldest first.
`setHostCallLive(live: boolean): void`	`void`	—	Called by GraineVoiceLauncher as the call it holds connects and ends. Used by the SDK's own components; apps do not need it.
`sendAudio(base64: string, sampleRate = 16_000): boolean`	`boolean`	—	Send one chunk of microphone audio. Used by the SDK's own components; apps do not need it.
`endUtterance(): boolean`	`boolean`	—	The customer stopped talking. Used by the SDK's own components; apps do not need it.
`setMuted(muted: boolean): void`	`void`	—	Mutes or unmutes the customer's microphone in a call on this client.
`get isMuted(): boolean`	`boolean`	—	
`setPlayoutClock(fn: (() => number) | null): void`	`void`	—	Tell the client how to find out what is still playing. Used by the SDK's own components; apps do not need it.
`get isAgentSpeaking(): boolean`	`boolean`	—	True while the agent is mid-reply.
`submitWidget(widgetId: string, data: Record<string, unknown>, summary: string): void`	`void`	—	Answer a generative-UI component the agent showed.
`close(): void`	`void`	—	Closes the connection (the chat).

GraineError

`class GraineError extends Error` — What a request to Graine rejects with: a code (branch on it), a message for the customer, the HTTP status (0 when nothing answered) and how long to wait for rate_limited.

Member	Returns	Errors	What it does
`readonly code: GraineErrorCode`	`GraineErrorCode`	—	Declared fields, not parameter properties: Node's type stripping cannot compile those (see RtcVoiceError).
`readonly status: number`	`number`	—	
`readonly retryAfterSeconds: number | null`	`number | null`	—	With rate_limited: how long the route asked to be left alone, when it said.
`constructor(code: GraineErrorCode, message: string, status = 0, retryAfterSeconds: number | null = null)`	`—`	—	

Hooks and functions

Hooks read the nearest GraineProvider's client.

Member	Returns	Errors	What it does
`function useDesign(scheme?: "light" | "dark" | "auto"): DesignLook`	`DesignLook`	—	
`function useDraggablePlacement(opts: DraggablePlacementOptions): DraggablePlacement`	`DraggablePlacement`	—	
`function useGraineAction(name: string, handler: ActionHandler): void`	`void`	—	
`function useGraineAgent(): GraineContextValue`	`GraineContextValue`	—	Connection state, the transcript, and how to send a turn.
`function useGraineClient(): GraineInAppClient`	`GraineInAppClient`	—	The client for this render: the provider's, else the registered default.
`function useGraineConnection(): { quality: ConnectionQuality; online: boolean | null; network: "online" | "offline" | "unknown"; }`	`{ quality: ConnectionQuality; online: boolean | null; network: "online" | "offline" | "unknown"; }`	—	The connection, for a bar that wants to say "weak connection" or "reconnecting" instead of letting the agent take the blame.
`function useGraineEvents(handler: (e: GraineEvent) => void): void`	`void`	—	Subscribe to the agent's lifecycle.
`function useGraineField(name: string, field: Omit<ScreenField, "name">): void`	`void`	—	Report ONE input, as it changes.
`function useGraineHighlight(name: string): { highlighted: boolean }`	`{ highlighted: boolean }`	—	Show somebody where, not just tell them.
`function useGraineIdentify(): (user: Record<string, unknown> | null, options?: { memory?: boolean }) => void`	`(user: Record<string, unknown> | null, options?: { memory?: boolean }) => void`	—	identify(), bound to the client — identify(user, { memory }), and identify(null) on sign-out.
`function useGraineInit(): { isInitialized: boolean; error: string | null }`	`{ isInitialized: boolean; error: string | null }`	—	Whether the SDK has initialised, and the error when it could not.
`function useGraineLoader(): LoaderLook &`	`LoaderLook &`	—	The dashboard's loader, resolved, for hosts that draw their own.
`function useGraineMemories(): { memories: GraineMemory[]; loading: boolean; error: GraineError | null; refresh: () => Promise<void>; }`	`{ memories: GraineMemory[]; loading: boolean; error: GraineError | null; refresh: () => Promise<void>; }`	—	What the agent remembers about the customer, for a screen that shows them.
`function useGraineReady(): { ready: boolean; error: string | null }`	`{ ready: boolean; error: string | null }`	—	

Member	Returns	Errors	What it does
`function useGraineScheduling(): { rules: GraineSchedulingRules; calls: GraineScheduledCall[]; schedule: (at: Date | string | number, reason?: string) => Promise<GraineScheduledCall>; cancel: (id: string) => Promise<GraineCallStatus>; refresh: () => Promise<void>; loading: boolean; error: GraineError | null; }`	`{ rules: GraineSchedulingRules; calls: GraineScheduledCall[]; schedule: (at: Date | string | number, reason?: string) => Promise<GraineScheduledCall>; cancel: (id: string) => Promise<GraineCallStatus>; refresh: () => Promise<void>; loading: boolean; error: GraineError | null; }`	—	Calls the customer books, for a screen that books and lists them.
`function useGraineScreen(context: ScreenContext | null): void`	`void`	—	Report what this screen shows, for as long as it is mounted.
`function useGraineSurface(): { surface: AgentSurface; surfaces: AgentSurface[]; canSwitch: boolean; setSurface: (surface: AgentSurface) => void; }`	`{ surface: AgentSurface; surfaces: AgentSurface[]; canSwitch: boolean; setSurface: (surface: AgentSurface) => void; }`	—	Which agent the customer is in, what else they may have, and how to move.
`function useGraineTap<T extends (...args: any[]) => any>( name: string, handler?: T, data?: Record<string, unknown> ): (...args: Parameters<T>) => ReturnType<T> | undefined`	`—`	—	Wrap a button's onPress so the agent knows it was tapped.
`function useGraineTheme(): ThemeTokens`	`ThemeTokens`	—	The design's tokens for now: the brand's light or dark, else the device's, re-picked when the device switches appearance.
`function useGraineTrack(): (name: string, data?: Record<string, unknown>) => void`	`(name: string, data?: Record<string, unknown>) => void`	—	Record a product event as context — the agent knows it, and stays quiet.
`function useGraineVariables(): (vars: Record<string, unknown> | null) => void`	`(vars: Record<string, unknown> | null) => void`	—	Prompt variables from the app's own data, while it runs.
`function useGraineVoice( adapter: AudioAdapter | null, options: VoiceSessionOptions = {} ): { listening: boolean; agentSpeaking: boolean; start: () => Promise<void>; stop: () => Promise<void> }`	`{ listening: boolean; agentSpeaking: boolean; start: () => Promise<void>; stop: () => Promise<void> }`	—	Talk to the agent instead of typing at it.
`function useGraineWidget(): { widget: GraineWidget | null; answered: { widget: GraineWidget; data: Record<string, unknown>; summary: string } | null; loading: { id: string; kind: string } | null; awaitingReply: boolean; closed: GraineWidget | null; reopen: () => void; submit: (data: Record<string, unknown>, summary: string) => void; dismiss: () => void; }`	`{ widget: GraineWidget | null; answered: { widget: GraineWidget; data: Record<string, unknown>; summary: string } | null; loading: { id: string; kind: string } | null; awaitingReply: boolean; closed: GraineWidget | null; reopen: () => void; submit: (data: Record<string, unknown>, summary: string) => void; dismiss: () => void; }`	—	The component the agent just put on screen, and the way to answer it.
`function useKeyboardLift(): number`	`number`	—	Pixels to add to a bottom-anchored view's bottom while the keyboard is up.
`function useReduceMotion(): boolean`	`boolean`	—	The OS "reduce motion" setting, live.
`function addMaskRule(pattern: RegExp, token = "[REDACTED]"): void`	`void`	—	
`function maskDeep<T>(value: T): T`	`T`	—	Walk any payload and mask every string leaf.
`function maskString(input: string): string`	`string`	—	Replace identifiers in a single string.

Member	Returns	Errors	What it does
`function requestMicrophonePermission( rationale: { title?: string; message?: string; buttonPositive?: string } = {} ): Promise<boolean>`	`Promise<boolean>`	—	
`function setDefaultClient(client: GraineInAppClient | null): void`	`void`	—	Register the client hooks use where no <GraineProvider> is above them.

GraineContextValue

What useGraineAgent() returns: the nearest provider's conversation — its state, the transcript and the controls.

Field / prop	Type		Meaning
`client`	`GraineInAppClient`	required	
`connected`	`boolean`	required	
`connecting`	`boolean`	required	
`error`	`string | null`	required	
`open`	`boolean`	required	
`setOpen`	`(open: boolean) => void`	required	
`messages`	`Turn[]`	required	
`widgets`	`any[]`	required	
`send`	`(text: string) => void`	required	Send what the customer typed.
`typing`	`boolean`	required	A message reached the agent and no word of the reply has arrived yet — draw a typing indicator from it.
`reconnecting`	`boolean`	required	The chat dropped and is coming back by itself.
`muted`	`boolean`	required	True while the customer's voice is being withheld.
`setMuted`	`(muted: boolean) => void`	required	
`agentSpeaking`	`boolean`	required	True while the agent is mid-reply — drive a speaking indicator from it.
`caption`	`Caption | null`	required	The line being said RIGHT NOW, either side, for captions.
`launcherVisible`	`boolean`	required	May the launcher be on screen right now, after route rules and delays.
`launcherInset`	`LauncherInset`	required	Where to sit, so the launcher clears tab bars and sheets per screen.
`currentScreen`	`string | null`	required	The route the SDK believes the customer is on, or null without tracking.
`appearance`	`Record<string, any> | null`	required	How this agent should LOOK, as configured in the dashboard.
`design`	`ResolvedDesign`	required	How this agent should look, as the brand chose it in the dashboard's Design editor: the config and its tokens for light and dark.
`fonts`	`Record<string, string> | null`	required	The app's registered fonts, from the provider's fonts.
`faces`	`LoadedFaces`	required	The design's faces the SDK has loaded through the provider's loadFont: "DM Sans|700" → the fontFamily it registered.
`svg`	`SvgKit | null`	required	react-native-svg's Svg and Path, from the provider's svg, for the SDK's own icons.
`endOnBackground`	`boolean`	required	The provider's endOnBackground, for the voice launcher: a call ends two seconds after the app leaves the foreground unless the app turned it off.
`sessionReady`	`boolean`	required	The session (agent config, appearance, actions) has resolved.
`onEvent`	`(fn: (e: GraineEvent) => void) => () => void`	required	Subscribe to the lifecycle stream.

Field / prop	Type		Meaning
track	`(name: string, data?: Record<string, unknown>) => void`	required	Record a product event as context, without making the agent speak.
identify	`(user: Record<string, unknown> | null, options?: { memory?: boolean }) => void`	required	Tell the agent who this is.
newConversation	`() => Promise<void>`	required	A fresh thread: the chat hangs up, Graine lets go of the conversation it kept, the chat clears and opens again with the welcome.
surface	`AgentSurface`	required	Which agent the customer is in right now — the call, or the chat.
surfaces	`AgentSurface[]`	required	The surfaces the dashboard allows, in the order to offer them.
canSwitch	`boolean`	required	Whether to draw a switch at all.
setSurface	`(surface: AgentSurface) => void`	required	Move the customer between them.
chatSize	`ChatSize`	required	How big the chat window opens — compact, large or full: the size the last openChat() asked for, else the provider's chatSize, else the design's.
openChat	`(options?: { size?: ChatSize }) => void`	required	Move to the chat and open it, at size for this open.
endChatOpen	`() => void`	required	For the SDK's own bar: the chat window closed, so an open's own size is over.
chatScreenShown	`boolean`	required	A GraineChatScreen is on screen: the bar's own chat steps aside.
noteChatScreen	`(delta: number) => void`	required	For GraineChatScreen: +1 as one appears, −1 as it goes.
connectionQuality	`ConnectionQuality`	required	How good the connection is, in one word: excellent · good · poor · lost, and unknown outside a conversation.
network	`"online" | "offline" | "unknown"`	required	Whether the device is online, as the SDK last learned.
setVariables	`(vars: Record<string, unknown> | null) => void`	required	Set prompt variables while the app runs.

GraineVoiceLauncher

`GraineVoiceLauncher(props: GraineVoiceLauncherProps)` — A component of the SDK.

Field / prop	Type		Meaning
transport	`"ws" | "rtc"`	optional	Which transport carries the AUDIO.
onTransport	`(info: { transport: "ws" | "rtc"; requested: "ws" | "rtc"; reason?: "not_configured" | "unreachable" | "screen_channel"; message?: string; }) => void`	optional	Told which transport actually carried the call, including after a fallback.
webView	`React.ComponentType<any>`	required	react-native-webview's WebView, passed in rather than imported.
autoStart	`boolean`	optional	Start talking as soon as the page is ready.
active	`boolean`	optional	Keep the engine mounted before the customer needs it.
onCaption	`(c: { role: string; text: string; live: boolean }) => void`	optional	Called with each caption line, for your own UI.
onCallState	`(s: { connected: boolean; connecting: boolean }) => void`	optional	Called when the call connects or ends.
onEnded	`() => void`	optional	The call is over.

Field / prop	Type		Meaning
onMicDenied	`(reason: string) => void`	optional	The microphone was refused or is unavailable — show a prompt.
onError	`(message: string) => void`	optional	Voice could not be set up at all — the key was rejected, the agent is not enabled for apps, the network is gone.
timing	`{ readyTimeoutMs?: number; retryDelaysMs?: number[]; stallMs?: number; bootMs?: number; hardCapMs?: number; endWatchdogMs?: number; slowConnectMs?: number; }`	optional	How long to wait for the page, and how often to try again.
children	`(api: GraineVoiceApi) => React.ReactNode`	optional	

GraineVoiceBar

`GraineVoiceBar(props: GraineVoiceBarProps)` — A component of the SDK.

Field / prop	Type		Meaning
api	`GraineVoiceApi`	required	The call — GraineVoiceLauncher's render prop hands it over.
onClose	`() => void`	optional	After a hang up or a close, once the goodbye has left the speaker.
bottomInset	`number`	optional	How far above the bottom edge it sits — clear of your tab bar.
draggable	`boolean`	optional	Let the customer drag it up and down.
placement	`{ bottomOffset?: number } | null`	optional	A position the app persisted, to restore.
onPlacementChange	`(placement: { bottomOffset: number }) => void`	optional	The customer let go somewhere new.
name	`string`	optional	What spoken labels call the agent — "Start talking to Ring".
accessory	`React.ReactNode`	optional	Anything that should ride on the bar, above it — a "Call" button that moves the conversation to a phone call, say.
safeArea	`{ top?: number; bottom?: number }`	optional	The phone's safe areas, for a chat that opens as a sheet or the whole screen (Window size in the dashboard, or chatSize).

GraineBarAction

`GraineBarAction(props: GraineBarActionProps)` — A button for the bar's accessory, drawn in the brand's design.

Field / prop	Type		Meaning
label	`string`	required	What the button says, and what a screen reader reads — "Call Ring", say.
icon	`Extract<SdkIconName, "call" | "chat">`	optional	The SDK icon in front of the label, in the design's icon style.
onPress	`() => void`	required	

GraineChatScreen

`GraineChatScreen(props: GraineChatScreenProps)` — The chat as A SCREEN OF YOUR OWN — the conversation filling a route of your app, drawn in the brand's design as the bar draws its full-screen chat: the header with the chevron leading it, the thread, the components the agent draws, the composer above the keyboard.

Field / prop	Type		Meaning
onClose	`() => void`	optional	The header's chevron: leave the screen (navigation.goBack, say).
safeArea	`{ top?: number; bottom?: number }`	optional	The phone's safe areas — react-native-safe-area-context's useSafeAreaInsets(); a typical phone's without.

GraineLauncherButton

`GraineLauncherButton(props: GraineLauncherButtonProps)` — A component of the SDK.

Field / prop	Type		Meaning
onPress	`() => void`	required	
style	`ViewStyle`	optional	Where it sits, for example { position: "absolute", right: 16, bottom: 24 }.
label	`string`	optional	What a screen reader says.
size	`number`	optional	

GraineAgentBar

`GraineAgentBar(props: GraineAgentBarProps)` — A component of the SDK.

Field / prop	Type		Meaning
accent	`string`	optional	
name	`string`	optional	Shown beside the status.
avatarUrl	`string`	optional	A face.
bottomInset	`number`	optional	Lifted clear of your tab bar, so it never covers the primary action.
hidden	`boolean`	optional	Hide it without unmounting the provider — for example on auth or payment screens.
captionsOn	`boolean`	optional	Start with captions on.
scheme	`"dark" | "light" | "auto"`	optional	Which palette to draw.
theme	`Partial<GraineBarTheme>`	optional	Override individual surfaces.
draggable	`boolean`	optional	Let the customer drag the bar up and down.
placement	`{ bottomOffset?: number } | null`	optional	A position the app persisted, to restore.
onPlacementChange	`(placement: { bottomOffset: number }) => void`	optional	The customer let go somewhere new.

GraineLauncher

`GraineLauncher(props: GraineLauncherProps)` — A component of the SDK.

No props interface.

GraineFab

`GraineFab(props: GraineFabProps)` — A component of the SDK.

Field / prop	Type		Meaning
webView	`ComponentType<any>`	required	WebView from react-native-webview — the voice engine needs it.
position	`"bottom-right" | "bottom-left"`	optional	Which corner.
size	`number`	optional	Diameter in px.
accent	`string`	optional	Override the dashboard accent for the button only.
label	`string`	optional	Screen-reader label.
overlay	`ComponentType<{ children?: React.ReactNode }>`	optional	Draw the button inside this wrapper.
onOpen	`() => void`	optional	
onClose	`() => void`	optional	
prewarm	`boolean`	optional	Load the voice page BEFORE the customer taps.

GraineTouchCapture

`GraineTouchCapture(props: GraineTouchCaptureProps)` — A component of the SDK.

Field / prop	Type		Meaning
`children`	`React.ReactNode`	required	
`enabled`	`boolean`	optional	Off switches capture without unmounting the wrapper.
`style`	`StyleProp<ViewStyle>`	optional	

GraineWidget

`GraineWidget(props: GraineWidgetProps)` — A component of the SDK.

Field / prop	Type		Meaning
`theme`	`Partial<GraineWidgetTheme>`	optional	All optional, because the dashboard already decided.
`accent`	`string`	optional	Buttons and badges.
`radius`	`number`	optional	Corner rounding.

GraineLoader

`GraineLoader(props: GraineLoaderProps)` — A component of the SDK.

Field / prop	Type		Meaning
`moment`	`LoaderMoment`	optional	Which words to show, from Appearance.
`label`	`string`	optional	Overrides the moment's configured words.
`kind`	`LoaderKind`	optional	Host overrides; the dashboard's choice is the default.
`color`	`string`	optional	
`size`	`number`	optional	
`stacked`	`boolean`	optional	Draw the label to the right (default) or under the mark.

GraineVoiceApi

What GraineVoiceLauncher's render prop receives: the voice call's state and controls.

Field / prop	Type		Meaning
`phase`	`GrainePhase`	required	
`connectivity`	`GraineConnectivity`	required	See GraineConnectivity.
`retry`	`() => void`	required	Reload the page and start the retry ladder from zero.
`connected`	`boolean`	required	
`connecting`	`boolean`	required	
`muted`	`boolean`	required	
`ready`	`boolean`	required	Whether a call can actually be started right now.
`start`	`() => void`	required	
`end`	`() => void`	required	
`setMuted`	`(m: boolean) => void`	required	
`captions`	`Caption[]`	required	What has been said so far, oldest first — the agent's turns and the customer's, as the engine reported them.
`caption`	`Caption | null`	required	The turn being said right now, or the last one said.
`captionsOn`	`boolean`	required	The CC switch, owned here so every host spells it the same way.
`setCaptionsOn`	`(on: boolean) => void`	required	
`clearCaptions`	`() => void`	required	Forget the transcript without ending the call.
`quality`	`ConnectionQuality`	required	The connection in one word: excellent, good, poor, lost or unknown.

Field / prop	Type		Meaning
network	`"online" | "offline" | "unknown"`	required	Whether the device is online, from the engine page's own view of the network.
reconnecting	`boolean`	required	A call was interrupted by the network and is being re-established.
error	`string | null`	required	Set when the call cannot work until a setting is fixed — for example a key or an agent not set up for in-app calls.
micDenied	`boolean`	required	The customer refused the microphone on the last start.
connectStage	`"registering" | "dialling" | "retrying" | "switching" | null`	required	WHERE A CALL BEING PLACED IS, as the engine reports it.
slowConnect	`boolean`	required	The connect has run past SLOW_CONNECT_MS.
callFailed	`boolean`	required	The last start could not be connected at all — not on WebRTC, not on the fallback.

GraineScheduledCall

A type the SDK hands your app.

Field / prop	Type		Meaning
id	`string`	required	
at	`Date | null`	required	When it rings.
status	`GraineCallStatus`	required	
source	`GraineCallSource | null`	required	
reason	`string | null`	required	
isUpcoming	`boolean`	required	It has not rung yet, or it is ringing now.
canCancel	`boolean`	required	It can still be cancelled: it has not started ringing.

GraineCallStatus

Where a booked call is.

```
type GraineCallStatus = | "scheduled" | "ringing" | "answered" | "completed" | "declined" | "missed" | "failed" | "cancelled" | "expired" | "noDevice" | (string & {})
```

GraineCallSource

Who booked it.

```
type GraineCallSource = "agent" | "app" | "dashboard" | "api" | (string & {})
```

GraineSchedulingRules

What the customer may book, from the session — schedulingRules.

Field / prop	Type		Meaning
usersCanBook	`boolean`	required	
minLeadMinutes	`number`	required	
maxDaysAhead	`number`	required	
maxPendingPerUser	`number`	required	
ringTimeoutSeconds	`number`	required	
callName	`string | null`	required	The name the phone shows when Graine rings.
hostedRinging	`boolean`	required	Graine rings the customer's devices itself (the workspace set up in-app calling).

Field / prop	Type		Meaning
`platforms`	`Array<"ios" | "android">`	required	
`earliest`	`method`	required	The earliest time a call may be booked for: now + minLeadMinutes.
`latest`	`method`	required	The latest: now + maxDaysAhead days.

GraineMemory

A type the SDK hands your app.

Field / prop	Type		Meaning
`id`	`string`	required	
`text`	`string`	required	
`createdAt`	`Date | null`	required	
`updatedAt`	`Date | null`	required	

GraineErrorCode

What the routes answer with ({ error, code }), plus network (no answer) and unknown.

```
type GraineErrorCode = | "invalid_key" | "origin_refused" | "rate_limited" | "invalid_request" |
"identity_required" | "device_limit" | "scheduling_off" | "too_soon" | "too_far" | "outside_window" |
"too_many_pending" | "not_found" | "unavailable" | "network" | "unknown" | (string & {})
```

IdentityChannel

The identity channel, as every SDK hands it to whatever carries the conversation.

Field / prop	Type		Meaning
`value`	`string | null`	required	The raw value of the agent's identity trait, or null when nobody is identified.
`installId`	`string | null`	required	This install's id, or null when it cannot be kept between launches.
`memory`	`boolean`	required	Whether this customer's conversations may be remembered.
`timezone`	`string | null`	required	The device's IANA time zone.

Turn

A type the SDK hands your app.

Field / prop	Type		Meaning
`role`	`"agent" | "customer"`	required	
`text`	`string`	required	
`live`	`boolean`	optional	True while the agent is still streaming this turn.
`pending`	`boolean`	optional	The customer's message is waiting for a connection; it goes the moment there is one.
`id`	`string`	optional	Ties a waiting message to the moment it is sent; a restored turn's id is Graine's.
`proactive`	`boolean`	optional	The agent started this turn unasked.
`at`	`number`	optional	When it was said, in ms since the epoch.
`restored`	`true`	optional	From an earlier conversation: the chat continues where the customer left it (an identified customer, memory on).
`channel`	`"voice"`	optional	Said on a call: a run of these is drawn as one "Voice call" row, never as bubbles.

Caption

A type the SDK hands your app.

Field / prop	Type		Meaning
`role`	`"agent" | "customer"`	required	
`text`	`string`	required	
`live`	`boolean`	required	Still being recognised or streamed — render it as provisional.

GraineEvent

Everything the host might want to react to, as one stream.

```
type GraineEvent = | { type: "conversation_started" }
```

ScreenContext

A type the SDK hands your app.

Field / prop	Type		Meaning
`screen`	`string`	required	Stable id for the screen — used for analytics, so keep it constant.
`title`	`string`	optional	Human title, spoken about by the agent.
`journey`	`{ name: string; step?: number; of?: number; next?: string }`	optional	Where this screen sits in a flow, and where the flow goes next.
`fields`	`ScreenField[]`	optional	
`error`	`string`	optional	A screen-level error banner, distinct from a per-field one.
`data`	`Record<string, unknown>`	optional	Anything else the agent should know.
`availableActions`	`string[]`	optional	Which actions work HERE.

ScreenField

One field on the screen, as the customer sees it.

Field / prop	Type		Meaning
`name`	`string`	required	
`label`	`string`	optional	
`value`	`string | number | null`	optional	Masked before it leaves the device.
`status`	`"empty" | "filled" | "invalid" | "pending" | "locked"`	optional	
`error`	`string`	optional	The validation message actually shown.
`optional`	`boolean`	optional	Leaving this one blank is fine.

AppEvent

Something worth the agent speaking up about, unprompted.

Field / prop	Type		Meaning
`name`	`string`	required	for example "stalled_on_step", "upload_failed", "application_rejected".
`detail`	`string`	optional	Plain English, given to the agent verbatim as the situation to help with.
`cooldown_seconds`	`number`	optional	Seconds before another intervention is allowed.
`max_per_session`	`number`	optional	Hard cap on unsolicited interventions per session.

Field / prop	Type		Meaning
quiet	boolean	optional	State, not a nudge: record it and put it in the agent's live context, but do not make the agent speak.
data	Record<string, unknown>	optional	Structured detail for the runtime.

AppActionResult

What your handler returns.

Field / prop	Type		Meaning
status	"ok" | "error" | "refused"	required	
data	Record<string, unknown>	optional	Structured result the agent can reason about.
message	string	optional	Shown to the agent, not the customer — it decides how to phrase it.

ActionHandler

A handler you register for one action the agent may call.

```
type ActionHandler = ( args: Record<string, any> ) => AppActionResult | void | Promise<AppActionResult | void>
```

ActionDiagnosis

What the app can do, set against what the agent is allowed to ask for.

Field / prop	Type		Meaning
usable	string[]	required	Registered here and declared on the agent.
undeclared	string[]	required	Registered here, NOT declared on the agent.
unhandled	string[]	required	Declared on the agent, nothing registered here.

SessionConfig

A type the SDK hands your app.

Field / prop	Type		Meaning
agentId	string	required	
webcallAgentId	string	optional	
wsHost	string	optional	
appearance	Record<string, any>	optional	
design	ResolvedDesign	optional	How the agent looks: the brand's design and the tokens it resolves to, for light and for dark, worked out by the server.
variables	Record<string, string>	optional	
components	any[]	optional	
appActions	Array<{ tool_name?: string; toolName?: string; name?: string }>	optional	The actions the AGENT is configured to call, from the dashboard.
friction	FrictionConfig	optional	Friction rules configured in the dashboard for this agent.
pageVersion	string	optional	Which deployment is serving the voice engine page.
identity	{ field?: string; devicesPerUser?: number }	optional	Who the customer is, as the agent's Memory section decides (Who is the user?): the trait to key on (user_id by default) and how many devices one customer may use.

Field / prop	Type		Meaning
memory	`{ enabled?: boolean }`	optional	Whether this agent remembers customers at all.
calls	`{ hosted?: boolean; platforms?: string[]; ringTimeoutSeconds?: number; scheduling?: { usersCanBook?: boolean; maxDaysAhead?: number; minLeadMinutes?: number; maxPendingPerUser?: number }; }`	optional	Calls Graine rings, and what customers may book from the app.

GraineStorage

Key-value storage, the shape AsyncStorage and localStorage share.

Field / prop	Type		Meaning
getItem	`method`	required	
setItem	`method`	required	
removeItem	`method`	optional	

ChatTurn

A type the SDK hands your app.

Field / prop	Type		Meaning
role	`"agent" | "customer"`	required	
text	`string`	required	
live	`boolean`	optional	The agent is still streaming this turn.
pending	`boolean`	optional	The customer's message is waiting for a connection; it is sent the moment there is one.
id	`string`	optional	Matches a queued message to its customer_sent; a restored turn's id is the one Graine gave it.
proactive	`boolean`	optional	The agent started this turn unasked.
at	`number`	optional	When it was said, in ms since the epoch — when the event that began it carried a time.
restored	`true`	optional	From an earlier conversation, restored by the history frame: delivered, and not said just now.
channel	`"voice"`	optional	Said on a call ("voice"); absent for the chat.

ChatSize

How big the chat window opens: compact is the window above the voice bar; large a sheet from the bottom on a phone (a panel on a tablet or a computer); full the whole screen.

```
type ChatSize = "compact" | "large" | "full"
```

Also exported

The building blocks the SDK's components are made of — design tokens, protocol constants, audio helpers, voice sessions, launcher visibility. Public so a custom UI can reuse them; an integration does not need them.

Name	Kind	What it is
AGENT_PCM_RATE	const	What headerless PCM from the agent is, on a voice session: 24 kHz.
AWAITING_REPLY_MS	const	How long the answered line waits for the agent before it stops saying so.
AgentChunk	interface	
AgentSurface	type	A conversation is one or the other at any moment.
AppActionRequest	interface	
AudioAdapter	interface	

Name	Kind	What it is
AudioFormat	interface	What a stream turned out to be.
CAPTION_HISTORY	const	How many turns the launcher keeps.
CAPTURE_SAMPLE_RATE	const	16 kHz mono.
CHAT_HISTORY_CAP	const	Turns kept in a chat.
CHAT_SIZES	const	
CHAT_TYPING_TIMEOUT_MS	const	How long "the reply is coming" may stand with nothing arriving, then typing_timeout.
CHAT_WINDOW	const	The numbers a large or full chat window is drawn with.
ChatEvent	type	
ChatPieceTokens	interface	The chat tokens that leave a piece out, as a session may carry them — a value that is not a boolean, or not a string, is the default.
ChatPieces	interface	What a chat window draws of those pieces.
ChatPresentation	type	Where a chat window of a size goes: window — above the voice bar, as always; sheet — up from the bottom, 92% of a phone; panel — 420 wide in the bottom corner of a tablet or a computer; screen — all of it.
ChatRow	type	WHAT A CHAT WINDOW DRAWS, in order: a bubble per chat turn; ONE "Voice call" row for a run of voice turns on one day (the window shows that a call happened, never its transcript); and a day separator — Today, Yesterday, a date — before the first row when it is not from today, and wherever the device's day changes between rows.
ChatRowOptions	interface	
ChatState	interface	
ConnectionQuality	type	
DEFAULT_DESIGN_PRESET	const	
DEFAULT_FRICTION	const	
DEFAULT_IDENTITY_FIELD	const	The trait an agent keys on when its Identity setting says nothing else.
DEFAULT_LOADER_KIND	const	
DEFAULT_LOADER_TEXT	const	
DEFAULT_UPCOMING_TITLE	const	The booked-calls card's title where the brand wrote none: one call, or more.
DEFAULT_WIDGET_PRESET	const	
DESIGN_CHROMES	const	
DESIGN_FONTS	const	Fonts offered in the dashboard: "system" and a curated set of Google Fonts, each with the scripts it covers — the product speaks Hindi and other Indian languages, and a Latin-only face must be paired with an Indic one for those.
DESIGN_PRESETS	const	
DRAG_SLOP_PX	const	A drag that moved less than this is a tap, and the tap must still work.
DecodedAudio	interface	
DesignBubbles	type	
DesignChrome	type	How a surface that holds content is framed.
DesignConfig	interface	What the dashboard saves under appearance.design, and what the session carries back.
DesignPreset	interface	A philosophy: a complete look, before any knob.
EMPTY_CHAT	const	
EchoGuard	class	Stops the agent from hearing itself.
FONT_ASSET_VERSION	const	The version of the typeface files Graine serves, part of their URL.
FontFace	interface	One face of one family, as registered with the platform.
FontLoader	type	Registers fonts by name — { "Graine-DMSans-700": "https://…/DMSans_700Bold.ttf" }.
FrictionConfig	interface	
FrictionDetector	class	
FrictionKind	type	
FrictionNudge	interface	

Name	Kind	What it is
FrictionScreenRules	interface	What a single screen may change.
FrictionSignal	interface	
GraineBarTheme	interface	The surfaces the bar draws on.
GraineWidgetTheme	interface	The colours a component is drawn in.
IconStyle	type	
LAUNCHER_ICONS	const	Glyphs for the launcher, as 24×24 SVG path data every platform can draw: Android's PathParser, iOS via an SVG-path parser, Flutter's path parsing, the web as-is.
LOADER_KINDS	const	
LOADER_MOMENTS	const	
LOADER_SIZES	const	
LOADER_SPEEDS	const	
LOADER_TEXT_MAX	const	
LOGO_PLATES	const	
LOGO_PLATE_WHITE	const	
LauncherContinuity	type	Whether the launcher stays put while the customer moves inside one flow.
LauncherDelayPolicy	type	When the delay is spent.
LauncherGroup	interface	
LauncherInset	interface	Distance from each screen edge.
LauncherShape	type	
LauncherVisibility	interface	
LauncherVisibilityTracker	class	Decides where and when the launcher may appear, and remembers enough between screens to honour the delay policies.
LoaderKind	type	How waiting looks — chosen in Embed & Widgets → Appearance, carried in the session's appearance.loader, drawn by the SDK's GraineLoader and by the web embed.
LoaderLook	interface	The effective loader: numbers a renderer can use directly.
LoaderMoment	type	
LoaderSize	type	
LoaderSpeed	type	
LoaderStyle	interface	
LoaderText	interface	
LogoPlate	type	What a logo sits on.
MAX_SELECT_DEFAULT	const	
MIN_LOADER_MS	const	Once the loader is on screen, it stays for at least this long.
MotionEasing	type	
NAVIGATE_ACTION	const	The one handler an app registers so the dashboard can open any screen.
NativeAppearance	interface	
NativeAppearanceFallback	interface	
PENDING_WIDGET_MS	const	How long a "component coming" skeleton may stand with nothing arriving.
PLATFORM_GRAMMAR	const	
PlatformGrammar	interface	
QualitySample	interface	
ResolvedDesign	interface	
RestoredTurn	interface	One turn of an earlier conversation, as the history frame restores it.
RtcVoiceError	class	
RtcVoiceErrorCode	type	
RtcVoiceOptions	interface	
RtcVoiceSession	interface	What the mint returns.
RtcVoiceSessionController	class	

Name	Kind	What it is
`RtcVoiceState`	type	
`SDK_ICONS`	const	
`SELECT_ARG_MAX`	const	
`SELECT_VALUE_MAX`	const	
`STREAM_END`	const	
`STREAM_START`	const	
`SdkIconName`	type	The sdk's own icons — microphone, muted, hang up, send, chat, call, chevron, close — as 24×24 drawings every surface draws the same: the dashboard's preview, the web widget and each SDK.
`SurfacePolicy`	interface	
`SvgKit`	interface	react-native-svg's two components, handed over rather than imported: the SDK must build without it.
`TAP_MAX_MOVE_PX`	const	A touch that moved further than this is a scroll or a drag, not a tap.
`TAP_MAX_MS`	const	A touch held longer than this is a long-press, not a tap.
`TICKET_VARIABLE_LIMITS`	const	—— Prompt variables on the ticket Two roads carry a variable to the runtime, and they have different limits.
`ThemeColors`	interface	
`ThemeShadow`	interface	
`ThemeTokens`	interface	Everything a renderer needs, as flat numbers and #RRGGBB strings.
`UNNAMED_TAP`	const	The name an automatically captured tap is tracked under.
`VisibilityDecision`	interface	
`VoiceSession`	class	Wires a microphone to an agent, both ways.
`VoiceSessionOptions`	interface	
`WIDGET_BUTTONS`	const	
`WIDGET_DENSITIES`	const	
`WIDGET_IMAGE_SIZES`	const	
`WIDGET_PLATFORMS`	const	
`WIDGET_PRESETS`	const	
`WIDGET_SELECTS`	const	
`WIDGET_SURFACE_CHOICES`	const	
`WidgetButton`	type	
`WidgetDensity`	type	
`WidgetImageSize`	type	
`WidgetLook`	interface	The effective look: preset defaults, then the knobs on top.
`WidgetMode`	type	—— THE COLOURS A COMPONENT IS DRAWN IN Appearance already carried accent, radius, logo and heading, and the dashboard's Appearance panel also has a light/dark/auto setting that nothing native ever read.
`WidgetPalette`	interface	
`WidgetPlatform`	type	Whose design grammar.
`WidgetPreset`	type	HOW A COMPONENT LOOKS — chosen on the component in the dashboard's Web components section, carried with it to the agent, drawn by the SDK.
`WidgetSelect`	type	One, or several.
`WidgetSelectAction`	interface	What the app does when the customer taps — beyond telling the agent.
`WidgetStyle`	interface	
`WidgetSurface`	type	
`activeRouteName`	function	The deepest active route — the screen the customer is actually looking at.
`base64ToBytes`	function	Decode without depending on atob or Buffer.
`base64ToPcm16`	function	base64 → PCM16, for an adapter that already knows it has PCM.
`chatClock`	function	"6:05 PM" in the device's own words.

Name	Kind	What it is
chatDate	function	A day that is neither today nor yesterday: "Tue 14 Oct", with the year when it is not this one.
chatEventFromClient	function	The chat event a client event stands for, or null.
chatIsNew	function	The brand's starters belong to a new chat: not once the customer has said something, and not on a conversation that continues.
chatPanelSize	function	A panel's size on a screen this tall.
chatPieces	function	The pieces a chat window draws for these turns and booked calls: the same rule in every SDK.
chatPresentation	function	How a size is drawn on a screen this big.
chatRows	function	The rows for items (oldest first).
chatSizeOf	function	The size to open at: the first of candidates that is a size — the open's own, then the app's config, then the design's token — else compact.
chatTurnRows	function	chatRows for the chat's own turns.
clampOffset	function	Clamp a proposed offset to what the screen allows.
classifyQuality	function	One sample → one word.
contextToken	function	The variables the agent's prompt reads, on the one channel the runtime takes.
contrastRatio	function	WCAG contrast ratio, 1–21.
decodeAgentAudio	function	Decode one agent audio frame.
decodeMuLaw	function	
decodePcm16	function	Little-endian PCM16.
designFromAppearance	function	The design an embed's appearance describes.
designPreset	function	
faceFor	function	The face to register for family drawn at weight, or null when the family is the system font or one this build has no files for.
facesForDesign	function	Every face a design can draw with, in either scheme.
fillSelectArgs	function	
fillSelectTemplate	function	Fill {id}/{title}/... from the tapped item.
foldChat	function	
fontUrl	function	The URL of one typeface file served by Graine.
glyphUrl	function	Where an app fetches a launcher glyph as a PNG.
isTap	function	A movement smaller than the slop, in both axes, is a tap.
liftFor	function	
liveAudioStreamAdapter	function	react-native-live-audio-stream for capture, plus any PCM sink you pass.
logoBox	function	The square a logo is fitted into — whole, proportions kept, centred, never cropped — inside a shape size across (a pill: its height).
logoPlateLook	function	The fill and outline under a logo, for its plate.
manySummary	function	The sentence the agent hears when several were chosen — and, in a text chat, the customer's own bubble: the button's label, then the names.
normaliseDesignConfig	function	The config as it may be trusted: known keys, allowed values, nothing else.
normaliseLoaderStyle	function	
normaliseLogoPlate	function	
normaliseSelectAction	function	
normaliseWidgetStyle	function	Only the keys and values the renderers read.
offsetDuringDrag	function	The offset during a drag: the offset the drag started from, plus how far the finger moved UP (a gesture's dy grows downward, so it is subtracted).
pickTokens	function	Which tokens to draw: the brand's fixed scheme, else the device's.
readAgentChunk	function	Read one text frame: what to show, and what it says about the turn.
readHistoryFrame	function	The turns a history frame restores, oldest first — or null when the frame is not one (its turns is not a list), which is then ignored.
readableOn	function	Whichever of the candidates reads best on bg.
resolveDesign	function	Both schemes at once: what the session carries.

Name	Kind	What it is
`resolveFriction`	function	Defaults ← dashboard ← app, each layer only changing what it names.
`resolveLoader`	function	
`resolveNativeAppearance`	function	
`resolvePlatform`	function	"auto" → the phone's own grammar.
`resolveSurface`	function	What the dashboard allows, and what opens first.
`resolveTokens`	function	The tokens for one scheme.
`resolveWidgetImages`	function	
`resolveWidgetLook`	function	
`resolveWidgetPalette`	function	
`rulesFor`	function	The rules that apply on one screen: the top level with that screen's overrides on it.
`selectCall`	function	The action to run and the arguments to run it with, for one tap.
`sessionDesign`	function	The design to draw for a session: the server's, read defensively, or the one its appearance implies when the server sent none.
`variablesForTicket`	function	
`voiceCallLabel`	function	The row for an earlier call: "Voice call · 6:05 PM".
`widgetSurface`	function	An unknown kind is a CARD, because what gets drawn for it is the agent's fallback sentence.

12.3 React Native — @graineai/inapp-react-native-calls 0.2.0

TypeScript, `@graineai/inapp-react-native-calls` : the native call engine of the iOS and Android SDKs, driven from JavaScript. It needs a native build (it is not available in Expo Go).

GraineCalls

`const GraineCalls` — The calls object. Every method is safe to call before configure; the engine is the app's one call engine.

Member	Returns	Errors	What it does
`get isAvailable(): boolean`	`boolean`	—	Whether the native module is linked into this build.
`configure(options: GraineCallsOptions): Promise<void>`	`Promise<void>`	—	Configures the call engine: the default agent, or several agents at once, or one agent by name.
`configure(options: GraineCallsAgentsOptions): Promise<void>`	`Promise<void>`	—	Configures the call engine: the default agent, or several agents at once, or one agent by name.
`configure(name: string, options: GraineCallsAgentOptions | GraineCallsOptions): Promise<void>`	`Promise<void>`	—	Configures the call engine: the default agent, or several agents at once, or one agent by name.
`get agents(): string[]`	`string[]`	—	The configured agents' names: "default" (and its own name, when it has one), then the others.
`enableVoIPPushes(onToken: (token: string | null) => void): () => void`	`() => void`	—	iOS: register for VoIP pushes.
`handlePush(data: Record<string, string>): Promise<boolean>`	`Promise<boolean>`	—	A call push's data, from your push handler.
`registerPushToken(push: { token: string; kind?: "fcm" | "voip"; environment?: "production" | "sandbox" }): Promise<boolean>`	`Promise<boolean>`	—	Android: FCM's token for this install, from your messaging service's onNewToken (and once at launch with getToken()): the engine keeps it and registers the device, so the calls Graine rings reach this phone.
`installId(): Promise<string | null>`	`Promise<string | null>`	—	This install's id — the one the native engine mints its calls with, which the in-app SDK uses too, so a call and a chat are one device.
`startCall(options: GraineStartCallOptions = {}): Promise<string | null>`	`Promise<string | null>`	—	The customer calls the agent — the default one, or agent by the name it was configured under.
`answer(): Promise<void>`	`Promise<void>`	—	Answer a ringing call from your own screen (the system ring answers by itself).

Member	Returns	Errors	What it does
`decline(): Promise<void>`	`Promise<void>`	—	Decline the ringing call.
`end(): Promise<void>`	`Promise<void>`	—	Hang up — or decline, while it rings.
`setMuted(muted: boolean): void`	`void`	—	Stops sending the customer's voice without ending the call.
`setSpeaker(on: boolean): void`	`void`	—	The loudspeaker (true) or the earpiece / a connected headset (false).
`attach(client: GraineAttachableClient, options: { agent?: string } = {}): () => void`	`() => void`	—	Hand the call what your app already tells the agent: who the customer is, your prompt variables, the screen they are on — kept in step as they change — and run the agent's in-app actions through the client's own handlers.
`onOutcome(fn: (o: GraineCallOutcome) => void): () => void`	`() => void`	—	The outcome of every call — tell your server (answered, declined, missed, ended, failed).
`onIncomingCall(fn: (c: GraineIncomingCall) => void): () => void`	`() => void`	—	A call started ringing — your own ringing screen, if you draw one.
`getSnapshot(): GraineCallSnapshot`	`GraineCallSnapshot`	—	The call as it stands now (GraineCallSnapshot).
`subscribe(fn: () => void): () => void`	`() => void`	—	Calls fn on every change to the snapshot; returns a function that unsubscribes.

useGraineCall

The call's snapshot as React state; redraws on every change.

Member	Returns	Errors	What it does
`function useGraineCall(): GraineCallSnapshot`	`GraineCallSnapshot`	—	

GraineCallsOptions

Exported by the calls package.

Field / prop	Type		Meaning
`publishableKey`	`string`	required	Agent → Embed & Widgets.
`baseUrl`	`string`	optional	Defaults to https://www.graine.ai
`displayName`	`string`	required	The name on the ring and the system call screen, for example "Acme assistant".
`ringtoneSound`	`string`	optional	iOS: a sound file of 30 s or less in the app bundle.
`ringTimeoutMs`	`number`	optional	How long a call rings before it is missed.
`appVersion`	`string`	optional	Which build placed the call; it lands on the call record.
`callScreen`	`"sdk" | "app"`	optional	Who draws the call while your app is open.
`debug`	`boolean`	optional	The native SDK's debug log, in Xcode's console or logcat: how a call connected and what its audio did, for finding out why one did not.
`android`	`{ smallIcon?: string; }`	optional	
`ios`	`{ iconTemplate?: string; includesCallsInRecents?: boolean; }`	optional	
`agents`	`Record<string, GraineCallsAgentOptions>`	optional	More agents, by name — a support agent and a sales agent, each with its own publishable key.

GraineCallsAgentOptions

One more agent: its own publishable key, and its host when it is not the default agent's.

Field / prop	Type		Meaning
publishableKey	string	required	
baseUrl	string	optional	

GraineCallsAgentsOptions

Agents only, without a default key: the FIRST agent listed is the default — the one startCall() uses when no agent is named, and the one a ring for an agent this app has not configured comes in as.

Field / prop	Type		Meaning
publishableKey	undefined	optional	
agents	Record<string, GraineCallsAgentOptions>	required	

GraineStartCallOptions

What startCall takes.

Field / prop	Type		Meaning
reason	string	optional	Why the customer is calling — the call_reason prompt variable.
variables	Record<string, unknown>	optional	
agent	string	optional	The configured agent to call (see configure).

GraineCallSnapshot

Exported by the calls package.

Field / prop	Type		Meaning
state	GraineCallState	required	
callId	string | null	required	Your server's id for a call it rang; a generated one for a call the customer placed.
direction	"incoming" | "outgoing" | null	required	
reason	string | null	required	
callerName	string | null	required	
muted	boolean	required	
speakerOn	boolean	required	
agentSpeaking	boolean	required	The agent's voice is playing right now.
caption	GraineCaption | null	required	
connectedAt	number | null	required	When the conversation opened, in ms since the epoch — for a call timer.
error	string | null	required	Why the last call could not connect, when it could not.
agent	string | null	required	Which configured agent the call is with — "default", or the name it was configured under (see configure).

GraineCallState

Exported by the calls package.

```
type GraineCallState = "idle" | "ringing" | "connecting" | "active" | "ended"
```

GraineCaption

Exported by the calls package.

Field / prop	Type		Meaning
role	`"agent" | "customer"`	required	
text	`string`	required	
live	`boolean`	required	Still being said.

GraineCallOutcome

What became of a call.

```
type GraineCallOutcome = ( | { callId: string; outcome: "ringing" } | { callId: string; outcome: "answered" } |
{ callId: string; outcome: "declined" } | { callId: string; outcome: "missed" } | { callId: string; outcome:
"ended"; reason: "local" | "remote" | "failed" } | { callId: string; outcome: "failed"; error: string } ) & {
durationSeconds?: number | null; hosted?: boolean; agent?: string | null }
```

GraineIncomingCall

Exported by the calls package.

Field / prop	Type		Meaning
callId	`string`	required	
reason	`string | null`	required	
callerName	`string | null`	required	
agent	`string | null`	optional	The configured agent that is calling, when the engine says (see GraineCallSnapshot.agent).

GraineActionResult

Exported by the calls package.

Field / prop	Type		Meaning
status	`"ok" | "refused" | "error"`	required	
message	`string`	optional	
data	`Record<string, unknown>`	optional	

GraineAttachableClient

What attach reads from the React Native SDK's client — structurally, so this package does not pin a version of it.

Field / prop	Type		Meaning
on	`method`	required	
getIdentity	`method`	required	
getRawTraits	`method`	optional	The traits exactly as identify() was given them.
memoryEnabled	`boolean`	optional	Whether the current customer's conversations may be remembered.
buildContextFrame	`method`	optional	The context frame the client would send right now — the screen, the customer, recent events and taps, masked.
getVariables	`method`	required	
getScreen	`method`	required	Any object: the in-app SDK's own ScreenContext is an interface, which a Record type would refuse.
executeAction	`method`	required	
getAvailableActions	`method`	optional	The actions this app registered handlers for.
setNativeCallLive	`method`	optional	A native call is live — ringing, connecting or connected — or is over.

Field / prop	Type		Meaning
`opts`	`{ readonly publishableKey?: string }`	optional	The client's options — read for its publishable key, so a client is matched to the configured agent with the same key.

DEFAULT_AGENT

The name configure(options) gives the agent it configures.

```
const DEFAULT_AGENT = "default";
```

EVENTS

Event names, shared with the native modules.

```
const EVENTS = {
```

12.4 iOS — GraineInApp 0.7.0

GraineInApp

The SDK's entry point: one instance per configured agent, made by `GraineInApp.configure(_:name:)` at launch and reached as `GraineInApp.shared` (the default agent) or `GraineInApp.named(_:)`. Its methods may be called from any thread; `voice`, `calls` and `chat` are main-thread objects, and listeners are called on the main thread.

Member	Returns	Errors	What it does
`@discardableResult static func configure(_ config: GraineConfig, name: String = "default") -> GraineInApp`	`GraineInApp`	—	Configures an agent. Call it once, at launch (`App.init` or `application(_:didFinishLaunchingWithOptions:)`). Without a name it is the default agent — the one `shared` returns and every view draws unless handed another; an app with several agents configures each under its own name and publishable key. The name is trimmed, and an empty name is the default. A second call with a name already configured returns that agent unchanged. A new agent takes on who is signed in, the memory choice and the current screen, and starts loading its settings from Graine.
`static func named(_ name: String) -> GraineInApp?`	`GraineInApp?`	—	The agent configured under `name` (trimmed), or nil.
`static var all: [String: GraineInApp] { get }`	`[String: GraineInApp]`	—	Every configured agent, keyed by name.
`static var current: GraineInApp? { get }`	`GraineInApp?`	—	The default agent — or, when every agent was given a name, the first one configured. Nil before any agent is configured.
`static var shared: GraineInApp { get }`	`GraineInApp`	—	The same agent as `current`. Stops the app with a message if no agent has been configured: call `configure` first.
`typealias ActionHandler = ([String: Any]) throws -> GraineActionResult`	—	—	The handler type for `registerAction(_:handler:)`: it receives the agent's arguments and returns the answer, and may throw.
`typealias DeferredActionHandler = ([String: Any], @escaping (GraineActionResult) -> Void) -> Void`	—	—	The handler type for `registerAction(_:deferred:)`: it receives the arguments and a `done` callback to answer with later.

Member	Returns	Errors	What it does
`let config: GraineConfig`	`GraineConfig`	—	The configuration this agent was made with.
`let name: String`	`String`	—	The name this agent was configured under; `"default"` for the default agent.
`var session: GraineSessionConfig? { get }`	`GraineSessionConfig?`	—	The agent's settings from the dashboard, once loaded (once per launch); nil before. `onSessionReady` reports it.
`var design: GraineDesign? { get }`	`GraineDesign?`	—	The brand's design from the session: light and dark tokens and which to use. Nil before the session loads, and from a server that sends none; the SDK's views then draw from the dashboard's appearance settings.
`var theme: GraineThemeTokens? { get }`	`GraineThemeTokens?`	—	The design tokens to draw right now: the brand's fixed scheme, else the device's current light or dark, picked again whenever the device switches. Nil without a design. `onTheme` reports each change.
`private(set) lazy var voice: GraineVoice`	`GraineVoice`	—	The in-app voice call that `GraineAgentBar` uses. Attach it to your window before starting a call. Main thread only.
`var calls: GraineCalls { get }`	`GraineCalls`	—	Native calls on CallKit and VoIP pushes: the customer calling your agent, or your agent ringing the customer. One object for the whole app — every agent's `calls` is the same object, and `startCall(reason:variables:agent:)` picks the agent. Main thread only.
`private(set) lazy var chat: GraineChat`	`GraineChat`	—	The typed conversation with this agent. `GraineChatView` and the bar use it themselves. Main thread.
`func addListener(_ listener: GraineListener)`	nothing	—	Hear everything this agent reports. Held weakly — keep your own reference. Adding the same listener twice has no effect. Callbacks arrive on the main thread.
`func removeListener(_ listener: GraineListener)`	nothing	—	Stops reporting to that listener.
`func identify(_ traits: [String: Any?]?, memory: Bool? = nil)`	nothing	—	Who is signed in, for every configured agent. The trait the agent's Identity setting names (`user_id` unless the dashboard says `email`, `phone` or a custom trait) identifies the customer for memory and for the calls Graine rings: its value travels only with the requests that start a conversation, never as a prompt variable. Every other trait becomes a prompt variable, masked on the device; nested objects and arrays are dropped. A placeholder value (`"guest"`, `"null"`, a masked `"[EMAIL]"`) is no identity, and those conversations are anonymous. `memory: false` means this customer's conversations are neither remembered nor recalled; nil keeps their last choice (true until they make one). The identity is fixed per conversation: a change applies to the next one. `identify(nil)` signs out: the device stops being that customer's, Graine is told, and later conversations are anonymous. When a different customer signs in, or the customer signs out, the chat transcript on screen is cleared.
`func setMemory(_ enabled: Bool)`	nothing	—	Whether the current customer's conversations are remembered, for every agent — the same as calling `identify` again with `memory: enabled`. Kept on the device for that customer; applies to the next conversation.

Member	Returns	Errors	What it does
`var memoryChoice: Bool { get }`	`Bool`	—	The current customer's memory choice: true until they turn it off. When false, nothing is recalled or stored.
`var installId: String { get }`	`String`	—	This install's id: a UUID made on first launch and kept in the Keychain (this device only), surviving reinstalls. The same for every agent. Graine counts a customer's devices by it; the agent never sees it. If the Keychain cannot keep it, that launch uses a new one.
`@discardableResult func forgetMemory() async throws -> Int`	`Int`	`GraineServiceError`: identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	Deletes everything this agent's memory holds about the signed-in customer. Waits for the session first. Throws `identity_required` before sending anything when nobody is identified, and `device_limit` from a device that is not one of the customer's. Returns how many memories were deleted (0 when Graine does not say). The kept conversation goes too, and the chat on screen clears — only when it succeeds; a refusal leaves both as they were.
`func memories() async throws -> [GraineMemory]`	`[GraineMemory]`	`GraineServiceError`: identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	What the agent remembers about the signed-in customer, newest first, at most 100, already redacted — to show people what is kept about them. Waits for the session first, and throws `identity_required` before sending anything when nobody is identified.
`func setVariables(_ values: [String: Any?]?)`	nothing	—	Prompt variables from the app's own data, while it runs. Merged, not replaced: nil under a key removes it, nil for the whole map clears them. Values become strings (nested objects and arrays are dropped) and are masked. They reach the next call; an engine page already loaded is updated too.
`func currentVariables() -> [String: String]`	`[String: String]`	—	Every variable the agent will receive, merged in precedence order: the dashboard's defaults, then `GraineConfig.variables`, then `setVariables`, then the traits from `identify` (without the identifying trait).
`func setScreen(_ next: GraineScreen?)`	nothing	—	What the customer is looking at; call it on every meaningful change. Shared by every agent. An unchanged screen is not resent, and bursts are coalesced to one frame per 400 ms — the first goes at once. nil means no screen.
`func updateField(_ field: GraineField)`	nothing	—	Reports one changed field, merged by name into the current screen (added when new), so a form need not describe itself again on every keystroke. Does nothing without a current screen, or for a field with an empty name.
`func removeField(_ name: String)`	nothing	—	Removes a field from the current screen. Nothing happens if it was never reported.
`func clearScreen(_ screenId: String?)`	nothing	—	Forgets the screen only if it is still the one with that id — safe in a screen's teardown, which can run after the next screen has reported itself. nil clears whatever screen is set.
`func track(_ name: String, data: [String: Any] = [:])`	nothing	—	A product event that informs the agent without interrupting. It rides the next context frame; the last 10 are kept. Every agent hears it. An empty name is ignored.
`func tap(_ name: String, data: [String: Any] = [:])`	nothing	—	Records a tap on a named control as `tap: <name>` (the name trimmed, inner spaces collapsed, cut to 60 characters; an empty name is ignored). Taps are context, not news: they ride the next frame, at the end of its 400 ms window. Every agent hears it.

Member	Returns	Errors	What it does
`func noteInteraction()`	nothing	—	The customer did something that is not a field edit — scrolled a list, opened a picker, sent a message — so the idle clock that detects a stuck customer restarts. Field edits do not count.
`func configureFriction(_ override: GraineFrictionConfig?)`	nothing	—	Replaces the app's layer of friction rules, applied over the dashboard's; nil removes it. Safe at any time.
`@discardableResult func reopenComponent() -> Bool`	`Bool`	—	Puts back the last component the customer closed without answering, exactly as it was — for a "show that again" control. Returns false when there is none.
`@discardableResult func registerAction(_ name: String, handler: @escaping ActionHandler) -> () -> Void`	`() -> Void` — unregisters	—	Something the agent can do in your app; declare it on the agent in the dashboard too — a handler with no declaration is never offered, and a declaration with no handler is refused. The handler runs on the main thread and must answer within six seconds, or the agent is told it was not done; a handler that throws answers `error` with the error's description. Registering the same name again replaces the handler. The returned function unregisters it, if it is still this registration.
`@discardableResult func registerAction(_ name: String, deferred handler: @escaping DeferredActionHandler) -> () -> Void`	`() -> Void` — unregisters	—	The same, for work that finishes later — an upload, a network call. Call `done` exactly once, from any thread; later calls are ignored. After six seconds the agent is told the app did not answer, so slower work should answer at once and report its outcome later with `setScreen` or `track`.
`func checkActions() -> GraineActionCheck`	`GraineActionCheck`	—	Which registered actions the agent can call, and which it cannot, compared with the actions declared on the agent. Log it once at startup.
`var schedulingRules: GraineSchedulingRules { get }`	`GraineSchedulingRules`	—	The rules for booking calls, from the session — or, before it loads, from the last session kept on the device; the defaults (no booking) when there is neither. `scheduleCall` checks them first.
`func scheduleCall(at date: Date, reason: String? = nil) async throws -> GraineScheduledCall`	`GraineScheduledCall`	`GraineServiceError`: scheduling_off, too_soon, too_far (on the device); outside_window, too_many_pending, identity_required, device_limit, invalid_request, rate_limited, network, unavailable, invalid_key, origin_refused (from Graine); unknown when the booking was accepted but its answer could not be read	Asks the agent to call this customer at `date`; Graine rings this device then, on the system's call screen. Waits for the session, then checks on the device before sending anything: `scheduling_off` when the agent does not let customers book, `too_soon` before `schedulingRules.earliest(now:)`, `too_far` after `latest(now:)`. `reason` (trimmed, cut to 120 characters) is what the agent is told the call is for; it never appears in the push. The time is sent with the device's offset and time zone; the booking is the identified customer's, or this install's when nobody is identified. Returns the booked call (status `scheduled`) and refreshes the chat's upcoming calls.
`func scheduledCalls() async throws -> [GraineScheduledCall]`	`[GraineScheduledCall]`	`GraineServiceError`: network, rate_limited, unavailable, invalid_key, origin_refused	This customer's calls still to come — this install's when nobody is identified — upcoming first. Waits for the session first.
`@discardableResult func cancelScheduledCall(id: String) async throws -> GraineCallStatus`	`GraineCallStatus`	`GraineServiceError`: not_found, network, rate_limited, unavailable, invalid_key, origin_refused	Calls off one of this customer's booked calls (this install's when nobody is identified); `not_found` for anything else. Returns the call's status afterwards — `cancelled`, also when Graine's answer names none.

GraineConfig

Everything the SDK needs up front, passed to `GraineInApp.configure(_:name:)` and kept as `GraineInApp.config`. A value type; only `baseUrl` and `publishableKey` are required.

Field / case	Type	Default	Meaning
`var baseUrl: String`	`String`	required	Graine's address: `https://www.graine.ai` unless self-hosted.
`var publishableKey: String`	`String`	required	From Agent → Embed & Widgets; selects the agent. It is not a secret and cannot place a call on its own: add `app://` to that agent's Allowed domains, or every request is refused.
`var variables: [String: Any]`	`[String: Any]`	`[:]`	Prompt variables known at build time. `identify` and `setVariables` win over these.
`var transport: GraineTransport?`	`GraineTransport?`	`nil`	Forces how the in-app call's audio travels. nil lets the dashboard's Call audio setting decide.
`var appVersion: String?`	`String?`	`nil`	Lands on the call record, to tell builds apart.
`var endOnBackground: Bool`	`Bool`	`true`	Ends a live in-app call when the app goes to the background (after two seconds, if it is still there). The chat also hangs up in the background and comes back with the app.
`var timing: Timing`	`GraineConfig.Timing`	`Timing()`	Load timing for the engine page, for very slow networks.
`var debug: Bool`	`Bool`	`false`	Logs every message between the SDK and its engine page, both ways, and the page's own console, under the subsystem `ai.graine.inapp`. Variables are logged as keys only and screen context as the screen's name only. The engine page can also be inspected in Safari (iOS 16.4 and later). Turn it on in debug builds.
`var friction: GraineFrictionConfig?`	`GraineFrictionConfig?`	`nil`	The app's friction rules, applied over the dashboard's — for screens only the app knows about.
`var fonts: [String: String]`	`[String: String]`	`[:]`	Fonts the app ships itself. The SDK downloads the design's typeface from Graine; a family named here, or one the app bundles under the family's own name, is used instead and never downloaded. Map a family to the name your app registered it under when the two differ: `["Inter": "InterVariable"]`.
`var captureTaps: Bool`	`Bool`	`true`	Tells the agent about taps: a tap on a control with an accessibility identifier — or a button's accessibility label — reaches it as `tap:<name>`, any other tap as `tap:screen` with where it landed. At most two a second, only while a screen is set, and never a field's value. The recogniser never takes a touch from the app.

Field / case	Type	Default	Meaning
`var chatSize: GraineChatSize?`	`GraineChatSize?`	`nil`	How big the chat window opens, over the dashboard's *Window size*: `.compact` above the bar, `.large` a sheet over most of a phone (a panel on an iPad), `.full` the whole screen. Nil follows the dashboard; a size passed to `chat.open(size:)` wins for that open.
`init(baseUrl: String, publishableKey: String, variables: [String: Any] = [:], transport: GraineTransport? = nil, appVersion: String? = nil, endOnBackground: Bool = true, timing: Timing = Timing(), debug: Bool = false, friction: GraineFrictionConfig? = nil, fonts: [String: String] = [:], captureTaps: Bool = true, chatSize: GraineChatSize? = nil)`	initialiser	—	Builds a configuration.

GraineConfig.Timing

Load timing for the engine page that carries the in-app call, in milliseconds, set as `GraineConfig.timing`. Most apps keep the defaults.

Field / case	Type	Default	Meaning
`var stallMs: Int`	`Int`	`15_000`	Silence — nothing downloaded — before a load is given up on.
`var bootMs: Int`	`Int`	`15_000`	Downloaded but still not ready after this long: the page is broken, not slow.
`var hardCapMs: Int`	`Int`	`120_000`	Ceiling on one load attempt, however steadily it is arriving.
`var retryDelaysMs: [Int]`	`[Int]`	`[1_000, 2_000, 4_000, 8_000, 15_000, 15_000]`	The waits between reloads; its length is the number of reloads before `connectivity` becomes `failed`.
`var endWatchdogMs: Int`	`Int`	`4_000`	How long `GraineVoice.end()` waits for the page to confirm before ending the call locally.
`var slowConnectMs: Int`	`Int`	`4_000`	A connect the customer has waited on this long sets `GraineVoice.slowConnect` ("Still connecting…").
`init(stallMs: Int = 15_000, bootMs: Int = 15_000, hardCapMs: Int = 120_000, retryDelaysMs: [Int] = [1_000, 2_000, 4_000, 8_000, 15_000, 15_000], endWatchdogMs: Int = 4_000, slowConnectMs: Int = 4_000)`	initialiser	—	Builds a timing.

GraineListener

Everything an agent reports, added with `GraineInApp.addListener(_:)`; every method has a default that does nothing, so implement only what you use. Called on the main thread; listeners are held weakly, so keep your own reference.

Member	Returns	Errors	What it does
`func onSessionReady(_ session: GraineSessionConfig)`	nothing	—	The agent's settings loaded (once per launch). `GraineInApp.theme` is already up to date when this is called.
`func onCallState(connected: Bool, connecting: Bool)`	nothing	—	The in-app call connected, started connecting, or stopped. Called only when either value changes.
`func onCaption(role: GraineRole, text: String, live: Bool)`	nothing	—	A line being said on the in-app call, again as it fills in; `live` is true while it is still being recognised or spoken.
`func onWidget(_ widget: [String: Any])`	nothing	—	The agent drew a component, in a call or in the chat (also when `reopenComponent()` puts one back). Mount `GraineWidgetView` to draw it, or answer it yourself with `GraineVoice.answerWidget(widgetId:data:summary:)`; `GraineWidget(frame:)` reads the map.
`func onWidgetPending(_ hint: [String: Any])`	nothing	—	A component is on its way and its content is loading: draw your loader.

Member	Returns	Errors	What it does
`func onEnded()`	nothing	—	The in-app call is over, however it ended. Once per call.
`func onMicDenied(reason: String)`	nothing	—	The microphone was refused — `NotAllowedError` when the permission is denied. Also called when a native call (`calls`) could not get the microphone.
`func onQuality(_ quality: GraineConnectionQuality)`	nothing	—	The connection's quality, in one word. Called only when the word changes.
`func onNetwork(online: Bool)`	nothing	—	The engine page saw the network go or come back.
`func onReconnecting(active: Bool)`	nothing	—	The network interrupted the call and it is being re-established (true), or that is over (false). Not an end.
`func onTransport(_ transport: GraineTransport, requested: GraineTransport, reason: String?, message: String?)`	nothing	—	What carries the call's audio, and why it is not the transport asked for. A `reason` of `not_configured` is a setting that never fixes itself, so it is also reported through `onError`.
`func onConnectivity(_ state: GraineConnectivity, attempt: Int, reason: String?)`	nothing	—	The engine page loading, ready, retrying (with the reload attempt and why) or failed.
`func onActionRequested(name: String)`	nothing	—	The agent asked for one of your actions.
`func onActionCompleted(name: String, result: GraineActionResult)`	nothing	—	The answer the agent received for that action — your handler's, or the SDK's `refused` or `error` for an action with no handler, a throw or a timeout.
`func onFriction(_ signal: GraineFrictionSignal)`	nothing	—	The customer looks stuck: on one screen, with a field outstanding, through a stretch of a live conversation. The agent is told right after; this is for the app — badge a control, count it.
`func onTheme(_ tokens: GraineThemeTokens?)`	nothing	—	The brand's design tokens for the device's current appearance — when the session arrives, and again when the device switches between light and dark. Never called when the session carries no design.
`func onChatMessage(_ turn: GraineChatTurn)`	nothing	—	The agent finished a message in the chat. `turn.proactive` is true when it spoke up without being asked — open or badge your chat for those if the customer is not looking at it.
`func onError(_ message: String)`	nothing	—	Something your team must fix — a rejected key, an agent that does not accept `app://` , an engine page that could not load. Meant for you, never shown to the customer.
`extension GraineListener`	—	—	An empty default implementation of every method above.

GraineActionCheck

Which registered actions the agent can call and which it cannot, returned by `GraineInApp.checkActions()`. Log it once at startup.

Field / case	Type	Default	Meaning
`let usable: [String]`	`[String]`	—	Registered here and declared on the agent: these work.
`let undeclared: [String]`	`[String]`	—	Registered here, not declared on the agent: never offered.
`let unhandled: [String]`	`[String]`	—	Declared on the agent, not registered here: refused mid-sentence. Sorted.

GraineChat

The typed conversation with an agent, on its own connection: `GraineInApp.shared.chat`, an `ObservableObject` to read on the main thread (its methods may be called from any thread and hop there). One conversation runs at a time across every agent: opening the chat ends any in-app call, starting a call closes it, a native call holds it down until it is over, and with `endOnBackground` it hangs up in the background and comes back with the app.

Member	Returns	Errors	What it does
`@Published private(set) var turns: [GraineChatTurn] = []`	`[GraineChatTurn]`	—	What has been said, oldest first: the customer's messages and the agent's replies, one bubble each (the last 200). Kept while the app runs.
`@Published private(set) var status: GraineChatStatus = .idle`	`GraineChatStatus`	—	Where the connection is.

Member	Returns	Errors	What it does
`@Published private(set) var typing = false`	`Bool`	—	The agent is working on a reply: from the customer's message until its first words, or 30 seconds.
`@Published private(set) var upcomingCalls: [GraineScheduledCall] = []`	`[GraineScheduledCall]`	—	This customer's calls still to come, soonest first, at most three — the chat window's "Upcoming calls" card. Fetched when the chat opens and after the agent finishes a reply (at most every 10 seconds), only where Graine rings this app.
`var online: Bool { get }`	`Bool`	—	Connected: a message goes now.
`var awaitingFirstMessage: Bool { get }`	`Bool`	—	The customer has said nothing yet in this conversation, and it is not one that continues (no turn was restored): the brand's starters are offered.
`private(set) var isOpen = false`	`Bool`	—	The customer is in the chat: `open()` was called and `close()` was not.
`@Published private(set) var openedSize: GraineChatSize?`	`GraineChatSize?`	—	The size the current open asked for (`open(size:)`), until `close()`; nil follows `GraineConfig.chatSize`, then the dashboard's. Moving to the call and back keeps it.
`func open(size: GraineChatSize? = nil)`	nothing	—	The customer is in the chat: connect, and stay connected. A mounted `GraineAgentBar` moves to the chat (where the agent offers one) and draws it at `size` until `close()`; nil follows `GraineConfig.chatSize`, then the dashboard's. Ends any in-app call and closes other agents' chats. A dropped connection is made again 1, 2, 4, 8, 15 and 15 seconds apart, then the status is `offline` until the customer sends something or you call `retry()`.
`func close()`	nothing	—	The customer left: the chat hangs up and stays down. What was said stays on screen.
`func send(_ text: String)`	nothing	—	Sends a message. It is shown at once and sent now, or as soon as the chat is connected (opening and connecting it if needed). Empty text is ignored. Up to 20 messages wait while the chat is down; older ones are marked failed.
`func retry()`	nothing	—	Connects now, and sends what could not be sent.
`func newConversation() async throws`	nothing	`GraineServiceError`: device_limit, network, …	Starts a fresh thread: the chat hangs up, Graine lets go of the conversation it kept for this agent and customer, the chat on screen clears and — when the customer was in the chat — a fresh chat opens with the welcome and the starters. Memories are untouched. With nobody identified only the screen clears. On a refusal the chat is as it was, connected again. `GraineChatView`'s header offers it while it shows an earlier conversation.
`func refreshUpcomingCalls()`	nothing	—	Fetches the upcoming calls again — after booking one elsewhere, say.
`func cancelUpcomingCall(id: String)`	nothing	—	Cancels one of the card's calls. Its row says "Cancelling…" until Graine has it, then goes; a failure leaves the row marked so the customer can try again. Nothing is thrown.

GraineChatSize

`public enum GraineChatSize: String, CaseIterable, Sendable` — how big the chat window opens. The dashboard's *Window size* chooses; `GraineConfig.chatSize` overrides it for the app, and `chat.open(size:)` for one open. Raw values are the design's words.

Field / case	Type	Default	Meaning
`case compact`	case	—	The window above the bar, as it has always been.
`case large`	case	—	A sheet over most of a phone; a panel in the corner of an iPad.
`case full`	case	—	The whole screen.

GraineChatTurn

One message in `GraineChat.turns`, made by the SDK. Each agent reply is its own turn.

Field / case	Type	Default	Meaning
`let id: String`	`String`	—	Stable for the life of the transcript: `t1`, `t2`, …
`let role: GraineRole`	`GraineRole`	—	`.agent` or `.customer`.
`internal(set) var text: String`	`String`	—	What was said; an agent's words are appended as they stream in.
`internal(set) var live: Bool`	`Bool`	—	The agent's words are still arriving: draw them as provisional.
`internal(set) var delivery: Delivery`	`GraineChatTurn.Delivery`	—	Where a customer's message is. An agent's turn is always `sent`.
`let at: Date`	`Date`	—	When it was said — for an agent's turn, its first words.
`let proactive: Bool`	`Bool`	—	The agent said this without being asked: its turn did not follow one of the customer's.
`let restored: Bool`	`Bool`	`false`	From an earlier conversation, restored when the chat opened (the customer came back): delivered, with the time it was said and Graine's id, not said just now.
`let channel: GraineChatChannel`	`GraineChatChannel`	`.chat`	Where it was said. A chat window draws a run of `.voice` turns as one "Voice call" row, never their words.
`enum Delivery: String, Sendable`	nested type	—	See `GraineChatTurn.Delivery`.

GraineChatChannel

`public enum GraineChatChannel: String, Sendable` — where a message was said: typed in the chat, or spoken on a call (only an earlier conversation's turn is).

Field / case	Type	Default	Meaning
`case chat`	case	—	Typed in the chat.
`case voice`	case	—	Said on a call with the agent.

GraineChatTurn.Delivery

Where a customer's chat message is. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case sending`	case	—	Waiting for the connection: typed while the chat was not connected, sent when it is.
`case sent`	case	—	On its way to the agent.
`case failed`	case	—	The chat could not reconnect to send it. `GraineChat.retry()` tries again.

GraineChatStatus

Where the chat's connection is: `GraineChat.status`.

Field / case	Type	Default	Meaning
`case idle`	case	—	Not in the chat: never opened, left, or the app is in the background.
`case connecting`	case	—	Opening.
`case online`	case	—	Connected.
`case reconnecting(attempt: Int)`	case	—	The connection dropped and is being made again; `attempt` of six.
`case offline`	case	—	It could not reconnect, or the conversation was ended by the other side. A message, or `retry()`, connects again.
`var isConnecting: Bool { get }`	`Bool`	—	Connecting or reconnecting: draw "Connecting…".

GraineSurface

Which conversation: a call or a chat, never both at once. Passed to `GraineAgentBar(surface:)`.

Field / case	Type	Default	Meaning
`case voice`	case	—	The call.
`case chat`	case	—	The chat.

GrainеChatView

The chat as a SwiftUI screen, drawn by the SDK in the brand's design: header, one bubble per reply, a typing bubble, the components the agent draws (answered in place) and the composer. It connects when it appears, hangs up when it goes, and draws nothing before an agent is configured.

Member	Returns	Errors	What it does
`init(graine: GraineInApp? = nil, accent: Color? = nil, onClose: (() -> Void)? = nil)`	`GraineChatView`	—	`graine`: the agent to chat with (default: the default agent), for example `GraineChatView(graine: .named("support"))`. `accent`: overrides the design's accent. `onClose`: the header's close control; without one it is not drawn, and the navigation you put the view in closes it.
`var body: some View`	`some View`	—	SwiftUI's view body; apps do not call it.

GrainеChatViewController

The chat screen for a UIKit app, to present or push: `present(GraineChatViewController(), animated: true)`. Presented, its close control dismisses it; pushed, the navigation bar's back button does and no close control is drawn.

Member	Returns	Errors	What it does
`init(graine: GraineInApp? = nil)`	`GraineChatViewController`	—	The agent to chat with; defaults to the default agent. Presents as a page sheet.
`override func viewDidLoad()`	nothing	—	UIKit's lifecycle callback, which builds the chat screen. Apps do not call it.

GrainеAgentBar

What the customer sees during a conversation, drawn by the SDK in the brand's design: avatar, name and a status line that names the network's state ("Weak connection", "No connection", "Reconnecting…"), microphone, hang-up, the line being said, the components the agent draws, and the chat — with a switch when the dashboard offers both. A SwiftUI view: mount it once at your root as an overlay, not inside a screen; it draws only the bar, lets other touches through, can be dragged up and down, and rises above the keyboard.

Member	Returns	Errors	What it does
`init(graine: GraineInApp? = nil, name: String? = nil, accent: Color? = nil, bottomInset: CGFloat = 16, draggable: Bool = true, initialOffset: CGFloat = 0, onPlacementChange: ((CGFloat) -> Void)? = nil, autoAttach: Bool = true, surface: GraineSurface? = nil, onClose: (() -> Void)? = nil)`	`GraineAgentBar<EmptyView>`	—	`graine`: the agent (default: the default agent; nothing draws before one is configured). `name`: the name beside the status and what VoiceOver calls the agent (default: the dashboard's heading). `accent`: overrides the design's and the dashboard's accent. `bottomInset`: points above the bottom of the area — lift it clear of your tab bar. `draggable`: let the customer move it up and down. `initialOffset`: a position you saved, in points above the anchor. `onPlacementChange`: the customer let go somewhere new — save it here; the SDK stores nothing. `autoAttach`: mount the voice engine in the key window when the bar appears, if nothing has yet. `surface`: which conversation opens, `.voice` or `.chat`, among those the dashboard offers — it can never add one the dashboard withheld; nil is the dashboard's choice. `onClose`: the customer hung up, or closed the bar with no call to end; called 400 ms later, after the goodbye has left the speaker — take the bar away here if your app shows it on demand.
`func accessory<Content: View>(@ViewBuilder _ content: () -> Content) -> GraineAgentBar<Content>`	`GraineAgentBar<Content>`	—	Something that rides above the bar and moves with it, above the keyboard included — a text field, say.
`var body: some View`	`some View`	—	SwiftUI's view body; apps do not call it.

GrainеWidgetView

The component the agent put on screen — product lists, plan comparisons, info cards, choices and quick actions, calendar slots, or the agent's fallback sentence for any other kind — drawn by the SDK in the dashboard's preset and

answered for you. A SwiftUI view: mount it once, above your bar; it shows inline only when there is something to show (never as a full-screen cover, which would take the voice engine out of the window), and while it is mounted the bar's own component slot steps aside.

Member	Returns	Errors	What it does
`init(graine: GraineInApp? = nil, accent: Color? = nil)`	`GraineWidgetView`	—	`graine`: the agent (default: the default agent). `accent`: overrides the component's and the dashboard's accent.
`var body: some View`	`some View`	—	SwiftUI's view body; apps do not call it.

GraineLauncher

The floating launcher button in the brand's design (56 points high; a pill is as wide as its label): a tap places the in-app call, mounting the engine first if needed, and during the call it is the end button. A SwiftUI view that draws only the button — place it yourself, for example `.overlay(alignment: .bottomTrailing) { GraineLauncher().padding(20) }`.

Member	Returns	Errors	What it does
`init(graine: GraineInApp? = nil, label: String? = nil, accent: Color? = nil, autoAttach: Bool = true, action: (() -> Void)? = nil)`	`GraineLauncher`	—	`graine`: the agent (default: the default agent; nothing draws before one is configured). `label`: a pill's words and the spoken label (default: the dashboard's heading). `accent`: overrides the design's and the dashboard's accent. `autoAttach`: mount the engine in the key window when the button appears, so a tap is only a call away. `action`: what a tap does instead of placing the call — the button then only draws and is never the end button.
`var body: some View`	`some View`	—	SwiftUI's view body; apps do not call it.

GraineLoader

How waiting looks, drawn from the dashboard's Appearance → loader: dots, pulse, ring, bars or the brand's logo breathing, at the configured size, speed and colour, with the dashboard's words for the moment. A SwiftUI view; nothing animates while it is off screen.

Member	Returns	Errors	What it does
`init(moment: String? = nil, label: String? = nil, size: CGFloat? = nil, stacked: Bool = false, graine: GraineInApp? = nil)`	`GraineLoader`	—	`moment`: `"connecting"`, `"reconnecting"`, `"thinking"` or `"component"` — which of the dashboard's words to show; omit for no label. `label`: overrides those words. `size`: overrides the dashboard's size, in points. `stacked`: the words under the mark instead of beside it. `graine`: the agent (default: the default agent). Pair `moment: "thinking"` with `GraineVoice.phase == .thinking`.
`var body: some View`	`some View`	—	SwiftUI's view body; apps do not call it.

GraineLoaderText

The words for each waiting moment, as carried in `GraineLoaderLook.text`. A value type.

Field / case	Type	Default	Meaning
`var connecting: String?`	`String?`	`nil`	While the page and the call are being set up.
`var reconnecting: String?`	`String?`	`nil`	While the page is being reloaded after a failure, or the call re-established.
`var thinking: String?`	`String?`	`nil`	Between the customer finishing and the agent starting.
`var component: String?`	`String?`	`nil`	While a component's content is loading.
`init(connecting: String? = nil, reconnecting: String? = nil, thinking: String? = nil, component: String? = nil)`	initialiser	—	Builds a set of words.
`func forMoment(_ moment: String) -> String?`	`String?`	—	The words for `"connecting"`, `"reconnecting"`, `"thinking"` or `"component"`; nil for anything else.

GraineLoaderLook

The loader in effect, as values a renderer can use. Used by the SDK's own views; apps do not need it.

Field / case	Type	Default	Meaning
`let kind: String`	`String`	`"dots"`	`"dots"`, `"pulse"`, `"ring"`, `"bars"` or `"mark"` (the brand's logo, breathing).
`let color: String?`	`String?`	`nil`	#RRGGBB, or nil for the accent.
`let px: Double`	`Double`	`20`	The mark's size in points: 14, 20 or 28 for small, medium and large.
`let periodMs: Double`	`Double`	`1_100`	One animation cycle: 1,600, 1,100 or 700 ms for calm, normal and quick.
`let text: GraineLoaderText`	`GraineLoaderText`	"Connecting…", "Reconnecting…", "Thinking…", "Loading…"	Every moment's words filled in; the dashboard's own words replace these (at most 40 characters each).
`let skeleton: Bool`	`Bool`	`false`	Also draw the shape of the coming component under the loader.

View.graineTap

A SwiftUI modifier that names a control for the agent, since SwiftUI views carry no accessibility identifier for the automatic tap capture to read. Available on every `View`.

Member	Returns	Errors	What it does
`func graineTap(_ name: String, graine: GraineInApp? = nil) -> some View`	`some View`	—	A tap on the view reaches the agent's context as `tap:<name>` ("you tapped Pay twice"), through `GraineInApp.tap(_:data:)`, without taking the tap from the control. `graine` defaults to the default agent.

GraineVoice

The in-app voice call: a page Graine hosts, in a web view the SDK owns (one invisible point big), so audio fixes reach your app without a release — reached as `GraineInApp.shared.voice` and driven for you by `GraineAgentBar`. An `ObservableObject`; every method belongs on the main thread, and one called from elsewhere hops there.

Member	Returns	Errors	What it does
`@Published private(set) var connected = false`	`Bool`	—	A call is up.
`@Published private(set) var connecting = false`	`Bool`	—	A call is being placed or re-established.
`@Published private(set) var muted = false`	`Bool`	—	The customer's voice is not being sent.
`@Published private(set) var ready = false`	`Bool`	—	The engine page is loaded and ready: a start now connects within moments.
`@Published private(set) var quality: GraineConnectionQuality = .unknown`	`GraineConnectionQuality`	—	The connection in one word.
`@Published private(set) var online: Bool? = nil`	`Bool?`	—	The engine page's view of the network; nil until it has said.
`@Published private(set) var reconnecting = false`	`Bool`	—	The network interrupted the call and it is being re-established. Not an end.
`@Published private(set) var connectivity: GraineConnectivity = .loading`	`GraineConnectivity`	—	Whether the engine page is there: loading, ready, retrying or failed.
`@Published private(set) var captions: [GraineCaption] = []`	`[GraineCaption]`	—	What has been said this call, oldest first. A live line is replaced as its words arrive, not appended; at most 12 are kept; cleared when a call starts.
`@Published private(set) var connectStage: GraineConnectStage? = nil`	`GraineConnectStage?`	—	Where a call being placed is: registering, dialling, retrying or switching. Nil outside a connect.
`@Published private(set) var slowConnect = false`	`Bool`	—	The customer has waited on this connect for `Timing.slowConnectMs` (4 s by default) — a bar open with the page not there yet, or a call they asked for not yet answered. Say so ("Still connecting…"). Cleared the moment it lands or they stop waiting.

Member	Returns	Errors	What it does
`@Published private(set) var callFailed = false`	`Bool`	—	The last start could not be connected on either transport. Not a setting to fix — the network may be better in a moment, so `start()` simply tries again. Cleared by a start, an answered call, `end()` and `detach()`.
`var isAttached: Bool { get }`	`Bool`	—	Whether the engine is mounted.
`var phase: GrainePhase { get }`	`GrainePhase`	—	What the call is doing in one word — idle, connecting, reconnecting, thinking, speaking or listening — for an app that draws its own bar. Draw `GraineLoader(moment: "thinking")` for `thinking`.
`func attach(to view: UIView)`	nothing	—	Mounts the engine in that view — pass your key window, not a view a screen owns — and loads the page idle: nothing registered, nothing dialled, no microphone, so a later `start()` connects quickly. Attaching somewhere new moves the engine and ends any call in progress.
`func attach()`	nothing	—	Mounts the engine in the app's key window. Call it once a window exists — from a view's `onAppear`, say; with no window yet it reports through `onError` and does nothing.
`func detach()`	nothing	—	Removes the engine, hanging up first: a call never outlives its UI.
`func start()`	nothing	—	Places the call. Asks for the microphone first if the app does not hold it; refused, no call is placed and `onMicDenied` is called. A start before the page is ready is held and placed when it is. Ends any other conversation — another agent's call, every chat. Refused through `onError` when the engine is not attached or a native call (`calls`) is live.
`func end()`	nothing	—	Hangs up. If the page does not confirm within `Timing.endWatchdogMs` (4 s), the call is ended locally — state, `onEnded`, and a reload that drops the audio. A start that was only being held is simply cancelled.
`func setMuted(_ value: Bool)`	nothing	—	Stops (or resumes) sending the customer's voice without ending the call.
`func answerWidget(widgetId: String, data: [String: Any], summary: String)`	nothing	—	The customer answered a component the agent drew. `summary` is the sentence the agent hears ("Let's compare the two plans."); without it the tap lands and the conversation does not move. `GraineWidgetView` does this itself.
`func reportEvent(_ name: String, detail: String? = nil, data: [String: Any] = [:])`	nothing	—	Something the agent may want to speak up about. Only during a call — otherwise it is dropped, so it never opens the next conversation. The current screen is sent first. `data` may carry `quiet`, `cooldown_seconds` and `max_per_session`, which pace the agent.
`func retry()`	nothing	—	Reloads the engine page and starts the reload ladder over — for a "Try again" button.

GraineCalls

Native calls like WhatsApp, on CallKit and VoIP pushes — the customer calls your agent, or your agent rings the customer — with the audio in your app's own process, so a call carries on from the lock screen and with the screen off. Reached as `GraineInApp.shared.calls` (one object for the whole app, one call at a time); an `ObservableObject` on the main thread whose methods hop there when called elsewhere.

Member	Returns	Errors	What it does
`@Published private(set) var state: GraineCallState = .idle`	`GraineCallState`	—	Where the call is. `ended` lasts one second before `idle`.
`@Published private(set) var callId: String?`	`String?`	—	The current (or just-ended) call's id: the push's `call_id`, or the one `startCall` made.
`@Published private(set) var direction: GraineCallDirection?`	`GraineCallDirection?`	—	Who rang whom.
`@Published private(set) var callerName = ""`	`String`	—	The name the call shows.
`@Published private(set) var muted = false`	`Bool`	—	The customer's voice is not being sent.

Member	Returns	Errors	What it does
`@Published private(set) var speakerOn = false`	`Bool`	—	Audio from the loudspeaker rather than the earpiece or a headset. Follows the audio route, so the system's call screen can change it too.
`@Published private(set) var agentSpeaking = false`	`Bool`	—	The agent's voice is playing — measured from the audio itself, not the agent's text.
`@Published private(set) var caption: GraineCaption?`	`GraineCaption?`	—	The latest line: the agent's words as they are said, or the customer's as they are heard.
`@Published private(set) var connectedAt: Date?`	`Date?`	—	When the conversation opened; nil before.
`@Published private(set) var transcript: [GraineCaption] = []`	`[GraineCaption]`	—	What has been said on this call, oldest first (up to 12 lines), a live line replaced as its words arrive. Cleared when the call is over.
`@Published private(set) var hosted = false`	`Bool`	—	Graine rang this call — one booked for this customer (the push said `hosted: "1"`). The call screen says "Scheduled call".
`@Published private(set) var agent: String?`	`String?`	—	The name of the agent the current call is with (`GraineInApp.configure(_:name:)`); nil at idle.
`@Published private(set) var missedCall: GraineMissedCall?`	`GraineMissedCall?`	—	The ring nobody answered, kept for 30 seconds after the call is over so a call screen can say "Missed call" and offer `callBack()` — or until the customer calls back, `dismissMissedCall()` is called, or another call rings or is placed.
`var showsCallScreen: Bool { get }`	`Bool`	—	Whether a call screen has something to show: a call, its last second, or a missed ring to call back.
`private(set) var config = GraineCallsConfig()`	`GraineCallsConfig`	—	The configuration in force.
`func configure(_ config: GraineCallsConfig)`	nothing	—	How calls ring and what they show. Call once, at launch, before `enableVoIPPushes`.
`func enableVoIPPushes(onToken: ((String?) -> Void)? = nil)`	nothing	—	Receive your agent's calls as VoIP pushes. Call it at launch, every launch: iOS wakes a terminated app for a push and delivers it once this has run. Every VoIP push is reported to CallKit before its completion runs, as iOS requires. `onToken` gets the device's VoIP token as hex — at once when already known, again when it changes, nil when iOS invalidates it — for your own server. When the workspace has hosted ringing (Graine rings this app), the SDK registers the token with Graine itself, for every configured agent whose workspace rings iOS — on a new token, a new signed-in customer, a new time zone or app version, and otherwise at most once a day — and reports what became of every ring.
`func addListener(_ listener: GraineCallsListener)`	nothing	—	Hear about calls. Held weakly; called on the main thread.
`func removeListener(_ listener: GraineCallsListener)`	nothing	—	Stops reporting to that listener.
`@discardableResult func startCall(reason: String? = nil, variables: [String: Any] = [:], agent: String? = nil) -> String?`	`String?`	—	The customer calls the agent. The agent hears `who_called = "customer"` and `call_reason` (`reason`), with `variables` (masked on the device) over the SDK's own; a call carries at most 12 variables, names up to 40 characters, values up to 200, about 1 KB in all. `agent` names which configured agent to call; nil is the default. Returns the call's id — `"call_"` and a lowercase UUID — or nil when a call is already live or no agent has that name. Asks for the microphone first if the app does not hold it; refused, the call ends `failed` and `onMicDenied` is called. Ends any in-app (`voice`) call.
`func reportIncomingCall(payload: [AnyHashable: Any], completion: (() -> Void)? = nil)`	nothing	—	For an app with its own `PKPushRegistry`: hands over a VoIP push's payload. It is reported to CallKit at once — before `completion` runs, as iOS requires — even when it is not a usable agent call or a call is already live; then it is ended at once, a live call is never disturbed, and a different call arriving mid-call is reported `missed`. The payload carries `type: "agent_call"` and a `call_id`, and optionally `reason`, `caller_name`, `variables`, `hosted` and `agent_id` (the agent the ring is for); the keys may sit under `data`.
`func voipTokenUpdated(_ token: String?)`	nothing	—	For an app with its own `PKPushRegistry`: the VoIP token (hex, or nil once invalidated), handled as `enableVoIPPushes` would handle it — registered with Graine where Graine rings this app.

Member	Returns	Errors	What it does
`func answer()`	nothing	—	Answers the ringing call from your own screen; the system's call screen answers by itself. The call is reported `answered` at once; if the microphone is then refused, or its prompt goes unanswered for a minute, the call ends `failed`.
`func end()`	nothing	—	Hangs up — or declines, while it rings.
`func setMuted(_ value: Bool)`	nothing	—	Stops (or resumes) sending the customer's voice without ending the call; the system's call screen shows it too.
`func setSpeaker(_ on: Bool)`	nothing	—	The loudspeaker (true) or the earpiece or headset (false). A call starts on the earpiece, or a connected headset.
`@discardableResult func callBack() -> String?`	`String?`	—	Returns the missed call: the customer calls the agent that rang, with the missed call's reason (or `returning_missed_call`) as `call_reason`. The new call's id, or nil when there is nothing to return or a call is live.
`func dismissMissedCall()`	nothing	—	The customer has seen the missed call: the screen can go.
`func providerDidReset(_ provider: CXProvider)`	nothing	—	CallKit's callback when the system resets every call; the SDK ends its call. Apps do not call it.
`func provider(_ provider: CXProvider, perform action: CXStartCallAction)`	nothing	—	CallKit's callback for a call being placed; the SDK opens the conversation. Apps do not call it.
`func provider(_ provider: CXProvider, perform action: CXAnswerCallAction)`	nothing	—	CallKit's callback for an answer; the SDK answers at once and settles the microphone after. Apps do not call it.
`func provider(_ provider: CXProvider, perform action: CXEndCallAction)`	nothing	—	CallKit's callback for a hang-up or decline. Apps do not call it.
`func provider(_ provider: CXProvider, perform action: CXSetMutedCallAction)`	nothing	—	CallKit's callback for mute from the system's call screen. Apps do not call it.
`func provider(_ provider: CXProvider, perform action: CXSetHeldCallAction)`	nothing	—	CallKit's callback for hold; an agent call is never put on hold, so the action fails and the system offers "End & Accept" instead. Apps do not call it.
`func provider(_ provider: CXProvider, didActivate audioSession: AVAudioSession)`	nothing	—	CallKit's callback when the system hands over the audio; the call's audio starts here. Apps do not call it.
`func provider(_ provider: CXProvider, didDeactivate audioSession: AVAudioSession)`	nothing	—	CallKit's callback when the system takes the audio back; the app's own audio session is restored after the call. Apps do not call it.
`func provider(_ provider: CXProvider, timedOutPerforming action: CXAction)`	nothing	—	CallKit's callback for an action that timed out; logged. Apps do not call it.
`func pushRegistry(_ registry: PKPushRegistry, didUpdate pushCredentials: PKPushCredentials, for type: PKPushType)`	nothing	—	PushKit's callback with a new VoIP token, handled as described for `enableVoIPPushes`. Apps do not call it.
`func pushRegistry(_ registry: PKPushRegistry, didInvalidatePushTokenFor type: PKPushType)`	nothing	—	PushKit's callback when the VoIP token is invalidated; `onToken` gets nil. Apps do not call it.
`func pushRegistry(_ registry: PKPushRegistry, didReceiveIncomingPushWith payload: PKPushPayload, for type: PKPushType, completion: @escaping () -> Void)`	nothing	—	PushKit's callback with a VoIP push, passed to `reportIncomingCall(payload:completion:)`. Apps do not call it.

GraineCallsConfig

How calls ring and what they show, passed to `GraineCalls.configure(_:)` once at launch. A value type.

Field / case	Type	Default	Meaning
`var displayName: String`	`String`	`""`	The name a call rings with and shows on the call screen — your agent's, or your brand's. Empty: the agent's call name from the dashboard, else your app's name. With one agent configured a non-empty value wins; with several, each agent's dashboard call name comes first, so a ring says who is calling. A push's `caller_name` wins for its own call.
`var ringtoneSound: String?`	`String?`	`nil`	A sound file in your app's bundle to ring with. nil: the brand's ring when the design chose one, else the system ringtone.

Field / case	Type	Default	Meaning
`var iconTemplateImageData: Data?`	`Data?`	`nil`	PNG data for a template image (40×40 points, transparent) on the system call screen's app button. nil: the brand's launcher glyph, from the design.
`var includesCallsInRecents: Bool`	`Bool`	`false`	List the calls in the Phone app's Recents.
`var ringTimeout: TimeInterval`	`TimeInterval`	`30`	Seconds an agent's call rings before it is missed (at least 1). A ring Graine sent lasts the agent's ring timeout from the dashboard instead.
`init(displayName: String = "", ringtoneSound: String? = nil, iconTemplateImageData: Data? = nil, includesCallsInRecents: Bool = false, ringTimeout: TimeInterval = 30)`	initialiser	—	Builds a calls configuration.

GraineCallsListener

What the calls report, for your server, added with `GraineCalls.addListener(_:)`; every method has a default that does nothing. Called on the main thread; held weakly.

Member	Returns	Errors	What it does
`func onIncomingCall(_ call: GraineIncomingCall)`	nothing	—	An agent's call is ringing on this device. The SDK shows the ring; this is for your own UI or analytics.
`func onCallState(_ state: GraineCallState, callId: String?)`	nothing	—	The call moved: idle, ringing, connecting, active or ended.
`func onCallOutcome(_ outcome: GraineCallOutcome, callId: String)`	nothing	—	What became of a call: answered, declined, missed, ended (local, remote, failed) or failed. Tell your server, with the call id; do not depend on its order relative to `onCallState`.
`extension GraineCallsListener`	—	—	An empty default implementation of every method above.

GraineCallView

A ready call screen for while your app is in the foreground, drawn in the design of the agent the call is with; the system's own call screen covers the lock screen and the background. A SwiftUI view — present it over your UI, for example `.fullScreenCover(isPresented: .constant(calls.showsCallScreen)) { GraineCallView() }`.

Member	Returns	Errors	What it does
`init(calls: GraineCalls? = nil)`	`GraineCallView`	—	The calls to draw; `GraineInApp.shared.calls` when nil, so an agent must be configured. The screen draws every moment of a call: a call Graine rang at a booked time ringing ("Scheduled call") and any other ring ("Incoming call"), each with Decline and Answer; connecting ("Connecting…"); on the call — the timer (with "· Muted" while muted), or "Muted", "Speaking" or "Listening" when the design has no timer, the line being said, and Mute, Speaker and End in the design's order; ended ("Call ended", "Call failed" or "Declined", the controls dimmed); and missed ("Missed call") with Close and Call back, for 30 seconds. The avatar is the brand's logo, its launcher glyph or the call name's initial; a ring pulses out while it rings and the avatar breathes while the agent speaks, not under Reduce Motion. A push that woke a closed app draws from the last session kept on the device.
`var body: some View`	`some View`	—	SwiftUI's view body; apps do not call it.

GraineCallState

Where a native call is: `GraineCalls.state` and `GraineCallsListener.onCallState(_:callId:)`.

Field / case	Type	Default	Meaning
`case idle`	case	—	No call.
`case ringing`	case	—	An agent's call is ringing on the device.
`case connecting`	case	—	Answered or placed; the conversation is being opened.
`case active`	case	—	Talking.
`case ended(GraineCallEndReason)`	case	—	Over, and why. Becomes `idle` once reported, a second later.

Field / case	Type	Default	Meaning
`var isLive: Bool { get }`	`Bool`	—	A call is ringing, connecting or active: no other may start.

GraineCallDirection

Who rang whom: `GraineCalls.direction`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case incoming`	case	—	The agent rang the customer (a VoIP push).
`case outgoing`	case	—	The customer called the agent (`startCall`).

GraineCallEndReason

Why a call is over, inside `GraineCallState.ended` and `GraineCallOutcome.ended`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case local`	case	—	The customer hung up, or the app called `end()`.
`case remote`	case	—	The agent hung up.
`case failed`	case	—	The line could not be kept, or never came up.
`case declined`	case	—	The customer declined the ring.
`case missed`	case	—	Nobody answered within the ring timeout.

GraineCallOutcome

What became of a call, reported once through `GraineCallsListener.onCallOutcome(_:callId:)` — send it to your server with the call id. For a call Graine rang, the SDK also reports it to Graine.

Field / case	Type	Default	Meaning
`case answered`	case	—	The customer answered the agent's call.
`case declined`	case	—	The customer declined it.
`case missed`	case	—	Nobody answered within the ring timeout — or it arrived while another call was live.
`case ended(GraineCallEndReason)`	case	—	A call that connected is over: `.local`, `.remote` or `.failed`.
`case failed(String)`	case	—	It never connected; the string says why, for your logs.
`var wire: String { get }`	`String`	—	One word for your server: `answered`, `declined`, `missed`, `ended` or `failed` — the same words on Android.
`var endReason: GraineCallEndReason? { get }`	`GraineCallEndReason?`	—	Who ended a call that connected (`.ended`): `local`, `remote` or `failed`. Nil otherwise.
`var error: String? { get }`	`String?`	—	Why a call never connected (`.failed`). Nil otherwise.

GraineIncomingCall

An agent's call as its push described it, delivered to `GraineCallsListener.onIncomingCall(_:)`. A value type.

Field / case	Type	Default	Meaning
`let callId: String`	`String`	required	The server's id for this call; every outcome carries it back.
`let reason: String?`	`String?`	`nil`	Why the agent is calling — the prompt's `call_reason`. A call Graine rang keeps its reason on Graine, so it is nil there.
`let callerName: String?`	`String?`	`nil`	The name to ring with, over `GraineCallsConfig.displayName`.
`let variables: [String: String]`	`[String: String]`	`[:]`	Extra prompt variables for this call, from the push.
`let hosted: Bool`	`Bool`	`false`	Graine rang this call itself (the push says `hosted: "1"`); the SDK reports its outcomes to Graine.

Field / case	Type	Default	Meaning
`let agentId: String?`	`String?`	`nil`	The agent the push says the call is from; the ring is routed to that configured agent (the default agent when none matches).
`init(callId: String, reason: String? = nil, callerName: String? = nil, variables: [String: String] = [:], hosted: Bool = false, agentId: String? = nil)`	initialiser	—	Builds an incoming call.

GraineMissedCall

A ring nobody answered, kept as `GraineCalls.missedCall` so a call screen can say so and offer to call back. A value type.

Field / case	Type	Default	Meaning
`let callId: String`	`String`	required	The missed call.
`let callerName: String`	`String`	required	The name it rang with.
`let reason: String?`	`String?`	`nil`	Why the agent called, when the push said.
`let hosted: Bool`	`Bool`	`false`	Graine rang it: a call booked for this customer.
`let at: Date`	`Date`	`Date()`	When it was missed.
`let agent: String?`	`String?`	`nil`	The name of the configured agent that rang; `callBack()` calls that agent.
`init(callId: String, callerName: String, reason: String? = nil, hosted: Bool = false, at: Date = Date(), agent: String? = nil)`	initialiser	—	Builds a missed call.

GraineCaption

One caption line: in `GraineVoice.captions`, `GraineCalls.caption` and `GraineCalls.transcript`. A value type.

Field / case	Type	Default	Meaning
`let role: GraineRole`	`GraineRole`	—	Who said it.
`let text: String`	`String`	—	The line.
`let live: Bool`	`Bool`	—	Still being recognised or spoken — draw it as provisional.
`init(role: GraineRole, text: String, live: Bool)`	initialiser	—	Builds a caption line.

GraineCallsHost

The bridge the React Native and Flutter packages use to drive `GraineCalls` with JSON-safe values (strings, numbers, booleans, null, lists and string-keyed maps); apps on native iOS do not call it — `GraineInApp.shared.calls` is the Swift API. Reached as `GraineCallsHost.shared`; it runs on the main thread, and `configure`, `restore`, `snapshot`, `startCall`, `installId` and `addSink` wait for their answer when called from another thread.

Member	Returns	Errors	What it does
`@objc static let shared = GraineCallsHost()`	`GraineCallsHost`	—	The one host per process. Used by the React Native and Flutter bridges.
`typealias Sink = (_ event: String, _ payload: [String: Any]) -> Void`	—	—	Receives every event on the main thread: its name — `state` (the snapshot, at most once per turn of the main loop), `outcome`, `incoming`, `voipToken` or `action` — and its payload.
`@objc @discardableResult func restore() -> Bool`	`Bool`	—	At launch, before any JavaScript or Dart runs: applies the options the last `configure` kept, and enables VoIP pushes again if they were, so a push that woke a terminated app rings. True when something was restored; false when nothing was kept or this launch is already configured.

Member	Returns	Errors	What it does
`func configure(_ options: [String: Any]?) throws`	nothing	`GraineCallsHostError`: invalid_options	Configures the calls and keeps the options for the next cold start; the first call makes the SDK instance. Options: `publishableKey` and `displayName` (required), `baseUrl`, `ringtoneSound`, `ringTimeoutMs` (30,000 by default), `appVersion`, `callScreen` (`"sdk"` or `"app"`), `debug`, `android`, `ios` (`iconTemplate`, `includesCallsInRecents`) and `agents` (more agents by name, each `{publishableKey, baseUrl?}`). Unknown keys are ignored; a known key of the wrong type is refused, naming it. A new key, base URL or debug setting takes effect at the next launch.
`func snapshot() -> [String: Any]`	`[String: Any]`	—	The call as a call screen draws it, every key always present: `state`, `callId`, `direction`, `reason`, `callerName`, `muted`, `speakerOn`, `agentSpeaking`, `caption`, `connectedAt` (milliseconds since the epoch), `error` and `agent`.
`func enableVoIPPushes()`	nothing	—	Receive the agent's calls as VoIP pushes, now and at every launch after. The token arrives as a `voipToken` event — again on every call, the last one known.
`func handlePush(_ data: [String: Any]?) -> Bool`	`Bool`	—	Always false on iOS, where a VoIP push reaches PushKit and the SDK directly. It exists for the Android side of the bridges.
`func startCall(_ options: [String: Any]?) -> String?`	`String?`	—	The customer calls the agent: `{reason?, variables?, agent?}`. The call's id, or nil when not configured, a call is live, or no agent has that name.
`func answer()`	nothing	—	Answers the ringing call.
`func decline()`	nothing	—	Declines the ringing call (on iOS, `end` while it rings); does nothing otherwise.
`func end()`	nothing	—	Hangs up.
`func setMuted(_ muted: Bool)`	nothing	—	Mutes or unmutes the customer.
`func setSpeaker(_ on: Bool)`	nothing	—	The loudspeaker or the earpiece.
`func setIdentity(_ traits: [String: Any]?)`	nothing	—	Who is signed in (`GraineInApp.identify`); nil on sign-out. Held until the instance exists.
`func setMemory(_ enabled: Bool)`	nothing	—	The current customer's memory choice (`GraineInApp.setMemory`). Held until the instance exists.
`func installId() -> String?`	`String?`	—	This install's id, kept in the Keychain — the one the native calls send, so a device is one install whichever layer speaks. Nil before `configure` or `restore`.
`func setVariables(_ values: [String: Any]?)`	nothing	—	The runtime variables become exactly this map: a key the last map set and this one leaves out is removed. Reaches every agent.
`func setScreen(_ screen: [String: Any]?)`	nothing	—	The screen, as `{screen, title?, journey?, fields?, error?, data?, availableActions?}`; nil, or no screen name, for none.
`func setActions(_ names: [String], sink: Int?)`	nothing	—	The actions the caller answers, and its sink's token: each request for one is sent as an `action` event to the sink that last listed a non-empty set, and waits for `resolveAction` within the SDK's six seconds. Names no longer listed are unregistered.
`func resolveAction(_ requestId: String, result: [String: Any]?)`	nothing	—	The answer to an `action` request: `{status: "ok" | "refused" | "error", message?, data?}`. The first answer wins.
`func addSink(_ sink: @escaping Sink) -> Int`	`Int`	—	Starts receiving events; returns the sink's token. Outcomes and rings that came while nothing listened (the last 20) are delivered to the first sink at once.
`func removeSink(_ token: Int)`	nothing	—	Stops that sink. Action requests it had not answered are refused.

GraineCallsHostError

Why `GraineCallsHost.configure(_:)` refused its options. Used by the React Native and Flutter bridges; native iOS apps do not see it.

Field / case	Type	Default	Meaning
`let code: String`	`String`	—	`"invalid_options"`.

Field / case	Type	Default	Meaning
`let message: String`	`String`	—	A sentence for the developer naming the option that is wrong, such as `callScreen must be "sdk" or "app"`.
`var errorDescription: String? { get }`	`String?`	—	The message.

GraineScheduledCall

A call the agent will make to this customer, booked for later: returned by `scheduleCall(at:reason:)` and `scheduledCalls()`, and listed in `GraineChat.upcomingCalls`. When its time comes, Graine rings this device on the system's call screen.

Field / case	Type	Default	Meaning
`let id: String`	`String`	required	Graine's id for the booking — what `cancelScheduledCall(id:)` takes.
`let at: Date?`	`Date?`	required	When it rings. Nil only when Graine's time for it could not be read.
`let status: GraineCallStatus`	`GraineCallStatus`	`.scheduled`	Where it is.
`let source: GraineCallSource?`	`GraineCallSource?`	`nil`	Who booked it; nil when Graine does not say.
`let reason: String?`	`String?`	`nil`	Why, as it was booked (up to 120 characters); nil when none was given.
`init(id: String, at: Date?, status: GraineCallStatus = .scheduled, source: GraineCallSource? = nil, reason: String? = nil)`	initialiser	—	Builds a booked call.
`var isUpcoming: Bool { get }`	`Bool`	—	Still to come: `scheduled`, or `ringing` now.
`var canCancel: Bool { get }`	`Bool`	—	It can still be called off: `scheduled`, not rung yet.

GraineCallStatus

Where a booked call is. A status this build does not know arrives as `unknown`, carrying its word.

Field / case	Type	Default	Meaning
`case scheduled`	case	—	Wire `scheduled`. Booked, waiting for its time.
`case ringing`	case	—	Wire `ringing`. Ringing on the customer's device.
`case answered`	case	—	Wire `answered`. Picked up; the conversation is on.
`case completed`	case	—	Wire `completed`. The conversation ended.
`case declined`	case	—	Wire `declined`. The customer declined it.
`case missed`	case	—	Wire `missed`. Nobody answered within the ring timeout.
`case failed`	case	—	Wire `failed`. It could not connect.
`case cancelled`	case	—	Wire `cancelled`. Called off.
`case expired`	case	—	Wire `expired`. It could not ring in time.
`case noDevice`	case	—	Wire `no_device`. Graine had no device of this customer's to ring.
`case unknown(String)`	case	—	A status this build does not know, with its word.
`init(wire: String)`	initialiser	—	The status for a wire word (trimmed, any case); `unknown` for any other word.
`var wire: String { get }`	`String`	—	The word on the wire: `scheduled`, …, `no_device`; an unknown status's own word.
`var description: String { get }`	`String`	—	The same as `wire`.

GraineCallSource

Who booked a call. A source this build does not know arrives as `unknown`, carrying its word.

Field / case	Type	Default	Meaning
`case agent`	case	—	Wire `agent`. The agent, in a conversation.
`case app`	case	—	Wire `sdk`. The app's own booking, through `scheduleCall`.
`case dashboard`	case	—	Wire `dashboard`. Your team, in the dashboard.
`case api`	case	—	Wire `api`. The API.
`case unknown(String)`	case	—	A source this build does not know, with its word.
`init(wire: String)`	initialiser	—	The source for a wire word (trimmed, any case); `sdk` and `app` are both `.app`.
`var wire: String { get }`	`String`	—	The word on the wire; `sdk` for `.app`.
`var description: String { get }`	`String`	—	The same as `wire`.

GraineSchedulingRules

The rules for booking calls, from the agent's settings: `GraineInApp.schedulingRules`. `scheduleCall` checks a time against them before it asks, and a booking screen can offer only times between `earliest(now:)` and `latest(now:)`.

Field / case	Type	Default	Meaning
`let usersCanBook: Bool`	`Bool`	`false`	Customers may book calls from the app. False when the agent's settings say nothing.
`let minLeadMinutes: Int`	`Int`	`5`	The soonest a booking may be, in minutes from now.
`let maxDaysAhead: Int`	`Int`	`14`	The furthest ahead a booking may be, in days.
`let maxPendingPerUser: Int`	`Int`	`3`	Calls one customer may have waiting.
`let ringTimeoutSeconds: Int`	`Int`	`45`	How long a ring Graine sends lasts before it is missed.
`let callName: String?`	`String?`	`nil`	The name the agent's calls ring with, when the dashboard set one.
`let hostedRinging: Bool`	`Bool`	`false`	Graine rings this app's devices (the workspace uploaded its push credentials).
`let platforms: [GraineRingPlatform]`	`[GraineRingPlatform]`	`[]`	Where Graine rings.
`init(usersCanBook: Bool = false, minLeadMinutes: Int = 5, maxDaysAhead: Int = 14, maxPendingPerUser: Int = 3, ringTimeoutSeconds: Int = 45, callName: String? = nil, hostedRinging: Bool = false, platforms: [GraineRingPlatform] = [])`	initialiser	—	Builds a set of rules.
`func earliest(now: Date = Date()) -> Date`	`Date`	—	The earliest a call can be booked for: `now` plus `minLeadMinutes`.
`func latest(now: Date = Date()) -> Date`	`Date`	—	The latest: `now` plus `maxDaysAhead` days.

GraineRingPlatform

A platform Graine rings, in `GraineSchedulingRules.platforms`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case ios`	case	—	iOS devices.
`case android`	case	—	Android devices.

GraineMemory

One thing the agent remembers about the signed-in customer, already redacted — returned by `GraineInApp.memories()`, to show people what is kept about them beside `forgetMemory()`. A value type.

Field / case	Type	Default	Meaning
`let id: String`	`String`	required	The memory's id.
`let text: String`	`String`	required	What is remembered.
`let createdAt: Date?`	`Date?`	`nil`	When it was stored.
`let updatedAt: Date?`	`Date?`	`nil`	When it last changed.
`init(id: String, text: String, createdAt: Date? = nil, updatedAt: Date? = nil)`	initialiser	—	Builds a memory.

GraineServiceError

What the methods that ask Graine throw — `memories()`, `forgetMemory()`, `scheduleCall`, `scheduledCalls()` and `cancelScheduledCall`: a refusal from Graine, or a request that got no answer. Branch on `code`; `message` is a sentence fit to show.

Field / case	Type	Default	Meaning
`let code: GraineErrorCode`	`GraineErrorCode`	required	Why — the stable part to branch on.
`let message: String`	`String`	required	A sentence fit to show the customer; Graine's own words when it gave some.
`let retryAfterSeconds: Int?`	`Int?`	`nil`	For `rateLimited`: the seconds Graine asked to wait, when it said.
`let status: Int`	`Int`	`0`	The HTTP status; 0 when no answer came back.
`var errorDescription: String? { get }`	`String?`	—	The message.
`init(code: GraineErrorCode, message: String, status: Int = 0, retryAfterSeconds: Int? = nil)`	initialiser	—	Builds an error — for tests and previews of your own error handling.

GraineErrorCode

Why Graine refused a request, as a stable code; each case has its wire word. A word this build does not know is `unknown`, carrying it.

Field / case	Type	Default	Meaning
`case invalidKey`	case	—	Wire `invalid_key`. The publishable key is not one, or belongs to no agent (HTTP 401).
`case originRefused`	case	—	Wire `origin_refused`. The app is not allowed: `app://` is missing from the agent's Allowed domains (HTTP 403).
`case rateLimited`	case	—	Wire `rate_limited`. Too many requests; wait `retryAfterSeconds` when Graine said how long (HTTP 429).
`case invalidRequest`	case	—	Wire `invalid_request`. A field is missing or malformed (HTTP 400 or 422), or `baseUrl` is not a URL.
`case identityRequired`	case	—	Wire `identity_required`. No customer is identified, or the identifier is a placeholder.
`case deviceLimit`	case	—	Wire `device_limit`. This install is not one of the customer's allowed devices.
`case schedulingOff`	case	—	Wire `scheduling_off`. Customers may not book calls with this agent; checked on the device first.
`case tooSoon`	case	—	Wire `too_soon`. Earlier than `schedulingRules.earliest(now:)`; checked on the device first.
`case tooFar`	case	—	Wire `too_far`. Later than `schedulingRules.latest(now:)`; checked on the device first.
`case outsideWindow`	case	—	Wire `outside_window`. Outside the agent's calling hours, in this device's time zone.
`case tooManyPending`	case	—	Wire `too_many_pending`. The customer already has the most calls waiting.

Field / case	Type	Default	Meaning
`case notFound`	case	—	Wire `not_found`. No such booking for this customer or install (HTTP 404).
`case unavailable`	case	—	Wire `unavailable`. Graine could not answer just now (HTTP 503); try again with backoff.
`case network`	case	—	Wire `network`. No answer came back: offline, or the connection dropped.
`case unknown(String)`	case	—	A code this build does not know, or an unexpected answer, carrying its word — for example `http_500`.
`init(wire: String)`	initialiser	—	The code for a wire word (trimmed, any case); `unknown` for any other word.
`var wire: String { get }`	`String`	—	The word on the wire: `too_soon`, …; an unknown code's own word.
`var description: String { get }`	`String`	—	The same as `wire`.

GraineScreen

What the customer is looking at, passed to `GraineInApp.setScreen(_:)`. Report real values: masking runs on the device before anything leaves it, and a screen that pre-redacts only blinds the agent.

Field / case	Type	Default	Meaning
`var screen: String`	`String`	required	A stable id for the screen; keep it constant — it is also used for analytics.
`var title: String?`	`String?`	`nil`	What the agent calls the screen.
`var journey: Journey?`	`GraineScreen.Journey?`	`nil`	Where the screen sits in a flow, and where the flow goes next.
`var fields: [GraineField]`	`[GraineField]`	`[]`	The inputs on the screen. Their `status` is what lets the agent name what needs attention after a pause.
`var error: String?`	`String?`	`nil`	A screen-level error banner, distinct from a field's own message.
`var data: [String: Any]`	`[String: Any]`	`[:]`	Anything else the agent should know, as JSON values.
`var availableActions: [String]?`	`[String]?`	`nil`	Which registered actions work here. nil allows every action this build registered; an empty list allows none. It can only narrow what the agent declares.
`init(screen: String, title: String? = nil, journey: Journey? = nil, fields: [GraineField] = [], error: String? = nil, data: [String: Any] = [:], availableActions: [String]? = nil)`	initialiser	—	Builds a screen.
`static func == (a: GraineScreen, b: GraineScreen) -> Bool`	`Bool`	—	Equality, comparing `data` by its JSON content; an unchanged screen is not sent again.
`struct Journey: Equatable`	nested type	—	See `GraineScreen.Journey`.

GraineScreen.Journey

Where a screen sits in a flow, and — `next` — where the flow goes after it. A value type.

Field / case	Type	Default	Meaning
`var name: String`	`String`	required	The flow's name, such as `"KYC"`.
`var step: Int?`	`Int?`	`nil`	This screen's step in the flow.
`var of: Int?`	`Int?`	`nil`	How many steps the flow has.
`var next: String?`	`String?`	`nil`	The next step's name.
`init(name: String, step: Int? = nil, of: Int? = nil, next: String? = nil)`	initialiser	—	Builds a journey.

GraineField

One input on a screen, as the agent should understand it, inside `GraineScreen.fields` or reported alone with `GraineInApp.updateField(_:)`. A value type.

Field / case	Type	Default	Meaning
`var name: String`	`String`	required	The field's id; fields merge by it.
`var label: String?`	`String?`	`nil`	What the field is called on screen.
`var value: String?`	`String?`	`nil`	The real value — masked on the way out; never pre-redact it.
`var status: Status?`	`GraineField.Status?`	`nil`	The field's state.
`var error: String?`	`String?`	`nil`	The validation message actually shown — what the agent can offer to fix.
`var isOptional: Bool`	`Bool`	`false`	Leaving it blank is fine, so the agent does not count it as work left. Sent as `optional`.
`init(name: String, label: String? = nil, value: String? = nil, status: Status? = nil, error: String? = nil, isOptional: Bool = false)`	initialiser	—	Builds a field.
`enum Status: String`	nested type	—	See `GraineField.Status`.

GraineField.Status

A field's state; raw values are the wire words. It is what lets the agent name the field that needs attention instead of asking a vague question.

Field / case	Type	Default	Meaning
`case empty`	case	—	Nothing entered.
`case filled`	case	—	Has a value.
`case invalid`	case	—	Rejected — pair it with `error`.
`case pending`	case	—	In flight.
`case locked`	case	—	Cannot be changed here; not the customer's to fix.

GraineActionResult

What an action handler answers. The agent speaks about exactly this, so say what actually happened.

Field / case	Type	Default	Meaning
`let status: Status`	`GraineActionResult.Status`	required	`ok`, `refused` or `error`.
`let message: String`	`String`	`""`	For the agent, not the customer — it decides how to phrase it.
`let data: [String: Any]`	`[String: Any]`	`[:]`	A structured result the agent can reason about.
`init(status: Status, message: String = "", data: [String: Any] = [:])`	initialiser	—	Builds a result.
`static func ok(_ message: String = "", data: [String: Any] = [:]) -> GraineActionResult`	`GraineActionResult`	—	It happened; the agent will tell the customer it is done.
`static func refused(_ message: String) -> GraineActionResult`	`GraineActionResult`	—	It will not happen, and why ("this is locked after submission") — often the most useful answer.
`static func error(_ message: String) -> GraineActionResult`	`GraineActionResult`	—	It was tried and failed.
`enum Status: String`	nested type	—	See `GraineActionResult.Status`.

GraineActionResult.Status

The outcome of an action. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case ok`	case	—	Done.

Field / case	Type	Default	Meaning
`case refused`	case	—	Will not be done — also the SDK's answer for an action this build has no handler for.
`case error`	case	—	Tried and failed — also the SDK's answer for a handler that threw or took longer than six seconds.

GraineFrictionConfig

One layer of friction (stuck-customer) rules — the app's own, applied over the dashboard's — set as `GraineConfig.friction` or with `GraineInApp.configureFriction(_:)`. Most apps set nothing; use it for screens only the app knows about.

Field / case	Type	Default	Meaning
`let json: [String: Any]`	`[String: Any]`	—	The rules as JSON, in the same shape as the dashboard's: `enabled`, `idle` (`{afterMs}` or `false`), `cooldownMs`, `message`, `nudge` (`cooldownSeconds`, `maxPerSession`, `openLauncher`), `requireOutstandingField` and `screens`. Unknown keys are dropped and bad numbers fall back.
`init(json: [String: Any])`	initialiser	—	Rules from JSON in that shape.
`init(enabled: Bool? = nil, idleAfterMs: Int? = nil, cooldownMs: Int? = nil, cooldownSeconds: Int? = nil, maxPerSession: Int? = nil, openLauncher: Bool? = nil, message: String? = nil, requireOutstandingField: Bool? = nil, screens: [String: Screen] = [:])`	initialiser	—	Rules from values; nil leaves the layer below in force. `idleAfterMs`: on one screen this long fires `stalled_on_step` (45 s by default; 0 or less is off). `cooldownMs`: the least gap between two signals on the same screen (60 s). `cooldownSeconds` and `maxPerSession`: the pacing of unprompted interventions (20 s and 3). `openLauncher`: for a host that draws its own launcher, open it for this. `message`: what the agent is told instead of the built-in sentence, with `{title}`, `{screen}`, `{seconds}`, `{field}` and `{error}`. `requireOutstandingField`: fire only while a field is outstanding — invalid, or empty and not optional — timed from when it became so (true by default). `screens`: per-screen overrides by screen id.
`struct Screen`	nested type	—	See `GraineFrictionConfig.Screen`.

GraineFrictionConfig.Screen

Friction rules for one screen, keyed by screen id in `GraineFrictionConfig(screens:)`. A value type.

Field / case	Type	Default	Meaning
`var ignore: Bool?`	`Bool?`	`nil`	True: no friction signals on this screen.
`var idleAfterMs: Int?`	`Int?`	`nil`	The idle time for this screen; 0 or less switches the idle rule off here.
`var message: String?`	`String?`	`nil`	What the agent is told for this screen, with the same placeholders.
`init(ignore: Bool? = nil, idleAfterMs: Int? = nil, message: String? = nil)`	initialiser	—	Builds a screen override.

GraineFrictionSignal

The customer looks stuck, as reported to `GraineListener.onFriction(_:)`. Made by the SDK; fired at most once per screen visit, only during a live conversation.

Field / case	Type	Default	Meaning
`let name: String`	`String`	—	`"stalled_on_step"`.
`let screen: String`	`String`	—	The screen's id.
`let detail: String`	`String`	—	Plain English, given to the agent verbatim — for example the field that is outstanding and the message it shows.
`let data: [String: Any]`	`[String: Any]`	—	`seconds`, `field` and `error`.

Field / case	Type	Default	Meaning
`let cooldownSeconds: Double`	`Double`	—	The pacing the agent applies to unprompted interventions.
`let maxPerSession: Double`	`Double`	—	The most unprompted interventions per conversation.
`let openLauncher: Bool`	`Bool`	—	For a host that draws its own launcher: open it for this.

GraineSessionConfig

The agent's settings from the dashboard, fetched once per launch: `GraineInApp.session` and `GraineListener.onSessionReady(_:)`. Missing fields are empty, never a crash.

Field / case	Type	Default	Meaning
`let agentId: String`	`String`	required	The agent's id.
`let webcallAgentId: String`	`String`	required	The id in-app calls are placed and filed under; the same as `agentId` when the settings name none.
`let appearance: [String: Any]`	`[String: Any]`	`[:]`	Embed & Widgets → Appearance, exactly as stored.
`let variables: [String: Any]`	`[String: Any]`	`[:]`	The agent's default prompt variables.
`let pageVersion: String?`	`String?`	`nil`	The engine page's version, used to cache it.
`let wsHost: String?`	`String?`	`nil`	Where the conversation server is, when the settings name it; nil uses Graine's default.
`let declaredActions: Set<String>`	`Set<String>`	`[]`	The in-app actions declared on this agent in the dashboard.
`let friction: [String: Any]?`	`[String: Any]?`	`nil`	The dashboard's friction rules for this agent, applied under `GraineConfig.friction`.
`let design: GraineDesign?`	`GraineDesign?`	`nil`	The brand's design, resolved into light and dark tokens. Nil from an older server: every surface then draws from `appearance`.
`let identity: GraineIdentitySettings?`	`GraineIdentitySettings?`	`nil`	The agent's Identity setting; nil from an older server, which means `user_id`.
`let memoryEnabled: Bool?`	`Bool?`	`nil`	Whether the agent remembers customers between conversations; nil from an older server.
`let calls: GraineHostedCallsSettings?`	`GraineHostedCallsSettings?`	`nil`	Whether Graine rings this app, and the booking settings; nil from an older server.
`init(agentId: String, webcallAgentId: String, appearance: [String: Any] = [:], variables: [String: Any] = [:], pageVersion: String? = nil, declaredActions: Set<String> = [], friction: [String: Any]? = nil, design: GraineDesign? = nil, wsHost: String? = nil, identity: GraineIdentitySettings? = nil, memoryEnabled: Bool? = nil, calls: GraineHostedCallsSettings? = nil)`	initialiser	—	Builds settings by hand — for previews and tests; the SDK makes the real ones.
`var schedulingRules: GraineSchedulingRules { get }`	`GraineSchedulingRules`	—	The rules for booking calls from `calls`; no booking, and the defaults, when there are none.
`var preferredTransport: GraineTransport { get }`	`GraineTransport`	—	The dashboard's Call audio setting.

Field / case	Type	Default	Meaning
`var accent: String? { get }`	`String?`	—	The dashboard's accent colour, as stored (with or without its "#").

GraineIdentitySettings

The agent's Identity setting: which trait passed to `identify` names the customer, and how many devices they may use. In `GraineSessionConfig.identity`.

Field / case	Type	Default	Meaning
`let field: String`	`String`	`"user_id"`	`user_id`, `email`, `phone`, or a custom trait such as `member_id`.
`let devicesPerUser: Int`	`Int`	`2`	How many devices one customer's memory may be used from, 1 to 5.
`init(field: String = "user_id", devicesPerUser: Int = 2)`	initialiser	—	Builds a setting; a field that is not a valid trait name becomes `user_id`, and `devicesPerUser` is held to 1–5.
`static let standard = GraineIdentitySettings()`	`GraineIdentitySettings`	—	What applies when the session says nothing: `user_id`, two devices.

GraineHostedCallsSettings

Whether Graine rings this app's devices, and the booking settings, as the session carries them: `GraineSessionConfig.calls`. Made by the SDK.

Field / case	Type	Default	Meaning
`let hosted: Bool`	`Bool`	`false`	Graine rings this app's devices (the workspace uploaded its push credentials).
`let platforms: [String]`	`[String]`	`[]`	Which platforms Graine rings: `ios`, `android`.
`let ringTimeoutSeconds: Int?`	`Int?`	`nil`	How long a ring Graine sent lasts before it is missed, up to 300 seconds; nil uses the app's own `ringTimeout`.
`let callName: String?`	`String?`	`nil`	The name the agent's calls ring with, when the dashboard set one.
`let usersCanBook: Bool`	`Bool`	`false`	Whether customers may book calls from the app (`scheduleCall`).
`let maxDaysAhead: Int?`	`Int?`	`nil`	The furthest ahead a booking may be, in days.
`let minLeadMinutes: Int?`	`Int?`	`nil`	The soonest a booking may be, in minutes.
`let maxPendingPerUser: Int?`	`Int?`	`nil`	Calls one customer may have waiting.
`var ringsThisPlatform: Bool { get }`	`Bool`	—	Graine rings iOS devices for this agent: `hosted`, and `platforms` is empty or includes `ios`.

GraineMasking

What never leaves the device unmasked: PAN, email, card, Indian mobile and Aadhaar numbers are replaced, in that order, in what the app reports — screens, events, traits and runtime variables — before it is sent. Static members, safe from any thread; masking is a set of rules, not a guarantee.

Member	Returns	Errors	What it does
`static func mask(_ input: String) -> String`	`String`	—	Replaces identifiers in one string with `[PAN]`, `[EMAIL]`, `[CARD]`, `[PHONE]` and `[AADHAAR]`, then applies the app's own rules.
`static func addRule(_ pattern: NSRegularExpression, token: String = "[REDACTED]")`	nothing	—	A rule for a field shape only your app knows — an account number, a policy id. Registered for the whole app, once at startup; every match is replaced. `token` is a template: `$1` refers to a capture group.
`@discardableResult static func addRule(pattern: String, token: String = "[REDACTED]") -> Bool`	`Bool`	—	The same, from a pattern string. Returns false, and adds nothing, when the pattern does not compile.
`static func clearCustomRules()`	nothing	—	Removes the app's own rules. For tests only.

Member	Returns	Errors	What it does
`static func deep(_ value: Any?) -> Any`	`Any`	—	Masks every string anywhere inside a JSON value. Keys are not masked; numbers and other non-string values are returned untouched.

GraineTransport

How the in-app call's audio travels; raw values are the wire words. Set `GraineConfig.transport` only to force one, and hear what was used through `onTransport`.

Field / case	Type	Default	Meaning
`case websocket = "ws"`	case	—	Audio on the conversation's WebSocket. Works everywhere; a lossy network sounds lossy.
`case webrtc = "rtc"`	case	—	WebRTC, with a jitter buffer and the device's echo canceller. Falls back to `.websocket`.
`var wire: String { get }`	`String`	—	The wire word: `ws` or `rtc`.
`static func from(_ s: String?) -> GraineTransport`	`GraineTransport`	—	`.webrtc` for `"rtc"`; anything else is `.websocket`, the transport that always works.

GraineRole

Who said a line, in captions and chat turns. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case agent`	case	—	The agent.
`case customer`	case	—	The customer.

GraineConnectionQuality

The connection in one word, with the same thresholds on every platform: `GraineVoice.quality` and `onQuality`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case excellent`	case	—	Under 150 ms round trip, 30 ms jitter and 2% loss.
`case good`	case	—	At or past one of those, but under 400 ms, 80 ms and 8%.
`case poor`	case	—	400 ms round trip, 80 ms jitter or 8% loss, or worse — any one is enough. The bar says "Weak connection".
`case lost`	case	—	The network is gone.
`case unknown`	case	—	Nothing measured yet, or outside a call.
`static func classify(rttMs: Double?, jitterMs: Double?, loss: Double?) -> GraineConnectionQuality`	`GraineConnectionQuality`	—	The word for one measurement (loss as a fraction, 0.08 = 8%); the worst value wins, and `unknown` when all three are missing.
`static func classify(sample: [String: Any]?) -> GraineConnectionQuality`	`GraineConnectionQuality`	—	The same, from a sample map `{rtt, jitter, loss}`; a missing, negative or non-numeric value is absent, not zero. Used by the SDK; apps rarely need it.

GrainePhase

What the in-app call is doing, in one word, for an app that draws its own bar: `GraineVoice.phase`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case idle`	case	—	No call; ready to start.
`case connecting`	case	—	The page loading, or a call being placed.
`case reconnecting`	case	—	The page failed and is being reloaded, or the call is being re-established.
`case thinking`	case	—	The customer finished a sentence and the agent has not answered yet — draw `GraineLoader(moment: "thinking")`.

Field / case	Type	Default	Meaning
`case speaking`	case	—	The agent is talking.
`case listening`	case	—	The agent is waiting for the customer.

GraineConnectStage

Where a call being placed is: `GraineVoice.connectStage`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case registering`	case	—	A line is being set up (a cold start).
`case dialling`	case	—	The call is on its way to the agent.
`case retrying`	case	—	The first attempt went unanswered and a fresh connection is being made — on mobile data, usually one the phone dropped while idle. Say "Reconnecting…".
`case switching`	case	—	The call is moving to the other transport.

GraineConnectProgress

One progress report from the engine page: a stage, or `failed`. The SDK turns it into `GraineVoice.connectStage` and `callFailed`.

Field / case	Type	Default	Meaning
`case stage(GraineConnectStage)`	case	—	The connect reached this stage.
`case failed`	case	—	Nothing could carry the call on either transport; `start()` tries again.

GraineConnectivity

Whether the engine page that carries the in-app call is there: `GraineVoice.connectivity` and `onConnectivity`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case loading`	case	—	First load in progress.
`case ready`	case	—	The page is ready.
`case retrying`	case	—	A load failed or never became ready; a reload is scheduled.
`case failed`	case	—	Every reload used up — say so, and offer `GraineVoice.retry()`.

GraineWidget

A component the agent asked to show, read from the map `GraineListener.onWidget(_:)` delivers — for an app that draws components itself (`GraineWidgetView` draws them for you). A value type.

Field / case	Type	Default	Meaning
`let id: String`	`String`	—	The component's id, which an answer names (`answerWidget(widgetId:…)`).
`let kind: String`	`String`	—	`"product_list"`, `"plan_comparison"`, `"choices"`, … — whatever the agent declares.
`let data: [String: Any]`	`[String: Any]`	—	Its content, with root-relative image paths resolved against `baseUrl`.
`let fallbackText: String`	`String`	—	What the agent would have said instead — draw it for a kind you do not render.
`let raw: [String: Any]`	`[String: Any]`	—	The map exactly as it arrived.
`init?(frame: [String: Any], baseUrl: String? = nil)`	failable initialiser	—	Reads a component map (`widget_id` or `id`, `kind`, `data`, `fallback_text`, `style`, `on_select`). Nil when it has no id. Pass the agent's `baseUrl` to resolve images.
`var preset: GraineWidgetPreset { get }`	`GraineWidgetPreset`	—	The preset the dashboard chose, or `.cards`.

GraineWidgetSurface

Where a component goes on screen — one rule shared with every platform, so the same component is not a sheet in one place and a card in another. Used by the SDK's component view; apps do not need it.

Field / case	Type	Default	Meaning
`case screen`	case	—	The component is the screen — pictures, several items to compare (product lists).
`case sheet`	case	—	A choice to finish, then back to the app (plan comparisons, choices, quick actions, calendar slots).
`case card`	case	—	A fact, or a kind this build cannot draw.

GraineWidgetPreset

How a component looks: the preset chosen in the dashboard's Web components, as `GraineWidget.preset`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case gallery`	case	—	Large pictures.
`case cards`	case	—	Cards with medium pictures; the default.
`case list`	case	—	Rows with small pictures.
`case spotlight`	case	—	One item at a time, with a large picture.
`case compact`	case	—	Small pictures and outlined buttons.

GraineNativeAppearance

The dashboard's appearance settings, translated for a phone. Used by the SDK's own views; apps do not need it.

Field / case	Type	Default	Meaning
`let accent: String`	`String`	—	Always a valid `#rgb` or `#rrggbb`.
`let heading: String`	`String`	—	The panel heading ("Talk to our AI assistant") — not the assistant's name.
`let logoUrl: String?`	`String?`	—	The brand's logo as an absolute URL, or nil when none is set.
`let logoPlate: GraineLogoPlate`	`GraineLogoPlate`	—	What the logo sits on where it stands in for the launcher glyph.
`let radius: Double`	`Double`	—	Corner rounding, clamped to what the view draws.

GraineLogoPlate

What the brand's logo sits on wherever it stands in for the launcher glyph — the launcher, the bar's avatar, the chat header. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case accent`	case	—	The shape's own accent fill — for a white or light mark. The default.
`case white`	case	—	White with a hairline border — for a coloured or dark logo, or an opaque image.

GraineWidgetPalette

The colours a component is drawn in when the session has no design, as #RRGGBB. Used by the SDK's own views; apps do not need it.

Field / case	Type	Default	Meaning
`let surface: String`	`String`	—	The card's fill.
`let border: String`	`String`	—	Hairlines.
`let text: String`	`String`	—	Primary text.

Field / case	Type	Default	Meaning
`let muted: String`	`String`	—	Secondary text.
`let onAccent: String`	`String`	—	Readable on the accent — chosen by luminance, so a yellow accent still has a label.
`let mode: String`	`String`	—	`"light"` or `"dark"`, after resolving "auto" against the device.

GraineDesign

The brand's design: two complete token sets, light and dark, and the brand's choice between them — `GraineInApp.design`. Resolved by Graine, so a new design style reaches the SDK as tokens with no release.

Field / case	Type	Default	Meaning
`let scheme: GraineDesignScheme`	`GraineDesignScheme`	`.auto`	A fixed scheme, or `.auto` to follow the device.
`let preset: String?`	`String?`	—	The design's preset name — informational; the tokens are already resolved.
`let light: GraineThemeTokens`	`GraineThemeTokens`	—	The light set.
`let dark: GraineThemeTokens`	`GraineThemeTokens`	—	The dark set.
`func tokens(deviceDark: Bool?) -> GraineThemeTokens`	`GraineThemeTokens`	—	Which tokens to draw: the brand's fixed scheme, else the device's current appearance — light when the device has not said (nil). Ask again when the device switches.
`static func parse(_ raw: Any?) -> GraineDesign?`	`GraineDesign?`	—	Reads a session's design JSON. Nil when there is none worth drawing — absent, not an object, or either scheme without a complete set of colours; nil is not an error. Used by the SDK; apps rarely need it.

GraineDesignScheme

Which of a design's two token sets to draw. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case auto`	case	—	Follow the device's light or dark.
`case light`	case	—	Always the light set.
`case dark`	case	—	Always the dark set.

GraineThemeTokens

One scheme's design tokens — flat numbers and #RRGGBB strings, lengths in points — for an app that draws its own UI in the brand's design: `GraineInApp.theme`, `onTheme` or `GraineDesign.tokens(deviceDark:)`. A missing or malformed value falls back to Graine Classic's, and a scheme without a complete set of colours makes the whole design nil.

Field / case	Type	Default	Meaning
`let preset: String`	`String`	—	The design style these tokens came from.
`let scheme: String`	`String`	—	`"light"` or `"dark"`: which of the design's two sets this is.
`let colors: Colors`	`GraineThemeTokens.Colors`	—	The palette.
`let font: Typography`	`GraineThemeTokens.Typography`	—	Typefaces, sizes, weights, spacing.
`let shape: Shape`	`GraineThemeTokens.Shape`	—	Corners, outline width, the launcher's outline.

Field / case	Type	Default	Meaning
`let spacing: Spacing`	`GraineThemeTokens.Spacing`	—	Spacing units.
`let elevation: Elevation`	`GraineThemeTokens.Elevation`	—	Shadows, or a solid offset.
`let motion: Motion`	`GraineThemeTokens.Motion`	—	How things move.
`let icons: Icons`	`GraineThemeTokens.Icons`	—	Icon style and the launcher glyph.
`let glass: Glass?`	`GraineThemeTokens.Glass?`	—	Translucent surfaces; nil for opaque ones.
`let bubbles: GraineBubbleStyle`	`GraineBubbleStyle`	—	How chat messages sit on the page.
`let chrome: GraineDesignChrome`	`GraineDesignChrome`	—	`.window`: the chat window, the caption card and every component card open as desktop windows.
`let chat: GraineChatTokens?`	`GraineChatTokens?`	—	The chat window's own tokens; nil from an older server, when the chat draws from `colors`.
`let call: GraineCallTokens?`	`GraineCallTokens?`	—	The call screen's tokens; nil from an older server, when the call screen draws as it did before designs.
`func launcherGlyphUrl(origin: String?, points: Double, scale: Double) -> String?`	`String?`	—	The launcher glyph as an image URL, in `colors.onAccent`, for a host that draws its own launcher and would rather not parse SVG. `points` × `scale` is the physical size, so it is sharp on every screen. Nil when the brand chose its logo.

GraineThemeTokens nested structs

The value types inside `GraineThemeTokens`, each with its fields. All are made by the SDK; colours are #RRGGBB strings and lengths are points.

Field / case	Type	Default	Meaning
`struct Colors`	struct	—	`background` (behind everything in the chat window), `surface` (the voice bar, the chat header, sheets), `surfaceRaised` (cards, the composer field), `text`, `textMuted` (secondary text, captions, status lines), `border` (hairlines and outlines), `accent` (primary buttons, the launcher, the send button), `onAccent` (what sits on the accent), `accentSoft` (chips, selected rows, the focus halo), `success`, `warning`, `danger` (the end-call button), `userBubble`, `userBubbleText`, `agentBubble`, `agentBubbleText`, `scrim` (behind a modal sheet, at 40% opacity) — all `String`.
`struct Typography`	struct	—	`body: String`, `heading: String` (`"system"` is the platform's UI font; anything else is a family the SDK downloads unless the app ships it), `scale: Scale`, `weights: Weights`, `letterSpacing: Pair` (in em: multiply by the size for points), `lineHeight: Pair` (multipliers of the size).
`struct Typography.Scale`	struct	—	`caption`, `body`, `title`, `display` — `Double` sizes.
`struct Typography.Weights`	struct	—	`body`, `strong`, `title` — `Int` CSS weights, 100–900.
`struct Typography.Pair`	struct	—	`body`, `title` — `Double`.
`struct Radius`	struct	—	`sm`, `md`, `lg`, `xl` — `Double` corner radii.
`struct Shape`	struct	—	`radius: Radius`, `borderWidth: Double` (the width of every outline), `launcher: GraineLauncherShape`.
`struct Spacing`	struct	—	`unit`, `cardPadding`, `gap` — `Double`.
`struct Shadow`	struct	—	`y: Double`, `blur: Double`, `spread: Double`, `opacity: Double` (0–1, applied to `color`), `color: String`.
`struct Offset`	struct	—	`x`, `y` — `Double`: a solid offset shadow, drawn in `colors.text`.
`struct Elevation`	struct	—	`card: Shadow?`, `floating: Shadow?`, `hardOffset: Offset?`.

Field / case	Type	Default	Meaning
`struct Motion`	struct	—	`durationMs: Double` (0 means no transitions), `easing: String` (`"standard"`, `"emphasized"`, `"spring"`, `"linear"` or `"none"`; a newer word is drawn as `"standard"`), `bounce: Double` (0–1: how much a spring overshoots), and `func effectiveDurationMs(reduceMotion: Bool) -> Double` — how long a transition runs, 0 for none; the system's Reduce Motion always wins over the brand's choice.
`struct Glyph`	struct	—	`id: String`, `path: String` (a 24×24 SVG path), `stroke: Bool` (drawn with `icons.stroke`; otherwise filled).
`struct Icons`	struct	—	`style: String` (`"outline"`, `"filled"`, `"duotone"` or `"sharp"`, for the SDK's own icons), `stroke: Double`, `launcher: Glyph?` (nil when the launcher shows the brand's logo).
`struct Glass`	struct	—	`blur: Double`, `surfaceOpacity: Double`: `surface` at that opacity over a blurred background.

GrainеChatTokens

How the chat window is drawn — its header, its composer, its words: `GraineThemeTokens.chat`, nil from an older server. A missing or malformed field falls back on its own, as listed.

Field / case	Type	Default	Meaning
`let title: String`	`String`	`""`	The header's title; empty means the dashboard's heading.
`let subtitle: String`	`String`	`""`	Under the title; empty means "Online" or "Connecting…".
`let avatar: String`	`String`	`"logo"`	The header's mark: `logo`, `glyph` or `none`.
`let placeholder: String`	`String`	`"Message…"`	The composer's placeholder.
`let showTimestamps: Bool`	`Bool`	`false`	Show each message's time.
`let suggestions: [String]`	`[String]`	`[]`	Up to four replies (40 characters each) offered before the customer has said anything.
`let background: String`	`String`	`colors.background`	Behind the messages.
`let pattern: String`	`String`	`"none"`	Over the thread: `none` or `dots`.
`let patternColor: String`	`String`	`colors.border`	The dots' colour (unused with no pattern).
`let headerBackground: String`	`String`	`colors.surface`	The header's fill.
`let headerText: String`	`String`	`colors.text`	The header's title colour.
`let headerSubtext: String`	`String`	`colors.textMuted`	The header's subtitle colour.
`let composerBackground: String`	`String`	`colors.surfaceRaised`	The composer's fill.
`let composerText: String`	`String`	`colors.text`	The composer's text colour.
`let composerPlaceholder: String`	`String`	`colors.textMuted`	The placeholder's colour.
`let composerBorder: String`	`String`	`colors.border`	The composer's outline.
`let size: GraineChatSize`	`GraineChatSize`	`.compact`	How big the window opens: the dashboard's *Window size*.
`let showUpcomingCalls: Bool`	`Bool`	`true`	The card of the customer's booked calls at the top of the chat (*Booked calls* in the dashboard).
`let upcomingTitle: String`	`String`	`""`	That card's title (*Card title*); empty means "Upcoming call" / "Upcoming calls".
`let showHistory: Bool`	`Bool`	`true`	The conversation kept from before, on screen (*Earlier messages*). Off, the agent still has it.
`let newConversation: Bool`	`Bool`	`true`	The header's compose control (*New conversation button*). Off, `GraineChat.newConversation()` still works.

GrainеCallTokens

How the SDK's call screen (`GraineCallView`) is drawn while the app is open — the system's own call screen is the system's: `GraineThemeTokens.call`, nil from an older server. A missing or malformed field falls back to the screen the

SDK drew before designs, as listed.

Field / case	Type	Default	Meaning
`let background: String`	`String`	`"#0E0F11"`	The screen's fill (the gradient's top, with `backgroundTo`).
`let backgroundTo: String?`	`String?`	`nil`	With one, the background is a gradient from `background` (top) to this (bottom).
`let text: String`	`String`	`"#F5F5F4"`	The name and the agent's caption.
`let textMuted: String`	`String`	`"#A0A5AE"`	The status line, button labels and the customer's caption.
`let avatar: String`	`String`	`"logo"`	`logo`, `glyph` or `initials`; a logo or glyph the brand lacks falls back to the other, then to the initial.
`let avatarShape: String`	`String`	`"circle"`	`circle`, `squircle` or `rounded-square`.
`let avatarFill: String`	`String`	`"#318CE7"`	Behind a glyph or initial.
`let avatarText: String`	`String`	`"#FFFFFF"`	The glyph or initial.
`let buttonFill: String`	`String`	`"#333435"`	A control's fill.
`let buttonIcon: String`	`String`	`"#F5F5F4"`	A control's icon.
`let buttonActiveFill: String`	`String`	`"#F5F5F4"`	A control that is on (muted, the speaker).
`let buttonActiveIcon: String`	`String`	`"#0E0F11"`	That control's icon.
`let endFill: String`	`String`	`"#DC2626"`	The end button's fill.
`let endIcon: String`	`String`	`"#FFFFFF"`	The end button's icon.
`let endShape: String`	`String`	`"round"`	`round` or `pill`.
`let acceptFill: String`	`String`	`"#16A34A"`	The Answer button's fill.
`let acceptIcon: String`	`String`	`"#FFFFFF"`	The Answer button's icon.
`let declineFill: String`	`String`	`"#DC2626"`	The Decline button's fill.
`let declineIcon: String`	`String`	`"#FFFFFF"`	The Decline button's icon.
`let callbackFill: String`	`String`	`"#318CE7"`	The Call back button's fill, on a missed call.
`let callbackIcon: String`	`String`	`"#FFFFFF"`	The Call back button's icon.
`let captions: Bool`	`Bool`	`true`	Show the line being said.
`let timer: Bool`	`Bool`	`true`	Show the call's timer.
`let controls: [String]`	`[String]`	`["mute", "speaker", "chat", "end"]`	The controls in order, from `mute`, `speaker`, `chat` and `end`; `end` is always added. The iOS call screen leaves `chat` out, because the chat is held during a native call.
`let ringtone: String`	`String`	`"system"`	`system`, `graine` or `soft` — the system's call screen rings with it when the app sets no `ringtoneSound`.
`let statusText: StatusText`	`GraineCallTokens.StatusText`	see below	The status words.

GrainCallTokens.StatusText

The status words on the SDK's call screen. A value type made by the SDK.

Field / case	Type	Default	Meaning
`let ringing: String`	`String`	`"Incoming call"`	While an agent's call rings.
`let connecting: String`	`String`	`"Connecting…"`	While the conversation opens.
`let ended: String`	`String`	`"Call ended"`	After a normal end.
`let scheduled: String`	`String`	`"Scheduled call"`	While a ring Graine sent for a call booked for this customer rings.
`let missed: String`	`String`	`"Missed call"`	A ring nobody answered; the screen then offers to call back.

GrainLauncherShape

The floating launcher's outline: `GraineThemeTokens.Shape.launcher`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case circle`	case	—	Fully round.

Field / case	Type	Default	Meaning
`case squircle`	case	—	Continuous corners, the way iOS draws its own icons.
`case roundedSquare = "rounded-square"`	case	—	A square with the design's large corner radius.
`case square`	case	—	A square with the design's small corner radius.
`case pill`	case	—	56 points high and as wide as its label.

GraineBubbleStyle

How chat messages sit on the page: `GraineThemeTokens.bubbles`. Raw values are the wire words.

Field / case	Type	Default	Meaning
`case filled`	case	—	Each speaker's messages on their own colour.
`case outline`	case	—	Both transparent and outlined: the customer's in the accent, the agent's in the border colour.
`case minimal`	case	—	The agent's are plain text on the page; the customer's sit on the soft accent.

GraineDesignChrome

How a surface that holds content is framed: `GraineThemeTokens.chrome`. A value this build does not know, or none at all, is `none`.

Field / case	Type	Default	Meaning
`case none`	case	—	A plain surface.
`case window`	case	—	The chat window, the caption card and every component card open as desktop windows: a title bar with window controls and a monospaced, lower-case title.

GraineGlyph

The launcher glyph as an image URL, for an app that cannot draw SVG path data. Static members, safe from any thread.

Member	Returns	Errors	What it does
`static let assetVersion = 2`	`Int`	—	Raised by Graine whenever a glyph's drawing changes, so a long-cached image never shows an old one.
`static func url(origin: String?, id: String, color: String, sizePx: Double, stroke: Double = 2) -> String`	`String`	—	The image URL for glyph `id` in `color`. `origin`: where Graine is served (`https://www.graine.ai` when empty). `sizePx`: physical pixels (points × screen scale), held to 16–512. `stroke`: held to 0.5–4, in quarter steps.

GraineVersion

The SDK's version, sent with the SDK's requests so the call record tells builds apart.

Field / case	Type	Default	Meaning
`static let name = "0.7.0"`	`String`	`"0.7.0"`	This build's version.

GraineProtocol

The message names the SDK and Graine's engine page exchange, and the helpers that build and read those messages. Used by the SDK itself; apps do not need it.

Member	Returns	Errors	What it does
`enum Out`	—	—	Messages the SDK sends into the page, as `static let` constants: `startCall = "graine:start-call"`, `endCall = "graine:end-call"`, `setMuted = "graine:set-muted"`, `warm = "graine:warm"`, `variables = "graine:variables"`, `appContext = "graine:app-context"`, `appActionResult = "graine:app-action-result"`, `appEvent = "graine:app-event"`, `widgetResult = "graine:widget-result"`, `identity = "graine:identity"` (who the customer is for memory).

Member	Returns	Errors	What it does
`enum In`	—	—	Messages the page sends out, as `static let` constants: `ready = "graine:ready"`, `call = "graine:call"`, `muted = "graine:muted"`, `caption = "graine:caption"`, `widget = "graine:widget"`, `widgetPending = "graine:widget-pending"`, `appAction = "graine:app-action"`, `ended = "graine:ended"`, `micDenied = "graine:mic-denied"`, `transport = "graine:transport"`, `quality = "graine:quality"`, `network = "graine:network"`, `reconnecting = "graine:reconnecting"`, `progress = "graine:progress"` (registering, dialling, retrying, switching or failed).
`static let bridgeName = "ReactNativeWebView"`	`String`	—	The name of the bridge object the engine page looks for, so one page serves every platform.
`static let messageHandlerName = "graine"`	`String`	—	The web view message handler the bridge object forwards to.
`static let actionDeadlineMs = 6_000`	`Int`	—	How long an action handler may take before the agent is told it was not done.
`static func engineUrl(baseUrl: String, voiceAgentId: String, transport: GraineTransport, cacheKey: String, idle: Bool, publishableKey: String, accent: String?) -> String`	`String`	—	The engine page's URL for an agent, transport and page version; `idle` loads it without connecting anything.
`static func deliver(_ message: [String: Any]) -> String?`	`String?`	—	The JavaScript that delivers one message into the page; nil when the message cannot be serialised (dropped, never a crash).
`static func out(_ type: String, _ fields: [String: Any?] = [:]) -> [String: Any]`	`[String: Any]`	—	One outgoing message: its type and its fields as JSON values.
`static func parse(_ raw: String?) -> PageMessage?`	`PageMessage?`	—	One message from the page, typed; nil for anything that is not one of the page's messages.

PageMessage

Every message the engine page sends, typed — returned by `GraineProtocol.parse(_:)`. Used by the SDK itself; apps do not need it.

Field / case	Type	Default	Meaning
`case ready`	case	—	The page is mounted and ready.
`case call(connected: Bool, connecting: Bool)`	case	—	The call's state.
`case muted(Bool)`	case	—	The microphone was muted or unmuted.
`case caption(role: GraineRole, text: String, live: Bool)`	case	—	A caption line.
`case widget([String: Any])`	case	—	A component the agent drew.
`case widgetPending([String: Any])`	case	—	A component on its way.
`case appAction(actionId: String, name: String, arguments: [String: Any])`	case	—	The agent asked for an action.
`case ended`	case	—	The call is over.
`case micDenied(reason: String)`	case	—	The microphone was refused.
`case transport(transport: GraineTransport, requested: GraineTransport, reason: String?, message: String?)`	case	—	What carries the audio, and why.
`case quality(rttMs: Double?, jitterMs: Double?, loss: Double?)`	case	—	One quality measurement.
`case network(online: Bool)`	case	—	The network went or came back.
`case reconnecting(active: Bool)`	case	—	The call is being re-established, or that is over.
`case progress(GraineConnectProgress)`	case	—	Where a connect is.

The Objective-C facade (GraineObjC...)

The same SDK for Objective-C apps (and the bridge the Kotlin Multiplatform SDK binds): NSObject classes over the Swift API, with completion handlers instead of `async`. Swift apps use the API above.

GraineObjCRegistration

What register… and observe… return: unregister stops it.

Member	What it does
`public func unregister()`	

GraineObjCActionResult

What an action handler answers.

Member	What it does
`@objc public let status: String`	ok, refused or error.
`@objc public let message: String`	ok, refused or error.
`@objc public let data: [String: Any]`	ok, refused or error.
`public static func ok(_ message: String) -> GraineObjCActionResult`	It happened.
`public static func ok(_ message: String, data: [String: Any]) -> GraineObjCActionResult`	It happened, with a structured result the agent can reason about.
`public static func refused(_ message: String) -> GraineObjCActionResult`	It will not happen, and why.
`public static func error(_ message: String) -> GraineObjCActionResult`	It was tried and failed.

GraineObjCMemory

One thing the agent remembers about the signed-in user (already redacted).

Member	What it does
`@objc public let identifier: String`	
`@objc public let text: String`	
`@objc public let createdAt: Date?`	
`@objc public let updatedAt: Date?`	

GraineObjCScheduledCall

A call booked to ring this user (or this install) later.

Member	What it does
`@objc public let identifier: String`	
`@objc public let at: Date?`	When it rings; nil only when the server's time could not be read.
`@objc public let status: String`	scheduled, ringing, answered, completed, declined, missed, failed, cancelled, expired, no_device — or a newer word.
`@objc public let source: String?`	agent, sdk (booked from the app), dashboard, api; nil when the server does not say.
`@objc public let reason: String?`	agent, sdk (booked from the app), dashboard, api; nil when the server does not say.
`@objc public let isUpcoming: Bool`	Still to come: scheduled, or ringing right now.
`@objc public let canCancel: Bool`	It can still be called off.

GraineObjCSession

What the dashboard says about the agent.

Member	What it does
`@objc public let agentId: String`	
`@objc public let memoryEnabled: NSNumber?`	Whether the agent remembers its users; nil when the server does not say.
`@objc public let identityField: String`	The trait that names the user: user_id unless the dashboard chose another.
`@objc public let devicesPerUser: Int`	The trait that names the user: user_id unless the dashboard chose another.
`@objc public let usersCanBook: Bool`	Users may book calls from the app.
`@objc public let hostedRinging: Bool`	Graine rings this workspace's users itself.

Member	What it does
`@objc public let callName: String?`	The name the agent's calls ring with, when the dashboard set one.
`@objc public let json: String`	The session as the facade's JSON (sessionJSON).

GraineObjCListener

Everything the SDK reports that is not a state, for an Objective-C app: name and its payload (json is the same payload as JSON text).

GraineObjC

Member	What it does
`@objc public private(set) lazy var chat = GraineObjCChat(graine: graine)`	The chat.
`@objc public private(set) lazy var calls = GraineObjCCalls(graine: graine)`	Native calls (CallKit and VoIP pushes).
`@objc public private(set) lazy var voice = GraineObjCVoice(graine: graine)`	The voice bar's call.
`public static var errorDomain: String`	The error domain of every NSError the facade hands out.
`public static func configure(baseUrl: String, publishableKey: String) -> GraineObjC`	Start the default agent — GraineInApp.configure.
`public static func configure(baseUrl: String, publishableKey: String, variables: [String: Any]?) -> GraineObjC`	Start the SDK with build-time prompt variables.
`public static func configure( baseUrl: String, publishableKey: String, appVersion: String?, debug: Bool, endOnBackground: Bool, captureTaps: Bool, transport: String?, variablesJSON: String? ) -> GraineObjC`	Start the SDK with every option, for a bridge: transport is ws, rtc or nil (the dashboard decides); variablesJSON a JSON object.
`public static func configure( baseUrl: String, publishableKey: String, appVersion: String?, debug: Bool, endOnBackground: Bool, captureTaps: Bool, transport: String?, variablesJSON: String?, name: String ) -> GraineObjC?`	Start an agent by name, with every option, for a bridge — its own key, its own chat, actions and requests (GraineInApp.configure(_:name:)).
`public static func configure( baseUrl: String, publishableKey: String, appVersion: String?, debug: Bool, endOnBackground: Bool, captureTaps: Bool, transport: String?, variablesJSON: String?, name: String, chatSize: String? ) -> GraineObjC?`	configure…name: with the chat window's size, for a bridge: compact, large or full (nil, or any other word, follows the dashboard's Window size).
`public static func named(_ name: String) -> GraineObjC?`	The facade over the agent configured under name (here or from Swift), or nil.
`public static var current: GraineObjC?`	The facade over the default agent (else the first configured); nil before any is configured (from here, or from Swift).
`@objc public var name: String`	The name this agent was configured under: default for the default agent.
`@objc public var installId: String`	This install's id, kept in the Keychain.
`@objc public var memoryChoice: Bool`	The current user's memory choice (true until they turn it off).
`public func identify(_ traits: [String: Any]?)`	Who is signed in; nil signs out.
`public func identify(_ traits: [String: Any]?, memory: Bool)`	Who is signed in, and whether their conversations are remembered.
`public func identifyJSON(_ traitsJSON: String?)`	identify: for a bridge: the traits as a JSON object; nil or null signs out.
`public func identifyJSON(_ traitsJSON: String?, memory: Bool)`	
`public func setMemory(_ enabled: Bool)`	
`public func forgetMemory(completion: @escaping (Int, NSError?) -> Void)`	Forget everything the agent remembers about the signed-in user.
`public func memories(completion: @escaping ([GraineObjCMemory]?, NSError?) -> Void)`	What the agent remembers about the signed-in user, newest first.
`public func memoriesJSON(completion: @escaping (String?, NSError?) -> Void)`	memoriesWithCompletion: for a bridge: a JSON array of {id, text, createdAt, updatedAt} (epoch ms).
`public func setVariables(_ values: [String: Any]?)`	Prompt variables from the app's data.

Member	What it does
`public func setVariablesJSON(_ json: String?)`	
`public func setScreen(_ screen: [String: Any]?)`	What the user is looking at: screen, title, journey, fields, error, data, availableActions.
`public func setScreenJSON(_ json: String?)`	
`public func updateField(_ field: [String: Any])`	One field changed: name, label, value, status, error, optional.
`public func updateFieldJSON(_ json: String)`	
`public func removeField(_ name: String)`	
`public func clearScreen(_ screenId: String?)`	The user left a screen; with an id, only if it is still the one described.
`public func track(_ name: String, data: [String: Any]?)`	A product event that informs without interrupting.
`public func trackEvent(_ name: String, dataJSON: String?)`	track:data: for a bridge: the data as a JSON object.
`public func tap(_ name: String)`	
`public func noteInteraction()`	
`public func registerAction( _ name: String, handler: @escaping ([String: Any], @escaping (GraineObjCActionResult) -> Void) -> Void ) -> GraineObjCRegistration`	Something the agent can do in the app.
`public func registerJSONAction( _ name: String, handler: @escaping (String, @escaping (String, String, String?) -> Void) -> Void ) -> GraineObjCRegistration`	registerAction:handler: for a bridge: the arguments arrive as a JSON object; answer takes the status (ok, refused, error), the message and the structured result as a JSON object (or nil).
`public func scheduleCall(at: Date, reason: String?, completion: @escaping (GraineObjCScheduledCall?, NSError?) -> Void)`	Book a call for at; the block hears the booked call or the refusal (scheduling_off, too_soon, outside_window…).
`public func scheduleCallJSON(at: Date, reason: String?, completion: @escaping (String?, NSError?) -> Void)`	scheduleCallAt:reason:completion: for a bridge: the call as JSON {id, at, status, source, reason}.
`public func scheduledCalls(completion: @escaping ([GraineObjCScheduledCall]?, NSError?) -> Void)`	The calls still to come, soonest first.
`public func scheduledCallsJSON(completion: @escaping (String?, NSError?) -> Void)`	
`public func cancelScheduledCall(_ callId: String, completion: @escaping (String?, NSError?) -> Void)`	Call one off.
`@objc public var session: GraineObjCSession?`	The agent's configuration, once it has loaded.
`@objc public var sessionJSON: String?`	The session as JSON, for a bridge; nil before it has loaded.
`public func addListener(_ listener: GraineObjCListener)`	Hear every event (held weakly).
`public func removeListener(_ listener: GraineObjCListener)`	
`public func observeEvents(_ observer: @escaping (String, String) -> Void) -> GraineObjCRegistration`	Every event as its name and its payload's JSON, for a bridge, until the registration is stopped.

GraineObjCCallOutcome

What became of a call, as, failed (reason: why) or ended (reason local, remote or failed, and how long it lasted).

Member	What it does
`@objc public let callId: String`	
`@objc public let outcome: String`	
`@objc public let reason: String?`	
`@objc public let durationSeconds: NSNumber?`	Seconds a call that connected lasted (ended only).
`@objc public let hosted: Bool`	Graine rang it: a call booked by the agent, the user or the dashboard.
`@objc public let agent: String?`	The agent the call was with, by name (default for the default agent); nil when it is not known.
`@objc public let json: String`	The same, as JSON.

GraineObjCCallsListener

Hears the calls: any change to the call (state, controls, caption), each outcome, each ring.

GraineObjCCalls

Native calls, for objective-c — GraineCalls (CallKit and VoIP pushes): the user calls the agent (startCallWithReason:variables:), or the agent rings the user (enableVoIPPushes at every launch).

Member	What it does
`public func configure(displayName: String, ringtoneSound: String?, ringTimeout: Double, includesCallsInRecents: Bool)`	How calls ring and what they show.
`public func enableVoIPPushes()`	Receive the agent's calls as VoIP pushes.
`public func startCall(reason: String?, variables: [String: Any]?) -> String?`	The user calls this facade's agent.
`public func startCallJSON(reason: String?, variablesJSON: String?) -> String?`	startCallWithReason:variables: for a bridge: the variables as a JSON object.
`public func answer()`	
`public func decline()`	Decline the ringing call.
`public func end()`	Hang up — or decline, while it rings.
`public func setMuted(_ muted: Bool)`	
`public func setSpeaker(_ on: Bool)`	
`public func callBack() -> String?`	Return the missed call.
`public func dismissMissedCall()`	
`@objc public var state: String`	idle, ringing, connecting, active or ended.
`@objc public var endReason: String?`	Who ended the call, while state is ended: local, remote, failed, declined or missed.
`@objc public var callId: String?`	Who ended the call, while state is ended: local, remote, failed, declined or missed.
`@objc public var callerName: String`	Who ended the call, while state is ended: local, remote, failed, declined or missed.
`@objc public var direction: String?`	incoming or outgoing; nil with no call.
`@objc public var muted: Bool`	incoming or outgoing; nil with no call.
`@objc public var speakerOn: Bool`	incoming or outgoing; nil with no call.
`@objc public var agentSpeaking: Bool`	incoming or outgoing; nil with no call.
`@objc public var hosted: Bool`	incoming or outgoing; nil with no call.
`@objc public var agent: String?`	The agent the call is with, by name (default for the default agent): the one a call was placed with, or a ring routed to.
`@objc public var showsCallScreen: Bool`	A call screen has something to show: a call, or a missed ring to call back.
`@objc public var snapshotJSON: String`	The call as JSON, for a bridge: every field above, plus caption and missedCall.
`public func addListener(_ listener: GraineObjCCallsListener)`	
`public func removeListener(_ listener: GraineObjCCallsListener)`	
`public func observe(_ observer: @escaping () -> Void) -> GraineObjCRegistration`	For a bridge: observer runs after every change to the call, until the registration is stopped.
`public func observeOutcomes(_ observer: @escaping (String) -> Void) -> GraineObjCRegistration`	For a bridge: each outcome as JSON {callId, outcome, reason, durationSeconds, hosted, agent} (agent when known).
`public func observeIncoming(_ observer: @escaping (String) -> Void) -> GraineObjCRegistration`	For a bridge: each ring as JSON {callId, callerName, reason, hosted, agent} (agent: the configured agent it rings for, when known).

GraineObjCVoice

The voice bar's call, for objective-c — GraineVoice: the agent talking inside the app.

Member	What it does
`public func attach()`	Mount the engine in the key window, idle.
`public func start()`	
`public func end()`	

Member	What it does
`public func setMuted(_ muted: Bool)`	
`@objc public var connected: Bool`	
`@objc public var connecting: Bool`	
`@objc public var muted: Bool`	
`@objc public var snapshotJSON: String`	{connected, connecting, muted} as JSON, for a bridge.
`public func observe(_ observer: @escaping () -> Void) -> GraineObjCRegistration`	observer runs on the main thread after every change, until the registration is stopped.

GraineObjCChatTurn

One message in the chat: the user's, or one reply of the agent's.

Member	What it does
`@objc public let identifier: String`	Stable for the life of the transcript: t1, t2, …
`@objc public let role: String`	user or agent.
`@objc public let text: String`	user or agent.
`@objc public let live: Bool`	The agent's words are still arriving.
`@objc public let delivery: String`	The user's messages: sending, sent or failed.
`@objc public let at: Date`	The user's messages: sending, sent or failed.
`@objc public let proactive: Bool`	The agent spoke up without being asked.
`@objc public let restored: Bool`	From an earlier conversation, restored when the chat opened (the user came back).
`@objc public let channel: String`	chat, or voice for a turn said on a call (drawn as one "Voice call" row).

GraineObjCChatListener

Hears the chat change — its turns, its status or the typing indicator.

GraineObjCChat

The chat, for objective-c — GraineChat: open when the user is in the chat, send:, close when they leave, and the state (turns, status, typing) read whenever a listener hears a change.

Member	What it does
`@objc public var turns: [GraineObjCChatTurn]`	What has been said, oldest first.
`@objc public var status: String`	idle, connecting, online, reconnecting or offline.
`@objc public var reconnectAttempt: Int`	While reconnecting, which attempt this is (of six); 0 otherwise.
`@objc public var typing: Bool`	The agent is working on a reply.
`@objc public var isOpen: Bool`	The user is in the chat.
`@objc public var snapshotJSON: String`	The whole chat as JSON, for a bridge: {turns: {id, role, text, live, delivery, at, proactive, restored, channel}, status, attempt, typing, open}.
`public func open()`	The user is in the chat: connect, and stay connected.
`public func open(size: String?)`	open, at a size for this open: compact, large or full.
`public func newConversation(completion: @escaping (NSError?) -> Void)`	A fresh thread (GraineChat.newConversation()): the error is Graine's refusal, the chat then as it was.
`public func close()`	The user left: hang up.
`public func send(_ text: String)`	Send a message: shown at once, sent now or when the chat connects.
`public func retry()`	Connect now, and send what could not be sent.
`public func addListener(_ listener: GraineObjCChatListener)`	
`public func removeListener(_ listener: GraineObjCChatListener)`	
`public func observe(_ observer: @escaping () -> Void) -> GraineObjCRegistration`	For a bridge: observer runs on the main thread after every change, until the registration is stopped.

12.5 Android — ai.graine:inapp-android 0.7.0

Kotlin, usable from Java (overloads and static companions are generated). Every public declaration of the library is listed below, except Android framework overrides inside the SDK's own views, activities and services (such as `onCreate` or `onLayout`), which are not for apps to call. Errors from requests to Graine arrive in the callback's `error` as a `GraineError` — never thrown.

GraineInApp

`class GraineInApp private constructor(internal val application: Application, val config: GraineConfig, val name: String, internal val env: AgentEnv)` — The agent. Create it once in Application.onCreate with GraineInApp.init(application, config) and reach it anywhere with GraineInApp.get(). Every member is called on the main thread; callbacks arrive on the main thread with exactly one of value and error set.

Field	Type	Default	Meaning
config	`GraineConfig`	required	
name	`String`	required	This agent's name: DEFAULT unless init() was given another.

Member	Returns	Errors	What it does
`var session: SessionConfig?`	`SessionConfig?`	—	The agent's settings once the SDK has loaded them (once per launch), else null.
`val voice: GraineVoice`	`GraineVoice`	—	The voice call.
`val calls: GraineCalls`	`GraineCalls`	—	Native calls, like WhatsApp: the agent rings the customer (a push → the call notification), or the customer calls the agent — with the screen off, from the lock screen, outcomes back to the app.
`val chat: GraineChat`	`GraineChat`	—	The text conversation with the agent — what the voice bar switches to when the dashboard offers chat, and what GraineChatView draws.
`fun reportEvent(name: String, detail: String? = null, data: Map<String, Any?> = emptyMap())`	`nothing`	—	Something happened that the agent may want to speak up about — on whichever conversation is live (a voice call, or the chat).
`var pendingWidget: JSONObject?`	`JSONObject?`	—	The component the agent drew that is still waiting to be answered, as the page sent it — retained because the view that draws it may not be mounted when it arrives (a panel that is closed), and an event has no replay.
`fun clearPendingWidget()`	`nothing`	—	Forget the retained component — answered, dismissed, or from a call that is over.
`fun addListener(l: GraineListener)`	`nothing`	—	Adds a GraineListener. Kept until removed; called on the main thread.
`fun removeListener(l: GraineListener)`	`nothing`	—	Removes a listener added with addListener.
`fun identify(traits: Map<String, Any?>?, memory: Boolean? = null)`	`nothing`	—	Who is signed in.
`fun setMemory(enabled: Boolean)`	`nothing`	—	Remember the current user's conversations, or stop — the same as identify(the same traits, memory = enabled).

Member	Returns	Errors	What it does
`val memoryEnabled: Boolean`	`Boolean`	—	Whether the current user's conversations are remembered: their last choice on this install, else on.
`fun forgetMemory(done: GraineCallback<Int>? = null)`	`nothing`	In the callback's error (GraineError): identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	Forget everything the agent remembers about the signed-in user — allowed only from a device that user is using.
`fun memories(done: GraineCallback<List<GraineMemory>>)`	`nothing`	In the callback's error (GraineError): identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	What the agent remembers about the signed-in user, newest first — for a privacy screen that shows it (and forgetMemory to clear it).
`val installId: String`	`String`	—	This install's id: a random UUID made on first launch and kept for as long as the app is installed (never backed up, so a restored phone is a new install).
`fun setVariables(values: Map<String, Any?>?)`	`nothing`	—	Prompt variables from the app's own data, while it runs.
`fun currentVariables() : Map<String, String>`	`Map<String, String>`	—	Every variable the agent will receive, in precedence order.
`fun setScreen(next: GraineScreen?)`	`nothing`	—	What the customer is looking at.
`val currentScreen: GraineScreen?`	`GraineScreen?`	—	The screen as last reported, on any agent, or null.
`fun updateField(field: GraineField)`	`nothing`	—	One field changed — report just that.
`fun removeField(name: String)`	`nothing`	—	A field left the screen.
`fun clearScreen(screenId: String? = null)`	`nothing`	—	The customer left a screen.
`fun configureFriction(override: FrictionConfig?)`	`nothing`	—	Replace the app's layer of friction rules.
`val frictionRules: FrictionRules`	`FrictionRules`	—	The rules in force right now — defaults, the dashboard's, the app's.
`fun noteInteraction()`	`nothing`	—	The customer did something that is not a field edit — scrolled a list, opened a picker, sent a message.
`fun track(name: String, data: Map<String, Any?> = emptyMap(), coalesce: Boolean = false)`	`nothing`	—	A product event that informs without interrupting.
`fun tap(name: String)`	`nothing`	—	A tap on a labelled control, noticed by TapCapture (or reported by an app for a control it names itself): tap:<name>, coalesced, at most two a second, and only while a screen is reported — a tap means something only against the screen it was on.
`fun registerAction(name: String, handler: (JSONObject) -> ActionResult): () -> Unit`	`() -> Unit`	—	Something the agent can do in your app.
`fun registerDeferredAction(name: String, handler: (args: JSONObject, answer: (ActionResult) -> Unit) -> Unit): () -> Unit`	`() -> Unit`	—	registerAction for an answer that is not ready when the handler returns: a network call, a confirmation the customer taps, code in another runtime (the React Native and Flutter call packages answer from JavaScript and Dart this way).
`fun checkActions(): Map<String, List<String>>`	`Map<String, List<String>>`	—	Which registered actions the agent can call, and which it cannot.

Member	Returns	Errors	What it does
`val design: ResolvedDesign?`	`ResolvedDesign?`	—	The design being drawn: a preview when one is set, else the session's — and before this launch's session has arrived, the last session's, kept on the device (so a call that rings a closed app is drawn in the brand's design, not a default).
`val schedulingRules: GraineSchedulingRules`	`GraineSchedulingRules`	—	The rules for booking calls — and for the calls Graine rings — from this launch's session, else the last one kept on the device (a push that woke the app), else the defaults: no booking, no hosted ringing.
`fun themeTokens(context: Context? = null, scheme: String? = null): ThemeTokens?`	`ThemeTokens?`	—	The tokens to draw in context right now: the brand's fixed light or dark, else whichever the phone is in (the context's own, so an activity that forces night mode gets dark).
`fun previewDesign(design: ResolvedDesign?)`	`nothing`	—	Draw this design instead of the session's — a preview, a screenshot, a test of how a design looks before the brand saves it.
`fun scheduleCall(at: java.util.Date, reason: String? = null, done: GraineCallback<GraineScheduledCall>? = null)`	`nothing`	In the callback's error (GraineError): on the device: scheduling_off, too_soon, too_far; from Graine: outside_window, too_many_pending, identity_required, device_limit, invalid_request, rate_limited, network, unavailable	Book a call from the agent for the signed-in user (or, with nobody signed in, this install) at at — "call me tomorrow at 6".
`fun scheduledCalls(done: GraineCallback<List<GraineScheduledCall>>)`	`nothing`	In the callback's error (GraineError): network, rate_limited, unavailable, invalid_key, origin_refused	The calls booked for the signed-in user (or this install) that are still to come, soonest first.
`fun cancelScheduledCall(id: String, done: GraineCallback<GraineCallStatus>? = null)`	`nothing`	In the callback's error (GraineError): not_found, network, rate_limited, unavailable	Cancel a booked call — only one booked for this user or install.
`companion · const val DEFAULT = "default"`	`–`	—	The name of the agent init() makes when it is given none, and get() returns.
`companion · fun init(application: Application, config: GraineConfig, name: String = DEFAULT): GraineInApp`	`GraineInApp`	—	Make the agent name — the default one unless named.
`companion · fun get(name: String = DEFAULT) : GraineInApp`	`GraineInApp`	—	The agent init() made under name — the default one unless named.
`companion · fun find(name: String) : GraineInApp?`	`GraineInApp?`	—	The agent init() made under name, or null.
`companion · val all: Map<String, GraineInApp>`	`Map<String, GraineInApp>`	—	Every agent init() made, by name, in the order they were made.

GraineConfig

`data class GraineConfig @JvmOverloads constructor(…)` — How the SDK starts: the base URL (https://www.graine.ai), the agent's publishable key, and options. Pass it to GraineInApp.init.

Field	Type	Default	Meaning
`baseUrl`	`String`	required	"https://www.graine.ai" unless self-hosted.
`publishableKey`	`String`	required	From Agent → Embed & Widgets.
`variables`	`Map<String, Any?>`	`emptyMap()`	Prompt {variables} known at build time.

Field	Type	Default	Meaning
transport	Transport?	null	Force a transport.
appVersion	String?	null	Lands on the call record, to tell builds apart.
endOnBackground	Boolean	true	End a live call when the app goes to the background.
timing	Timing	Timing()	Load timing, for a field app on a very slow network.
debug	Boolean	false	Log every message across the bridge, both ways, and the engine page's own console, to logcat under the tag "Graine".
friction	FrictionConfig?	null	This app's friction rules, over the dashboard's (Embed & Widgets → friction).
fonts	Map<String, Typeface>	emptyMap()	Optional: fonts this app already bundles, by the family names the dashboard offers — mapOf("Inter" to Typeface.createFromAsset(assets, "fonts/Inter.ttf")).
captureTaps	Boolean	true	Notice taps by itself: a tap on a labelled control (what TalkBack reads, its view id, or a button's words) becomes tap:<name> in the screen's recent events, at most twice a second and only while a screen is reported — so the agent knows what the customer just pressed.
chatSize	GraineChatSize?	null	How big the chat window opens, over the dashboard's choice (Embed & Widgets → Appearance → Chat window → Window size): COMPACT above the bar, LARGE a sheet over most of a phone (a panel on a tablet), FULL the whole screen.

GrainеConfig.Timing

`data class Timing @JvmOverloads constructor(…)` — Load and connect timing for very slow networks; the defaults suit almost every app.

Field	Type	Default	Meaning
stallMs	Long	15_000	
bootMs	Long	15_000	
hardCapMs	Long	120_000	
retryDelaysMs	List<Long>	listOf(1_000, 2_000, 4_000, 8_000, 15_000, 15_000)	
endWatchdogMs	Long	4_000	
slowConnectMs	Long	4_000	A connect the customer has waited on this long says "Still connecting…" (GraineVoice.slowConnect).

GrainеCallback

`fun interface GrainеCallback<T>` — The result of an asynchronous call, on the main thread: exactly one of value and error is set.

Member	Returns	Errors	What it does
fun onResult(value: T?, error: GrainеError?)	nothing	—	Called once with the value, or with the error.

GrainеListener

`interface GrainеListener` — Events from the agent. Every method has a default that does nothing, so a Java or Kotlin listener overrides only what it uses. Called on the main thread.

Member	Returns	Errors	What it does
fun onSessionReady(session: SessionConfig)	nothing	—	The agent's settings have loaded.

Member	Returns	Errors	What it does
`fun onCallState(connected: Boolean, connecting: Boolean)`	`nothing`	—	The bar's voice call is connected, or connecting.
`fun onCaption(role: Role, text: String, live: Boolean)`	`nothing`	—	A line being said, again as its words arrive (live) and once more when it is final.
`fun onWidget(widget: JSONObject)`	`nothing`	—	The agent drew a component.
`fun onWidgetPending(hint: JSONObject)`	`nothing`	—	A component is on its way; its content is loading.
`fun onEnded()`	`nothing`	—	The bar's voice call is over, however it ended. Once per call.
`fun onMicDenied(reason: String)`	`nothing`	—	The microphone was refused (NotAllowedError) or there is no usable one (NotFoundError).
`fun onQuality(quality: ConnectionQuality)`	`nothing`	—	One word for the connection.
`fun onNetwork(online: Boolean)`	`nothing`	—	The device went offline or came back.
`fun onReconnecting(active: Boolean)`	`nothing`	—	The network interrupted the call and it is being re-established.
`fun onConnectProgress(stage: ConnectStage?, slow: Boolean, failed: Boolean)`	`nothing`	—	Where a call being placed is (GraineVoice.connectStage), whether the customer has waited on it long enough to say so (GraineVoice.slowConnect), and whether it could not be connected at all (GraineVoice.callFailed).
`fun onTransport(transport: Transport, requested: Transport, reason: String?, message: String?)`	`nothing`	—	Which path carries the bar's audio, and why it is not the one asked for (not_configured, unreachable, screen_channel, media).
`fun onConnectivity(state: Connectivity, attempt: Int, reason: String?)`	`nothing`	—	The engine page is loading, ready, retrying or has failed.
`fun onActionRequested(name: String)`	`nothing`	—	The agent asked for one of your actions.
`fun onActionCompleted(name: String, result: ActionResult)`	`nothing`	—	Your handler answered an action; the result is what the agent was told.
`fun onFriction(signal: FrictionSignal)`	`nothing`	—	The customer looks stuck (stalled_on_step): on one screen, with a field outstanding, through a stretch of a live call.
`fun onError(message: String)`	`nothing`	—	Loud for you, invisible to the customer: a misconfigured agent is never the customer's problem.
`fun onChatState(connected: Boolean, connecting: Boolean)`	`nothing`	—	The text conversation (GraineChat) connected, is (re)connecting, or is off.
`fun onChatMessage(role: Role, text: String, live: Boolean)`	`nothing`	—	A line of the text conversation: the agent's turn as it streams in (live, the words so far), then once more when it is complete; the customer's line when they send it.

SessionConfig

`data class SessionConfig(…)` — What the dashboard says about this agent, fetched once per launch.

Field	Type	Default	Meaning
`agentId`	`String`	required	
`webcallAgentId`	`String`	required	
`appearance`	`JSONObject`	required	
`variables`	`Map<String, Any?>`	required	
`pageVersion`	`String?`	required	
`declaredActions`	`Set<String>`	required	
`friction`	`JSONObject?`	`null`	The dashboard's friction rules for this agent, as served.
`design`	`ResolvedDesign?`	`null`	The design the brand chose, resolved by the server into light and dark tokens.

Field	Type	Default	Meaning
wsHost	String?	null	Where this agent's voice socket lives (wss://…), when the server says; null means Graine's default.
identity	SessionIdentity?	null	Which trait names the user (the agent's Memory settings: Who is the user?); null from a server that predates it.
memoryEnabled	Boolean?	null	Whether the agent remembers its users; null when the server does not say.
schedulingRules	GraineSchedulingRules	GraineSchedulingRules()	The rules for booking calls, and for the calls Graine rings (GraineSchedulingRules); the defaults from an older server.

Member	Returns	Errors	What it does
val preferredTransport: Transport	Transport	—	The dashboard's Call audio setting.
val accent: String?	String?	—	The dashboard's accent colour (#RRGGBB), when one is set.

SessionIdentity

`data class SessionIdentity(val field: String, val devicesPerUser: Int)` — What the session says about who the user is: the trait that names them, and how many devices they may use.

Field	Type	Default	Meaning
field	String	required	A trait name: user_id, email, phone or a custom one such as patient_id.
devicesPerUser	Int	required	

ChatSource

`interface ChatSource` — What a chat view draws from. GraineChat implements it; an app can implement it to drive GraineChatView itself.

Member	Returns	Errors	What it does
val messages: List<GraineChatTurn>	List<GraineChatTurn>	—	The conversation, oldest first.
val online: Boolean	Boolean	—	Connected: "Online".
val typing: Boolean	Boolean	—	The agent is working on a reply: the window draws its typing indicator.
val failed: Boolean	Boolean	—	Nothing could connect the conversation: the window says so and a send tries again.
fun send(text: String)	nothing	—	Sends the customer's message. Empty text is ignored.
fun shown(visible: Boolean)	nothing	—	A chat window showing this conversation came on screen (true) or left it.
fun addChangeListener(listener: Runnable)	nothing	—	Hear about every change.
fun removeChangeListener(listener: Runnable)	nothing	—	Removes a change listener added with addChangeListener.

GraineChat

`class GraineChat internal constructor(internal val graine: GraineInApp) : ChatSource` — The typed conversation with the agent: GraineInApp.get().chat. What the bar's chat window and GraineChatView draw. Main thread.

Member	Returns	Errors	What it does
var connected	–	—	The conversation is up: lines go out as they are typed.
var connecting	–	—	It is being opened, or re-opened after a drop.
override var failed	–	—	Nothing could connect it, after every retry.
override val messages: List<GraineChatTurn>	List<GraineChatTurn>	—	The transcript, oldest first.
var upcomingCalls: List<GraineScheduledCall>	List<GraineScheduledCall>	—	This user's calls still to come (GraineInApp.scheduledCalls), soonest first, at most three — the chat window's Upcoming calls card.

Member	Returns	Errors	What it does
`override val online: Boolean`	`Boolean`	—	The chat is connected.
`override val typing: Boolean`	`Boolean`	—	The agent is typing: from a send until its first words, a component, the chat closing, or 12 seconds.
`var openedSize: GraineChatSize?`	`GraineChatSize?`	—	The size the last open asked for, until close; null follows GraineConfig.chatSize, then the dashboard's.
`fun open(size: GraineChatSize? = null) : Unit`	`Unit`	—	Open the chat — at size for this open, else GraineConfig.chatSize, else the dashboard's (Window size).
`fun connect() : Unit`	`Unit`	—	Open the conversation, if it is not open or opening.
`fun refreshUpcomingCalls() : Unit`	`Unit`	—	Fetch the upcoming calls again (the card refreshes itself; call this after booking one elsewhere).
`fun cancelUpcomingCall(id: String) : Unit`	`Unit`	—	Cancel one of the card's calls: its row says "Cancelling…" until it is done, then goes; a failure says so.
`fun close() : Unit`	`Unit`	—	End the conversation: the agent is told it is over (the stop frame), and no reconnect follows.
`override fun send(text: String)`	`nothing`	—	The customer's line: on screen at once, sent when it can be.
`fun clear() : Unit`	`Unit`	—	Forget the lines on screen — a sign-out, a new topic.
`fun newConversation(done: GraineCallback<Unit>? = null) : Unit`	`Unit`	—	START A NEW CONVERSATION — a fresh thread, as a support chat offers one.
`override fun shown(visible: Boolean) : Unit`	`Unit`	—	Tell the chat whether a view shows it, so it connects while it is on screen.
`override fun addChangeListener(listener: Runnable)`	`nothing`	—	Runs the listener on the main thread whenever anything the chat shows changes.
`override fun removeChangeListener(listener: Runnable)`	`nothing`	—	Removes a change listener.
`companion · const val CLOSE_UNSEEN_MS = 15_000L`	—	—	How long a conversation no window shows is kept for the next screen to take.
`companion · const val UPCOMING_EVERY_MS = 10_000L`	—	—	The upcoming calls are fetched again at most this often, unless asked.

GraineChatTurn

`data class GraineChatTurn @JvmOverloads constructor(…)` — One message in the chat.

Field	Type	Default	Meaning
`id`	`String`	required	Stable for the life of the transcript: t1, t2, …
`role`	`Role`	required	Role.CUSTOMER (the user) or Role.AGENT.
`text`	`String`	required	What was said; an agent's turn may be only a component.
`at`	`java.util.Date`	required	When the turn began.
`delivery`	`Delivery`	`Delivery.SENT`	Where a customer's message is; an agent's turn is always Delivery.SENT.
`proactive`	`Boolean`	`false`	The agent said this without being asked: its turn did not follow one of the customer's.
`component`	`GraineComponent?`	`null`	The component the agent drew with this turn, when it drew one.
`live`	`Boolean`	`false`	The agent's words still arriving: drawn as provisional until the turn ends.
`restored`	`Boolean`	`false`	From an earlier conversation, restored when the chat opened: delivered, with its own time and id, not said just now.
`channel`	`Channel`	`Channel.CHAT`	Where it was said.

GraineChatTurn.Delivery

`enum class Delivery`

Case	Meaning
SENDING	Waiting for the connection: typed while the chat was not connected, sent when it is.
SENT	On its way to the agent.
FAILED	The chat could not reconnect to send it; the next message, or a retry, tries again.

GraineChatView

`class GraineChatView @JvmOverloads constructor(context: Context, attrs: AttributeSet? = null) : FrameLayout(context, attrs), GraineListener` — The chat, as A WHOLE SCREEN — the text conversation with the agent, drawn by the SDK in the brand's design (the chat window's tokens: header, bubbles, composer, suggestions), with the agent's components right under the line that asked for them.

Field	Type	Default	Meaning
context	`Context`	required	
attrs	`AttributeSet?`	`null`	

Member	Returns	Errors	What it does
var source: ChatSource	`ChatSource`	—	The conversation drawn: the default agent's chat unless set — another agent's (GraineInApp.get("support").chat), drawn in that agent's design, or one the app runs itself.
var scheme: String?	`String?`	—	"light", "dark" or "auto" (the device).
var onClose: (() -> Unit)?	`(() -> Unit)?`	—	The close button was pressed.
val widgetView: GraineWidgetView	`GraineWidgetView`	—	The components the agent draws in the chat, between the messages and the composer.

GraineChatActivity

`class GraineChatActivity : Activity()` — The chat on A SCREEN OF ITS OWN — GraineChatView full screen, in the brand's design, above the keyboard, closed by its close button or Back.

Member	Returns	Errors	What it does
companion · fun open(context: Context, agent: String = GraineInApp.DEFAULT)	`nothing`	—	Open the chat screen — with agent, for an app that runs several (GraineInApp.init's name); the default one unless named.

GraineChatSize

`enum class GraineChatSize(val wire: String)` — How big the chat window opens.

Field	Type	Default	Meaning
wire	`String`	required	The design's word for it.

Case	Meaning
COMPACT	The window above the bar, as it has always been. — wire `compact`
LARGE	A sheet over most of a phone; a panel in the corner of a tablet. — wire `large`
FULL	The whole screen. — wire `full`

Member	Returns	Errors	What it does
companion · fun from(wire: Any?): GraineChatSize?	`GraineChatSize?`	—	The design's word for a size; null for anything else — absent, or a newer server's word.

GraineAgentBarView

`class GraineAgentBarView @JvmOverloads constructor(context: Context, attrs: AttributeSet? = null) : LinearLayout(context, attrs), GraineListener` — The voice bar — what the customer sees during a call, drawn by the SDK exactly as the dashboard's Garage draws it in its phone preview.

Field	Type	Default	Meaning
context	`Context`	required	
attrs	`AttributeSet?`	`null`	

Member	Returns	Errors	What it does
`var accent: String?`	`String?`	—	#RRGGBB for the avatar, the microphone and the meter.
`var name: String?`	`String?`	—	The name beside the status.
`var avatarUrl: String?`	`String?`	—	A picture for the avatar, filling its circle — the app's own, so it is the app's to crop.
`var scheme: String?`	`String?`	—	"light", "dark" or "auto" (the device).
`var radius: Int?`	`Int?`	—	Corner rounding in dp for the caption card (the bar takes it plus 8, at most 26).
`var onClose: (() -> Unit)?`	`(() -> Unit)?`	—	The end button was pressed with no call to end, or a call was just hung up (once its goodbye has left the speaker): close the bar — hide it, or show your launcher again.
`var draggable: Boolean`	`Boolean`	—	Let the customer drag it up and down.
`var placementOffset: Int`	`Int`	—	Where the customer left it: dp above its anchor.
`var onPlacementChange: ((bottomOffset: Int) -> Unit)?`	`((bottomOffset: Int) -> Unit)?`	—	The customer let go somewhere new.
`var keyboardAware: Boolean`	`Boolean`	—	Rise above the soft keyboard.
`fun resetPlacement()`	`nothing`	—	Put it back on its anchor, and say so.
`val widgetView: GraineWidgetView`	`GraineWidgetView`	—	The components the agent draws, mounted above the caption so they move with the bar.
`var graine: GraineInApp`	`GraineInApp`	—	The agent this bar talks to — the default one unless set: bar.graine = GraineInApp.get("sales") for another of the app's agents (GraineInApp.init's name).
`var chat: ChatSource?`	`ChatSource?`	—	The text conversation the bar opens beside the call, with the switch between them, as the dashboard decides (Appearance → mode and start: voice, chat, or both.

GraineWidgetView

`class GraineWidgetView @JvmOverloads constructor(context: Context, attrs: AttributeSet? = null) : LinearLayout(context, attrs), GraineListener` — The component the agent put on screen — drawn by the SDK, not by the host.

Field	Type	Default	Meaning
context	`Context`	required	
attrs	`AttributeSet?`	`null`	

Member	Returns	Errors	What it does
`var accent: String?`	`String?`	—	Buttons and badges.
`var radius: Int?`	`Int?`	—	Corner rounding in dp.
`var scheme: String?`	`String?`	—	"light", "dark" or "auto" (the device).
`var theme: Theme?`	`Theme?`	—	Overrides for how components are drawn; null uses the agent's design.
`var onChange: (() -> Unit)?`	`(() -> Unit)?`	—	Called after every change of what is drawn — to show your own "show that again", say.
`val widget: GraineWidget?`	`GraineWidget?`	—	The component on screen, if any.
`val answered: GraineAnswer?`	`GraineAnswer?`	—	What the customer last chose, kept as a line until the next component or a dismiss.
`val closedWidget: GraineWidget?`	`GraineWidget?`	—	The last component closed without an answer — reopen puts it back.
`val isLoading: Boolean`	`Boolean`	—	A component is on its way.

Member	Returns	Errors	What it does
`var graine: GraineInApp?`	`GraineInApp?`	—	The agent whose components this view draws and answers — the default one unless set.
`fun present(frame: JSONObject)`	`nothing`	—	Draw a frame received some other way — a preview, a test — exactly as if the agent sent it.
`fun presentPending(hint: JSONObject)`	`nothing`	—	Show the loader for a component on its way, as if the agent announced it.
`fun dismiss()`	`nothing`	—	Close the component without choosing.
`fun reopen()`	`nothing`	—	Put back the last component the customer closed, exactly as it was.

GraineWidgetView.Theme

`data class Theme(val surface: String? = null, val border: String? = null, val text: String? = null, val muted: String? = null, val onAccent: String? = null)` — Single-colour overrides, each #RRGGBB.

Field	Type	Default	Meaning
`surface`	`String?`	`null`	
`border`	`String?`	`null`	
`text`	`String?`	`null`	
`muted`	`String?`	`null`	
`onAccent`	`String?`	`null`	

GraineLauncherView

`class GraineLauncherView @JvmOverloads constructor(context: Context, attrs: AttributeSet? = null) : View(context, attrs), GraineListener` — The launcher — the floating button, drawn the way the brand designed it.

Field	Type	Default	Meaning
`context`	`Context`	required	
`attrs`	`AttributeSet?`	`null`	

Member	Returns	Errors	What it does
`var label: String?`	`String?`	—	The pill's words, and what TalkBack reads.
`var accent: String?`	`String?`	—	#RRGGBB fill.
`var scheme: String?`	`String?`	—	"light", "dark" or "auto" (the device).
`var graine: GraineInApp?`	`GraineInApp?`	—	The agent this launcher calls, and whose design it wears — the default one unless set (GraineInApp.get("sales")).
`override fun setOnClickListener(l: OnClickListener?)`	`nothing`	—	Replaces the default tap (start or end the call) with the app's own.

GraineLoaderView

`class GraineLoaderView @JvmOverloads constructor(context: Context, attrs: AttributeSet? = null) : View(context, attrs)` — How waiting looks, drawn from the dashboard's Appearance → loader.

Field	Type	Default	Meaning
`context`	`Context`	required	
`attrs`	`AttributeSet?`	`null) : View(context`	

Member	Returns	Errors	What it does
`fun configure(look: LoaderLook, color: Int, track: Int, sizeDp: Int? = null, logoUrl: String? = null)`	`nothing`	—	Draw this look.

GraineVoice

`class GraineVoice internal constructor(private val graine: GraineInApp)` — The assistant bar's voice call: GraineInApp.get().voice. Attach the engine once with attach(activity); the bar starts and ends calls itself. Main thread.

Member	Returns	Errors	What it does
`var connected`	`—`	—	A voice call is connected.
`var connecting`	`—`	—	A voice call is being placed.
`var muted`	`—`	—	The customer's microphone is muted.
`var ready`	`—`	—	The engine is loaded and a call can start now.
`var quality`	`—`	—	The connection in one word: EXCELLENT, GOOD, POOR, LOST or UNKNOWN.
`var online: Boolean?`	`Boolean?`	—	Whether the device is online, as the engine sees it; null before it knows.
`var reconnecting`	`—`	—	The call is being re-established after a network drop.
`var connectivity`	`—`	—	Whether the engine page loaded: LOADING, READY, RETRYING or FAILED.
`var error: String?`	`String?`	—	Set when the call cannot work until a setting is fixed — for example a key or an agent not set up for in-app calls. Show it to your team, not the customer.
`val connectStage: ConnectStage?`	`ConnectStage?`	—	Where a call being placed is, as the engine reports it — registering, dialling, retrying, switching (ConnectStage).
`val slowConnect: Boolean`	`Boolean`	—	The customer has waited on this connect past GraineConfig.Timing.slowConnectMs (4 s): say so — "Still connecting…" — honestly, not with a faster spinner.
`val callFailed: Boolean`	`Boolean`	—	The last start could not be connected at all — not on WebRTC, not on the fallback.
`val phase: CallPhase`	`CallPhase`	—	What the call is doing, in one word — see CallPhase.
`fun attach(activity: Activity)`	`nothing`	—	Mount the engine.
`fun detach()`	`nothing`	—	Remove the engine.
`fun start()`	`nothing`	—	Place the call.
`fun end()`	`nothing`	—	Hang up.
`fun setMuted(value: Boolean)`	`nothing`	—	Stop sending the customer's voice without ending the call.
`fun answerWidget(widgetId: String, data: Map<String, Any?>, summary: String)`	`nothing`	—	The customer answered a component the agent drew.
`fun reportEvent(name: String, detail: String? = null, data: Map<String, Any?> = emptyMap())`	`nothing`	—	Something happened that the agent may want to speak up about.
`fun retry()`	`nothing`	—	Reload the page and start the retry ladder over.
`fun onRequestPermissionsResult(requestCode: Int, grantResults: IntArray): Boolean`	`Boolean`	—	Forward from your activity.
`companion · const val MIC_REQUEST = 0x6E7A`	`—`	—	The request code the SDK uses for the microphone prompt.

GraineCalls

`class GraineCalls internal constructor(private val home: GraineInApp)` — Native calls: GraineInApp.get().calls (one engine for the whole app). Configure once in Application.onCreate; hand Firebase's token and messages to the companion functions; startCall from a foreground activity. Main thread, except handlePush.

Member	Returns	Errors	What it does
`var config: GraineCallsConfig`	`GraineCallsConfig`	—	How calls look and ring.
`var state: GraineCallState`	`GraineCallState`	—	Where the call is.
`var call: GraineCall?`	`GraineCall?`	—	The call in progress, or null.

Member	Returns	Errors	What it does
`var muted`	`—`	—	The customer's microphone is muted (the call goes on; nothing they say is sent).
`var speaker`	`—`	—	The customer chose the loudspeaker (else the call is on a headset, if one is connected, or the earpiece).
`var agentSpeaking`	`—`	—	The agent's voice is being heard.
`var caption: Triple<Role, String, Boolean>?`	`Triple<Role, String, Boolean>?`	—	The last line on the call, for a screen that opens mid-call: role, words, still arriving.
`var connectedAt`	`—`	—	When the call connected (SystemClock.elapsedRealtime millis, which keep counting in deep sleep), for a duration display; 0 before.
`val callAgent: GraineInApp?`	`GraineInApp?`	—	The agent the call in progress is with — for an app's own call screen to draw in its design.
`fun addListener(l: GraineCallListener)`	`nothing`	—	Adds a GraineCallListener; called on the main thread; kept until removed.
`fun removeListener(l: GraineCallListener)`	`nothing`	—	Removes a call listener.
`fun configure(config: GraineCallsConfig)`	`nothing`	—	Set how calls look and ring.
`fun handlePush(data: Map<String, String>): Boolean`	`Boolean`	—	Hand every FCM data message here first.
`fun callBack(activity: Activity, missedCallId: String?, reason: String? = null): String?`	`String?`	—	Call the agent back after a missed call — from the missed screen's Call back, or the notification's.
`fun answer()`	`nothing`	—	Pick up the ringing call.
`fun decline()`	`nothing`	—	Decline the ringing call.
`fun startCall(activity: Activity, reason: String? = null, variables: Map<String, Any?> = emptyMap(), agent: String = GraineInApp.DEFAULT): String?`	`String?`	—	Call the agent: the call screen opens and the call connects.
`fun end()`	`nothing`	—	Hangs up — or, while it rings, declines.
`fun setMuted(value: Boolean)`	`nothing`	—	Stop sending the customer's voice without ending the call.
`fun setSpeaker(on: Boolean)`	`nothing`	—	The loudspeaker on or off.
`fun onRequestPermissionsResult(requestCode: Int, grantResults: IntArray): Boolean`	`Boolean`	—	Forward from an activity of your own that shows the call: the SDK asks for the microphone from the call screen when it does not have it.
`companion · fun onNewToken(context: Context, token: String)`	`nothing`	—	FCM's new token for this install, from the app's FirebaseMessagingService.onNewToken — one line: override fun onNewToken(token: String) = GraineCalls.onNewToken(this, token) Kept on the device and registered with Graine when the workspace has Graine…
`companion · fun handlePush(context: Context, data: Map<String, String>): Boolean`	`Boolean`	—	Hand every FCM data message here first, from the app's FirebaseMessagingService.onMessageReceived — one line: override fun onMessageReceived(message: RemoteMessage) { if (GraineCalls.handlePush(this, message.data)) return } True when it was a call…
`companion · const val ACTION_ANSWER = "ai.graine.inapp.action.ANSWER_CALL"`	`—`	—	The call screen was opened to answer (from the notification's Answer).
`companion · const val ACTION_SHOW = "ai.graine.inapp.action.SHOW_CALL"`	`—`	—	The call screen was opened to show the call.
`companion · const val ACTION_DECLINE = "ai.graine.inapp.action.DECLINE_CALL"`	`—`	—	The ring notification's Decline (handled by the SDK's receiver).
`companion · const val ACTION_HANGUP = "ai.graine.inapp.action.HANG_UP"`	`—`	—	The ongoing-call notification's Hang up (handled by the SDK's receiver).
`companion · const val EXTRA_CALL_ID = "ai.graine.inapp.extra.CALL_ID"`	`—`	—	Which call an intent is about.
`companion · const val EXTRA_AGENT = "ai.graine.inapp.extra.AGENT"`	`—`	—	Which agent (its GraineInApp name) the call an intent is about is with.
`companion · const val ACTION_MISSED = "ai.graine.inapp.action.SHOW_MISSED_CALL"`	`—`	—	The call screen was opened on a missed call (the missed-call notification).

Member	Returns	Errors	What it does
`companion · const val ACTION_CALL_BACK = "ai.graine.inapp.action.CALL_BACK"`	`—`	—	The call screen was opened to call the agent back after a missed call.
`companion · const val EXTRA_CALLER_NAME = "ai.graine.inapp.extra.CALLER_NAME"`	`—`	—	Who a missed call was from, for the missed screen.
`companion · const val EXTRA_REASON = "ai.graine.inapp.extra.REASON"`	`—`	—	Why a missed call was placed, carried into the call back.
`companion · const val MISSED_REASON = "missed_call"`	`—`	—	What a call back to a missed call tells the agent when the missed call had no reason of its own.
`companion · const val MIC_REQUEST = 0x6E7B`	`—`	—	The request code the SDK uses for the call's microphone prompt.

GraineCallsConfig

`data class GraineCallsConfig @JvmOverloads constructor(…)` — How calls ring and what they show. Pass to calls.configure in Application.onCreate; it also creates the notification channels.

Field	Type	Default	Meaning
`displayName`	`String`	`"Assistant"`	The agent's name on the ring and the call screen, when the push names no caller.
`smallIcon`	`Int`	`0`	A monochrome drawable for the notifications' status-bar icon.
`ringtone`	`Uri?`	`null`	The ring.
`callActivity`	`Class<out Activity>?`	`null`	The app's own call screen.
`ringTimeoutMs`	`Long`	`30_000`	How long an incoming call rings before it is missed.

GraineCallListener

`interface GraineCallListener` — Everything a native call reports, on the main thread.

Member	Returns	Errors	What it does
`fun onIncomingCall(call: GraineCall)`	`nothing`	—	A push rang the phone.
`fun onCallState(state: GraineCallState, call: GraineCall?)`	`nothing`	—	The call moved — call is null once it is back to IDLE.
`fun onCallOutcome(callId: String, outcome: GraineCallOutcome)`	`nothing`	—	What became of a call.
`fun onCallCaption(role: Role, text: String, live: Boolean)`	`nothing`	—	The agent's words as they are said (live), then the line once its turn is complete; the customer's likewise.
`fun onAgentSpeaking(speaking: Boolean)`	`nothing`	—	The agent's voice started or stopped being heard.
`fun onCallControls(muted: Boolean, speaker: Boolean)`	`nothing`	—	Muted or speaker changed — from this app, or from the call screen.

GraineCall

`data class GraineCall @JvmOverloads constructor(val callId: String, val direction: Direction, val callerName: String?, val reason: String?, val variables: Map<String, String>, val hosted: Boolean = false)` — The call in progress: who placed it, why, and the id every outcome carries back.

Field	Type	Default	Meaning
`callId`	`String`	required	From the push for an incoming call; generated for one the customer places.
`direction`	`Direction`	required	
`callerName`	`String?`	required	What the ring shows: the push's caller_name, else GraineCallsConfig.displayName.
`reason`	`String?`	required	The push's reason, or startCall's — the prompt's call_reason.

Field	Type	Default	Meaning
variables	`Map<String, String>`	required	The call's own prompt variables (who_called, call_reason, the push's variables).
hosted	`Boolean`	`false`	Graine rang it (the push said hosted: "1"): a call booked with the agent — by the agent in a conversation, by the user, or from the dashboard.

GraineCall.Direction

`enum class Direction { INCOMING, OUTGOING }`

Case	Meaning
INCOMING	
OUTGOING	

GraineCallState

`enum class GraineCallState { IDLE, RINGING, CONNECTING, ACTIVE, ENDED }` — Where the one call is.

Case	Meaning
IDLE	
RINGING	
CONNECTING	
ACTIVE	
ENDED	

GraineCallOutcome

`data class GraineCallOutcome @JvmOverloads constructor(…)` — What became of a call, for the app's server.

Field	Type	Default	Meaning
kind	`Kind`	required	
endReason	`EndReason?`	`null`	
error	`String?`	`null`	
callId	`String`	`""`	The call's id — the same one GraineCallListener.onCallOutcome carries beside it.
durationSeconds	`Int?`	`null`	How long a call that connected lasted, in whole seconds (Kind.ENDED); null otherwise.
hosted	`Boolean`	`false`	Graine rang it (a call booked with the agent): Graine hears this outcome too.
busy	`Boolean`	`false`	A Kind.MISSED ring refused because another call was live.

Member	Returns	Errors	What it does
`val wire: String`	`String`	—	One word for the app's server: answered · declined · missed · ended · failed.
`val outcome: String`	`String`	—	The outcome as one word: answered, declined, missed, ended or failed.
`val reason: String?`	`String?`	—	Why, in one word or sentence: who ended a call (Kind.ENDED), why it failed (Kind.FAILED), busy for a refused ring.
`companion · val ANSWERED`	`—`	—	The answered outcome.
`companion · val DECLINED`	`—`	—	The declined outcome.
`companion · val MISSED`	`—`	—	The missed outcome.
`companion · fun ended(reason: EndReason, error: String? = null)`	`nothing`	—	An ended outcome, with how it ended (LOCAL, REMOTE or FAILED) and, for FAILED, why.
`companion · fun failed(error: String)`	`nothing`	—	A failed outcome — the call never connected — with why.

GraineCallOutcome.Kind

`enum class Kind(val wire: String) { ANSWERED("answered"), DECLINED("declined"), MISSED("missed"), ENDED("ended"), FAILED("failed") }`

Case	Meaning
`ANSWERED("answered")`	
`DECLINED("declined")`	
`MISSED("missed")`	
`ENDED("ended")`	
`FAILED("failed")`	

GraineCallOutcome.EndReason

`enum class EndReason(val wire: String) { LOCAL("local"), REMOTE("remote"), FAILED("failed") }` — LOCAL: this app hung up.

Case	Meaning
`LOCAL("local")`	
`REMOTE("remote")`	
`FAILED("failed")`	

GraineCallActivity

`class GraineCallActivity : Activity(), GraineCallListener` — The sdk's call screen — over the lock screen, for the ring and the call, drawn in the brand's design: the session's call tokens, exactly as

GraineCallService

`class GraineCallService : Service()` — The call's foreground service — a microphone foreground service, so the call keeps its microphone and its process with the screen off and with no activity at all, and the ongoing-call notification (with Hang up) is the way back to it.

GraineCallReceiver

`class GraineCallReceiver : BroadcastReceiver()` — The ring's Decline and the ongoing call's Hang up.

GraineCallsHost

`object GraineCallsHost` — The bridge the React Native and Flutter packages drive the native call engine through. Apps on native Android do not call it; it is listed because it is public.

Member	Returns	Errors	What it does
`fun restore(context: Context): Boolean`	`Boolean`	—	Apply the options the last configure kept, so a launch can ring before any JavaScript or Dart runs.
`fun configure(context: Context, options: Map<*, *>?)`	`nothing`	—	Configure the calls and keep the options for the next cold start.
`fun snapshot() : Map<String, Any?>`	`Map<String, Any?>`	—	The call as a call screen draws it: every key always present.
`fun enableVoIPPushes()`	`nothing`	—	iOS registers for VoIP pushes here.
`fun handlePush(context: Context, data: Map<*, *>?): Boolean`	`Boolean`	—	A push's data, from the app's messaging service: {String: String}.
`fun onNewToken(context: Context, token: String)`	`nothing`	—	FCM's new token for this install, from the app's messaging service: kept, and registered with Graine when the workspace has Graine ring its users (GraineCalls.onNewToken).
`fun startCall(activity: Activity?, options: Map<*, *>?) : String?`	`String?`	—	The customer calls the agent: {reason?, variables?, agent?}, from activity, which must be in the foreground; agent names one of the agents configure set up (the default one unless named).
`fun answer()`	`nothing`	—	Answers the ringing call (bridge).

Member	Returns	Errors	What it does
`fun decline()`	`nothing`	—	Declines the ringing call (bridge).
`fun end()`	`nothing`	—	Hang up — or, while it rings, decline.
`fun setMuted(muted: Boolean)`	`nothing`	—	Mutes or unmutes the call (bridge).
`fun setSpeaker(on: Boolean)`	`nothing`	—	Turns the loudspeaker on or off (bridge).
`fun setIdentity(traits: Map<*, *>?)`	`nothing`	—	Who is signed in (GraineInApp.identify), the RAW traits: the SDK masks them and reads the identity itself.
`fun setMemory(enabled: Boolean)`	`nothing`	—	Remember the signed-in user's conversations, or stop (GraineInApp.setMemory).
`fun installId(context: Context): String`	`String`	—	This install's id (GraineInApp.installId) — the one id the native SDK, React Native and Flutter share on a device.
`fun setVariables(values: Map<*, *>?)`	`nothing`	—	The runtime variables become exactly this map: a key the last map set and this one leaves out is removed.
`fun setScreen(screen: Map<*, *>?)`	`nothing`	—	The screen, as {screen, title?, journey?, fields?, error?, data?, availableActions?}; null, or no screen name, for none.
`fun setActions(names: List<*>?, sink: Int?)`	`nothing`	—	The actions the caller answers, and its sink (addSink's token): each request for one of them is sent as action to the sink that last listed a non-empty set, and waits for resolveAction — within the SDK's own six seconds.
`fun resolveAction(requestId: String, result: Map<*, *>?)`	`nothing`	—	The answer to an action request: {status: "ok" | "refused" | "error", message?, data?}.
`fun addSink(sink: Sink) : Int`	`Int`	—	Start receiving events.
`fun removeSink(token: Int)`	`nothing`	—	Stop.
`fun onRequestPermissionsResult(requestCode: Int, grantResults: IntArray): Boolean`	`Boolean`	—	Forward every permission result from the activity that draws the call when the app draws it (callScreen: "app"): the SDK asks for the microphone from that screen.
`fun onNewIntent(activity: Activity, intent: Intent?): Boolean`	`Boolean`	—	Forward the new intents of the activity that draws the call when the app draws it (callScreen: "app").
`fun needsMicrophonePermission(context: Context): Boolean`	`Boolean`	—	True when the app draws the call (callScreen: "app") and has no microphone permission yet.
`fun microphoneDenied()`	`nothing`	—	The customer refused the microphone a call needed: none was placed or answered, and the snapshot's error says why.

GraineCallsHost.Sink

`fun interface Sink` — Receives the bridge's events (bridge).

Member	Returns	Errors	What it does
`fun onEvent(event: String, payload: Map<String, Any?>)`	`nothing`	—	One event with its payload (bridge).

GraineCallsHostException

`class GraineCallsHostException(val code: String, message: String) : IllegalArgumentException(message)` — Why the host refused a call from React Native or Flutter: code for the platform's error (invalid_options) and a sentence for the developer that names what is wrong.

Field	Type	Default	Meaning
code	`String`	required	
message	`String) : IllegalArgumentException(message`	required	

GraineScheduledCall

`data class GraineScheduledCall @JvmOverloads constructor(…)` — A CALL THE AGENT WILL MAKE TO THIS USER, booked for later: Graine rings it at at, through the app's call screen.

Field	Type	Default	Meaning
id	`String`	required	

Field	Type	Default	Meaning
at	`java.util.Date?`	required	When it rings; null only when the server's time could not be read.
status	`GraineCallStatus`	`GraineCallStatus.SCHEDULED`	
source	`GraineCallSource?`	`null`	Who booked it; null when the server does not say.
reason	`String?`	`null`	Why, as it was booked; null when none was given (or the server keeps it to itself).

Member	Returns	Errors	What it does
`val isUpcoming: Boolean`	`Boolean`	—	Still to come: scheduled, or ringing now.
`val canCancel: Boolean`	`Boolean`	—	It can still be called off: it has not rung yet.

GraineCallStatus

`sealed class GraineCallStatus(val wire: String)` — Where a booked call is.

Field	Type	Default	Meaning
wire	`String`	required	

Case	Meaning
SCHEDULED	— wire `scheduled`
RINGING	— wire `ringing`
ANSWERED	— wire `answered`
COMPLETED	— wire `completed`
DECLINED	— wire `declined`
MISSED	— wire `missed`
FAILED	— wire `failed`
CANCELLED	— wire `cancelled`
EXPIRED	— wire `expired`
NO_DEVICE	Graine had no device of this user's to ring (wire: no_device). — wire `no_device`

Member	Returns	Errors	What it does
`companion · val KNOWN: List<GraineCallStatus>`	`List<GraineCallStatus>`	—	Every status this build knows.
`companion · fun from(wire: String): GraineCallStatus`	`GraineCallStatus`	—	The status for a wire word; Unknown(raw) for a word newer than this SDK.

GraineCallSource

`sealed class GraineCallSource(val wire: String)` — Who booked a call.

Field	Type	Default	Meaning
wire	`String`	required	

Case	Meaning
AGENT	— wire `agent`
APP	— wire `sdk`
DASHBOARD	— wire `dashboard`
API	— wire `api`

Member	Returns	Errors	What it does
`companion · fun from(wire: String) : GraineCallSource`	`GraineCallSource`	—	The source for a wire word ('sdk' is APP); Unknown(raw) for anything else.

GraineSchedulingRules

`data class GraineSchedulingRules @JvmOverloads constructor(…)` — The rules for booking a call, read from the agent's settings with the session. Before the settings arrive (or from an older server) they are the defaults, with booking off.

Field	Type	Default	Meaning
usersCanBook	`Boolean`	`false`	Users, from your app is on: scheduleCall is allowed.
minLeadMinutes	`Int`	`5`	The least notice a booking may have, in minutes.
maxDaysAhead	`Int`	`14`	How far ahead a booking may be, in days.
maxPendingPerUser	`Int`	`3`	How many calls a customer may have waiting.
ringTimeoutSeconds	`Int`	`45`	How long a ring Graine sends lasts before it is missed.
callName	`String?`	`null`	The name the agent's calls ring with, when the dashboard set one.
hostedRinging	`Boolean`	`false`	Graine rings this app's users (the workspace uploaded its push credentials).
platforms	`List<String>`	`emptyList()`	The platforms Graine rings: "ios", "android".

Member	Returns	Errors	What it does
`fun earliest(now: java.util.Date = java.util.Date()) : java.util.Date`	`java.util.Date`	—	The earliest a call can be booked for: now plus minLeadMinutes.
`fun latest(now: java.util.Date = java.util.Date()) : java.util.Date`	`java.util.Date`	—	The latest: now plus maxDaysAhead days.
`val ringsAndroid: Boolean`	`Boolean`	—	Graine rings this app on Android.
`companion · const val RING_SECONDS_MIN = 20`	`—`	—	How long a call Graine rings may ring, in seconds: the dashboard's range.
`companion · const val RING_SECONDS_MAX = 60`	`—`	—	Constant.
`companion · fun parse(raw: Any?): GraineSchedulingRules`	`GraineSchedulingRules`	—	From the session's calls; the defaults (and no booking, no hosted ringing) when there are none.

GraineMemory

`data class GraineMemory @JvmOverloads constructor(val id: String, val text: String, val createdAt: java.util.Date? = null, val updatedAt: java.util.Date? = null)` — One thing the agent remembers about the signed-in user, as Graine keeps it (already redacted).

Field	Type	Default	Meaning
id	`String`	required	
text	`String`	required	
createdAt	`java.util.Date?`	`null`	
updatedAt	`java.util.Date?`	`null`	

GraineError

`data class GraineError @JvmOverloads constructor(val code: GraineErrorCode, val message: String, val retryAfterSeconds: Int? = null)` — A refusal from Graine, or a request that never got an answer: code is the stable part to branch on; message is the server's sentence, fit to show; retryAfterSeconds, for a rate limit, how long the server asked to wait when it said.

Field	Type	Default	Meaning
code	`GraineErrorCode`	required	
message	`String`	required	
retryAfterSeconds	`Int?`	`null`	

GraineErrorCode

`sealed class GraineErrorCode(val wire: String)` — Why Graine refused a request, as the wire says it — or NETWORK when no answer came back.

Field	Type	Default	Meaning
wire	`String`	required	

Case	Meaning
INVALID_KEY	The publishable key is not one, or belongs to no agent. — wire `invalid_key`
ORIGIN_REFUSED	This agent does not accept the app: add app:// to its Allowed domains. — wire `origin_refused`
RATE_LIMITED	Too many requests: wait GraineError.retryAfterSeconds when the server said how long. — wire `rate_limited`
INVALID_REQUEST	A field is missing or malformed. — wire `invalid_request`
IDENTITY_REQUIRED	The request needs a signed-in user: identify() with the trait the agent's Identity setting names. — wire `identity_required`
DEVICE_LIMIT	This user is already using the agent on as many devices as it allows. — wire `device_limit`
SCHEDULING_OFF	The agent does not let users book calls (Calls & scheduling in the dashboard). — wire `scheduling_off`
TOO_SOON	Earlier than schedulingRules.earliest(now). — wire `too_soon`
TOO_FAR	Later than schedulingRules.latest(now). — wire `too_far`
OUTSIDE_WINDOW	Outside the agent's calling hours. — wire `outside_window`
TOO_MANY_PENDING	The customer already has the most calls waiting that the agent allows. — wire `too_many_pending`
NOT_FOUND	No such booking for this customer or install. — wire `not_found`
UNAVAILABLE	The service could not answer just now. — wire `unavailable`
NETWORK	No answer came back: no network, or Graine could not be reached. — wire `network`

Member	Returns	Errors	What it does
`companion · val KNOWN: List<GraineErrorCode>`	`List<GraineErrorCode>`	—	Every code this build knows.
`companion · fun from(wire: String): GraineErrorCode`	`GraineErrorCode`	—	The code for a wire word; Unknown for one this build does not know.

GraineScreen

`data class GraineScreen @JvmOverloads constructor(…)` — What the customer is looking at.

Field	Type	Default	Meaning
screen	`String`	required	
title	`String?`	`null`	
journey	`Journey?`	`null`	
fields	`List<GraineField>`	`emptyList()`	
error	`String?`	`null`	
data	`Map<String, Any?>`	`emptyMap()`	
availableActions	`List<String>?`	`null`	

GraineScreen.Journey

`data class Journey @JvmOverloads constructor(val name: String, val step: Int? = null, val of: Int? = null, val next: String? = null)`

Field	Type	Default	Meaning
name	`String`	required	
step	`Int?`	`null`	
of	`Int?`	`null`	

Field	Type	Default	Meaning
next	`String?`	`null`	

GraineField

`data class GraineField @JvmOverloads constructor(val name: String, val label: String? = null, val value: String? = null, val status: Status? = null, val error: String? = null, val optional: Boolean = false)` — One input on a screen, as the agent should understand it.

Field	Type	Default	Meaning
name	`String`	required	
label	`String?`	`null`	
value	`String?`	`null`	
status	`Status?`	`null`	
error	`String?`	`null`	
optional	`Boolean`	`false`	

GraineField.Status

`enum class Status(val wire: String) { EMPTY("empty"), FILLED("filled"), INVALID("invalid"), PENDING("pending"), LOCKED("locked") }`

Case	Meaning
EMPTY("empty")	
FILLED("filled")	
INVALID("invalid")	
PENDING("pending")	
LOCKED("locked")	

ActionResult

`data class ActionResult(val status: Status, val message: String = "", val data: Map<String, Any?> = emptyMap())` — What an action handler answers: ok, refused or error, with a message for the agent and optional data.

Field	Type	Default	Meaning
status	`Status`	required	
message	`String`	`""`	
data	`Map<String, Any?>`	`emptyMap()`	

Member	Returns	Errors	What it does
companion · fun ok(message: String = "", data: Map<String, Any?> = emptyMap())	`nothing`	—	The action was done; the message says what happened.
companion · fun refused(message: String)	`nothing`	—	The action was not done, and why.
companion · fun error(message: String)	`nothing`	—	The action failed, and why.

ActionResult.Status

`enum class Status(val wire: String) { OK("ok"), REFUSED("refused"), ERROR("error") }`

Case	Meaning
OK("ok")	
REFUSED("refused")	
ERROR("error")	

Masking

`object Masking` — The masking applied to screens, variables and traits before they leave the device. Built in: PAN, Aadhaar, card and account numbers, phone numbers, email addresses.

Member	Returns	Errors	What it does
`fun mask(input: String): String`	`String`	—	Masks a string the way the SDK does before sending it.
`fun addRule(pattern: Regex, token: String = "[REDACTED]")`	`nothing`	—	Add a rule for a field shape only your app knows — an account number, a policy id.
`fun clearCustomRules()`	`nothing`	—	Removes the rules added with addRule.
`fun deep(value: Any?) : Any?`	`Any?`	—	Masks every string anywhere inside a JSON value.

FrictionConfig

`data class FrictionConfig @JvmOverloads constructor(…)` — The app's layer of friction rules, over the dashboard's (Embed & Widgets → friction JSON, served with the session) over the built-in defaults.

Field	Type	Default	Meaning
`enabled`	`Boolean?`	`null`	
`idleAfterMs`	`Long?`	`null`	On the same screen this long → stalled_on_step.
`cooldownMs`	`Long?`	`null`	Minimum gap between two signals on the same screen.
`cooldownSeconds`	`Int?`	`null`	Runtime pacing for the agent's unsolicited interventions.
`maxPerSession`	`Int?`	`null`	
`openLauncher`	`Boolean?`	`null`	Carried to GraineListener.onFriction for a host that opens its own launcher.
`message`	`String?`	`null`	What the agent is told instead of the built-in sentence: {title} {screen} {seconds} {field} {error}.
`requireOutstandingField`	`Boolean?`	`null`	Fire only while a field is outstanding (invalid, or empty and not optional), with the clock running from when it became so.
`screens`	`Map<String, Screen>`	`emptyMap()`	Per-screen overrides, keyed by the screen id the app reports.

Member	Returns	Errors	What it does
`companion · fun fromJson(o: JSONObject?): FrictionConfig?`	`FrictionConfig?`	—	The dashboard's layer, from the session's friction object.

FrictionConfig.Screen

`data class Screen(val ignore: Boolean? = null, val idleAfterMs: Long? = null, val message: String? = null)`

Field	Type	Default	Meaning
`ignore`	`Boolean?`	`null`	No friction detection at all on this screen — someone reading terms is not stuck.
`idleAfterMs`	`Long?`	`null`	
`message`	`String?`	`null`	

FrictionRules

`data class FrictionRules(…)` — Every layer applied: what is actually in force.

Field	Type	Default	Meaning
`enabled`	`Boolean`	`true`	
`idleAfterMs`	`Long?`	`45_000`	Null: the idle rule is off.
`cooldownMs`	`Long`	`60_000`	
`cooldownSeconds`	`Int`	`20`	
`maxPerSession`	`Int`	`3`	

Field	Type	Default	Meaning
openLauncher	`Boolean`	`false`	
message	`String`	`""`	
requireOutstandingField	`Boolean`	`true`	
screens	`Map<String, FrictionConfig.Screen>`	`emptyMap()`	

Member	Returns	Errors	What it does
`fun forScreen(id: String?): ForScreen`	`ForScreen`	—	The rules on one screen: the top level with that screen's overrides on it.

FrictionRules.ForScreen

`data class ForScreen(val ignore: Boolean, val idleAfterMs: Long?, val message: String)`

Field	Type	Default	Meaning
ignore	`Boolean`	required	
idleAfterMs	`Long?`	required	
message	`String`	required	

FrictionSignal

`data class FrictionSignal(val name: String, val screen: String, val detail: String, val data: Map<String, Any?>, val cooldownSeconds: Int, val maxPerSession: Int, val openLauncher: Boolean)` — The customer looks stuck.

Field	Type	Default	Meaning
name	`String`	required	
screen	`String`	required	
detail	`String`	required	
data	`Map<String, Any?>`	required	
cooldownSeconds	`Int`	required	
maxPerSession	`Int`	required	
openLauncher	`Boolean`	required	

GraineWidget

`data class GraineWidget(val id: String, val kind: String, val data: JSONObject, val fallbackText: String, val style: WidgetStyle, val onSelect: WidgetSelectAction)` — A component the agent put on screen, adopted from the page's frame.

Field	Type	Default	Meaning
id	`String`	required	
kind	`String`	required	"product_list", "plan_comparison", "choices", … or whatever the agent declares.
data	`JSONObject`	required	Root-relative pictures already resolved against the agent's origin.
fallbackText	`String`	required	What the agent would have said instead.
style	`WidgetStyle`	required	How the dashboard asked for it to look.
onSelect	`WidgetSelectAction`	required	What the app should do on a tap, beyond telling the agent.

Member	Returns	Errors	What it does
`companion · fun adopt(raw: JSONObject?, baseUrl: String): GraineWidget?`	`GraineWidget?`	—	The page's widget object → a component, or null when it names no widget_id (a frame the SDK cannot answer is not one to draw).

GraineAnswer

`data class GraineAnswer(val widget: GraineWidget, val data: Map<String, Any?>, val summary: String)` — A component the customer answered: what they chose, and the sentence the agent heard as their turn.

Field	Type	Default	Meaning
`widget`	`GraineWidget`	required	
`data`	`Map<String, Any?>`	required	
`summary`	`String`	required	

WidgetAnswer

`data class WidgetAnswer(val data: Map<String, Any?>, val summary: String)`

Field	Type	Default	Meaning
`data`	`Map<String, Any?>`	required	
`summary`	`String`	required	

Role

`enum class Role { AGENT, CUSTOMER }` — Who said something: AGENT or CUSTOMER.

Case	Meaning
`AGENT`	
`CUSTOMER`	

Transport

`enum class Transport(val wire: String)` — How the call's audio travels.

Field	Type	Default	Meaning
`wire`	`String`	required	

Case	Meaning
`WEBSOCKET`	Raw PCM on the conversation socket. — wire `ws`
`WEBRTC`	Opus over WebRTC with a jitter buffer and the device's echo canceller. — wire `rtc`

Member	Returns	Errors	What it does
`companion · fun from(s: String?) : Transport`	`Transport`	—	The transport for a wire word.

ConnectionQuality

`enum class ConnectionQuality` — The connection in one word: EXCELLENT, GOOD, POOR, LOST or UNKNOWN.

Case	Meaning
`EXCELLENT`	
`GOOD`	
`POOR`	
`LOST`	
`UNKNOWN`	

Member	Returns	Errors	What it does
`companion · fun classify(rttMs: Double?, jitterMs: Double?, loss: Double?): ConnectionQuality`	`ConnectionQuality`	—	The quality for a round-trip time, jitter and loss: POOR at 400 ms, 80 ms or 8 %; GOOD up to 150 ms, 30 ms, 2 %.

ConnectStage

`enum class ConnectStage(val wire: String)` — WHERE A CALL BEING PLACED IS, as the engine reports it (graine:progress, — the React Native launcher's connectStage.

Field	Type	Default	Meaning
`wire`	`String`	required	

Case	Meaning
`REGISTERING`	— wire `registering`
`DIALLING`	— wire `dialling`
`RETRYING`	— wire `retrying`
`SWITCHING`	— wire `switching`

Member	Returns	Errors	What it does
`companion · fun from(s: String?) : ConnectStage?`	`ConnectStage ?`	—	Null for anything else — failed, and any stage this build does not know.

Connectivity

`enum class Connectivity { LOADING, READY, RETRYING, FAILED }` — Whether the page that carries the call is actually there.

Case	Meaning
`LOADING`	
`READY`	
`RETRYING`	
`FAILED`	

CallPhase

`enum class CallPhase` — What the call is doing, in one word — the React Native launcher's phase.

Case	Meaning
`IDLE`	
`CONNECTING`	
`RECONNECTING`	
`THINKING`	
`SPEAKING`	
`LISTENING`	

Member	Returns	Errors	What it does
`companion · fun of(reconnecting: Boolean, connectivity: Connectivity, connecting: Boolean, ready: Boolean, connected: Boolean, agentSpeaking: Boolean, awaitingAgent: Boolean) : CallPhase`	`CallPha se`	—	Thinking is the gap between the customer's final line and the agent's first: kept as a flag rather than a timer, because the transcript is the truth about who last spoke.

GraineVersion

`object GraineVersion` — The SDK's version.

Member	Returns	Errors	What it does
`const val NAME = "0.7.0"`	–	—	"0.7.0".

GraineChatTurn.Channel

`enum class Channel`

Case	Meaning
`CHAT`	Typed, in the chat.
`VOICE`	Said on a call with the agent — only a restored turn is.

GraineErrorCode.Unknown

`class Unknown(raw: String) : GraineErrorCode(raw)` — A code this build does not know, as the server sent it.

Field	Type	Default	Meaning
raw	`String) : GraineErrorCode(raw`	required	

GraineCallStatus.Unknown

`class Unknown(raw: String) : GraineCallStatus(raw)`

Field	Type	Default	Meaning
raw	`String) : GraineCallStatus(raw`	required	

GraineCallSource.Unknown

`class Unknown(raw: String) : GraineCallSource(raw)`

Field	Type	Default	Meaning
raw	`String) : GraineCallSource(raw`	required	

Design and drawing types

Read-only values the SDK resolves from the agent's design (the Garage) and uses to draw. Apps drawing their own UI beside the agent can read them; most apps never need to.

Type	Fields and members	What it is
`WidgetStyle`	`preset, imageSize, density, button, facts, columns, surface, radius, accent, platform, select, maxSelect`	HOW A COMPONENT LOOKS — chosen on the component in the dashboard, carried with it to the agent, drawn by the SDK.
`WidgetLook`	`preset, imageSize, density, button, facts, columns, surface, radius, accent, platform, select, maxSelect`	The effective look: the preset's defaults, then the knobs on top.
`PlatformGrammar`	`radius, chipRadius, buttonRadius, pillButtons, separatorInset, grabberWidth, grabberHeight, titleWeight, padBoost, chipOutlined`	WHAT A PLATFORM'S DESIGN GRAMMAR CHANGES — the shapes people on that phone already know.
`WidgetSelectAction`	`path, action, args, say, isEmpty`	What the app does when the customer taps, beyond telling the agent: an in-app action (or a path for the one navigate handler), with arguments filled from the tapped item.
`SelectCall`	`name, args`	One tap's in-app call: the action to run and what to run it with.
`LoaderText`	`connecting, reconnecting, thinking, component, forMoment`	Words for each moment a customer waits.
`LoaderStyle`	`kind, color, size, speed, text, skeleton`	How waiting looks, as chosen in Embed & Widgets → Appearance.
`LoaderLook`	`kind, color, px, periodMs, text, skeleton`	The effective loader: numbers a renderer can use directly.
`Loaders`	`KINDS, SIZES, SPEEDS, MOMENTS, DEFAULT_KIND = "dots", TEXT_MAX = 40, DEFAULT_TEXT, normalise, resolve`	
`Placement`	`DRAG_SLOP = 8.0, clampOffset, offsetDuringDrag, isTap`	Where the bar may sit — arithmetic only.

Type	Fields and members	What it is
`Placement.Bounds`	`windowHeight, baseBottom, barHeight, topMargin`	
`KeyboardLift`	`liftFor`	How far A BOTTOM-ANCHORED BAR MUST RISE TO STAY ABOVE THE KEYBOARD: the keyboard's height minus whatever the window already shrank.
`NativeAppearance`	`accent, heading, logoUrl, radius, logoPlate`	The dashboard's appearance, translated: see Appearance.resolve.
`LogoPlate`	`wire`	What the brand's logo sits on — appearance.logoPlate,
`Logo`	`PLATE_WHITE = "#FFFFFF", ON_WHITE = "#101318", CLASSIC_HAIRLINE = "#E5E7EB", box, plate`	The brand's logo, whole and on its plate — theme.ts's logoBox and logoPlateLook, for every place the logo stands in for the launcher glyph: the launcher, the voice bar's avatar, the chat header's mark.
`Logo.Stroke`	`width, color`	An outline: a width in the shape's units and a #RRGGBB colour.
`Logo.Plate`	`fill, outline`	The fill under a logo, and its outline — null for none.
`WidgetPalette`	`surface, border, text, muted, onAccent, mode`	The colours a component is drawn in, as #RRGGBB.
`Appearance`	`DEFAULT_ACCENT = "#318CE7", DEFAULT_ORIGIN = "https, resolve, isDark, palette, luminance, argb, tint, encodeURIComponent`	
`ThemeColors`	`background, surface, surfaceRaised, text, textMuted, border, accent, onAccent, accentSoft, success, warning, danger, userBubble, userBubbleText, agentBubble, agentBubbleText, scrim`	Every colour a surface is drawn in, as "#RRGGBB".
`FontScale`	`caption, body, title, display`	Type sizes, in sp.
`FontWeights`	`body, strong, title`	CSS weights, 100–900.
`FontPair`	`body, title`	One number for body text and one for titles: letter spacing in em, or line height as a multiplier.
`ThemeFont`	`body, heading, scale, weights, letterSpacing, lineHeight`	
`ThemeRadius`	`sm, md, lg, xl`	Corner radii, dp.
`ThemeShape`	`radius, borderWidth, launcher`	
`ThemeSpacing`	`unit, cardPadding, gap`	dp.
`ThemeShadow`	`y, blur, spread, opacity, color`	A soft shadow.
`ThemeOffset`	`x, y`	A solid offset shadow, drawn in colors.text.
`ThemeElevation`	`card, floating, hardOffset`	
`ThemeMotion`	`durationMs, easing, bounce`	
`LauncherGlyph`	`id, path, stroke`	A launcher glyph: 24×24 SVG path data, stroked or filled.
`ThemeIcons`	`style, stroke, launcher`	
`ThemeGlass`	`blur, surfaceOpacity`	Translucent surfaces: surface at surfaceOpacity, what is behind blurred by blur where the platform can.
`ThemeTokens`	`v, preset, scheme, colors, font, shape, spacing, elevation, motion, icons, glass, bubbles, chrome, chat, call, isDark, toJson`	Everything a renderer needs for one scheme, as flat numbers and "#RRGGBB" strings.
`DesignConfig`	`preset, accent, scheme, roundness, font, headingFont, density, shadow, motion, neutral, glass, bubbles, launcherIcon, launcherShape, iconStyle, chrome, chat, call, toJson`	The knobs the brand turned, as the session carries them beside the tokens.
`ResolvedDesign`	`config, light, dark, pick, toJson`	The session's design: the config and both token sets.

Type	Fields and members	What it is
`Design`	`DEFAULT_PRESET = "graine-classic", GLYPH_ASSET_VERSION = 2, LAUNCHER_SIZE = 56.0, LAUNCHER_GLYPH = 0.46, LAUNCHER_SHAPES, BUBBLES, EASINGS, ICON_STYLES, CHROMES, normaliseHex, glyphUrl, launcherCorner, motionMs, androidShadow, fontFamily, <T> fontFor, CLASSIC_LIGHT, CLASSIC_DARK`	The design's rules that are arithmetic: parsing, the launcher's shape, the glyph's address, motion.
`ChatTokens`	`title, subtitle, avatar, placeholder, showTimestamps, suggestions, background, pattern, patternColor, headerBackground, headerText, headerSubtext, composerBackground, composerText, composerPlaceholder, composerBorder, size, showUpcomingCalls, upcomingTitle, showHistory, newConversation`	The chat window's resolved tokens (one scheme).
`CallStatusText`	`ringing, connecting, ended, scheduled, missed, scheduledText, missedText`	What the call screen says while it rings, connects, and once it is over — and, when the server sends them, for a call Graine rang at a booked time (scheduled) and for a ring nobody answered (missed).
`CallTokens`	`background, backgroundTo, text, textMuted, avatar, avatarShape, avatarFill, avatarText, buttonFill, buttonIcon, buttonActiveFill, buttonActiveIcon, endFill, endIcon, endShape, acceptFill, acceptIcon, declineFill, declineIcon, callbackFill, callbackIcon, captions, timer, controls, ringtone, statusText`	The call screen's resolved tokens (one scheme): Android's full-screen ring and in-call screen.

12.6 Flutter — graine_inapp 0.7.0

Dart; everything `package:graine_inapp/graine_inapp.dart` exports. Requests to Graine throw `GraineException`; state objects are `ChangeNotifier`s and `ValueListenable`s; call them on the UI isolate.

GraineInApp

The in-app agent: one instance per publishable key, configured with `GraineInApp.init(config)` before `runApp` and reached afterwards as `GraineInApp.instance` (or `GraineInApp.named('support')` for an agent configured under a name). Accessing `instance` before `init` throws `StateError`. It is a `WidgetsBindingObserver`. Use it on the main (UI) isolate only.

Member	Returns	Errors	What it does
`static const String defaultName = 'default'`	`String`	—	The name of the default agent: the one `init` configures when given no name, which `instance` returns and every view draws unless handed another.
`static GraineInApp init(GraineConfig config, {String name = defaultName})`	`GraineInApp`	—	Configures an agent and starts fetching its configuration at once. Call it before `runApp` or in your first widget; it initialises the Flutter binding itself. Without a `name` it is the default agent. The name is trimmed, and an empty name is the default agent. A later call with a name already configured returns that agent unchanged (the new `config` is not applied).
`static GraineInApp? named(String name)`	`GraineInApp?`	—	The agent configured under `name` (trimmed; empty means the default agent), or null.
`static Map<String, GraineInApp> get all`	`Map<String, GraineInApp>`	—	Every configured agent, by name, in the order they were configured. Unmodifiable.
`static GraineInApp get instance`	`GraineInApp`	`StateError` before `init`	The default agent: the one configured without a name, or, in an app that named every agent, the first one configured.

Member	Returns	Errors	What it does
`static GraineInApp? get maybeInstance`	`GraineInApp?`	—	The default agent, or null before `init`. Use it to ask whether the SDK has been configured.
`@visibleForTesting static GraineInApp forTesting(GraineConfig config, {http.Client? httpClient, int Function()? clock, bool install = false, String name = defaultName, Directory? fontCacheDir, Future<void> Function(String name, Uint8List bytes)? fontRegistrar, GraineChatConnector? chatConnector})`	`GraineInApp`	—	For tests only: an instance with an injected HTTP client, clock, font cache directory, font registrar and chat connector, and no lifecycle observer. With `install: true` it joins the app's agents under `name`; otherwise it shares nothing with them.
`@visibleForTesting static void resetForTesting()`	nothing	—	For tests only: forgets every agent and what they share, disposing each.
`final GraineConfig config`	`GraineConfig`	—	The configuration this agent was initialised with.
`final String name`	`String`	—	The name this agent was configured under; `defaultName` for the default agent.
`late final GraineVoice voice`	`GraineVoice`	—	This agent's voice call. `GraineEngine` attaches it; `voice.start()` places a call.
`late final GraineComponents components`	`GraineComponents`	—	The components the agent draws (from the call or the chat), as state a `GraineWidgetView` renders.
`GraineChatSession get chat`	`GraineChatSession`	—	This agent's text chat, on its own connection; created on first use. `GraineAgentBar` and `GraineChat` open it while the customer is in it and close it when they leave; an app drawing its own chat calls `open()` and `close()` itself. Opening it ends any other agent's call and closes its chat.
`void addListener(GraineListener l)`	nothing	—	Adds a listener for this agent's events. It is kept until removed; adding the same object twice has no further effect. A listener that throws is logged and does not stop the others.
`void removeListener(GraineListener l)`	nothing	—	Removes a listener.
`void identify(Map<String, Object?>? traits, {bool? memory})`	nothing	—	Who is signed in. Traits become prompt variables, masked on the device, and land on the call record — except the one trait the agent's Identity setting names (`user_id` unless it names `email`, `phone` or a custom trait), which is kept raw and sent only where Graine needs it to recognise the user, never as a prompt variable or in the screen context. A placeholder (such as "guest", "null" or an already-masked "[EMAIL]"), null, a boolean or a fraction identifies nobody: that conversation is anonymous, with no memory. `memory: false` : this user's conversations are neither remembered nor recalled; the choice is kept on the device for this user, and when left out their last choice stands (on until they make one). Identity is fixed per conversation: a change reaches the next one. `null` signs out: Graine is told this device is no longer the user's, a chat open for them ends and its transcript is cleared, and later conversations are anonymous. A different user signing in without a sign-out first signs the previous one out the same way. One user for the whole app: every agent the app configured hears it (each keeps the trait its own Identity setting names), and the native call engine receives it too.

Member	Returns	Errors	What it does
`void setMemory(bool enabled)`	nothing	—	Whether the current user's conversations are remembered — the same as `identify(currentTraits, memory: enabled)`. Kept on the device for that user (for an anonymous user, for this run only). Applies to every agent and reaches the native call engine.
`Future<bool> memoryEnabled()`	`Future<bool>`	—	The current user's memory choice: true unless they turned it off. Waits for the agent's configuration and the choices kept on the device.
`Future<int> forgetMemory()`	`Future<int>` — how many memories were deleted	`GraineException`: identityRequired (checked on the device first), deviceLimit, rateLimited, unavailable, invalidKey, originRefused, network, unknown	Deletes everything this agent's memory holds about the identified user. Allowed only from a device that is theirs. Completes with the number deleted (0 when Graine does not say).
`Future<List<GraineMemory>> memories()`	`Future<List<GraineMemory>>`	`GraineException`: identityRequired (checked on the device first), deviceLimit, rateLimited, unavailable, invalidKey, originRefused, network, unknown	What this agent remembers about the identified user, already redacted, in Graine's order: newest first, at most 100. For a "what you remember about me" screen. Only from a device that is theirs.
`Future<String> get installId`	`Future<String>`	—	This install's id: a UUID made on the app's first launch and kept by the platform (the Keychain on iOS, surviving reinstalls; storage excluded from backups on Android). The same id the native call engine sends, and the same for every agent. Graine counts a user's devices by it; the agent never sees it. Where there is no platform side (a test, a desktop build) it is a random UUID kept in memory for the run.
`GraineSchedulingRules get schedulingRules`	`GraineSchedulingRules`	—	What the agent lets users book, and how Graine rings, from the agent's configuration. No request. Before the configuration loads, and from a server that says nothing, the defaults — which allow no booking.
`Future<GraineScheduledCall> scheduleCall({required DateTime at, String? reason})`	`Future<GraineScheduledCall>` — the booked call, status scheduled	`GraineException` — on the device: invalidRequest, schedulingOff, tooSoon, tooFar; from Graine: outsideWindow, tooManyPending, identityRequired, deviceLimit, invalidRequest, rateLimited, unavailable, invalidKey, originRefused, network; unknown (`codeName` "bad_response") when the booking is not confirmed	Books a call from the agent at `at` ("call me tomorrow at 6"). The agent rings this user's devices (with nobody identified, this install) through the app. `reason` is trimmed, at most 120 characters, kept with the call for the agent, and never put in the push that rings it. Checks made on the device before the booking request is sent: a reason over 120 characters (invalidRequest, checked first); then, once the agent's configuration has loaded, schedulingOff when users may not book, tooSoon before `schedulingRules.earliest(now)` and tooFar after `schedulingRules.latest(now)`, where now is the device clock. The time is sent as ISO-8601 with the device's offset. An open chat's Upcoming calls card shows the booking at once.

Member	Returns	Errors	What it does
`Future<List<GraineScheduledCall>> scheduledCalls()`	`Future<List<GraineScheduledCall>>`	`GraineException`: rateLimited, unavailable, invalidKey, originRefused, network, notFound (a server with no booked calls to list), unknown	This user's calls (with nobody identified, this install's), in Graine's order: upcoming first. Entries that are not calls are skipped.
`Future<GraineCallStatus> cancelScheduledCall(String id)`	`Future<GraineCallStatus>` — the call's status afterwards	`GraineException`: invalidRequest (an empty id, on the device), notFound (not this user's or install's), rateLimited, unavailable, invalidKey, originRefused, network, unknown	Cancels one of this user's booked calls. The id is trimmed. Completes with the status Graine reports afterwards — `GraineCallStatus.cancelled` when it does not say. An open chat's Upcoming calls card drops the call.
`void setVariables(Map<String, Object?>? values)`	nothing	—	Prompt variables from the app's own data, for this agent. Merged into what is there: null under a key removes it; null for the whole map clears them all. Values become strings (an integer never gains ".0"; maps and lists are dropped) and are masked on the device. They reach the next conversation, and a ready engine page receives them at once.
`Map<String, String> currentVariables()`	`Map<String, String>`	—	Every variable the agent will receive, merged in precedence order: the dashboard's defaults, then `GraineConfig.variables`, then `setVariables`, then the traits from `identify` (without the identifying trait once the configuration has loaded).
`void setScreen(GraineScreen? next)`	nothing	—	What the customer is looking at; null forgets it. Call it on every meaningful change. Values are masked on the device before they leave, and an unchanged screen is not resent. The first change goes out at once; a burst after it is coalesced into one more update at the end of a 400 ms window, so the agent's opening line is never a screen behind. The screen is the app's: every agent hears it and tells its own conversation while one is live. A new screen id starts a new visit for friction detection.
`void updateField(GraineField field)`	nothing	—	Replaces the current screen's field of the same name, or adds it after the others. Does nothing before a screen is set (the next `setScreen` carries the field itself) or for a field with an empty name.
`void removeField(String name)`	nothing	—	Removes a field from the current screen. Does nothing if no screen is set or the field was never reported.
`void clearScreen([String? screenId])`	nothing	—	Forgets the screen, but only if `screenId` (when given) is still the current screen's id. Call it from a screen's cleanup: a navigator keeps the old screen through its exit transition, and an unconditional `setScreen(null)` there would wipe the screen that has just replaced it.
`void track(String name, [Map<String, Object?> data = const <String, Object?>{}])`	nothing	—	A product event the agent is told about without being interrupted. It rides the next context update; the last ten events are kept. Every agent records it. An empty name is ignored.

Member	Returns	Errors	What it does
`void tap(String name)`	nothing	—	The customer tapped a control you name: recorded as "tap:<name>" in the recent events (the name cut to one line of at most 40 characters), so the agent knows what they just did. Context, not news: it goes out at the end of the 400 ms window. Every agent records it. An empty name is ignored.
`void captureTap({required double x, required double y, String? label})`	nothing	—	A tap anywhere in the app. `GraineEngine` reports every tap it sees; an app with its own gesture layer may call it too. On a control with an accessibility `label` it is recorded as "tap:<label>" (one line, at most 40 characters); elsewhere as "tap:screen" with `x` and `y` in logical pixels. Recorded only by agents with `GraineConfig.captureTaps` on that have a screen set, and at most two taps a second across the app. Context, not news: it rides the next update.
`void configureFriction(FrictionConfig? override)`	nothing	—	Replaces the app's layer of friction rules (the layer `GraineConfig.friction` set). Safe at any time. The rules in force become the defaults, then the dashboard's, then `override`.
`FrictionRules get frictionRules`	`FrictionRules`	—	The friction rules in force now: the defaults, the dashboard's, then the app's.
`void noteInteraction()`	nothing	—	The customer did something that is not a field edit — scrolled a list, opened a picker. Restarts every agent's idle clock. Field edits do not count.
`void Function() registerAction(String name, GraineActionHandler handler)`	`void Function()` — unregisters it	—	Something the agent can do in the app, for this agent. Declare the same action on the agent in the dashboard: a handler with no declaration is never offered, and a declaration with no handler is refused. The handler may answer synchronously or with a `Future` and must answer within six seconds, or the agent is told it was not done; returning null means ok, and a throw is answered as error. Registering a name again replaces its handler. The returned function unregisters it, if it is still the one registered.
`Map<String, List<String>> checkActions()`	`Map<String, List<String>>`	—	Compares what this build registered with what the agent declares: `usable` (both), `undeclared` (registered, not declared on the agent), `unhandled` (declared, not registered). Declarations come from the agent's configuration, so read it after the session has loaded; log it once at startup.
`Future<ActionResult> executeAction(String name, Map<String, Object?> args)`	`Future<ActionResult>`	—	Runs one registered action exactly as the agent would, against the six-second deadline, and reports `onActionRequested` and `onActionCompleted` to listeners. Always completes: an unknown name answers refused; a throw or a timeout answers error.
`Map<String, Object?>? get pendingWidget`	`Map<String, Object?>?`	—	The component the agent drew that is still waiting to be answered, as Graine sent it, so a view mounted later can still draw it. Cleared when it is answered or dismissed, and when a new call starts.
`void clearPendingWidget()`	nothing	—	Forgets the retained component.
`SessionConfig? get session`	`SessionConfig?`	—	What the dashboard says about this agent; null until it has loaded.
`ValueListenable<SessionConfig?> get sessionListenable`	`ValueListenable<SessionConfig?>`	—	The session, for a widget that rebuilds when it arrives.

Member	Returns	Errors	What it does
`Future<SessionConfig?> loadSession()`	`Future<SessionConfig?>`	—	Fetches the agent's configuration once; concurrent callers share one request, and a failure is retried by the next caller. Completes with null on failure, which is also reported through `GraineListener.onError`. `init` calls it.
`ResolvedDesign? get design`	`ResolvedDesign?`	—	The design the brand chose in the dashboard, with both token sets. Null until the session loads, and from a server that predates designs.
`ThemeTokens? get tokens`	`ThemeTokens?`	—	The tokens to draw with right now: the brand's fixed scheme, else the device's light or dark, re-picked when the device switches. Null without a design. For an app drawing its own UI; the SDK's widgets pick for themselves.
`ValueListenable<ThemeTokens?> get tokensListenable`	`ValueListenable<ThemeTokens?>`	—	The tokens, for a widget that rebuilds when they change.
`@override void didChangePlatformBrightness()`	nothing	—	Lifecycle callback: the device switched between light and dark, so the tokens are picked again. Called by Flutter; apps do not call it.
`ValueListenable<Map<String, String>> get fontFaces`	`ValueListenable<Map<String, String>>`	—	The design's typefaces the SDK has loaded: `family|weight` (the design's family at the weight it really has) → the family name the face is registered under, for example `"DM Sans|700"` → `"Graine-DMSans-700"`. Empty until one loads, and for "system", which is never fetched. To draw your own UI in the brand's typeface, set `fontFamily` to the registered name with `FontWeight.normal`: a loaded face is its weight.
`Future<void> loadDesignFonts(ResolvedDesign? design)`	`Future<void>`	—	Fetches and registers the faces `design` draws with. The SDK does this for the session's design itself; call it for a design you hand a widget directly. Families listed in `GraineConfig.fonts` are the app's own and are never fetched. Safe to call again: each face is fetched once.
`bool get foreground`	`bool`	—	Whether the app is in the foreground, as the lifecycle last reported.
`@override void didChangeAppLifecycleState(AppLifecycleState state)`	nothing	—	Lifecycle callback: `resumed` is the foreground; `inactive` is ignored (iOS reports it for the app switcher, a pulled-down notification and the microphone permission dialog); anything else is the background. Called by Flutter; apps do not call it.
`@visibleForTesting void setForegroundForTesting(bool foreground)`	nothing	—	For tests only: what the lifecycle callback calls.
`@visibleForTesting void dispose()`	nothing	—	For tests only: stops every timer, detaches the engine, closes the chat and removes the agent from the app's agents. An app keeps its agents for its lifetime.

GraineConfig

Everything the SDK needs up front, passed to `GraineInApp.init`. Immutable.

Field / case	Type	Default	Meaning
`const GraineConfig({required this.baseUrl, required this.publishableKey, this.variables = const <String, Object?>{}, this.transport, this.appVersion, this.endOnBackground = true, this.timing = const GraineTiming(), this.debug = false, this.friction, this.requestMicrophone, this.fonts = const <String>[], this.captureTaps = true, this.chatSize})`	—	—	The constructor.
`final String baseUrl`	`String`	required	`"https://www.graine.ai"` unless self-hosted.
`final String publishableKey`	`String`	required	From Agent → Embed & Widgets. Selects the agent. It is not a secret and cannot place a call on its own: add `app://` to that agent's Allowed domains, or every request is refused.
`final Map<String, Object?> variables`	`Map<String, Object?>`	empty	Prompt variables known at build time. The dashboard's defaults sit under them; `setVariables` and `identify` win over them. Unlike runtime variables and traits, the SDK does not mask these.
`final Transport? transport`	`Transport?`	null	Forces a transport. Null lets the dashboard's Call audio setting decide.
`final String? appVersion`	`String?`	null	Lands on the call record and in the screen context, to tell builds apart.
`final bool endOnBackground`	`bool`	`true`	Ends a live call, and hangs up a connected chat, once the app has been in the background for two seconds (a moment away, such as a permission dialog, does not count). The chat reconnects when the app returns, if the customer is still in it. You want it on.
`final GraineTiming timing`	`GraineTiming`	`GraineTiming()`	The page-load watchdog, retry ladder, end-call watchdog and "Still connecting…" threshold.
`final bool debug`	`bool`	`false`	Logs, prefixed "[Graine]", every message across the engine bridge both ways, the chat's frames and the engine page's console. Outgoing variables and screen context are logged by key names and screen id only; incoming messages are logged as received, cut to 600 characters. Turn it on in debug builds.
`final FrictionConfig? friction`	`FrictionConfig?`	null	This app's layer of friction rules, over the dashboard's. Most apps set nothing here.
`final Future<bool> Function()? requestMicrophone`	`Future<bool> Function()?`	null	Makes sure the app holds the microphone before a call opens; this package has no permission plugin of its own, so pass the one the app already uses (for example `() async => (await Permission.microphone.request()).isGranted`). Called by `GraineVoice.start()` — and so by the bar's microphone button and `GraineLauncher` — before the call is placed. `false`: no call is opened and `onMicDenied("NotAllowedError")` is reported. If it throws, the error is logged and the call is attempted anyway. Null: the SDK assumes the app asked already; if it did not, the engine reports the refusal.
`final List<String> fonts`	`List<String>`	empty	The font families the app bundles itself, by the family name its pubspec declares. The SDK fetches the brand's typeface by itself; a family listed here is the app's own copy, wins over the download and is never fetched. Empty: the SDK fetches every family the design uses, drawing the platform font until each arrives.

Field / case	Type	Default	Meaning
`final bool captureTaps`	`bool`	`true`	Tells the agent what the customer taps while a screen is set: a tap on a control with an accessibility label (`Semantics(label: …)`) as "tap:<label>", anywhere else as "tap:screen" with the position — in the recent events, at most two a second, masked like everything else. `GraineEngine` (given a child) sees the taps.
`final GraineChatSize? chatSize`	`GraineChatSize?`	`null`	How big the chat window opens, over the dashboard's *Window size*: `compact` above the bar, `large` a sheet over most of a phone (a panel on a tablet), `full` the whole screen. Null follows the dashboard; a size passed to `GraineChat.open` wins for that open.

GraineTiming

Load and connect timing, for an app used on very slow networks. Passed as `GraineConfig.timing`; all values in milliseconds.

Field / case	Type	Default	Meaning
`const GraineTiming({this.stallMs = 15000, this.bootMs = 15000, this.hardCapMs = 120000, this.retryDelaysMs = const <int>[1000, 2000, 4000, 8000, 15000, 15000], this.endWatchdogMs = 4000, this.slowConnectMs = 4000})`	—	—	The constructor.
`final int stallMs`	`int`	`15000`	Silence — nothing downloaded — before a page load is given up on and retried.
`final int bootMs`	`int`	`15000`	Downloaded but still not ready after this long: the page is broken, not slow.
`final int hardCapMs`	`int`	`120000`	The ceiling on one load attempt, however steadily it is arriving.
`final List<int> retryDelaysMs`	`List<int>`	1, 2, 4, 8, 15, 15 s	The whole retry ladder, each delay jittered by ±25%. Its length is the number of reloads before the SDK gives up and reports `GraineConnectivity.failed`. The chat's reconnect backoff uses the same ladder.
`final int endWatchdogMs`	`int`	`4000`	How long an end-call may go unconfirmed before the call is torn down locally.
`final int slowConnectMs`	`int`	`4000`	How long a connect the customer is waiting on runs before `slowConnect` turns true ("Still connecting…").

GraineListener

Everything an agent reports. An `abstract mixin class`: use it with `extends` or `with`; every method has an empty default, so override only what you use. Register it with `GraineInApp.addListener`. Delivered on the main isolate.

Member	Returns	Errors	What it does
`void onSessionReady(SessionConfig session) {}`	nothing	—	The agent's configuration has loaded.
`void onCallState(bool connected, bool connecting) {}`	nothing	—	The voice call connected, started connecting, or stopped. Also fires when a connect attempt ends because nothing could carry the call.
`void onCaption(Role role, String text, bool live) {}`	nothing	—	A line of the voice call: repeated with `live: true` as it fills in, then once settled. The chat's lines are in `GraineChatSession.turns`.
`void onWidget(Map<String, Object?> widget) {}`	nothing	—	The agent drew a component, in the call or the chat, as Graine sent it. `GraineComponents` already holds it for `GraineWidgetView`; an app drawing its own answers it with `GraineVoice.answerWidget` (call) or `GraineChatSession.answerWidget` (chat).
`void onWidgetPending(Map<String, Object?> hint) {}`	nothing	—	A component is on its way and its content is loading; the hint carries its `widget_id` and `kind` when Graine gives them.
`void onEnded() {}`	nothing	—	The voice call is over, however it ended (including a hang-up the engine never confirmed). Once per call.

Member	Returns	Errors	What it does
`void onMicDenied(String reason) {}`	nothing	—	The microphone was refused: "NotAllowedError" when `requestMicrophone` answered false (or the engine asked for it after that), otherwise the engine's reason, such as "NotFoundError".
`void onQuality(ConnectionQuality quality) {}`	nothing	—	One word for the connection; fires only when the word changes. `lost` when the network goes; `unknown` when a call ends.
`void onNetwork(bool online) {}`	nothing	—	The engine page's view of the network changed.
`void onReconnecting(bool active) {}`	nothing	—	The network interrupted the call and it is being re-established (`true`), or that is over (`false`). Not an end.
`void onTransport(Transport transport, Transport requested, String? reason, String? message) {}`	nothing	—	Which transport carries the call, which was asked for, and why they differ. With reason "not_configured", `message` is also reported to `onError` and held as `GraineVoice.error` .
`void onConnectivity(GraineConnectivity state, int attempt, String? reason) {}`	nothing	—	The engine page is loading, ready, retrying or has failed. `attempt` is the number of reloads so far; `reason` is a short word such as "stalled", "boot_timeout", "load_cap" or "session".
`void onActionRequested(String name) {}`	nothing	—	The agent asked the app to run an action; fires before the handler runs.
`void onActionCompleted(String name, ActionResult result) {}`	nothing	—	The answer the agent was given for that action.
`void onFriction(FrictionSignal signal) {}`	nothing	—	The customer looks stuck on a step (`stalled_on_step`): on one screen, with a field outstanding, through a stretch of a live conversation. The conversation is told at the same moment; this is for the app — badge a control, count it.
`void onAgentVolunteered(String text) {}`	nothing	—	In the chat, the agent spoke without being asked — its welcome, or something it noticed on screen; `text` is the turn's first words. Open the chat if it is closed.
`void onError(String message) {}`	nothing	—	Something your team must fix, for your logs and never shown to the customer: the agent's configuration could not load, a call was started with no `GraineEngine` for the agent, a call was started while a native call holds the line, the engine page could not load after every retry, or a transport is not configured.

GraineException

What a request to Graine fails with: a typed code, a sentence written for whoever has to fix it, and, with a rate limit, how long to wait. Implements `Exception` ; thrown by the memory and scheduling methods of `GraineInApp` .

Field / case	Type	Default	Meaning
`const GraineException(this.message, {this.code = GraineErrorCode.unknown, this.codeName, this.statusCode, this.retryAfterSeconds})`	—	—	The constructor; the SDK builds these.
`final String message`	`String`	required	The sentence Graine (or the SDK) gave, written for whoever has to fix it.
`final GraineErrorCode code`	`GraineErrorCode`	`GraineErrorCode.unknown`	Why — branch on this. `network` when Graine could not be reached; `unknown` for a code this build does not know (see `codeName`).
`final String? codeName`	`String?`	null	The code as Graine said it, including a newer one this build does not know. Null when it said none.
`final int? statusCode`	`int?`	null	The HTTP status of a refusal, when there was one.
`final int? retryAfterSeconds`	`int?`	null	With `rateLimited` : how long Graine asked the app to wait, from the `Retry-After` header.
`@override String toString()`	`String`	—	The message.

GraineErrorCode

Why Graine said no: the codes Graine sends, typed, plus `network` and `unknown` . When a refusal carries no code, the HTTP status decides: 400 invalidRequest, 401 invalidKey, 403 originRefused, 404 notFound, 429 rateLimited, 503

unavailable, anything else unknown.

Field / case	Type	Default	Meaning
`invalidKey('invalid_key')`	`GraineErrorCode`	—	The publishable key was rejected. Check it against Embed & Widgets in the dashboard.
`originRefused('origin_refused')`	`GraineErrorCode`	—	The agent does not accept connections from a mobile app: add `app://` to its Allowed domains.
`rateLimited('rate_limited')`	`GraineErrorCode`	—	Too many requests. Wait `retryAfterSeconds` when given.
`invalidRequest('invalid_request')`	`GraineErrorCode`	—	A field is missing or malformed. Also raised on the device for a booking reason over 120 characters and for an empty id to cancel.
`identityRequired('identity_required')`	`GraineErrorCode`	—	Nobody is identified. Raised on the device by `memories` and `forgetMemory`, and by Graine.
`deviceLimit('device_limit')`	`GraineErrorCode`	—	This install is not one of the user's devices.
`schedulingOff('scheduling_off')`	`GraineErrorCode`	—	The agent does not take bookings from the app. Checked on the device.
`tooSoon('too_soon')`	`GraineErrorCode`	—	Earlier than `schedulingRules.earliest(now)`. Checked on the device.
`tooFar('too_far')`	`GraineErrorCode`	—	Later than `schedulingRules.latest(now)`. Checked on the device.
`outsideWindow('outside_window')`	`GraineErrorCode`	—	Outside the agent's calling window.
`tooManyPending('too_many_pending')`	`GraineErrorCode`	—	The user already has the most calls waiting that the agent allows.
`notFound('not_found')`	`GraineErrorCode`	—	No such booking for this user or install.
`unavailable('unavailable')`	`GraineErrorCode`	—	Graine could not answer just now. Try again with backoff.
`network('network')`	`GraineErrorCode`	—	Graine could not be reached: offline, timed out, or the agent's configuration could not be loaded.
`unknown('unknown')`	`GraineErrorCode`	—	A code this build does not know (its word is in `GraineException.codeName`), or an unexpected answer.
`final String wire`	`String`	—	The word on the wire, as listed above.
`static GraineErrorCode fromWire(String? s)`	`GraineErrorCode`	—	The value for a wire word (trimmed, any case); `unknown` for anything else.

GraineVoice

`class GraineVoice extends ChangeNotifier implements GraineCallState, GraineCallFault, GraineConnectProgress` — The assistant bar's voice call: GraineInApp.instance.voice (a ChangeNotifier). GraineEngine attaches it once, above the navigator; the bar starts and ends calls itself.

Member	Returns	Errors	What it does
`GraineConfig get config`	`GraineConfig`	—	The configuration this call was set up with.
`GraineInApp get app`	`GraineInApp`	—	The instance this call belongs to.
`CallPhase get phase`	`CallPhase`	—	Where the call stands in one word — idle, connecting, reconnecting, thinking, speaking, listening — as the React Native launcher's phase.
`Transport? get transport`	`Transport?`	—	The transport that actually carries the call, once the page has said.
`bool get attached`	`bool`	—	Whether an engine page is attached (GraineEngine is mounted).
`void attach(GraineEnginePage page)`	`nothing`	—	Mount the engine: GraineEngine calls this with its WebView page; a test calls it with a fake.
`void detachPage(GraineEnginePage page)`	`nothing`	—	Detach page if it is the one attached — what GraineEngine does on unmount.

Member	Returns	Errors	What it does
`void detach()`	`nothing`	—	Remove the engine.
`void viewOpened()`	`nothing`	—	A UI showing this call came on screen — GraineAgentBar says so itself; a host drawing its own bar may too, paired with viewClosed.
`void viewClosed()`	`nothing`	—	A UI from viewOpened left the screen.
`void answerWidget(String widgetId, Map<String, Object?> data, String summary)`	`nothing`	—	The customer answered a component.
`void reportEvent(String name, {String? detail, Map<String, Object?> data = const <String, Object?>{}})`	`nothing`	—	Something happened that the agent may want to speak up about.

GraineCallState

`abstract class GraineCallState implements Listenable` — The call, as A UI SEES IT.

Member	Returns	Errors	What it does
`bool get connected`	`bool`	—	
`bool get connecting`	`bool`	—	
`bool get muted`	`bool`	—	
`bool get ready`	`bool`	—	The engine page said graine:ready: a start is an INVITE away.
`bool get reconnecting`	`bool`	—	The network interrupted the call and it is being re-established.
`bool? get online`	`bool?`	—	From the engine page's own view of the network; null before it said.
`ConnectionQuality get quality`	`ConnectionQuality`	—	
`GraineConnectivity get connectivity`	`GraineConnectivity`	—	
`List<Caption> get captions`	`List<Caption>`	—	What has been said so far, oldest first, folded (see Captions).
`Caption? get caption`	`Caption?`	—	The turn being said now, or the last one.
`bool get agentSpeaking`	`bool`	—	The agent is mid-sentence (its caption is live).
`bool get micDenied`	`bool`	—	The last start was refused the microphone.
`Map<String, Object?> get appearance`	`Map<String, Object?>`	—	The dashboard's Appearance for this agent, empty until the session loads.
`Future<void> start()`	`Future<void>`	—	
`void end()`	`nothing`	—	
`void setMuted(bool muted)`	`nothing`	—	
`void retry()`	`nothing`	—	

GraineConnectProgress

`abstract interface class GraineConnectProgress` — A CALL THAT SAYS WHERE A CONNECT IS — beside GraineCallState, as GraineCallFault is, so a host's own call state keeps compiling.

Member	Returns	Errors	What it does
`ConnectStage? get connectStage`	`ConnectStage?`	—	Where the call being placed is, as the engine reports it (graine:progress); null outside a connect.
`bool get slowConnect`	`bool`	—	A connect the customer is waiting on — a bar on screen while the page loads, or a call being placed — has run past timing.slowConnectMs.
`bool get callFailed`	`bool`	—	The last start could not be connected at all — not on WebRTC, not on the fallback.

GraineChatState

`abstract class GraineChatState implements Listenable` — A CHAT WITH THE AGENT, as the bar sees it.

Member	Returns	Errors	What it does
`List<Caption> get messages`	`List<Caption>`	—	The conversation so far, oldest first; live while a line is still arriving.

Member	Returns	Errors	What it does
`bool get connected`	`bool`	—	The chat is connected: its header says "Online".
`void send(String text)`	`nothing`	—	The customer typed text and sent it.

GraineComponents

`class GraineComponents extends ChangeNotifier` — The components the agent draws, as state a GraineWidgetView renders: GraineInApp.instance.components.

Member	Returns	Errors	What it does
`GraineComponents({ required String baseUrl, this.onAnswer, this.onAction, this.onReport, int Function()? clock})`	`constructor`	—	
`final void Function(String widgetId, Map<String, Object?> data, String summary)`	`nothing`	—	The customer answered: send it to the agent (the SDK: to the call or the chat, whichever drew it).
`final Future<ActionResult> Function(String name, Map<String, Object?> args)`	`Future<ActionResult>`	—	Run a component's in-app action (the SDK: GraineInApp.executeAction).
`final void Function(String name, String detail, Map<String, Object?> data)`	`nothing`	—	Tell the agent something quietly (the SDK: the live call and chat).
`GraineInApp? get app`	`GraineInApp?`	—	The agent whose components these are; null for a standalone one.
`bool get fromChat`	`bool`	—	The component on screen arrived in the typed chat, not on a call: nothing is listening, so its view says nothing about listening.
`GraineWidget? get widget`	`GraineWidget?`	—	The component on screen, or null.
`WidgetContent? get content`	`WidgetContent?`	—	What widget reduces to — rows, chips, days — and the answer each tap sends.
`WidgetLook? get look`	`WidgetLook?`	—	How widget asked to look.
`WidgetPending? get loading`	`WidgetPending?`	—	A component announced and still loading, or null.
`WidgetAnswered? get answered`	`WidgetAnswered?`	—	The last answer, kept on screen as one line.
`bool get awaitingReply`	`bool`	—	The customer answered and the agent has not started replying.
`GraineWidget? get closed`	`GraineWidget?`	—	The last component closed without an answer — for a "show it again".
`void present(Map<String, Object?> frame)`	`nothing`	—	Draw a frame received some other way — a preview, a test — exactly as if the agent had sent it.
`void presentPending(Map<String, Object?> hint)`	`nothing`	—	Show the loader for a component on its way, as if the agent announced it.
`void answer(WidgetAnswer a)`	`nothing`	—	The customer answered.
`void dismiss()`	`nothing`	—	Close without choosing (or "Clear" the answered line).
`void reopen()`	`nothing`	—	Put the last closed component back, exactly as it was.
`void reset()`	`nothing`	—	Forget everything drawn (a call ended, or a new one began).

GraineChatSession

`class GraineChatSession extends ChangeNotifier implements GraineChatState, GraineChatConnection, GraineChatTyping` — The typed conversation: GraineInApp.instance.chat (a ChangeNotifier). The bar's chat window and GraineChat open it while the customer is in it.

Member	Returns	Errors	What it does
`GraineInApp get app`	`GraineInApp`	—	The agent this chat is with.

Member	Returns	Errors	What it does
`List<GraineChatTurn> get turns`	`List<GraineChatTurn>`	—	The conversation so far, oldest first, whether an agent turn was volunteered, and the component it drew.
`bool get failed`	`bool`	—	Every reconnect was spent: the chat stays offline until the customer sends something, or open is called again.
`String? get error`	`String?`	—	Why the last connect failed, for a log; null once connected.
`bool get isOpen`	`bool`	—	The customer is in the chat (open was called and close was not).
`List<GraineScheduledCall> get upcomingCalls`	`List<GraineScheduledCall>`	—	The user's booked calls still to ring, soonest first — what the chat window's Upcoming calls card shows (it draws the first three).
`GraineUpcomingCancel cancelStateOf(String id)`	`GraineUpcomingCancel`	—	Where a booked call's Cancel is.
`Future<void> refreshUpcoming()`	`Future<void>`	—	Read the booked calls again now.
`Future<void> cancelUpcoming(String id)`	`Future<void>`	—	The card's Cancel: "Cancelling…" until Graine answers, then the row goes; on a failure the row says so, and a tap tries again.
`void answerWidget(String widgetId, Map<String, Object?> data, String summary)`	`nothing`	—	Answer a component the agent drew in the chat: the structured answer, then summary as the customer's turn — without it the conversation does not move.
`void clear()`	`nothing`	—	Forget the conversation on screen (a "new chat").
`Future<void> newConversation()`	`Future<void>`	—	START A NEW CONVERSATION — a fresh thread, as a support chat offers one.

GraineChatTurn

`class GraineChatTurn` — One turn of the chat

Member	Returns	Errors	What it does
`const GraineChatTurn({ required this.id, required this.role, required this.text, required this.at, this.delivery, this.proactive = false, this.component, this.live = false, this.restored = false, this.channel = GraineChatChannel.chat})`	`constructor`	—	
`final String id`	`String`	—	This chat's id for the turn: t1, t2, … (a restored turn: Graine's)
`final GraineChatRole role`	`GraineChatRole`	—	
`final String text`	`String`	—	What was said.
`final DateTime at`	`DateTime`	—	When it was said (an agent turn: its first words).
`final GraineChatDelivery? delivery`	`GraineChatDelivery?`	—	A user's turn only: sending, sent or failed.
`final bool proactive`	`bool`	—	An agent's turn nobody asked for: its welcome, or something it noticed.
`final GraineComponent? component`	`GraineComponent?`	—	The component the agent drew with this turn, if any.
`final bool live`	`bool`	—	An agent turn still arriving.
`final bool restored`	`bool`	—	From an earlier conversation, restored when the chat opened (the user came back): delivered, with the time it was said, not said just now.
`final GraineChatChannel channel`	`GraineChatChannel`	—	Where it was said.
`Caption get caption`	`Caption`	—	The turn as a caption — what a Caption-drawing window shows.

Member	Returns	Errors	What it does
`GraineChatTurn copyWith({String? text, bool? live, GraineChatDelivery? delivery, GraineComponent? component})`	`GraineChatTurn`	—	

GraineChatRole

`enum GraineChatRole` — Who said a turn of the chat.

GraineChatDelivery

`enum GraineChatDelivery` — Where a user's turn is: typed and waiting to go out, on the wire, or not sent (the chat could not connect; it goes when the chat does).

GraineChatChannel

`enum GraineChatChannel` — Where a turn was said: typed in the chat, or spoken on a call (only an earlier conversation's turn is).

GraineCalls

`class GraineCalls extends ChangeNotifier implements ValueListenable<GraineCallSnapshot>` — Native calls: GraineCalls.instance, a ValueListenable<GraineCallSnapshot>. Configure once at launch; hand pushes to it; startCall from a button.

Member	Returns	Errors	What it does
`static final GraineCalls instance`	`GraineCalls`	—	The one instance.
`bool get isSupported`	`bool`	—	Native calls run on Android and iOS.
`Stream<GraineCallOutcome> get outcomes`	`Stream<GraineCallOutcome>`	—	What became of every call — tell your server.
`Stream<GraineIncomingCall> get incomingCalls`	`Stream<GraineIncomingCall>`	—	An agent's call started ringing — for your own ringing UI or analytics.
`Future<void> configure(GraineCallsOptions options)`	`Future<void>`	—	Once, at launch, before a push can arrive.
`Future<void> registerPushToken(String token)`	`Future<void>`	—	Hosted ringing on android: the token your Firebase messaging hands the app — call it from FirebaseMessaging.instance.getToken() and onTokenRefresh.
`void Function() enableVoIPPushes(void Function(String? token) onToken)`	`void Function()`	—	iOS: receive your agent's calls as VoIP pushes.
`Future<bool> handlePush(Map<String, Object?> data)`	`Future<bool>`	—	Android: a push's data, from your Firebase message handlers — foreground and background.
`Future<String?> startCall({String? reason, Map<String, Object?> variables = const <String, Object?>{}, String? agent})`	`Future<String?>`	—	The customer calls the agent — agent, by the name the app configured it under (GraineInApp.init); the default agent without one.
`Future<void> answer()`	`Future<void>`	—	Answer the ringing call from your own screen.
`Future<void> decline()`	`Future<void>`	—	Decline the ringing call.
`Future<void> end()`	`Future<void>`	—	Hang up — or decline, while it rings.
`Future<void> setMuted(bool muted)`	`Future<void>`	—	Stop sending the customer's voice without ending the call.
`Future<void> setSpeaker(bool on)`	`Future<void>`	—	The loudspeaker (true) or the earpiece / headset (false).

Member	Returns	Errors	What it does
`void Function() attach([GraineInApp? graine])`	`void Function()`	—	Hand the call what the app already tells the agent through graine — who the customer is, the prompt variables, the screen and the actions the app registered — kept in step as they change, and answer the agent's actions through the app's own handlers.
`void resetForTesting()`	`nothing`	—	For tests: forget everything this instance holds.

GraineCallsOptions

`class GraineCallsOptions` — How calls ring and what they show; kept natively so a push that wakes a closed app rings before Dart runs.

Member	Returns	Errors	What it does
`const GraineCallsOptions({ required this.publishableKey, required this.displayName, this.baseUrl = 'https://www.graine.ai', this.ringtoneSound, this.ringTimeout = const Duration(seconds: 30), this.appVersion, this.callScreen = GraineCallScreen.sdk, this.debug = false, this.androidSmallIcon, this.iosIconTemplate, this.iosIncludesCallsInRecents = false})`	`constructor`	—	
`final String publishableKey`	`String`	—	Agent → Embed & Widgets.
`final String displayName`	`String`	—	The name on the calls the customer places, and on a ring whose push names nobody.
`final String baseUrl`	`String`	—	
`final String? ringtoneSound`	`String?`	—	iOS: a sound file of 30 seconds or less in the app bundle.
`final Duration ringTimeout`	`Duration`	—	How long an agent's call rings before it is missed.
`final String? appVersion`	`String?`	—	Which build placed the call; it lands on the call record.
`final GraineCallScreen callScreen`	`GraineCallScreen`	—	Who draws the call while the app is open.
`final bool debug`	`bool`	—	The native SDK's debug log, in Xcode's console or logcat: how a call connected and what its audio did, for finding out why one did not.
`final String? androidSmallIcon`	`String?`	—	Android: a monochrome drawable for the call notifications, by name.
`final String? iosIconTemplate`	`String?`	—	iOS: a template image in the app bundle, by name, for the system call screen.
`final bool iosIncludesCallsInRecents`	`bool`	—	iOS: list the calls in the Phone app's Recents.
`Map<String, Object?> toMap()`	`Map<String, Object?>`	—	

GraineCallScreen

`enum GraineCallScreen` — Who draws the call while the app is open: the SDK's call screen, or yours.

GraineCallSnapshot

`class GraineCallSnapshot` — The call to draw, as GraineCalls.instance.value.

Member	Returns	Errors	What it does
`const GraineCallSnapshot({ this.status = GraineNativeCallStatus.idle, this.callId, this.direction, this.reason, this.callerName, this.muted = false, this.speakerOn = false, this.agentSpeaking = false, this.caption, this.connectedAt, this.error, this.agent})`	`constructor`	—	
`static const GraineCallSnapshot idle`	`GraineCallSnapshot`	—	No call.
`factory GraineCallSnapshot.fromMap(Map<Object?, Object?>? m)`	`constructor`	—	From the engine's snapshot.
`final GraineNativeCallStatus status`	`GraineNativeCallStatus`	—	
`final String? callId`	`String?`	—	Your server's id for a call it rang; a generated one for a call the customer placed.
`final GraineCallDirection? direction`	`GraineCallDirection?`	—	
`final String? reason`	`String?`	—	The push's reason, or the one startCall was given.
`final String? callerName`	`String?`	—	The name the call shows.
`final bool muted`	`bool`	—	
`final bool speakerOn`	`bool`	—	Audio out of the loudspeaker rather than the earpiece or a headset.
`final bool agentSpeaking`	`bool`	—	The agent's voice is playing right now.
`final GraineCallCaption? caption`	`GraineCallCaption?`	—	The latest line: the agent's words as they are said, or the customer's as they are heard.
`final DateTime? connectedAt`	`DateTime?`	—	When the conversation opened — for a call timer.
`final String? error`	`String?`	—	Why the last call could not connect.
`final String? agent`	`String?`	—	The agent the call is with, by the name the app configured it under (GraineInApp.init; 'default' for the default agent) — the one GraineCalls.startCall named, or the one a ring was routed to by its push.

GraineNativeCallStatus

`enum GraineNativeCallStatus` — Where a native call is.

Case	Meaning
`idle('idle')`	
`ringing('ringing')`	
`connecting('connecting')`	
`active('active')`	
`ended('ended')`	

Member	Returns	Errors	What it does
`const GraineNativeCallStatus(this.wire)`	`constructor`	—	
`final String wire`	`String`	—	
`bool get isLive`	`bool`	—	Ringing, connecting or connected.
`static GraineNativeCallStatus fromWire(Object? s)`	`GraineNativeCallStatus`	—	

GraineCallDirection

`enum GraineCallDirection` — Who placed the call: the agent (a push rang the phone) or the customer.

GraineCallCaption

`class GraineCallCaption` — The line being said on a native call.

Member	Returns	Errors	What it does
`const GraineCallCaption(this.role, this.text, {this.live = false})`	`constructor`	—	
`final Role role`	`Role`	—	
`final String text`	`String`	—	
`final bool live`	`bool`	—	Still being said.

GraineCallOutcome

`class GraineCallOutcome` — What became of A CALL

Member	Returns	Errors	What it does
`const GraineCallOutcome(this.callId, this.outcome, {this.reason, this.durationSeconds, this.hosted = false, this.agent})`	`constructor`	—	
`factory GraineCallOutcome.fromMap(Map<Object?, Object?> m)`	`constructor`	—	
`final String callId`	`String`	—	
`final GraineCallOutcomeKind outcome`	`GraineCallOutcomeKind`	—	
`final String? reason`	`String?`	—	With ended: local (this app hung up), remote (the agent did) or failed (the connection broke mid-call).
`final int? durationSeconds`	`int?`	—	How long the conversation lasted, when the engine says.
`final bool hosted`	`bool`	—	Graine rang this call (hosted ringing); the SDK has already told Graine what became of it.
`final String? agent`	`String?`	—	The agent the call was with, by the name the app configured it under ('default' for the default agent); null when the engine does not know.
`String get wire`	`String`	—	One word for your server: ringing · answered · declined · missed · ended · failed.
`GraineCallOutcomeKind get kind`	`GraineCallOutcomeKind`	—	outcome, by its 0.6 name.
`String? get endReason`	`String?`	—	reason of an ended call, by its 0.6 name.
`String? get error`	`String?`	—	reason of a failed call, by its 0.6 name.

GraineCallOutcomeKind

`enum GraineCallOutcomeKind` — What became of a call.

GraineIncomingCall

`class GraineIncomingCall` — An agent's call ringing on this device.

Member	Returns	Errors	What it does
`const GraineIncomingCall(this.callId, {this.reason, this.callerName, this.hosted = false, this.agent})`	`constructor`	—	
`final String callId`	`String`	—	
`final String? reason`	`String?`	—	
`final String? callerName`	`String?`	—	
`final bool hosted`	`bool`	—	Graine rang it (hosted ringing) rather than your own server's push.

Member	Returns	Errors	What it does
`final String? agent`	`String?`	—	The agent ringing, by the name the app configured it under ('default' for the default agent): the native engine matches the push's agent_id to a configured agent's — the default agent when none matches.

GraineScheduledCall

`class GraineScheduledCall` — A booked call, returned by scheduleCall and scheduledCalls.

Member	Returns	Errors	What it does
`const GraineScheduledCall({ required this.id, this.at, this.status = GraineCallStatus.scheduled, String? statusName, this.source, this.sourceName, this.reason})`	`constructor`	—	
`final String id`	`String`	—	Graine's id for the booking — what GraineScheduledCalls are cancelled by.
`final DateTime? at`	`DateTime?`	—	When it rings, in UTC (.toLocal() for the user's clock); null only when the server's time was unreadable.
`final GraineCallStatus status`	`GraineCallStatus`	—	
`final String statusName`	`String`	—	The status as the server said it — with GraineCallStatus.unknown, the word this build does not know.
`final GraineCallSource? source`	`GraineCallSource?`	—	Who booked it; null when the server does not say.
`final String? sourceName`	`String?`	—	The source as the server said it (sdk, agent, ...), or null.
`final String? reason`	`String?`	—	What it is about, as booked.
`bool get isUpcoming`	`bool`	—	Still to come: scheduled or ringing.
`bool get canCancel`	`bool`	—	It can still be called off: scheduled.
`String get statusWire`	`String`	—	The status word: statusName when the server gave one, else status's.
`static GraineScheduledCall? fromJson(Object? raw)`	`GraineScheduledCall?`	—	Null for anything that is not a call with an id.
`Map<String, Object?> toJson()`	`Map<String, Object?>`	—	

GraineCallStatus

`enum GraineCallStatus` — Where a booked call stands (wire: no_device is noDevice).

Case	Meaning
`scheduled('scheduled')`	Booked, waiting for its time.
`ringing('ringing')`	The device is ringing now.
`answered('answered')`	
`completed('completed')`	
`declined('declined')`	
`missed('missed')`	Rang out with no answer.
`failed('failed')`	
`cancelled('cancelled')`	
`expired('expired')`	Its time passed while it could not ring (outside the calling hours, or held too long).
`noDevice('no_device')`	No device of the user's could be rung.
`unknown('unknown')`	A status this build does not know — the call's statusName says it.

Member	Returns	Errors	What it does
`const GraineCallStatus(this.wire)`	`constructor`	—	

Member	Returns	Errors	What it does
`final String wire`	`String`	—	The word on the wire.
`static GraineCallStatus fromWire(Object? s)`	`GraineCallStatus`	—	
`bool get upcoming`	`bool`	—	Still to come: it will ring, or is ringing.

GraineCallSource

`enum GraineCallSource` — Who booked a call (wire: sdk is app — this app).

Case	Meaning
`agent('agent')`	The agent, in a conversation.
`app('sdk')`	The app, through this SDK.
`dashboard('dashboard')`	
`api('api')`	
`unknown('unknown')`	A source this build does not know — the call's sourceName says it.

Member	Returns	Errors	What it does
`const GraineCallSource(this.wire)`	`constructor`	—	
`final String wire`	`String`	—	The word on the wire.
`static GraineCallSource? fromWire(Object? s)`	`GraineCallSource?`	—	Null for no source at all.

GraineSchedulingRules

`class GraineSchedulingRules` — The booking rules, read from the agent's settings with the session (GraineInApp.instance.schedulingRules). Defaults before they load, with booking off.

Member	Returns	Errors	What it does
`const GraineSchedulingRules({ this.usersCanBook = false, this.minLeadMinutes = 5, this.maxDaysAhead = 14, this.maxPendingPerUser = 3, this.ringTimeoutSeconds = 45, this.callName, this.hostedRinging = false, this.platforms = const <String>[]})`	`constructor`	—	
`factory GraineSchedulingRules.of(CallsSetting? calls)`	`constructor`	—	The rules of calls (the session's calls; null: an older server).
`final bool usersCanBook`	`bool`	—	scheduleCall is allowed at all.
`final int minLeadMinutes`	`int`	—	
`final int maxDaysAhead`	`int`	—	
`final int maxPendingPerUser`	`int`	—	
`final int ringTimeoutSeconds`	`int`	—	How long a ring lasts before it is missed.
`final String? callName`	`String?`	—	What the phone shows for the agent's calls; null — the agent's name.
`final bool hostedRinging`	`bool`	—	The workspace has Graine ring its users' devices.
`final List<String> platforms`	`List<String>`	—	Where it rings: ios, android.
`DateTime earliest(DateTime now)`	`DateTime`	—	The soonest a call can be booked for: now + minLeadMinutes.
`DateTime latest(DateTime now)`	`DateTime`	—	The latest a call can be booked for: now + maxDaysAhead days.

GraineMemory

`class GraineMemory` — One thing the agent remembers about the signed-in customer, already redacted.

Member	Returns	Errors	What it does
`const GraineMemory({required this.id, required this.text, this.createdAt, this.updatedAt})`	`constructor`	—	
`final String id`	`String`	—	
`final String text`	`String`	—	What is remembered, in a sentence.
`final DateTime? createdAt`	`DateTime?`	—	When it was first remembered, in UTC; null when the server does not say.
`final DateTime? updatedAt`	`DateTime?`	—	When it last changed, in UTC; null when the server does not say.
`static GraineMemory? fromJson(Object? raw)`	`GraineMemory?`	—	Null for anything that is not a memory with an id and some words.
`static List<GraineMemory> listFrom(Object? body)`	`List<GraineMemory>`	—	{memories: ...} (or a bare list), in the server's order — newest first.
`Map<String, Object?> toJson()`	`Map<String, Object?>`	—	

GrainеScreen

`class GraineScreen` — What the customer is looking at.

Member	Returns	Errors	What it does
`const GraineScreen(this.screen, { this.title, this.journey, this.fields = const <GraineField>[], this.error, this.data = const <String, Object?>{}, this.availableActions})`	`constructor`	—	
`final String screen`	`String`	—	Stable id for the screen — used for analytics, so keep it constant.
`final String? title`	`String?`	—	
`final GraineJourney? journey`	`GraineJourney?`	—	
`final List<GraineField> fields`	`List<GraineField>`	—	
`final String? error`	`String?`	—	A screen-level error banner, distinct from a per-field one.
`final Map<String, Object?> data`	`Map<String, Object?>`	—	Anything else the agent should know.
`final List<String>? availableActions`	`List<String>?`	—	Which actions work HERE.
`GraineScreen copyWith({List<GraineField>? fields})`	`GraineScreen`	—	This screen with fields in place of its own.
`GraineScreen withField(GraineField field)`	`GraineScreen`	—	This screen with field in place of the field of the same name — or added, after the others, when it has none.
`GraineScreen withoutField(String name)`	`GraineScreen`	—	This screen without the field named name (itself when it has none).

GraineJourney

`class GraineJourney` — Where a screen sits in a flow, and where the flow goes next.

Member	Returns	Errors	What it does
`const GraineJourney(this.name, {this.step, this.of, this.next})`	`constructor`	—	
`final String name`	`String`	—	
`final int? step`	`int?`	—	
`final int? of`	`int?`	—	

Member	Returns	Errors	What it does
`final String? next`	`String?`	—	What happens after this step — asked, the agent repeats it rather than inventing one.

GraineField

`class GraineField` — One input on a screen, as the agent should understand it.

Member	Returns	Errors	What it does
`const GraineField(this.name, {this.label, this.value, this.status, this.error, this.optional = false})`	`constructor`	—	
`final String name`	`String`	—	
`final String? label`	`String?`	—	
`final String? value`	`String?`	—	The real value.
`final FieldStatus? status`	`FieldStatus?`	—	
`final String? error`	`String?`	—	The validation message actually shown — what the agent can offer to fix.
`final bool optional`	`bool`	—	Leaving this one blank is fine: kept in the frame, out of the count.

FieldStatus

`enum FieldStatus` — How an input on a screen is doing.

Case	Meaning
`empty('empty')`	
`filled('filled')`	
`invalid('invalid')`	
`pending('pending')`	
`locked('locked')`	

Member	Returns	Errors	What it does
`const FieldStatus(this.wire)`	`constructor`	—	
`final String wire`	`String`	—	
`static FieldStatus? fromWire(String? s)`	`FieldStatus?`	—	

ActionResult

`class ActionResult` — What an action handler answers.

Member	Returns	Errors	What it does
`const ActionResult(this.status, {this.message = '', this.data = const <String, Object?>{}})`	`constructor`	—	
`const ActionResult.ok([this.message = '', this.data = const <String, Object?>{}])`	`constructor`	—	
`const ActionResult.refused(this.message)`	`constructor`	—	
`const ActionResult.error(this.message)`	`constructor`	—	
`final ActionStatus status`	`ActionStatus`	—	
`final String message`	`String`	—	
`final Map<String, Object?> data`	`Map<String, Object?>`	—	

ActionStatus

`enum ActionStatus` — What an action handler's answer means to the agent.

Case	Meaning
`ok('ok')`	
`refused('refused')`	
`error('error')`	

Member	Returns	Errors	What it does
`const ActionStatus(this.wire)`	`constructor`	—	
`final String wire`	`String`	—	

GraineActionHandler

`typedef GraineActionHandler = FutureOr<ActionResult?> Function( Map<String, Object?> args);` — A handler for one thing the agent may do in the app.

Masking

`abstract final class Masking` — The masking applied before anything leaves the device. Built in: PAN, Aadhaar, card and account numbers, phone numbers, email addresses.

Member	Returns	Errors	What it does
`static String mask(String input)`	`String`	—	Replace identifiers in one string: the five built-in rules, then the app's own (addRule).
`static void addRule(RegExp pattern, [String token = '[REDACTED]'])`	`nothing`	—	A rule for a field shape only your app knows — an account number, a policy id.
`static void clearCustomRules()`	`nothing`	—	Only for tests — production code registers rules once, at startup.
`static Object? deep(Object? value)`	`Object?`	—	Masks every string anywhere inside a JSON value.
`static String expandReplacement(String token, Match m)`	`String`	—	JavaScript's GetSubstitution: what String.prototype.replace does with $ in a replacement string, so a custom rule written for the RN SDK means the same thing here.

SessionConfig

`class SessionConfig`

Member	Returns	Errors	What it does
`const SessionConfig({ required this.agentId, required this.webcallAgentId, this.appearance = const <String, Object?>{}, this.variables = const <String, Object?>{}, this.pageVersion, this.declaredActions = const <String>{}, this.friction, this.design, this.wsHost, this.identity, this.memory, this.calls})`	`constructor`	—	
`final String agentId`	`String`	—	
`final String webcallAgentId`	`String`	—	The agent the call is filed under.
`final Map<String, Object?> appearance`	`Map<String, Object?>`	—	Embed & Widgets → Appearance, exactly as stored.
`final Map<String, Object?> variables`	`Map<String, Object?>`	—	The dashboard's variable defaults.
`final String? pageVersion`	`String?`	—	The deployment id: the engine page's cache key.
`final Set<String> declaredActions`	`Set<String>`	—	The in-app actions the dashboard declares.
`final Map<String, Object?>? friction`	`Map<String, Object?>?`	—	The dashboard's friction rules, as served.

Member	Returns	Errors	What it does
`final ResolvedDesign? design`	`ResolvedDesign?`	—	The design the brand chose, resolved by the server into light and dark tokens.
`final String? wsHost`	`String?`	—	Where the chat socket goes when there is no ticket.
`final IdentitySetting? identity`	`IdentitySetting?`	—	Which trait identifies a user.
`final MemorySetting? memory`	`MemorySetting?`	—	The agent's memory switch; null from an older server.
`final CallsSetting? calls`	`CallsSetting?`	—	Hosted ringing and what users may book; null from an older server.
`IdentitySetting get identitySetting`	`IdentitySetting`	—	identity, or what a server that does not say means.
`Transport get preferredTransport`	`Transport`	—	The dashboard's Call audio setting.
`String? get accent`	`String?`	—	The dashboard's accent, as stored (it may lack its "#").
`static SessionConfig fromJson(Map<String, Object?> data)`	`SessionConfig`	—	The session response, parsed exactly as the Android SDK parses it.
`static String? errorFor(int status, Map<String, Object?> body)`	`String?`	—	The sentence for a refused session request — written for whoever has to fix it — or null for a 2xx.

IdentitySetting

`class IdentitySetting` — The agent's identity setting, as the session serves it.

Member	Returns	Errors	What it does
`const IdentitySetting({this.field = Identity.defaultField, this.devicesPerUser = 2})`	`constructor`	—	
`final String field`	`String`	—	Which trait identifies a user: user_id, email, phone or a custom name.
`final int devicesPerUser`	`int`	—	How many devices one user may hold at a time (1–5).
`static const IdentitySetting standard`	`IdentitySetting`	—	What an older server, which says nothing, means: the user_id trait.
`static IdentitySetting? fromJson(Object? raw)`	`IdentitySetting?`	—	Null when raw is not an object — the server did not say.

MemorySetting

`class MemorySetting` — The agent's memory switch, as the session serves it.

Member	Returns	Errors	What it does
`const MemorySetting({this.enabled = false})`	`constructor`	—	
`final bool enabled`	`bool`	—	The agent remembers identified users between conversations.
`static MemorySetting? fromJson(Object? raw)`	`MemorySetting?`	—	

SchedulingSetting

`class SchedulingSetting` — What users may book from the app (Agent → Calls & scheduling).

Member	Returns	Errors	What it does
`const SchedulingSetting({ this.usersCanBook = false, this.maxDaysAhead = 14, this.minLeadMinutes = 5, this.maxPendingPerUser = 3})`	`constructor`	—	
`final bool usersCanBook`	`bool`	—	scheduleCall is allowed at all.
`final int maxDaysAhead`	`int`	—	
`final int minLeadMinutes`	`int`	—	
`final int maxPendingPerUser`	`int`	—	
`static SchedulingSetting? fromJson(Object? raw)`	`SchedulingSetting?`	—	

CallsSetting

`class CallsSetting` — Calls Graine rings on this workspace's apps (hosted ringing).

Member	Returns	Errors	What it does
`const CallsSetting({ this.hosted = false, this.platforms = const <String>[], this.ringTimeoutSeconds = 45, this.callName, this.scheduling})`	`constructor`	—	
`final bool hosted`	`bool`	—	The workspace uploaded push credentials: Graine rings its users' devices.
`final List<String> platforms`	`List<String>`	—	Where it can ring: ios, android.
`final int ringTimeoutSeconds`	`int`	—	
`final String? callName`	`String?`	—	What the phone shows for the agent's calls; null — the agent's display name.
`final SchedulingSetting? scheduling`	`SchedulingSetting?`	—	
`static CallsSetting? fromJson(Object? raw)`	`CallsSetting?`	—	

GraineConnectivity

`enum GraineConnectivity` — Whether the page that carries the call is actually there.

Transport

`enum Transport` — How the call's audio travels.

Case	Meaning
`websocket('ws')`	Raw PCM on the conversation socket.
`webrtc('rtc')`	Opus over WebRTC with a jitter buffer and the device's echo canceller.

Member	Returns	Errors	What it does
`const Transport(this.wire)`	`constructor`	—	
`final String wire`	`String`	—	The word on the wire and in the URL.
`static Transport from(String? s)`	`Transport`	—	

Role

`enum Role` — Who said a caption.

ConnectionQuality

`enum ConnectionQuality` — The connection, in one word: excellent, good, poor or lost (unknown before any sample).

Case	Meaning
excellent	
good	
poor	
lost	
unknown	

Member	Returns	Errors	What it does
static ConnectionQuality classify(num? rttMs, num? jitterMs, num? loss)	ConnectionQuality	—	One sample → one word.

CallPhase

enum CallPhase — Where the call stands, in one word — the React Native launcher's GrainePhase, derived from the same state by callPhase.

Case	Meaning
idle	No call; ready to start one.
connecting	The page loading, or a call being placed.
reconnecting	The page failed and is being reloaded, or the network dropped the call.
thinking	The customer finished a sentence and the agent has not answered yet.
speaking	The agent is talking.
listening	The agent is waiting for the customer.

ConnectStage

enum ConnectStage — WHERE A CALL BEING PLACED IS, as the engine reports it (graine:progress).

Case	Meaning
registering	A credential and a line are being set up (a cold start).
dialling	The call is on its way to the agent.
retrying	The first attempt went unanswered; a fresh connection is being made — on mobile data, usually a connection the phone dropped while idle.
switching	The call is moving to the other transport.

Member	Returns	Errors	What it does
static ConnectStage? from(String? wire)	ConnectStage?	—	The stage for a word on the wire, or null for one this SDK does not know — and for "failed", which ends a connect rather than being a stage of one.
String get wire	String	—	The word on the wire.

Caption

class Caption — One caption line: who, what, and whether it is still being said.

Member	Returns	Errors	What it does
const Caption(this.role, this.text, {this.live = false})	constructor	—	
final Role role	Role	—	
final String text	String	—	
final bool live	bool	—	Still being recognised or streamed — render it as provisional.
Caption settled()	Caption	—	

FrictionConfig

class FrictionConfig — The app's layer of friction rules, over the dashboard's (Embed & Widgets → friction, served with the session) over the built-in defaults.

Member	Returns	Errors	What it does
`const FrictionConfig({ this.enabled, this.idleAfterMs, this.cooldownMs, this.cooldownSeconds, this.maxPerSession, this.openLauncher, this.message, this.requireOutstandingField, this.screens = const <String, FrictionScreenRule>{}})`	`constructor`	—	
`final bool? enabled`	`bool?`	—	
`final int? idleAfterMs`	`int?`	—	On the same screen this long → stalled_on_step.
`final int? cooldownMs`	`int?`	—	Minimum gap between two signals on the same screen.
`final int? cooldownSeconds`	`int?`	—	Runtime pacing for the agent's unsolicited interventions.
`final int? maxPerSession`	`int?`	—	
`final bool? openLauncher`	`bool?`	—	Carried to GraineListener.onFriction for a host that opens its own launcher.
`final String? message`	`String?`	—	What the agent is told instead of the built-in sentence: {title} {screen} {seconds} {field} {error}.
`final bool? requireOutstandingField`	`bool?`	—	Fire only while a field is outstanding (invalid, or empty and not optional), with the clock running from when it became so.
`final Map<String, FrictionScreenRule> screens`	`Map<String, FrictionScreenRule>`	—	
`static FrictionConfig? fromJson(Map<String, Object?>? o)`	`FrictionConfig?`	—	The dashboard's layer, from the session's friction object.

FrictionScreenRule

`class FrictionScreenRule` — Per-screen friction overrides, keyed by the screen id the app reports.

Member	Returns	Errors	What it does
`const FrictionScreenRule({this.ignore, this.idleAfterMs, this.message})`	`constructor`	—	
`final bool? ignore`	`bool?`	—	No friction detection at all on this screen — someone reading terms is not stuck.
`final int? idleAfterMs`	`int?`	—	
`final String? message`	`String?`	—	

FrictionSignal

`class FrictionSignal` — The customer looks stuck.

Member	Returns	Errors	What it does
`const FrictionSignal({ required this.name, required this.screen, required this.detail, required this.data, required this.cooldownSeconds, required this.maxPerSession, required this.openLauncher})`	`constructor`	—	
`final String name`	`String`	—	
`final String screen`	`String`	—	
`final String detail`	`String`	—	
`final Map<String, Object?> data`	`Map<String, Object?>`	—	
`final int cooldownSeconds`	`int`	—	
`final int maxPerSession`	`int`	—	
`final bool openLauncher`	`bool`	—	

GraineAgentBar

`class GraineAgentBar extends StatefulWidget` — The assistant bar, drawn in the brand's design. Place it once above the navigator, inside GraineEngine.

Member	Returns	Errors	What it does
`const GraineAgentBar({ super.key, this.call, this.components, this.accent, this.name, this.avatarUrl, this.scheme, this.radius, this.design, this.bottomInset = 96, this.horizontalInset = 12, this.draggable = true, this.initialOffset, this.onPlacementChange, this.keyboardAware = true, this.showComponents = true, this.topMargin, this.showCaptions = true, this.onClose, this.chat, this.builtInChat = true, this.surface, this.onSurfaceChange, this.app})`	`constructor`	—	
`final GraineCallState? call`	`GraineCallState?`	—	The call to draw.
`final GraineComponents? components`	`GraineComponents?`	—	The components to draw above the bar.
`final GraineInApp? app`	`GraineInApp?`	—	The agent this bar talks to, in an app with several (GraineAgentBar(app: GraineInApp.named('support'))): its call, chat, components, session and design.
`final String? accent`	`String?`	—	#RRGGBB for the avatar, the microphone, the meter and components.
`final String? name`	`String?`	—	The name beside the status.
`final String? avatarUrl`	`String?`	—	A face for the avatar, over the design's glyph.
`final String? scheme`	`String?`	—	"light", "dark" or "auto" (the device).
`final double? radius`	`double?`	—	Corner rounding for the caption and components.
`final ResolvedDesign? design`	`ResolvedDesign?`	—	The design to draw.
`final double bottomInset`	`double`	—	Where the bar sits with no offset: above your tab bar.
`final double horizontalInset`	`double`	—	
`final bool draggable`	`bool`	—	Let the customer drag it up and down.
`final int? initialOffset`	`int?`	—	A position the app persisted, in logical pixels above the anchor.
`final ValueChanged<int>? onPlacementChange`	`ValueChanged<int>?`	—	The customer let go somewhere new.
`final bool keyboardAware`	`bool`	—	Rise above the soft keyboard.
`final bool showComponents`	`bool`	—	Mount a GraineWidgetView above the bar.
`final double? topMargin`	`double?`	—	Space to keep clear at the top.
`final bool showCaptions`	`bool`	—	Show the line being said above the bar during a call.
`final VoidCallback? onClose`	`VoidCallback?`	—	The end button was pressed: after a hang-up, or — off a call — to close the bar.
`final GraineChatState? chat`	`GraineChatState?`	—	The chat, for an agent the dashboard offers as a chat, or as both a call and a chat (Embed & Widgets → mode).
`final bool builtInChat`	`bool`	—	Use the SDK's own chat when chat is null.
`final AgentSurface? surface`	`AgentSurface?`	—	Which to open when both are offered — the dashboard's start otherwise.

Member	Returns	Errors	What it does
`final ValueChanged<AgentSurface>? onSurfaceChange`	`ValueChanged<AgentSurface>?`	—	The customer switched between the call and the chat, or the dashboard's setting moved them.
`static const Key barKey`	`Key`	—	The bar itself — the 60-point frame with the avatar and the buttons.
`static const Key callButtonKey`	`Key`	—	The button that starts the call, retries a load that failed, or ends the call: the microphone off a call, the end button on one.
`static const Key muteButtonKey`	`Key`	—	The microphone during a call, where it mutes.
`static const Key closeButtonKey`	`Key`	—	The end button off a call, where it closes.
`static const Key switchButtonKey`	`Key`	—	The switch between the call and the chat (only when both are offered).
`static const Key statusKey`	`Key`	—	The status line.
`static const Key meterKey`	`Key`	—	The four-bar meter beside "Listening" and "Speaking".
`static const Key avatarKey`	`Key`	—	The avatar.
`static const Key captionKey`	`Key`	—	The live caption above the bar.
`static const Key chatKey`	`Key`	—	The chat window above the bar.

GraineChat

`class GraineChat extends StatefulWidget` — The chat window as a widget, for an app that places the chat itself.

Member	Returns	Errors	What it does
`const GraineChat({ super.key, this.chat, this.components, this.design, this.scheme, this.accent, this.name, this.onClose, this.showComponents = true, this.app})`	`constructor`	—	
`final GraineChatState? chat`	`GraineChatState?`	—	The chat to draw.
`final GraineComponents? components`	`GraineComponents?`	—	The components to draw above the composer.
`final GraineInApp? app`	`GraineInApp?`	—	The agent this chat is with: its chat, components, session and design.
`final ResolvedDesign? design`	`ResolvedDesign?`	—	The design to draw.
`final String? scheme`	`String?`	—	"light", "dark" or "auto".
`final String? accent`	`String?`	—	#RRGGBB over the design's accent.
`final String? name`	`String?`	—	The heading.
`final VoidCallback? onClose`	`VoidCallback?`	—	The chevron in the header — usually a pop.
`final bool showComponents`	`bool`	—	Draw the components the agent shows.
`static const Key screenKey`	`Key`	—	The whole screen.
`static Future<void> open(BuildContext context, {GraineInApp? app, GraineChatSize? size})`	`Future<void>`	—	Open the chat, at size for this open — over the app's (GraineConfig.chatSize) and the dashboard's (Embed & Widgets → Appearance → Chat window → Window size).

GraineChatSize

`enum GraineChatSize` — How big the chat window opens: the dashboard's Window size, GraineConfig.chatSize over it, and GraineChat.open(size:) over both for one open.

Case	Meaning
`compact`	The window above the bar, as it has always been.
`large`	A sheet over most of a phone; a panel in the corner of a tablet.
`full`	The whole screen.

Member	Returns	Errors	What it does
`String get wire`	`String`	—	The design's word for it.
`static GraineChatSize? fromWire(Object? raw)`	`GraineChatSize?`	—	The design's word for a size; null for anything else — absent, or a newer server's word.

GraineEngine

`class GraineEngine extends StatefulWidget` — Holds the voice engine. Place it once, above the navigator (MaterialApp.builder), so a call outlives every route.

Member	Returns	Errors	What it does
`const GraineEngine({super.key, this.child, this.voice, this.app})`	`constructor`	—	
`final Widget? child`	`Widget?`	—	
`final GraineVoice? voice`	`GraineVoice?`	—	The call to attach.
`final GraineInApp? app`	`GraineInApp?`	—	The agent whose calls this engine carries.

GraineLauncher

`class GraineLauncher extends StatelessWidget` — The launcher — the floating button that stands for the agent.

Member	Returns	Errors	What it does
`const GraineLauncher({ super.key, this.call, this.onPressed, this.label, this.design, this.scheme, this.app})`	`constructor`	—	
`final GraineCallState? call`	`GraineCallState?`	—	The call a tap starts.
`final GraineInApp? app`	`GraineInApp?`	—	The agent a tap calls: its call, session and design.
`final VoidCallback? onPressed`	`VoidCallback?`	—	What a tap does instead of starting the call.
`final String? label`	`String?`	—	A pill's words, and what a screen reader says.
`final ResolvedDesign? design`	`ResolvedDesign?`	—	The design to draw.
`final String? scheme`	`String?`	—	"light", "dark" or "auto" (the device).
`static const Key launcherKey`	`Key`	—	The button, for a test to find.

GraineLoaderView

`class GraineLoaderView extends StatefulWidget` — How waiting looks, drawn from the dashboard's Appearance → loader.

Member	Returns	Errors	What it does
`const GraineLoaderView({ super.key, required this.look, required this.color, this.trackColor = const Color(0x332C2E33), this.size, this.label, this.labelColor = const Color(0xFF5F6672), this.labelStyle, this.stacked = false, this.logoUrl})`	`constructor`	—	
`final LoaderLook look`	`LoaderLook`	—	From Loaders.resolve.
`final Color color`	`Color`	—	
`final Color trackColor`	`Color`	—	The ring's unlit part.

Member	Returns	Errors	What it does
`final double? size`	`double?`	—	Overrides the look's size — the bar's status line wants a small one.
`final String? label`	`String?`	—	The words for this moment; null for the mark alone.
`final Color labelColor`	`Color`	—	
`final TextStyle? labelStyle`	`TextStyle?`	—	The words' type, whole — the voice bar sets them in the design's face.
`final bool stacked`	`bool`	—	The label under the mark rather than beside it.
`final String? logoUrl`	`String?`	—	

GraineTranscriptView

`class GraineTranscriptView extends StatelessWidget` — The conversation so far, drawn as the brand's chat window.

Member	Returns	Errors	What it does
`const GraineTranscriptView({ super.key, this.call, this.title, this.showHeader = true, this.design, this.scheme, this.maxHeight = 320, this.app})`	`constructor`	—	
`final GraineCallState? call`	`GraineCallState?`	—	The call whose words to show.
`final GraineInApp? app`	`GraineInApp?`	—	The agent whose call this is: its call, session and design.
`final String? title`	`String?`	—	The header's words.
`final bool showHeader`	`bool`	—	
`final ResolvedDesign? design`	`ResolvedDesign?`	—	The design to draw.
`final String? scheme`	`String?`	—	"light", "dark" or "auto" (the device).
`final double maxHeight`	`double`	—	The most height it takes; the conversation scrolls inside.
`static const Key headerKey`	`Key`	—	
`static Key bubbleKey(int index)`	`Key`	—	The indexth turn's bubble (oldest is 0).

GraineWidgetView

`class GraineWidgetView extends StatefulWidget` — The component the agent put on screen — drawn by the SDK, not by the host.

Member	Returns	Errors	What it does
`const GraineWidgetView({ super.key, this.components, this.accent, this.radius, this.scheme, this.theme, this.appearance, this.design, this.maxHeight = 460, this.app})`	`constructor`	—	
`final GraineComponents? components`	`GraineComponents?`	—	The state to draw.
`final GraineInApp? app`	`GraineInApp?`	—	The agent whose components these are: its components, session and design.
`final String? accent`	`String?`	—	#RRGGBB for buttons and badges.
`final double? radius`	`double?`	—	Corner rounding.
`final String? scheme`	`String?`	—	"light", "dark" or "auto" (the device).
`final GraineWidgetTheme? theme`	`GraineWidgetTheme?`	—	

Member	Returns	Errors	What it does
`final Map<String, Object?>? appearance`	`Map<String, Object?>?`	—	The dashboard's Appearance.
`final ResolvedDesign? design`	`ResolvedDesign?`	—	The design.
`final double maxHeight`	`double`	—	The most height the component may take; its body scrolls inside.

GraineWidgetTheme

`class GraineWidgetTheme` — Single-colour overrides for GraineWidgetView.

Member	Returns	Errors	What it does
`const GraineWidgetTheme({this.surface, this.border, this.text, this.muted, this.onAccent})`	`constructor`	—	
`final Color? surface`	`Color?`	—	Behind the component.
`final Color? border`	`Color?`	—	Hairlines and chip fills.
`final Color? text`	`Color?`	—	Titles and prices.
`final Color? muted`	`Color?`	—	Subtitles, facts, secondary labels.
`final Color? onAccent`	`Color?`	—	Readable ON the accent.

Lower-level exported types

The package also exports the building blocks the SDK draws and connects with — the design tokens, the engine page's protocol, chat framing, loaders, placement and friction detection. They are public so a custom UI can reuse them; an app integrating the SDK does not need them. Each is listed with its members.

Type	Members	What it is
`GraineCallFault`	`error`	A CALL THAT CAN SAY IT IS MISCONFIGURED — beside GraineCallState rather than in it, so a host's own call state keeps compiling.
`GraineEnginePage`	`bind, load, reload, run, dispose`	What GraineVoice needs from the page that carries the call.
`GraineEngineSink`	`pageStarted, progress, pageFinished, mainFrameError, message, microphoneRequested`	How a page reports to the call.
`GraineChatConnection`	`connecting, open, close`	A CHAT THAT IS OPENED AND CLOSED with the customer — beside GraineChatState, as GraineCallFault is beside the call state, so a host's own chat keeps compiling.
`GraineChatTyping`	`agentTyping`	
`GraineChatSocket`	`frames, send, close`	One open chat socket, as the chat sees it.
`GraineChatConnector`		Opens a GraineChatSocket to url, offering protocols.
`NativeAppearance`	`NativeAppearance, accent, heading, logoUrl, logoPlate, radius`	The dashboard's appearance, translated for a phone: see Appearance.resolve.
`WidgetPalette`	`WidgetPalette, surface, border, text, muted, onAccent, mode`	The colours a component is drawn in, as #RRGGBB.
`Appearance`	`defaultAccent, defaultOrigin, resolve, isDark, palette, luminance, argb, tint, encodeURIComponent`	
`Captions`	`history, fold`	What has been said so far, oldest first — folded the way the RN launcher folds it.
`Top-level functions and constants`	`agentBarStatus, callPhase, connectingLook, voiceBarStatus`	
`BarStatus`	`BarStatus, text, waiting, meter, label`	What the voice bar's status line says (voiceBarStatus).

Type	Members	What it is
`ConnectProgress`	`slowConnectMs, stillConnecting, couldNotConnect`	The numbers and words of a connect that says where it is.
`VoiceMeter`	`heights, height, barWidth, gap, low, staggerMs, legMs, still, scale`	The meter — four bars beside "Listening" and "Speaking" that move at the design's own pace, so the motion knob is visible on the one surface a customer watches.
`AgentSurface`		The two ways a customer can be with the agent.
`SurfacePolicy`	`SurfacePolicy, allowed, start, canSwitch`	Which surfaces the dashboard allows, which to open, and whether to offer a switch between them.
`Surfaces`	`resolve`	The call or the chat — the React Native SDK's resolveSurface.
`ChatWire`	`subprotocol, pingMs, openTimeoutMs, stopGraceMs, client, defaultWsHost, callType, directUrl, withContextToken, init, ping, stop, text, widgetResult, actionResult, appContext, appEvent`	The constants and outgoing frames of the chat socket.
`AgentChunk`	`AgentChunk, streamStart, streamEnd, text, startsTurn, endsTurn, read`	ONE text FRAME: the words to show, and what it says about the turn.
`ChatFrame`	`ChatFrame, parse`	Everything the chat socket can say, typed.
`ChatHistory`	`ChatHistory, turns`	The earlier conversation, restored : the turns, oldest first — possibly none.
`ChatHistoryReader`	`read, time`	The conversation continues .
`ChatText`	`ChatText, chunk`	The agent's words (and turn markers).
`ChatWidget`	`ChatWidget, widget`	A component the agent drew.
`ChatWidgetPending`	`ChatWidgetPending, hint`	A component on its way.
`ChatAction`	`ChatAction, actionId, name, arguments`	Something the agent asks the app to do; answered with ChatWire.actionResult.
`ChatStream`	`ChatStream, start`	An explicit start or end of the agent's turn.
`ChatClear`	`ChatClear`	The turn was cut off.
`ChatTranscript`	`ChatTranscript, max, turns, messages, agentTyping, pending, customer, sent, failPending, retryFailed, expectReply, turnStarted, agent, component, turnEnded, closed, clear, opened, restore`	
`ChatDay`		Which day a separator names.
`ChatRow`	`ChatRow, key`	One row of the thread.
`ChatDayRow`	`ChatDayRow, day, date, at, contractKey`	A day separator: which day, the local date (YYYY-MM-DD), and the first message's time on it.
`ChatVoiceRow`	`ChatVoiceRow, id, at, first, last`	An earlier call: when its first turn was said, and which turns (indices into the conversation) it stands for.
`ChatTurnRow`	`ChatTurnRow, turn, index`	A bubble.
`ChatRows`	`build, dayLabel, callLabel, callSpoken, offersNewConversation, offersStarters`	
`ChatPieces`	`turns, starters, newConversation, upcoming, upcomingTitle, of, ofChat`	
`EmbedRefusal`	`EmbedRefusal, message, code, codeName, statusCode, retryAfterSeconds, of`	A refusal from one of the SDK's routes: { error, code }, with the sentence written for whoever has to act on it.
`ContextFrame`	`build, availableActions`	The context frame the runtime reads — the same keys as the React Native SDK's buildContextFrame() and the Android SDK's ContextFrame, masked on the way out.
`ActionRegistry`	`ActionRegistry, deadline, Function, names, has, execute, describeError`	The actions the agent may take in this app, and the one way they are run.

Type	Members	What it is
Top-level functions and constants	`deepEquals, deepHash`	
ThemeShadow	`ThemeShadow, y, blur, spread, opacity, color, fromJson, toJson`	One drop shadow: { y, blur, spread, opacity, color }.
ThemeOffset	`ThemeOffset, x, y, fromJson, toJson`	A solid offset shadow (neo-brutalist), drawn in colors.text.
ThemeColors	`ThemeColors, background, surface, surfaceRaised, text, textMuted, border, accent, onAccent, accentSoft, success, warning, danger, userBubble, userBubbleText, agentBubble, agentBubbleText, scrim, fromJson, toJson`	Every colour a surface uses, as #RRGGBB.
ThemeFontScale	`ThemeFontScale, caption, body, title, display, fromJson, toJson`	font.scale: sizes in logical pixels.
ThemeFontWeights	`ThemeFontWeights, body, strong, title, fromJson, toJson`	font.weights: CSS weights, 100–900.
ThemeFontPair	`ThemeFontPair, body, title, fromJson, toJson`	A body value and a title value — font.letterSpacing (em) and font.lineHeight (multipliers).
ThemeFont	`ThemeFont, body, heading, scale, weights, letterSpacing, lineHeight, fromJson, toJson`	Type: families, sizes, weights, spacing.
ThemeRadius	`ThemeRadius, sm, md, lg, xl, fromJson, toJson`	shape.radius: corner radii in logical pixels.
ThemeShape	`ThemeShape, radius, borderWidth, launcher, fromJson, toJson`	
ThemeSpacing	`ThemeSpacing, unit, cardPadding, gap, fromJson, toJson`	
ThemeElevation	`ThemeElevation, card, floating, hardOffset, fromJson, toJson`	Depth.
ThemeMotion	`ThemeMotion, durationMs, easing, bounce, fromJson, none, toJson`	
LauncherGlyph	`LauncherGlyph, id, path, stroke, fromJson, toJson`	A launcher glyph: 24×24 SVG path data.
ThemeIcons	`ThemeIcons, style, stroke, launcher, fromJson, toJson`	
ThemeGlass	`ThemeGlass, blur, surfaceOpacity, fromJson, toJson`	Translucent, blurred surfaces.
ThemeChat	`ThemeChat, avatars, patterns, maxSuggestions, title, subtitle, avatar, placeholder, showTimestamps, suggestions, background, pattern, patternColor, headerBackground, headerText, headerSubtext, composerBackground, composerText, composerPlaceholder, composerBorder, size, showUpcomingCalls, upcomingTitle, showHistory, newConversation, ThemeChat.fallback, fromJson, toJson`	The chat window's tokens : the brand's words — or "" for the default named on each — and colours already checked against what they sit on.
ThemeCallStatusText	`ThemeCallStatusText, ringing, scheduled, connecting, ended, missed, fromJson, toJson`	The words a call screen says at each moment.

Type	Members	What it is
ThemeCall	ThemeCall, avatars, avatarShapes, endShapes, ringtones, allControls, defaultControls, background, backgroundTo, text, textMuted, avatar, avatarShape, avatarFill, avatarText, buttonFill, buttonIcon, buttonActiveFill, buttonActiveIcon, endFill, endIcon, endShape, acceptFill, acceptIcon, declineFill, declineIcon, callbackFill, callbackIcon, captions, timer, controls, ringtone, statusText, ThemeCall.fallback, fromJson, toJson	The call screen's tokens — the same for ringing, connecting, on a call and ended.
RingNote	RingNote, hz, ms, toJson	One note of a ring: a frequency (0 is a rest) for a duration.
Ringtone	Ringtone, label, wave, gain, notes, pauseMs, all, toJson	A RING EVERY SDK PLAYS THE SAME : the notes once, then pauseMs of silence, until the call is answered, declined or stops ringing — each note a wave at gain (0–1 of full scale), with a 10 ms rise and a 40 ms fall inside it.
ThemeTokens	ThemeTokens, v, preset, scheme, colors, font, shape, spacing, elevation, motion, icons, glass, bubbles, chrome, servedChat, servedCall, dark, window, chat, call, fromJson, toJson	EVERYTHING A RENDERER NEEDS for one scheme, as flat numbers and #RRGGBB strings — the ThemeTokens of
ResolvedDesign	ResolvedDesign, config, light, dark, scheme, pick, fromJson, toJson	The session's design: the brand's config and both token sets.
Design	schemes, launcherShapes, bubbleStyles, iconStyles, easings, chromes, launcherSize, launcherGlyphScale, squircleScale, glyphAssetVersion, defaultOrigin, normaliseHex, mixHex, fontFamily, launcherRadius, glyphUrl	The rules every renderer shares: allowlists, colour, fonts, the launcher's geometry and the glyph image URL.
DesignFontFace	DesignFontFace, family, weight, name, url, key, toJson	One face of one family, as the SDK registers it.
DesignFonts	assetVersion, weights, key, nearestWeight, canonicalFamily, faceName, url, faceFor, facesFor, facesForDesign	
FrictionForScreen	FrictionForScreen, ignore, idleAfterMs, message	The rules on one screen.
FrictionRules	FrictionRules, enabled, idleAfterMs, cooldownMs, cooldownSeconds, maxPerSession, openLauncher, message, requireOutstandingField, screens, forScreen	Every layer applied: what is actually in force.
Friction	resolve	
FrictionDetector	FrictionDetector, rules, configure, onScreen, onInteraction, onCall, idleDueIn, onIdle, outstanding, fill	Stalled on A STEP — the idle half of the React Native SDK's friction detector, with time as an argument so every rule is a plain test.
SdkIcon	SdkIcon, body, lines, toJson	One icon: a closed body (may be filled) and lines (only stroked).
IconTreatment	IconTreatment, fillBody, bodyFillOpacity, bodyStroke, lineStroke, square	How one icon style strokes and fills: SdkIcons.treatment.
SdkIcons	mic, micOff, end, send, chat, call, chevron, close, =, all, treatment	
Identity	defaultField, placeholders, maxValueLength, normaliseField, valueOf, pick, normaliseInstallId, normaliseTimezone, uuidV4, serverLowercase, serverTrim, normaliseValue, memoryKey, withoutIdentity, requestFields	The rules every SDK applies on the device before an identity leaves it.

Type	Members	What it is
LoaderText	LoaderText, connecting, reconnecting, thinking, component, forMoment, isEmpty, toMap	Words for each moment a customer waits.
LoaderStyle	LoaderStyle, kind, color, size, speed, text, skeleton, toMap	How waiting looks, as chosen in Embed & Widgets → Appearance.
LoaderLook	LoaderLook, kind, color, px, periodMs, text, skeleton	The effective loader: numbers a renderer can use directly.
Loaders	kinds, sizes, speeds, moments, defaultKind, textMax, defaultText, normalise, resolve, resolveStyle	
LoadVerdict	wait, stalled, bootTimeout, loadCap, LoadVerdict, reason	What the load watchdog concluded.
LoadWatchdog	LoadWatchdog, stallMs, bootMs, hardCapMs, start, progress, loaded, check, nextCheckInMs	Waiting is not the same as being stuck.
RetryLadder	RetryLadder, attempt, exhausted, next, reset	How often to reload a page that failed, and when to stop.
LogoPlate	accent, white, from, wire	What the brand's logo sits on
LogoPlateLook	LogoPlateLook, fill, outlineWidth, outlineColor, hasOutline	The fill under a logo, and its outline (none on the accent).
BrandLogo	plateWhite, labelOnWhite, box, outlineWidth, plateLook	The logo's geometry and plate — theme.ts's logoBox and logoPlateLook.
PlacementBounds	PlacementBounds, windowHeight, baseBottom, barHeight, topMargin	The bounds a dragged bar is clamped to.
Placement	dragSlop, clampOffset, offsetDuringDrag, isTap	Where the bar may sit — arithmetic only, ported from the RN SDK's placement-math.ts.
KeyboardLift	liftFor	How far A BOTTOM-ANCHORED BAR MUST RISE TO STAY ABOVE THE KEYBOARD: the keyboard's height minus whatever the window already shrank.
Protocol	bridgeName, actionDeadlineMs, engineUrl, deliver, out, toJson, parse, decodeJson, contextToken, ticketMaxKeys, ticketMaxKeyLength, ticketMaxValueLength, ticketMaxBytes, ticketMetadata, trimTrailingSlashes	The wire protocol — the same one the React Native and Android SDKs speak, byte for byte.
ProtocolOut	startCall, endCall, setMuted, warm, variables, appContext, appActionResult, appEvent, widgetResult, identity	Messages this SDK sends into the page.
ProtocolIn	ready, call, muted, caption, widget, widgetPending, appAction, ended, micDenied, transport, quality, network, reconnecting, progress	Messages the page sends out.
PageMessage	PageMessage	Everything the page can say, typed.
PageReady	PageReady	
PageCall	PageCall, connected, connecting	
PageMuted	PageMuted, muted	
PageCaption	PageCaption, role, text, live	
PageWidget	PageWidget, widget	
PageWidgetPending	PageWidgetPending, widget	
PageAppAction	PageAppAction, actionId, name, arguments	
PageEnded	PageEnded	
PageMicDenied	PageMicDenied, reason	
PageTransport	PageTransport, transport, requested, reason, message	
PageQuality	PageQuality, rttMs, jitterMs, loss	
PageNetwork	PageNetwork, online	

Type	Members	What it is
PageReconnecting	PageReconnecting, active	
PageProgress	PageProgress, stage, failed	The engine's account of a connect in progress.
ScheduledCalls	maxReasonLength, atWire, parseTime, oneFrom, listFrom, statusAfterCancel	The rules for what goes out and what comes back.
GraineUpcomingCancel	none, cancelling, failed	Where one booked call's Cancel is, on the chat's Upcoming calls card.
ChatTime	clock, booked	The times A CHAT SHOWS, in the device's own zone : the short clock under a message, and a booked call's day.
Sha256	hash, ofText, hex	
VariableStore	VariableStore, setDashboardDefaults, setRuntime, setIdentity, identity, merged, flatten, flattenOne	The prompt's {variables}, from every place they can come from, in one view.
GraineVersion	name	The SDK's version, sent nowhere secret.
GraineComponent		
GraineWidget	GraineWidget, id, kind, data, fallbackText, style, onSelect, adopt	A component the agent put on screen, adopted from the page's frame exactly as the React Native SDK's useGraineWidget adopts it.
WidgetAnswer	WidgetAnswer, data, summary	What a tap sends: the structured payload, and the sentence that is the customer's turn.
WidgetRow	WidgetRow, id, title, subtitle, price, facts, image, unavailable, selectLabel	One normalised row: the shape every dashboard kind is reduced to.
WidgetChip	WidgetChip, id, label	
WidgetDay	WidgetDay, label, slots	
WidgetContent	WidgetContent, widget, isQuickActions, selectLabel, rows, isProducts, isChips, isCalendar, drawable, chips, days, title, prompt, note, surface, many, pick, pickMany, pickChip, pickSlot, toggle, manyButtonLabel, static, closeLabel	One kind, reduced to what the renderer draws — widget.tsx's normalisation, line for line.
WidgetPending	WidgetPending, id, kind	A component announced and still loading.
WidgetAnswered	WidgetAnswered, widget, data, summary	The customer's answer, kept on screen as one line.
WidgetSubmission	WidgetSubmission, widgetId, data, summary, action	What to put on the wire for an answer, and the app's own action to run first.
WidgetDismissal	WidgetDismissal, toolName, kind, title, eventData	The data of the quiet component_dismissed event, exactly as the RN SDK sends it.
WidgetFlow	WidgetFlow, baseUrl, widget, answered, loading, awaitingReply, closed, seed, onPending, onWidget, onAgentStarted, submit, dismiss, reopen, reset, tick, nextDeadline	useGraineWidget, as a state machine with time passed in.
WidgetStyle	WidgetStyle, preset, imageSize, density, button, facts, columns, surface, radius, accent, platform, select, maxSelect, toMap	HOW A COMPONENT LOOKS — chosen on the component in the dashboard, carried with it to the agent, drawn by the SDK.
WidgetLook	WidgetLook, preset, imageSize, density, button, facts, columns, surface, radius, accent, platform, select, maxSelect, toMap	The effective look: the preset's defaults, then the knobs on top.
PlatformGrammar	PlatformGrammar, radius, chipRadius, buttonRadius, pillButtons, separatorInset, grabberWidth, grabberHeight, titleWeight, padBoost, chipOutlined	WHAT A PLATFORM'S DESIGN GRAMMAR CHANGES — the shapes people on that phone already know.
WidgetSelectAction	WidgetSelectAction, path, action, args, say, isEmpty, toMap	What the app does when the customer taps, beyond telling the agent: an in-app action (or a path for the one navigate handler), with arguments filled from the tapped item.
SelectCall	SelectCall, name, args	One tap's in-app call: the action to run and what to run it with.

Type	Members	What it is
`Widgets`	`presets, imageSizes, densities, buttons, surfaceChoices, platforms, selects, maxSelectDefault, defaultPreset, navigateAction, selectArgMax, selectValueMax, minLoaderMs, pendingWidgetMs, awaitingReplyMs, surface, normaliseStyle, resolveLook, grammar, resolvePlatform, manySummary, normaliseSelectAction, fillSelectTemplate, fillSelectArgs, selectCall, resolveImages`	The component rules, one function each — ports of the RN SDK's protocol.ts.
`ChromeSurface`		Which surface wears the title bar — the caption card's is the small one.
`WindowChrome`	`barHeight, lightsInset, lightSize, lightsGap, lights, lightsWidth, onlineDot, onlineDotGap, chevronIcon, chevronBox, chevronInset, titleGap, titleInset, monospace, monospaceFallback, captionTitle, title, innerRadius`	
`WebViewEnginePage`	`WebViewEnginePage, controller`	The engine page in a webview_flutter WebView: JavaScript on, media without a gesture (nothing on a 1x1 view can be tapped), the bridge the page looks for (window.ReactNativeWebView), and the microphone granted to the page only when the app holds it.

12.7 Kotlin Multiplatform — ai.graine:inapp-kmp 0.7.0

One API in shared Kotlin over the Android and iOS SDKs. Suspending functions throw `GraineException`; states are `StateFlow`s that always hold the latest value; events are `SharedFlow`s that are not replayed.

Graine

`object Graine` — The SDK, and the default agent. Graine.configure(config) once at launch (Application.onCreate on Android, the app's init on iOS); every member may be called from any thread, and the suspending ones never block the main thread. Who the user is, the install, the memory choice and the screen are the app's and shared by every agent.

Member	Returns	Errors	What it does
`const val DEFAULT_AGENT = "default"`	`–`	—	The default agent's name.
`fun configure(config: GraineConfig, name: String = DEFAULT_AGENT): GraineAgent`	`GraineAgent`	—	Start an agent.
`fun agent(name: String = DEFAULT_AGENT) : GraineAgent?`	`GraineAgent?`	—	The agent configured under name; null when it is not configured.
`val agents: Map<String, GraineAgent>`	`Map<String, GraineAgent>`	—	Every configured agent, by name.
`val isConfigured: Boolean`	`Boolean`	—	Whether configure has run.
`val session: StateFlow<GraineSession?>`	`StateFlow<GraineSession?>`	—	The default agent's configuration, once it has loaded; null before.
`val events: SharedFlow<GraineEvent>`	`SharedFlow<GraineEvent>`	—	Everything the default agent reports that is not a state.
`val chat: GraineChat`	`GraineChat`	—	The typed conversation with the default agent.
`val voice: GraineVoice`	`GraineVoice`	—	The default agent's voice bar call.

Member	Returns	Errors	What it does
`val schedulingRules: GraineSchedulingRules`	`GraineSchedulingRules`	—	What the default agent allows when a user books a call.
`val calls: GraineCalls`	`GraineCalls`	—	The app's one call engine.
`fun setVariables(values: Map<String, Any?>?)`	`nothing`	—	The default agent's prompt variables.
`fun registerAction(name: String, handler: suspend (args: Map<String, Any?>) -> GraineActionResult): () -> Unit`	`() -> Unit`	—	An action for the default agent.
`suspend fun forgetMemory() : Int`	`Int`	GraineException: identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	Forget what the default agent remembers about the signed-in user.
`suspend fun memories() : List<GraineMemory>`	`List<GraineMemory>`	GraineException: identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	What the default agent remembers about the signed-in user.
`suspend fun scheduleCall(at: Instant, reason: String? = null) : GraineScheduledCall`	`GraineScheduledCall`	GraineException: on the device: scheduling_off, too_soon, too_far; from Graine: outside_window, too_many_pending, identity_required, device_limit, invalid_request, rate_limited, network, unavailable	Book a call from the default agent.
`suspend fun scheduledCalls() : List<GraineScheduledCall>`	`List<GraineScheduledCall>`	GraineException: network, rate_limited, unavailable, invalid_key, origin_refused	The default agent's booked calls still to come.
`suspend fun cancelScheduledCall(id: String) : GraineCallStatus`	`GraineCallStatus`	GraineException: not_found, network, rate_limited, unavailable	Call off one of the default agent's booked calls.
`val installId: String`	`String`	—	This install's id: a random UUID made on first launch and kept for as long as the app is installed.
`val memoryEnabled: Boolean`	`Boolean`	—	Whether the current user's conversations are remembered: their last choice on this install, else true.
`fun identify(traits: Map<String, Any?>?, memory: Boolean? = null)`	`nothing`	—	Who is signed in — for every agent.
`fun setMemory(enabled: Boolean)`	`nothing`	—	Remember the current user's conversations, or stop — identify(the same traits, memory = enabled).
`fun setScreen(screen: GraineScreen?)`	`nothing`	—	What the user is looking at — the app's, heard by whichever agent has a live conversation.
`fun updateField(field: GraineField)`	`nothing`	—	One field changed: merged by name into the current screen (added when new).
`fun removeField(name: String)`	`nothing`	—	A field left the screen.
`fun clearScreen(screenId: String? = null)`	`nothing`	—	The user left a screen.
`fun track(name: String, data: Map<String, Any?> = emptyMap())`	`nothing`	—	A product event that informs without interrupting.
`fun tap(name: String)`	`nothing`	—	A tap on a control the agent should know about (tap:<name>), for a control the SDK cannot name itself.

Member	Returns	Errors	What it does
`fun noteInteraction()`	`nothing`	—	The user did something that is not a field edit (scrolled, opened a picker): the stuck-user clock restarts.

GraineAgent

`class GraineAgent internal constructor(val name: String)` — One configured agent (Graine.configure(config, name) or Graine.agent(name)): its session, chat, voice bar, actions, scheduling and memory requests go to that agent's key.

Field	Type	Default	Meaning
`name`	`String`	required	The name it was configured under; "default" for the default agent.

Member	Returns	Errors	What it does
`val session: StateFlow<GraineSession?>`	`StateFlow<GraineSession?>`	—	This agent's configuration, once it has loaded; null before.
`val events: SharedFlow<GraineEvent>`	`SharedFlow<GraineEvent>`	—	Everything this agent reports that is not a state.
`val chat: GraineChat`	`GraineChat`	—	The typed conversation with this agent.
`val voice: GraineVoice`	`GraineVoice`	—	This agent's voice bar call.
`val schedulingRules: GraineSchedulingRules`	`GraineSchedulingRules`	—	What this agent allows when a user books a call: its session's rules, or the defaults (no booking) before it loads.
`val isConfigured: Boolean`	`Boolean`	—	Whether this agent has been configured.
`fun setVariables(values: Map<String, Any?>?)`	`nothing`	—	Prompt variables from the app's own data, for this agent.
`fun registerAction(name: String, handler: suspend (args: Map<String, Any?>) -> GraineActionResult): () -> Unit`	`() -> Unit`	—	Something this agent can do in your app.
`suspend fun forgetMemory() : Int`	`Int`	GraineException: identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	Forget everything this agent remembers about the signed-in user.
`suspend fun memories() : List<GraineMemory>`	`List<GraineMemory>`	GraineException: identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	What this agent remembers about the signed-in user, newest first (at most 100), already redacted — to show people what is kept about them.
`suspend fun scheduleCall(at: Instant, reason: String? = null): GraineScheduledCall`	`GraineScheduledCall`	GraineException: on the device: scheduling_off, too_soon, too_far; from Graine: outside_window, too_many_pending, identity_required, device_limit, invalid_request, rate_limited, network, unavailable	Book a call from this agent for the signed-in user (or, with nobody signed in, this install) at at — "call me tomorrow at 6".
`suspend fun scheduledCalls() : List<GraineScheduledCall>`	`List<GraineScheduledCall>`	GraineException: network, rate_limited, unavailable, invalid_key, origin_refused	The calls this agent has booked for the signed-in user (or this install) that are still to come, soonest first.
`suspend fun cancelScheduledCall(id: String): GraineCallStatus`	`GraineCallStatus`	GraineException: not_found, network, rate_limited, unavailable	Call one off — only one booked for this user or install.
`fun startCall(reason: String? = null, variables: Map<String, Any?> = emptyMap()): String?`	`String?`	—	The user calls this agent: Graine.calls.startCall(reason, variables, agent = name).

GraineConfig

`data class GraineConfig(…)` — How the SDK starts: the base URL, the agent's publishable key, and options.

Field	Type	Default	Meaning
baseUrl	`String`	required	"https://www.graine.ai" unless self-hosted.
publishableKey	`String`	required	From Agent → Embed & Widgets.
variables	`Map<String, Any?>`	`emptyMap()`	Prompt {variables} known at build time.
appVersion	`String?`	`null`	Lands on the call record, to tell builds apart.
endOnBackground	`Boolean`	`true`	End a live conversation when the app goes to the background.
debug	`Boolean`	`false`	Log the SDK's traffic with Graine — keys only, never values — to logcat (tag "Graine") and the iOS console.
captureTaps	`Boolean`	`true`	Notice taps on labelled controls by themselves (what a screen reader reads, a view's id, a button's words), at most two a second and only while a screen is reported, so the agent knows what the customer just pressed.
transport	`GraineTransport?`	`null`	Force how the voice bar's audio travels.
chatSize	`GraineChatSize?`	`null`	How big the chat window opens, over the dashboard's choice (Embed & Widgets → Appearance → Chat window → Window size).

GraineTransport

`enum class GraineTransport(val wire: String)` — How the voice bar's audio travels.

Field	Type	Default	Meaning
wire	`String`	required	

Case	Meaning
WEBSOCKET	Raw audio on the conversation socket. — wire `ws`
WEBRTC	WebRTC with a jitter buffer and the device's echo canceller; falls back to WEBSOCKET. — wire `rtc`

GraineSession

`data class GraineSession(…)` — The agent's settings as loaded with the session.

Field	Type	Default	Meaning
agentId	`String`	required	
memoryEnabled	`Boolean?`	required	Whether the agent remembers its users; null when the server does not say.
identityField	`String`	`"user_id"`	The trait that names the user (Agent → Memory → Who is the user?): user_id unless the dashboard chose another.
devicesPerUser	`Int`	`2`	How many devices one user may use the agent on.
schedulingRules	`GraineSchedulingRules`	`GraineSchedulingRules()`	
declaredActions	`Set<String>`	`emptySet()`	The actions the agent is given (Embed & Widgets → actions).

GraineEvent

`sealed class GraineEvent` — Events that are not states. Not replayed: collect Graine.events from app start.

Case	Meaning
VoiceEnded	The voice bar's call is over.

GraineChat

`class GraineChat internal constructor(private val agent: GraineAgent)` — The chat — a typed conversation with an agent: Graine.chat (the default agent's), Graine.agent("support")?.chat (another's).

Member	Returns	Errors	What it does
`val turns: StateFlow<List<GraineChatTurn>>`	`StateFlow<List<GraineChatTurn>>`	—	What has been said, oldest first.
`val status: StateFlow<GraineChatStatus>`	`StateFlow<GraineChatStatus>`	—	The chat's connection: idle, connecting, online, reconnecting or offline.
`val typing: StateFlow<Boolean>`	`StateFlow<Boolean>`	—	The agent is working on a reply: from the user's message until its first words.
`fun open(size: GraineChatSize? = null)`	`nothing`	—	The user is in the chat: connect, and stay connected.
`fun close()`	`nothing`	—	The user left: the chat hangs up and stays down.
`fun send(text: String)`	`nothing`	—	Send a message.
`fun retry()`	`nothing`	—	Connect now, and send what could not be sent.
`suspend fun newConversation()`	`nothing`	—	START A NEW CONVERSATION — a fresh thread, as a support chat offers one.

GraineChatTurn

`data class GraineChatTurn(…)` — One message in the chat: the user's, or one reply of the agent's.

Field	Type	Default	Meaning
`id`	`String`	required	
`role`	`GraineRole`	required	
`text`	`String`	required	
`at`	`Instant`	required	When the message began.
`delivery`	`GraineDelivery?`	`null`	The user's messages only: where it is.
`proactive`	`Boolean`	`false`	The agent's messages only: it spoke up without being asked (its turn did not follow one of the user's).
`component`	`GraineComponent?`	`null`	A component the agent drew with this message, when the platform SDK ties one to it.
`live`	`Boolean`	`false`	The agent's words are still arriving: draw them as provisional.
`restored`	`Boolean`	`false`	From an earlier conversation, restored when the chat opened — the user came back, and the agent picks up where it left off.
`channel`	`GraineChatChannel`	`GraineChatChannel.CHAT`	Where it was said: typed in the chat, or on a call (a restored turn only).

GraineDelivery

`enum class GraineDelivery` — Where one of the user's messages is.

Case	Meaning
`SENDING`	On its way: typed while the chat was not connected, sent as soon as it is.
`SENT`	
`FAILED`	The chat could not reconnect to send it.

GraineChatStatus

`enum class GraineChatStatus` — Where the chat's connection is.

Case	Meaning
`IDLE`	Not in the chat: never opened, left, or the app is in the background.
`CONNECTING`	

Case	Meaning
`ONLINE`	
`RECONNECTING`	The connection dropped and is being made again.
`OFFLINE`	It could not reconnect, or the other side ended the conversation.

GraineComponent

`data class GraineComponent(val id: String, val kind: String?, val title: String?, val payload: Map<String, Any?>)` — A component the agent drew — a product list, choices, time slots — as the conversation sent it.

Field	Type	Default	Meaning
`id`	`String`	required	
`kind`	`String?`	required	product_list, plan_comparison, choices, quick_actions, calendar_slots, info_card — or a newer one.
`title`	`String?`	required	
`payload`	`Map<String, Any?>`	required	The whole component as it arrived.

GraineVoice

`class GraineVoice internal constructor(private val agent: GraineAgent)` — The voice bar's call — the agent talking inside the app, beside the screen it is talking about.

Member	Returns	Errors	What it does
`val state: StateFlow<GraineVoiceInfo>`	`StateFlow<GraineVoiceInfo>`	—	The voice bar's call as it stands: connected, connecting, muted.
`fun start()`	`nothing`	—	Place the call.
`fun end()`	`nothing`	—	Hang up.
`fun setMuted(muted: Boolean)`	`nothing`	—	Stop sending the user's voice without ending the call.

GraineVoiceInfo

`data class GraineVoiceInfo(val connected: Boolean = false, val connecting: Boolean = false, val muted: Boolean = false)` — The voice bar's call, as a screen draws it.

Field	Type	Default	Meaning
`connected`	`Boolean`	`false`	
`connecting`	`Boolean`	`false`	
`muted`	`Boolean`	`false`	

GraineCalls

`class GraineCalls internal constructor()` — Calls like whatsapp — the agent rings the user, or the user calls the agent, on the phone's own call screen and over the lock screen, with the call going on with the screen off.

Member	Returns	Errors	What it does
`val state: StateFlow<GraineCallInfo>`	`StateFlow<GraineCallInfo>`	—	The call as it stands (GraineCallInfo), including whether a call screen has something to show.
`val outcomes: SharedFlow<GraineCallOutcome>`	`SharedFlow<GraineCallOutcome>`	—	Every outcome as it happens. Not replayed: collect from app start.
`val incoming: SharedFlow<GraineIncomingCall>`	`SharedFlow<GraineIncomingCall>`	—	Every ring on this device. Not replayed.
`fun configure(config: GraineCallsConfig)`	`nothing`	—	How calls look and ring.

Member	Returns	Errors	What it does
`fun enableVoIPPushes()`	`nothing`	—	iOS: receive the agent's calls as VoIP pushes — call at every launch (iOS wakes a closed app for a VoIP push and hands it over once this has run).
`fun startCall(reason: String? = null, variables: Map<String, Any?> = emptyMap(), agent: String? = null): String?`	`String?`	—	The user calls an agent — agent by name, the default agent when null: the call screen opens and the call connects.
`fun answer()`	`nothing`	—	Pick up the ringing call (the SDK's call screen does this itself).
`fun decline()`	`nothing`	—	Decline the ringing call.
`fun end()`	`nothing`	—	Hang up — or, while it rings, decline.
`fun setMuted(muted: Boolean)`	`nothing`	—	Stop sending the user's voice without ending the call.
`fun setSpeaker(on: Boolean)`	`nothing`	—	The loudspeaker on or off.
`fun callBack() : String?`	`String?`	—	Return the last missed call: the user calls the agent back.
`fun dismissMissedCall()`	`nothing`	—	The user has seen the missed call: a call screen can go.
`fun onNewToken(token: String)`	`nothing`	—	Android: your FirebaseMessagingService's new token — one line, override fun onNewToken(token: String) = Graine.calls.onNewToken(token).
`fun handlePush(data: Map<String, String>) : Boolean`	`Boolean`	—	Android: hand every FCM data message here first, from your FirebaseMessagingService — if (Graine.calls.handlePush(message.data)) return.

GraineCallsConfig

`data class GraineCallsConfig(val displayName: String = "", val ringTimeoutSeconds: Int = 30, val ringtone: String? = null, val smallIcon: Int = 0, val includeInRecents: Boolean = false)` — How native calls look and ring.

Field	Type	Default	Meaning
`displayName`	`String`	`""`	The name a call rings with and shows, when the push names nobody.
`ringTimeoutSeconds`	`Int`	`30`	Seconds an agent's call rings before it is missed (a call Graine rang uses the agent's own ring time).
`ringtone`	`String?`	`null`	The ring: on Android a content URI, on iOS a sound file in the app's bundle.
`smallIcon`	`Int`	`0`	Android: a monochrome drawable resource for the status-bar icon.
`includeInRecents`	`Boolean`	`false`	iOS: list the calls in the Phone app's Recents.

GraineCallInfo

`data class GraineCallInfo(…)` — The call to draw, as a state that always holds the latest value.

Field	Type	Default	Meaning
`state`	`GraineCallState`	`GraineCallState.IDLE`	
`callId`	`String?`	`null`	
`callerName`	`String?`	`null`	
`incoming`	`Boolean`	`false`	The agent rang (a push); false for a call the user placed.
`hosted`	`Boolean`	`false`	
`muted`	`Boolean`	`false`	
`speakerOn`	`Boolean`	`false`	The loudspeaker (else a headset, if one is connected, or the earpiece).
`agentSpeaking`	`Boolean`	`false`	
`caption`	`GraineCaption?`	`null`	The last line said, while a call is live.
`missedCall`	`GraineMissedCall?`	`null`	The last ring nobody answered, until it is called back or dismissed.

Field	Type	Default	Meaning
agent	`String?`	`null`	The agent the call is with, by the name it was configured under ("default" for the default agent) — the one startCall named, or the one a ring was routed to.

Member	Returns	Errors	What it does
`val showsCallScreen: Boolean`	`Boolean`	—	A call screen has something to show: a call, or a missed ring to call back.

GraineCallState

`enum class GraineCallState { IDLE, RINGING, CONNECTING, ACTIVE, ENDED }` — Where the one native call is.

Case	Meaning
IDLE	
RINGING	
CONNECTING	
ACTIVE	
ENDED	

GraineCallOutcome

`data class GraineCallOutcome(val callId: String, val outcome: Kind, val reason: String? = null, val durationSeconds: Int? = null, val hosted: Boolean = false, val agent: String? = null)` — What became of a call — for your server, and for your own screens.

Field	Type	Default	Meaning
callId	`String`	required	
outcome	`Kind`	required	
reason	`String?`	`null`	
durationSeconds	`Int?`	`null`	
hosted	`Boolean`	`false`	Graine rang it: a call booked by the agent, the user or the dashboard.
agent	`String?`	`null`	The agent the call was with, by the name it was configured under ("default" for the default agent); null when the platform does not know.

GraineCallOutcome.Kind

`enum class Kind(val wire: String)`

Field	Type	Default	Meaning
wire	`String`	required	

Case	Meaning
RINGING	— wire `ringing`
ANSWERED	— wire `answered`
DECLINED	— wire `declined`
MISSED	— wire `missed`
FAILED	— wire `failed`
ENDED	— wire `ended`

GraineIncomingCall

`data class GraineIncomingCall(val callId: String, val callerName: String?, val reason: String?, val hosted: Boolean, val agent: String? = null)` — An agent's call ringing on this device.

Field	Type	Default	Meaning
callId	String	required	
callerName	String?	required	What the ring shows: the push's caller name, else the agent's call name or the app's display name.
reason	String?	required	Why the agent is calling, when the push said (a call Graine rang keeps its reason on the server).
hosted	Boolean	required	
agent	String?	null	The agent ringing, by the name it was configured under ("default" for the default agent): the platform matches the push to a configured agent.

GraineMissedCall

`data class GraineMissedCall(val callId: String, val callerName: String?, val reason: String?, val hosted: Boolean, val at: Instant)` — A ring nobody answered, kept so a call screen can say so and offer GraineCalls.callBack.

Field	Type	Default	Meaning
callId	String	required	
callerName	String?	required	
reason	String?	required	
hosted	Boolean	required	
at	Instant	required	

GraineCaption

`data class GraineCaption(val role: GraineRole, val text: String, val live: Boolean)` — A line said on the call (or in the voice bar's call): the agent's as it is spoken, the user's as it is heard.

Field	Type	Default	Meaning
role	GraineRole	required	
text	String	required	
live	Boolean	required	

GraineScheduledCall

`data class GraineScheduledCall(val id: String, val at: Instant?, val status: GraineCallStatus, val source: GraineCallSource? = null, val reason: String? = null)` — A call booked to ring this user (or, with nobody signed in, this install) later.

Field	Type	Default	Meaning
id	String	required	
at	Instant?	required	When it rings.
status	GraineCallStatus	required	
source	GraineCallSource?	null	Who booked it; null when the server does not say.
reason	String?	null	Why it was booked, when the server shares it.

Member	Returns	Errors	What it does
val isUpcoming: Boolean	Boolean	—	Still to come: scheduled, or ringing right now.
val canCancel: Boolean	Boolean	—	It can still be called off: nothing has rung yet.

GraineCallStatus

`sealed class GraineCallStatus(val wire: String)` — Where a booked call is.

Field	Type	Default	Meaning
wire	`String`	required	

Case	Meaning
Scheduled	— wire `scheduled`
Ringing	— wire `ringing`
Answered	— wire `answered`
Completed	— wire `completed`
Declined	— wire `declined`
Missed	— wire `missed`
Failed	— wire `failed`
Cancelled	— wire `cancelled`
Expired	— wire `expired`
NoDevice	Nothing could be rung: the user has no device registered for calls. — wire `no_device`

Member	Returns	Errors	What it does
`companion · fun from(wire: String?): GraineCallStatus`	`GraineCallStatus`	—	The status for a word on the wire.

GrainеCallSource

`sealed class GraineCallSource(val wire: String)` — Who booked a call.

Field	Type	Default	Meaning
wire	`String`	required	

Case	Meaning
Agent	— wire `agent`
App	— wire `sdk`
Dashboard	— wire `dashboard`
Api	— wire `api`

Member	Returns	Errors	What it does
`companion · fun from(wire: String?): GraineCallSource?`	`GraineCallSource?`	—	Null when the server does not say.

GraineSchedulingRules

`data class GraineSchedulingRules(…)` — What the agent allows when a user books a call, from the session (Calls & scheduling in the dashboard).

Field	Type	Default	Meaning
usersCanBook	`Boolean`	`false`	Users may book calls from the app.
minLeadMinutes	`Int`	`5`	
maxDaysAhead	`Int`	`14`	
maxPendingPerUser	`Int`	`3`	
ringTimeoutSeconds	`Int`	`45`	How long a call Graine rings rings before it is missed.
callName	`String?`	`null`	The name the agent's calls ring with; null: the app's own display name.
hostedRinging	`Boolean`	`false`	Graine rings this workspace's users itself (its push credentials are uploaded).
platforms	`List<GrainePlatform>`	`emptyList()`	The platforms Graine rings.

Member	Returns	Errors	What it does
`fun earliest(now: Instant = Clock.System.now()) : Instant`	`Instant`	—	The soonest time a booking is accepted: now plus minLeadMinutes.
`fun latest(now: Instant = Clock.System.now()) : Instant`	`Instant`	—	The latest time a booking is accepted: now plus maxDaysAhead days.

GrainePlatform

`enum class GrainePlatform(val wire: String) { IOS("ios"), ANDROID("android") }` — A platform Graine can ring.

Case	Meaning
`IOS("ios")`	
`ANDROID("android")`	

GraineMemory

`data class GraineMemory(val id: String, val text: String, val createdAt: Instant? = null, val updatedAt: Instant? = null)` — One thing the agent remembers about the signed-in user, as Graine keeps it (already redacted).

Field	Type	Default	Meaning
`id`	`String`	required	
`text`	`String`	required	
`createdAt`	`Instant?`	`null`	
`updatedAt`	`Instant?`	`null`	

GraineException

`class GraineException(val code: GraineErrorCode, message: String, val retryAfterSeconds: Int? = null) : Exception(message)` — The error every suspending request throws: a code, Graine's sentence as message, and retryAfterSeconds when Graine asked to wait.

Field	Type	Default	Meaning
`code`	`GraineErrorCode`	required	
`message`	`String`	required	
`retryAfterSeconds`	`Int?`	`null) : Exception(message`	When the server asked to wait before trying again (Retry-After), in seconds; null when it did not say.

GraineErrorCode

`sealed class GraineErrorCode(val wire: String)` — Why a request to Graine did not do what was asked: one stable word a program branches on.

Field	Type	Default	Meaning
`wire`	`String`	required	

Case	Meaning
`InvalidKey`	The publishable key was refused. — wire `invalid_key`
`OriginRefused`	The agent does not accept this app: add app:// to its Allowed domains. — wire `origin_refused`
`RateLimited`	Too many requests; GraineException.retryAfterSeconds says when to try again, when the server said. — wire `rate_limited`
`InvalidRequest`	A field was missing or malformed. — wire `invalid_request`
`IdentityRequired`	Nobody is signed in: identify() the user with the trait the agent's Identity setting names. — wire `identity_required`
`DeviceLimit`	This user is already using the agent on as many devices as it allows. — wire `device_limit`
`SchedulingOff`	The agent does not let users book calls (Calls & scheduling). — wire `scheduling_off`
`TooSoon`	Sooner than the agent's minimum lead time. — wire `too_soon`

Case	Meaning
`TooFar`	Further ahead than the agent allows. — wire `too_far`
`OutsideWindow`	Outside the agent's calling hours, in the user's time zone. — wire `outside_window`
`TooManyPending`	The user already has as many calls booked as the agent allows. — wire `too_many_pending`
`NotFound`	— wire `not_found`
`Unavailable`	The service could not answer just now. — wire `unavailable`
`Network`	No answer: no network, or Graine could not be reached. — wire `network`

Member	Returns	Errors	What it does
`companion · fun from(wire: String?): GraineErrorCode`	`GraineErrorCode`	—	The code for a word on the wire (or a platform SDK's own); blank is Unknown with "unknown".

GrainеScreen

`data class GraineScreen(…)` — What the customer is looking at.

Field	Type	Default	Meaning
`screen`	`String`	required	A stable id for the screen; keep it constant.
`title`	`String?`	`null`	
`journey`	`Journey?`	`null`	
`fields`	`List<GraineField>`	`emptyList()`	
`error`	`String?`	`null`	A screen-level error banner, distinct from a field's own.
`data`	`Map<String, Any?>`	`emptyMap()`	Anything else the agent should know: strings, numbers, booleans, lists and maps of them.
`availableActions`	`List<String>?`	`null`	Which registered actions work here.

GrainеScreen.Journey

`data class Journey(val name: String, val step: Int? = null, val of: Int? = null, val next: String? = null)` — Where this screen sits in a flow: step 2 of 4 of "KYC".

Field	Type	Default	Meaning
`name`	`String`	required	
`step`	`Int?`	`null`	
`of`	`Int?`	`null`	
`next`	`String?`	`null`	

GrainеField

`data class GraineField(val name: String, val label: String? = null, val value: String? = null, val status: GraineFieldStatus? = null, val error: String? = null, val optional: Boolean = false)` — One input on a screen, as the agent should understand it.

Field	Type	Default	Meaning
`name`	`String`	required	
`label`	`String?`	`null`	
`value`	`String?`	`null`	The real value.
`status`	`GraineFieldStatus?`	`null`	
`error`	`String?`	`null`	The validation message actually shown — what the agent can offer to fix.
`optional`	`Boolean`	`false`	Leaving it blank is fine, so the agent does not count it as work left.

GraineFieldStatus

`enum class GraineFieldStatus(val wire: String)` — Where a field stands.

Field	Type	Default	Meaning
`wire`	`String`	required	

Case	Meaning
`EMPTY`	— wire `empty`
`FILLED`	— wire `filled`
`INVALID`	— wire `invalid`
`PENDING`	— wire `pending`
`LOCKED`	— wire `locked`

GraineActionResult

`data class GraineActionResult(val status: Status, val message: String = "", val data: Map<String, Any?> = emptyMap())` — What an action handler answers.

Field	Type	Default	Meaning
`status`	`Status`	required	
`message`	`String`	`""`	Shown to the agent, not the customer: it decides how to say it.
`data`	`Map<String, Any?>`	`emptyMap()`	A structured result the agent can reason about.

Member	Returns	Errors	What it does
`companion · fun ok(message: String = "", data: Map<String, Any?> = emptyMap())`	`nothing`	—	It happened.
`companion · fun refused(message: String)`	`nothing`	—	It will not happen, and why — often the most useful answer.
`companion · fun error(message: String)`	`nothing`	—	It was tried and failed.

GraineActionResult.Status

`enum class Status(val wire: String) { OK("ok"), REFUSED("refused"), ERROR("error") }`

Case	Meaning
`OK("ok")`	
`REFUSED("refused")`	
`ERROR("error")`	

GraineRole

`enum class GraineRole { USER, AGENT }` — Who said a line: the person using the app, or the agent.

Case	Meaning
`USER`	
`AGENT`	

Compose views (ai.graine:inapp-kmp-compose)

`(top-level)` — The SDK's own screens in a shared Compose UI, each hosting the platform SDK's native view in the brand's design. Nothing is drawn before Graine.configure(…).

Member	Returns	Errors	What it does
`expect fun GraineChat(modifier: Modifier = Modifier, onClose: (() -> Unit)? = null, agent: String = Graine.DEFAULT_AGENT)`	`nothing`	—	The chat with the agent, as a whole screen: header, one bubble per reply, the components the agent draws, the composer.
`expect fun GraineCallScreen(modifier: Modifier = Modifier)`	`nothing`	—	The call screen: the ring (Answer, Decline — "Scheduled call" for a call Graine rang), connecting, the call with its controls, and a missed ring with Call back.
`expect fun GraineAgentBar(modifier: Modifier = Modifier, name: String? = null, onClose: (() -> Unit)? = null, agent: String = Graine.DEFAULT_AGENT)`	`nothing`	—	The voice bar: the agent talking inside the app, beside the screen it talks about.

Android extras (androidMain)

`(top-level)` — For an Android app that removed the library's start-up provider, or wants the permission answers forwarded.

Member	Returns	Errors	What it does
`fun Graine.configure(context: Context, config: GraineConfig, name: String = Graine.DEFAULT_AGENT): GraineAgent`	`GraineAgent`	—	Android: start the SDK with an explicit context — for an app that removed GraineInitProvider from its manifest.
`fun Graine.onRequestPermissionsResult(requestCode: Int, grantResults: IntArray): Boolean`	`Boolean`	—	Android: forward your activity's onRequestPermissionsResult when it shows the voice bar or places calls from a screen of its own — the SDK asks for the microphone there.

GraineViews (iOS)

`object GraineViews` — The SDK's screens as UIViewControllers, for an iOS app that presents UIKit itself.

Member	Returns	Errors	What it does
`fun chat(onClose: (() -> Unit)? = null, agent: String = Graine.DEFAULT_AGENT): UIViewController`	`UIViewController`	—	An agent's chat as a screen.
`fun callScreen() : UIViewController`	`UIViewController`	—	The call screen for while the app is open: the ring, the call, and a missed ring with Call back.
`fun voiceBar(name: String? = null, onClose: (() -> Unit)? = null, agent: String = Graine.DEFAULT_AGENT): UIViewController`	`UIViewController`	—	An agent's voice bar, drawn at the bottom of the view it fills.

GraineKmpVersion

`object GraineKmpVersion` — The library's version.

Member	Returns	Errors	What it does
`const val NAME = "0.7.0"`	`–`	—	Constant.

GraineChatChannel

`enum class GraineChatChannel(val wire: String)` — Where a message was said.

Field	Type	Default	Meaning
`wire`	`String`	required	The word on the wire.

Case	Meaning
`CHAT`	Typed, in the chat. — wire `chat`
`VOICE`	Said on a call with the agent — only a restored turn is. — wire `voice`

GraineChatSize

`enum class GraineChatSize(val wire: String)` — How big the chat window opens: the dashboard's choice (Embed & Widgets → Appearance → Chat window → Window size), the app's (GraineConfig.chatSize), or one open's (GraineChat.open) — the open's wins, then the app's.

Field	Type	Default	Meaning
`wire`	`String`	required	The design's word for it.

Case	Meaning
`COMPACT`	The window above the voice bar, as it has always been. — wire `compact`
`LARGE`	A sheet over most of a phone; a panel in the corner of a tablet. — wire `large`
`FULL`	The whole screen. — wire `full`

GraineChatDay

`enum class GraineChatDay { TODAY, YESTERDAY, DATE }` — Which day a separator names.

Case	Meaning
`TODAY`	
`YESTERDAY`	
`DATE`	

GraineChatRow

`sealed class GraineChatRow` — One row of a chat window, as GraineChatRows.build lays the conversation out.

Member	Returns	Errors	What it does
`abstract val key: String`	`String`	—	Stable while the conversation grows: a separator by its date, a call by its first turn, a bubble by its turn.

GraineChatRow.Day

`data class Day(val day: GraineChatDay, val date: LocalDate, val at: Instant) : GraineChatRow()` — A day separator: which day, the device's date, and the first message's time on it.

Field	Type	Default	Meaning
`day`	`GraineChatDay`	required	
`date`	`LocalDate`	required	
`at`	`Instant) : GraineChatRow(`	required	

Member	Returns	Errors	What it does
`override val key: String`	`String`	—	

GraineChatRow.VoiceCall

`data class VoiceCall(val id: String, val at: Instant, val first: Int, val last: Int) : GraineChatRow()` — An earlier call, as ONE row — that it happened and when, never its words: when its first turn was said, and which turns (first to last, indices into the conversation) it stands for.

Field	Type	Default	Meaning
`id`	`String`	required	
`at`	`Instant`	required	
`first`	`Int`	required	
`last`	`Int) : GraineChatRow(`	required	

Member	Returns	Errors	What it does
`override val key: String`	`String`	—	

GraineChatRow.Turn

`data class Turn(val turn: GraineChatTurn, val index: Int) : GraineChatRow()` — A message bubble: the turn, and its index in the conversation.

Field	Type	Default	Meaning
`turn`	`GraineChatTurn`	required	
`index`	`Int) : GraineChatRow(`	required	

Member	Returns	Errors	What it does
`override val key: String`	`String`	—	

GraineChatRows

`object GraineChatRows` — WHAT A CHAT WINDOW DRAWS, in order — the rule every Graine SDK draws its own window by, for an app that draws its chat from Graine.chat.turns (the SDK's GraineChat() already does): a bubble per chat turn; ONE "Voice call · 6:05 PM" row for a run of voice…

Member	Returns	Errors	What it does
`fun build(turns: List<GraineChatTurn>, now: Instant = Clock.System.now(), zone: TimeZone = TimeZone.currentSystemDefault()): List<GraineChatRow>`	`List<GraineChatRow>`	—	The rows for turns (oldest first), on the days of zone — the device's own unless a test says otherwise.
`fun dayLabel(row: GraineChatRow.Day, now: Instant = Clock.System.now(), zone: TimeZone = TimeZone.currentSystemDefault()): String`	`String`	—	"Today", "Yesterday", else the date — "Sat 10 Oct", with the year when it is not this one ("Wed 31 Dec 2025").
`fun callLabel(clock: String) : String`	`String`	—	"Voice call · 6:05 PM": clock is the call's first turn in the device's short time.
`fun offersStarters(turns: List<GraineChatTurn>) : Boolean`	`Boolean`	—	The brand's starters belong to a new chat: not once the user has said something, nor on a conversation that continues.
`fun offersNewConversation(turns: List<GraineChatTurn>) : Boolean`	`Boolean`	—	"New conversation" belongs to a conversation that continues: offered while restored turns are on screen.

GraineEvent.SessionReady

`data class SessionReady(val session: GraineSession) : GraineEvent()` — The agent's configuration arrived (also in Graine.session).

Field	Type	Default	Meaning
`session`	`GraineSession) : GraineEvent(`	required	

GraineEvent.Caption

`data class Caption(val role: GraineRole, val text: String, val live: Boolean) : GraineEvent()` — A line of the voice bar's call: the agent's as it is spoken, the user's as it is heard.

Field	Type	Default	Meaning
`role`	`GraineRole`	required	
`text`	`String`	required	
`live`	`Boolean) : GraineEvent(`	required	

GraineEvent.Component

`data class Component(val component: GraineComponent) : GraineEvent()` — The agent drew a component, in the chat or on a call.

Field	Type	Default	Meaning
component	`GraineComponent) : GraineEvent(`	required	

GraineEvent.ComponentPending

`data class ComponentPending(val hint: Map<String, Any?>) : GraineEvent()` — A component is on its way; its content is loading.

Field	Type	Default	Meaning
hint	`Map<String, Any?>) : GraineEvent(`	required	

GraineEvent.ChatMessage

`data class ChatMessage(val turn: GraineChatTurn) : GraineEvent()` — The agent finished a message in the chat.

Field	Type	Default	Meaning
turn	`GraineChatTurn) : GraineEvent(`	required	

GraineEvent.ActionRequested

`data class ActionRequested(val name: String) : GraineEvent()` — The agent asked for one of your actions.

Field	Type	Default	Meaning
name	`String) : GraineEvent(`	required	

GraineEvent.ActionCompleted

`data class ActionCompleted(val name: String, val result: GraineActionResult) : GraineEvent()` — What one of your actions answered the agent — the handler's result, or the SDK's when it was late or unknown.

Field	Type	Default	Meaning
name	`String`	required	
result	`GraineActionResult) : GraineEvent(`	required	

GraineEvent.Friction

`data class Friction(val name: String, val screen: String?, val detail: String) : GraineEvent()` — The user looks stuck (stalled_on_step): on one screen, with a field outstanding, through a stretch of a live conversation.

Field	Type	Default	Meaning
name	`String`	required	
screen	`String?`	required	
detail	`String) : GraineEvent(`	required	

GraineEvent.MicrophoneDenied

`data class MicrophoneDenied(val reason: String) : GraineEvent()` — The microphone was refused, so the agent cannot hear the user.

Field	Type	Default	Meaning
reason	`String) : GraineEvent(`	required	

GraineEvent.Error

`data class Error(val message: String) : GraineEvent()` — Loud for you, invisible to the user: a misconfigured agent is never the user's problem.

Field	Type	Default	Meaning
`message`	`String) : GraineEvent(`	required	

GraineErrorCode.Unknown

`data class Unknown(val raw: String) : GraineErrorCode(raw)`

Field	Type	Default	Meaning
`raw`	`String) : GraineErrorCode(raw`	required	

GraineCallStatus.Unknown

`data class Unknown(val raw: String) : GraineCallStatus(raw)`

Field	Type	Default	Meaning
`raw`	`String) : GraineCallStatus(raw`	required	

GraineCallSource.Unknown

`data class Unknown(val raw: String) : GraineCallSource(raw)`

Field	Type	Default	Meaning
`raw`	`String) : GraineCallSource(raw`	required	

12.8 Website — the loader 1.2.0

The loader, `https://www.graine.ai/embed/v1.js`, defines one global, `window.GraineAgent`. Plain JavaScript (no build step). Every member below exists before `load()` is called; what needs the widget is held until it exists. Promises reject with an `Error` carrying `code` and `message` (and `status`, `retryAfterSeconds` when Graine answered).

GraineAgent

`window.GraineAgent` — loader version `1.2.0`.

Member	Returns	Errors	What it does
`load(options)`	`Promise<boolean>`	Never rejects; resolves `false` (the console says why)	Puts the agent on the page. `options.agentId` and `options.publishableKey` are required; `options.variables` (up to 12 keys, about 1 KB) are prompt variables. Fallbacks used only where the dashboard sets nothing: `position` (`"bottom-right"`), `accent` (`"#318CE7"`), `launcherLabel` (`"Chat with us"`), `mode` (`"both"`, `"voice"` or `"chat"`). `chatSize` (`"compact"`, `"large"` or `"full"`) sets this page's chat window size over the dashboard's *Window size*. Called again with another agent, it takes the one on the page off first and resolves `true` once the new one is on it; with the same agent it does nothing.
`version`	`"1.2.0"`	—	The loader's version.
`switchesAgents`	`true`	—	This loader switches agents with `load()`. An older loader has no flag: reload the page to change agent there.

Member	Returns	Errors	What it does
`open(options)`	nothing	—	Opens the panel. Nothing happens before `load()` has put an agent on the page. `options.mode` (`"chat"` or `"call"`) picks the tab among those the agent offers; `options.size` (`"compact"`, `"large"` or `"full"`) is for this open only. On a computer, large and full open a bigger panel (420 wide, up to 760 tall) above the launcher; on a page narrower than 640 px, large is a sheet over most of the page and full the whole page.
`close()`	nothing	—	Closes the panel.
`destroy()`	nothing	—	Takes the agent off the page (its frame, launcher and conversation). `load()` puts one back; the visitor stays identified.
`defineAction(name, handler)`	nothing	—	Implements an action declared on the agent. `handler(args)` returns `{ status: "ok" | "refused" | "error", message }` (or a promise of it). Answer within 5.5 seconds: after that the agent is told the page did not finish; a throw answers `error`; a declared action this page does not implement is refused. Safe before `load()`.
`removeAction(name)`	nothing	—	Stops offering an action — for example when a section unmounts.
`setScreen(context)`	nothing	—	What the visitor can see: `{ screen, title, journey, fields, error, data, availableActions }`. Coalesced, masked, and buffered until the widget exists.
`addMaskRule(pattern, token)`	nothing	—	Adds a masking rule (a `RegExp`, replaced by `token`, default `"[REDACTED]"`) applied before anything leaves the page, after the built-in ones.
`reportEvent(event)`	nothing	—	Something worth the agent speaking up about: `{ name, detail }`.
`identify(traits, options)`	nothing	—	Who the visitor is; `null` signs them out of every agent they used on the page. The trait the agent's *Who is the user?* setting names is the identifier (never a prompt variable); other traits are masked and become prompt variables. `options.memory: false`: not remembered. Applies to the next conversation. Safe before `load()`.
`setMemory(enabled)`	nothing	—	Remember this visitor's conversations, or not; kept for them in this browser.
`getInstallId()`	`string`	—	This browser's install ID, kept in `localStorage` (or for the page's life where storage is refused).
`installId` (getter)	`string`	—	The same, as a property.
`schedulingRules` (getter)	object	—	`{ usersCanBook, minLeadMinutes, maxDaysAhead, maxPendingPerUser, ringTimeoutSeconds, callName, hostedRinging, platforms, earliest(now), latest(now) }`; booking is off until `load()` has the agent's settings.
`memories()`	`Promise<Array<{ id, text, createdAt, updatedAt }>>`	identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	What the agent remembers about the visitor, newest first (dates as `Date` or `null`).
`forgetMemory()`	`Promise<number>`	identity_required, device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	Deletes what the agent remembers about the visitor, and the kept conversation; resolves how many were deleted. Only when it succeeds, the widget's chat clears.

Member	Returns	Errors	What it does
`newConversation()`	`Promise`	device_limit, rate_limited, network, unavailable, invalid_key, origin_refused	Starts a new conversation: Graine lets go of the conversation it kept for this visitor and agent, and the widget's chat clears and opens fresh (the welcome again). Memories are untouched. Nobody identified: nothing was kept, so only the chat starts over. On a refusal the chat stays as it was.
`scheduleCall(at, reason)` `scheduleCall({ at, reason })`	`Promise<GraineScheduledCall>`	On the page: invalid_request (no usable `at`), scheduling_off, too_soon (30 seconds' grace), too_far; from Graine: outside_window, too_many_pending, identity_required, device_limit, rate_limited, network, unavailable	Books a call that rings the identified customer's app (a browser cannot ring). `at`: a `Date`, an ISO-8601 string or a timestamp; `reason` is trimmed to 120 characters.
`scheduledCalls()`	`Promise<GraineScheduledCall[]>`	network, rate_limited, unavailable, invalid_key, origin_refused	The visitor's upcoming calls.
`cancelScheduledCall(id)`	`Promise<string>`	invalid_request (no id), not_found, network, rate_limited, unavailable	Cancels one of the visitor's calls; resolves the status afterwards, `"cancelled"`.

GraineScheduledCall (website)

A plain object: `{ id, at /* Date | null */, status, source, reason, isUpcoming, canCancel }` — the same fields as on every SDK; `status` and `source` are the words in GraineCallStatus and GraineCallSource.

13. Errors, limits and webhooks

13.1 Error codes and their fixes

A method that asks Graine on the customer's behalf fails with one error type carrying a **code**. Branch on the code; show the **message**, which is written for the customer. Errors never carry the identifier, traits or a push token.

Code	HTTP	Means	Raised by	Fix
`invalid_key`	401	The publishable key is not one, or belongs to no agent	Every request to Graine	Copy the key again from Agent → Embed & Widgets → Install
`origin_refused`	403	The app or site is not allowed: `app://` (apps) or the site's host is missing from Allowed domains	Every request	Add `app://`, or the exact host
`rate_limited`	429	Too many requests from this install, key or address	Every request	Wait `retryAfterSeconds` (the `Retry-After` header), then retry
`invalid_request`	400	A field is missing or malformed — a time without its offset, a reason over 120 characters	`scheduleCall`, device registration	Fix the request (Flutter refuses a long reason on the device)
`identity_required`	400	No customer is identified, or the identifier is a placeholder	`memories`, `forgetMemory`, bookings that need a customer	`identify()` with the identifier first
`device_limit`	403	This install is not one of the customer's allowed devices	`memories`, `forgetMemory`, `newConversation`, `scheduleCall`	Sign out on another device, raise *Devices per user*, or **Reset devices** for the customer
`scheduling_off`	403 (0 on the device)	*Users, from your app* is off	`scheduleCall`	Turn it on, or hide booking when `usersCanBook` is false
`too_soon`	400 (0 on the device)	Earlier than `earliest(now)`	`scheduleCall`	Offer times from `earliest(now)`
`too_far`	400 (0 on the device)	Later than `latest(now)`	`scheduleCall`	Offer times up to `latest(now)`
`outside_window`	400	Outside the agent's Calling Window (or, for agent and team bookings, inside quiet hours), in the customer's time zone	`scheduleCall`, agent and dashboard bookings	Show the message (it names a time that works); offer another time
`too_many_pending`	409	The customer already has the most calls waiting	`scheduleCall`	Cancel one first
`not_found`	404	No such booking for this customer or install	`cancelScheduledCall`	Refresh `scheduledCalls()`
`unavailable`	503 (504 on a timeout)	Graine could not answer just now — including a booking refused because the agent is switched off or out of credit	Every request	Retry with backoff; check the agent is on and the workspace has credit
`network`	—	No answer came back: offline, or the connection dropped	Every request	Retry when online
`unknown`	other	A code newer than the SDK, or an unexpected answer (carrying its word)	Every request	Log the code and message; show the message

When the bar or chat cannot start, the reason arrives through `onError` (and React Native's launcher `error`), not as a coded exception: `key_missing`, `key_malformed` (publishable keys start with `pk_live_`), `key_unknown` (401 — copy the key again); `no_origin` (403 — add `app://`); `no_domains_configured` (403 — the list is empty); `origin_not_allowed`

(403 — add the exact host); `not_configured` (high-quality audio is not enabled for the workspace; calls use the standard path). A refused microphone arrives as `onMicDenied("NotAllowedError")` (or `NotFoundError` for no usable microphone); React Native and Flutter calls configured wrongly reject with `invalid_options`, naming the option.

13.2 Rate limits and quotas

Limit	Value	Where it applies
Bookings from one install	10 an hour	`scheduleCall`, and the agent's bookings in that install's conversations
Bookings in a workspace	1,000 an hour	All in-app bookings
Calls waiting per customer	3 by default (1–10)	*At most this many upcoming calls per user*
Devices per customer	2 by default (1–5)	*Devices per user*; idle devices free their place after 30 days
Rings per customer per day	3 by default (1–10)	*Ring each user at most*, in the customer's own day
Quiet period after a decline	6 hours	No ring for that customer
Notice and horizon	5 minutes (2–1440) · 14 days (1–30)	*With at least this much notice · Booked at most this far ahead*
Ring time · re-ring	45 s (20–60) · once after 10 min (0–120)	*Ring for · After a missed call, ring once more after*
Lateness	10 minutes	A booked call more than 10 minutes late is not rung; credit and agent holds last at most 10 minutes
Booking reason · variables	120 characters · 12 variables, about 1 KB	Bookings and calls
Memories listed	At most 100, newest first	`memories()`
Conversation kept	50 messages, 7 days after the last	Conversations continue (0.7.0)
Requests per key and address, per minute	Starting a conversation 30 (and 300 per key); high-quality audio sessions 10 (120 per key); booking, listing, cancelling, device registration, memory list and sign-out 30 (300 per key); call outcomes 60 (600 per key); forgetting memory 10 (120 per key); listing memories also 30 per install	Answered `rate_limited` with `Retry-After`
Action handlers · component data	About 5 seconds · 8 seconds	After that, the agent is told the action did not finish; a component shows its authored content

13.3 Webhooks for in-app conversations

- **Where:** Agent → *Integrations & Tools* → *Custom Webhook*: *Webhook URL*, *Send these events*, and the *Signing secret*. A subscription receives only `all_processing_completed` (the complete record) unless you tick more.
- **In-app and website conversations** — chats, the bar's voice, native calls — are recorded in Call History like other calls, with their transcript, the screens and taps the app reported, the actions the agent took and, for a call Graine rang, the booking it came from.
- **Calls Graine rang that never became a conversation** add three opt-in events — under *In-app calls* in *Send these events*: *In-app call declined*, *In-app call missed*, *In-app call expired* — `call_declined` (the customer declined a call

the agent rang in your app), `call_missed` (it rang out unanswered) and `call_expired` (a scheduled in-app call could not ring in time — calling hours, credit, or the agent was off — so it was not rung). Each carries the standard envelope (`event_type` , `event_id` , `timestamp` , `call_id` , `agent_id` , `status` , `sub_status` , …); its status says how the ring ended. A webhook never carries the customer's identifier.

14. Data, privacy and security

14.1 What leaves the phone or the page

Data	When	How it is protected
Screen context, field values, traits, prompt variables, tracked events	As your app reports them	Masked on the device before they leave it: PAN, Aadhaar, card and account numbers, phone numbers, email addresses, and your own patterns
The identifier (the trait *Who is the user?* names)	Only with requests that must know the customer: starting a conversation or call, registering the device, booking, listing or cancelling calls, memory requests, signing out	Sent as it is (it must match), never a prompt variable, never in what the agent reads, never in a ring or a webhook
Install ID, time zone, memory choice, app and SDK version	With the same requests	The install ID is kept by Graine only as a one-way key
The customer's voice	Only while a conversation or call is connected	—
Typed chat messages	As they are sent	Not masked on the device: they are what the customer chose to say, kept in the conversation's transcript
Your actions' answers	When the agent asks	Your handler decides what it returns
Push token	Device registration, for hosted ringing	Stored encrypted

14.2 What a ring carries

The call's ID, the agent's call name, a flag that Graine rang it, and the booking agent's ID — no reason, no variables, nothing about the customer. The reason and variables reach the agent when the call connects.

14.3 What Graine stores, and for how long

What	Kept	Deleted
Push credentials (iOS Team ID, Key ID, `.p8`, environment; Android service account)	Encrypted with your workspace's own key; never shown again or logged. The bundle ID and package name stay visible	When an admin presses **Remove**
Devices (push token, platform, app, the install's one-way key, the customer's one-way key when the device is theirs, time zone, versions, last seen)	The token encrypted; the customer appears only as their one-way key	When Apple or Google reports the token dead; 90 days after the device was last seen. Unlinked from the customer on sign-out
Device bindings	Which installs act as a customer	Sign-out, **Reset devices**, or 30 days unseen
The identifier on conversation records	Normalised (trimmed, lower-cased email, last ten digits of a phone, ≤256 characters), so your team sees who each conversation was with	With the conversation record
Memory	Short facts under the one-way key, redacted before they are stored, for *Keep memories for* (default 1 year)	On `forgetMemory()`, **Forget this person**, a single **Delete this memory**, or when they expire. Deleting an agent does not delete its memories early

What	Kept	Deleted
Kept conversation (0.7.0)	The last 50 messages per agent and customer	7 days after the last message; on forgetting; on `newConversation()`
Bookings	Target (one-way keys; for a dashboard booking, the identifier typed), time, time zone, reason, status	—

14.4 The publishable key, and what protects it

- The key is public by design: it ships in your page source and your app. It selects the agent; it cannot read your data or place a phone call.
- A website request is accepted only from a host in Allowed domains; an app request only when `app://` is listed.
- Each conversation starts with a single-use connection ticket valid for two minutes.
- Every public route is rate-limited per key, per address and, for bookings and memory, per install.
- Memory and rings are bounded by the device limit and the customer's own sign-out. For accounts where recognising a returning customer is not enough, confirm the customer through one of your actions before the agent discusses the account.

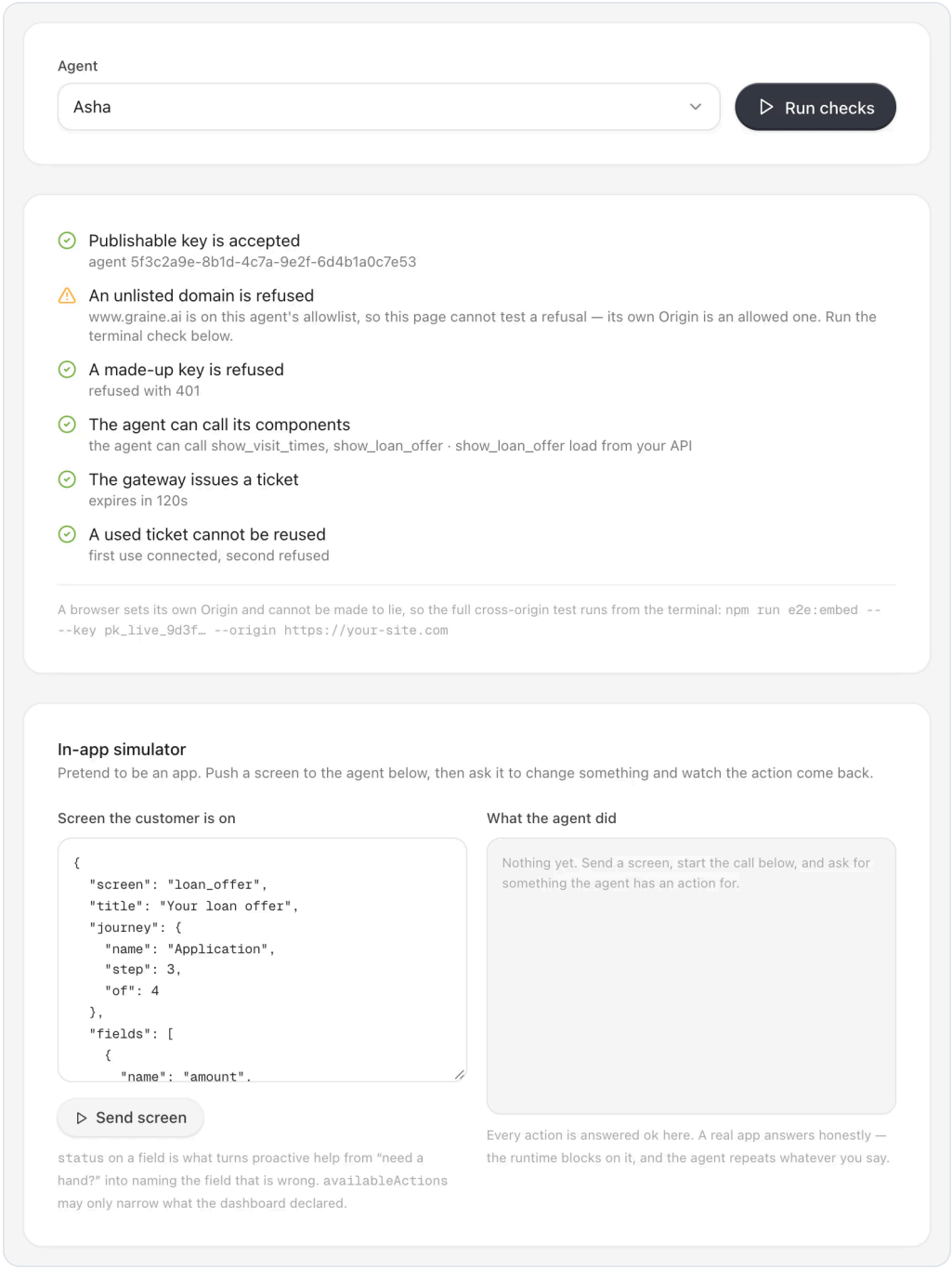
14.5 Erasure

From your app: `forgetMemory()` (memory and the kept conversation) and `identify(null)` (releases the device and unlinks it from the customer's calls). From the dashboard: **Forget this person**, **Delete this memory**, **Reset devices**, and **Remove** for push credentials.

15. Testing end to end

15.1 Before you start

- Two test customers whose identifiers you know (for example `u_test_1`, `u_test_2`), and *Devices per user* at its default (2).
- At least one iPhone and two Android phones from different manufacturers — calls do not ring in simulators — plus your website on a host in Allowed domains.
- Push credentials *Verified* for the exact bundle ID and package name you build (iOS development builds: environment *Automatic* or *Sandbox*).
- Calls & scheduling: both booking switches on; a Calling Window that includes now; quiet hours that do not.
- `debug` on in test builds: the SDKs log their main steps (iOS console, Android logcat tag `Graine`); audio, message text and variable values are never logged.
- The dashboard's **Playground** runs quick checks first: the key is accepted, a made-up key is refused, the agent can call its components, a connection ticket is issued and cannot be reused, and an in-app simulator sends a screen and shows the agent's actions.



The Playground after **Run checks**.

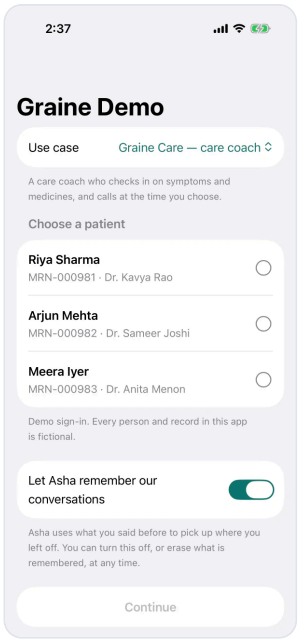
15.2 The QA script

#	Do this	Expect
1	Phone A: sign in as customer 1; open the bar	The launcher and bar in your design; the agent greets by name (from the `name` trait)
2	Ask "what screen am I on?"; ask it to do something you declared	It names the screen and fields; the action happens in the app and the agent says what happened
3	Ask for something a component shows; tap an option	The component appears in your design; the tap continues the conversation
4	Switch to the chat and back	One conversation at a time: the call ends when the chat opens
5	Tell the agent something to remember; end; start again	It remembers (allow a short while for a new fact to be stored)

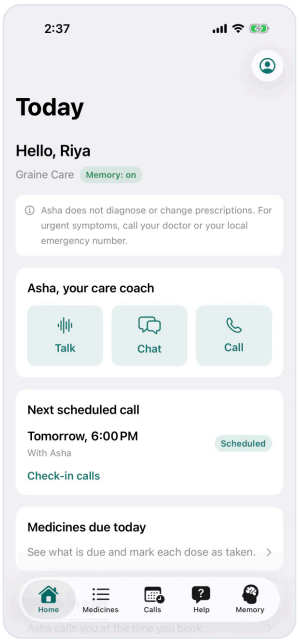
#	Do this	Expect
6	Chat a few messages; close the app; reopen the chat — then sign in as customer 1 on phone B and open the chat	The earlier messages are there on both phones (voice calls as one row each); no welcome repeated; the agent continues
7	Turn memory off for customer 1 (`setMemory(false)`); start again	Nothing new remembered; nothing restored
8	Sign out (`identify(null)`); start again; sign in as customer 2 on phone A	Anonymous after sign-out: no name, no memory; then customer 2's own conversation, nothing of customer 1's
9	Sign in as customer 1 on a third device	It works, but with no memory and `device_limit` from `memories()`; **Reset devices** in Memory → People frees the places
10	Tap your call button	The call screen (CallKit on iPhone, the SDK's screen on Android); the agent knows you called (`who_called = customer`, `call_reason`)
11	Ask the agent to call you back in 10 minutes	It confirms the time; Calls & scheduling → Scheduled calls lists it, *Booked by the agent*
12	Book from the app with your booking sheet, from the website with `scheduleCall`, and from the dashboard with **Schedule a call**	All three listed (*Booked by the app*, *Booked by Dashboard*); all ring the app, not the browser
13	Lock phone A and close the app; wait for a booked call	It rings on the lock screen with your call name (iPhone: the system call screen; Android: full screen, or heads-up without the full-screen permission). Answer: the agent knows why it called
14	Mid-call: lock the phone, open another app, connect a Bluetooth headset	The call goes on and follows the headset
15	Decline one booked call; let another ring out	`declined` then nothing for 6 hours; `missed` then one re-ring after your delay; webhooks `call_declined` / `call_missed` if subscribed; Android leaves a missed-call notification with Call back
16	Ring during a live call (dashboard **Ring now**)	Reported `missed`; the live call is untouched
17	Turn on airplane mode mid-call	The call ends (`ended`, reason `failed`); it does not reconnect
18	Deny the microphone; on Android 13+ deny notifications	The bar says it needs the microphone; a booked call does not ring with the app closed
19	Book for an identifier that has never used the app	The booking ends *No device to ring* (`no_device`)
20	Open the second agent's screen (for example Help); chat; book a call from it	Its own design, title and starters; its own conversation; the ring shows its call name and design; nothing from the first agent's chat
21	Open each conversation in Call History (Monitor → Call history → Web)	Each row shows **User ID · u_test_1** (your field's label and value), the transcript, the screens and taps, the actions, and for rung calls the booking's outcome

15.3 What each step should look like

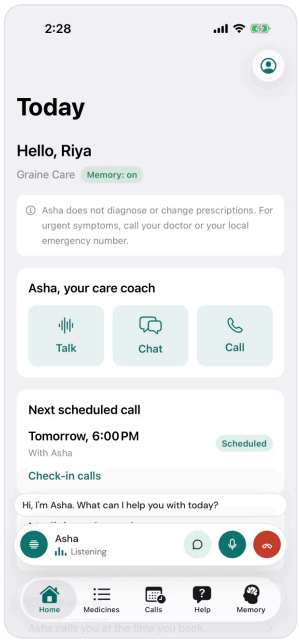
The demo apps, run offline, show the screens the steps above produce — here the demo's care-coach use case, one of its three packs (a care coach, a banking help desk, a shop):



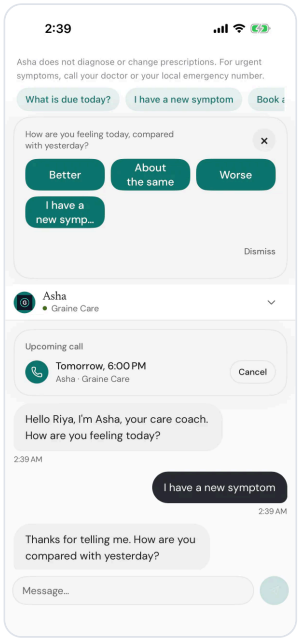
Sign-in: the app identifies the customer and asks whether to remember (steps 1, 7).



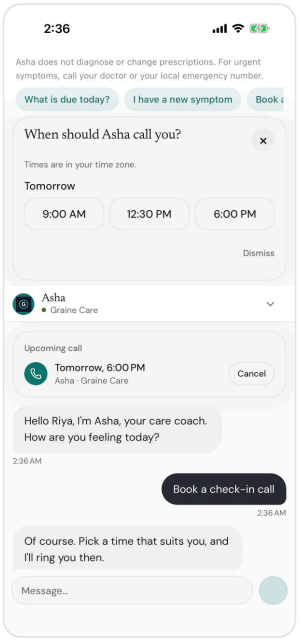
Home: talk, chat, call, and the next scheduled call (steps 1, 10).



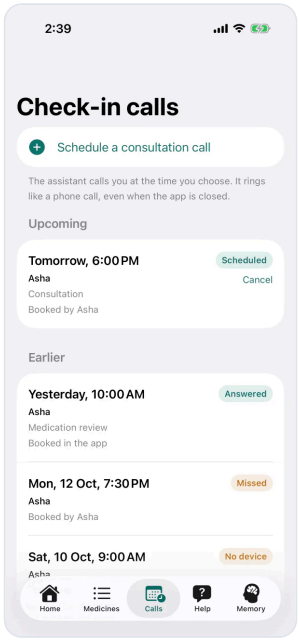
The assistant bar, listening (step 1).



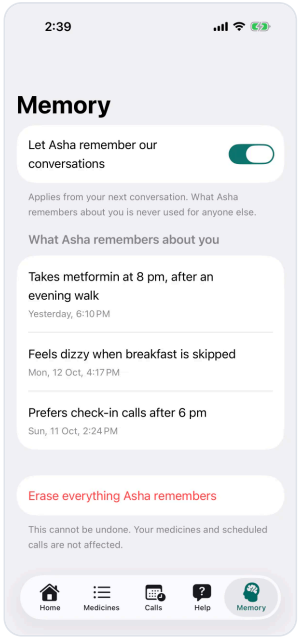
A component in the chat, with the upcoming call (step 3).



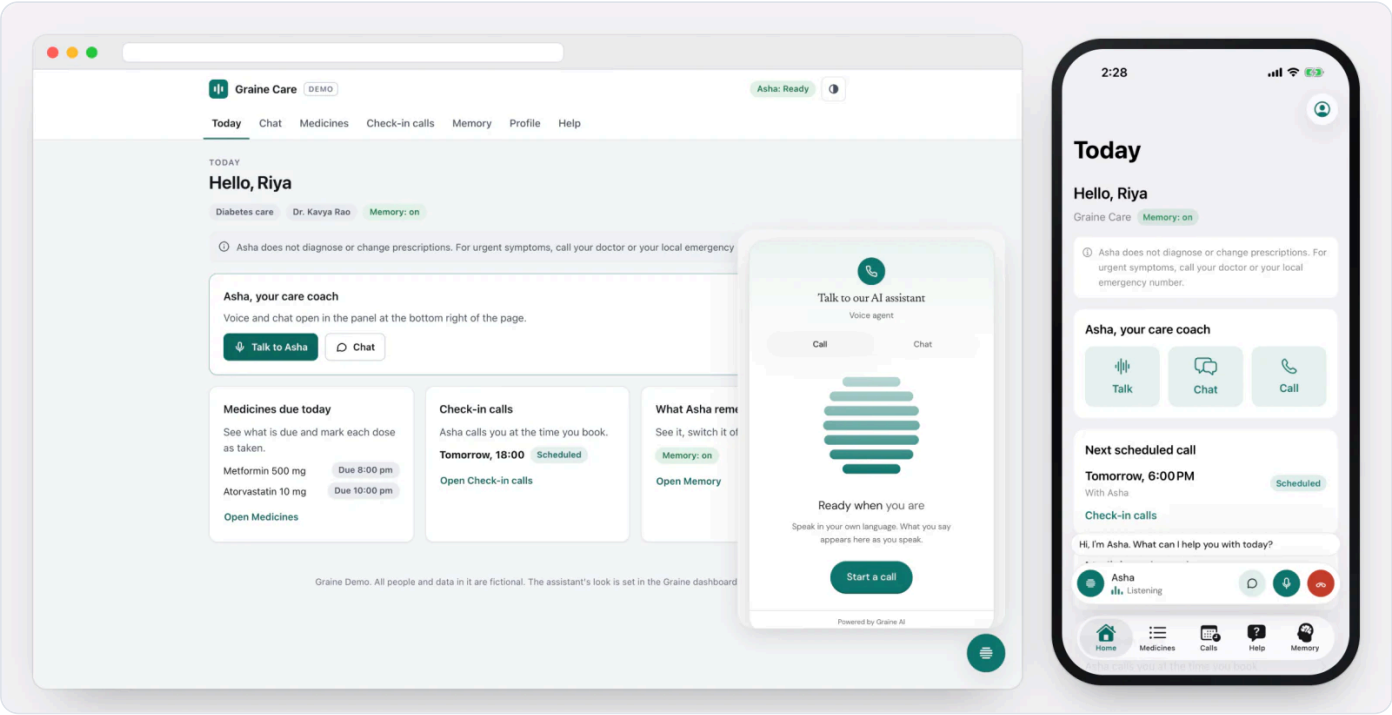
The agent books a call: time slots in the chat (step 11).



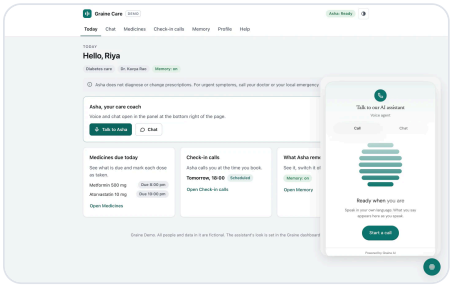
The booking list: upcoming with Cancel, earlier outcomes (steps 12, 15, 19).



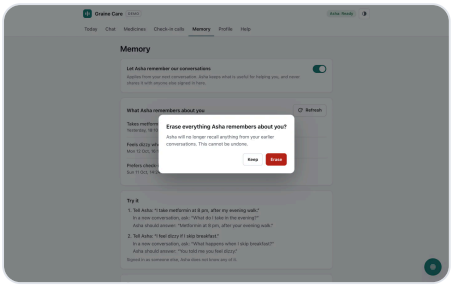
A privacy screen built on `memories()`, `setMemory` and `forgetMemory()` (steps 5, 7).



The same agent on the website (the widget) and in the app (the bar), in one design.



The website widget.



`GraineAgent.forgetMemory()` behind a confirmation.

15.4 The demo apps

Graine provides demo apps for React Native, Flutter, iOS, Android, Kotlin Multiplatform and the website, each driven by the same use-case packs (each pack is two agents: a main agent and a help agent for the Help screen). To run one against your own agent:

1. Create the agent from the pack's setup notes. Two settings must match the pack: Memory → *Who is the user?* → *Custom field* (the pack's field, for example `customer_id`) and Calls & scheduling → *Users, from your app* (on).
2. Allowed domains: `app://` for the apps; your local host for the website (add `localhost` explicitly while you test, and remove it later).
3. Put the publishable key (and optionally the help agent's) in the app's configuration file, which is not committed:

Demo	Configuration	Run
React Native (Expo development build)	`react-native/.env`: `EXPO_PUBLIC_GRAINE_PUBLISHABLE_KEY`, `EXPO_PUBLIC_GRAINE_HELP_KEY`; your bundle ID and package (`GRAINE_DEMO_IOS_BUNDLE_ID`, `GRAINE_DEMO_ANDROID_PACKAGE`) and `google-services.json` for rings	`npm install`, then `npx expo run:ios --device` or `npx expo run:android --device`
Flutter	`flutter/dart_defines.json`: `GRAINE_PUBLISHABLE_KEY`, `GRAINE_HELP_KEY`; your `applicationId` and bundle ID for rings	`flutter pub get`, then `flutter run --dart-define-from-file=dart_defines.json`
iOS	`ios/Config.xcconfig`: `GRAINE_PUBLISHABLE_KEY`, `GRAINE_HELP_PUBLISHABLE_KEY`, `DEVELOPMENT_TEAM`, `PRODUCT_BUNDLE_IDENTIFIER`	Open `GraineDemo.xcodeproj`, run the *GraineDemo* scheme on a device
Android	`android/local.properties`: `graine.publishableKey`, `graine.applicationId`, `graine.helpPublishableKey`	`./gradlew :app:installDebug`
Kotlin Multiplatform	See the demo's own README	Android Studio (Android) and Xcode (iOS)
Website	`web/config.js`	From the demo folder: `npx serve .`, then `http://localhost:3000/web/`

For rings, upload push credentials for the bundle ID and package **you** build, then sign in as one of the pack's customers, book a call a few minutes ahead, lock the phone, and answer.

Offline mode (for screenshots, or a demo without a network): a small stand-in server, `node scripts/offline-server.mjs` (Node 18+, `http://127.0.0.1:8790`; `PACK=…` for another pack), answers what the SDK asks — the agent's design and settings, booked calls, memories — and plays the chat and the voice bar from the pack's script. The iOS demo connects to it with a launch argument (`-GraineOffline http://127.0.0.1:8790`); the website points at a stand-in loader. Nothing in offline mode is real: no model, no audio, no ringing, and a simulator has no CallKit. The screenshots above were taken this way.

16. Go-live, troubleshooting and support

16.1 Go-live checklist

Dashboard

- [ ] An agent per surface; each key in the right build; `app://` and every website host (staging included) in Allowed domains; `localhost` removed
- [ ] The design saved in the Garage (*Assistant bar*, *Chat window*, *Call screen*) and the *Setup* card saved
- [ ] Actions declared and synced; components ticked under *On screen*; Friction tuned
- [ ] Memory: on where wanted; *Who is the user?* the same field on the app's and the website's agents; *Devices per user*; retention and instructions
- [ ] Calls & scheduling: call name, ring time, re-ring, rings a day, quiet hours, who may book, limits; the agent's Calling Window
- [ ] Workspace settings → In-app calling: iOS and Android *Verified* for the production bundle ID and package (iOS environment *Automatic* or *Production*)
- [ ] The prompt handles `who_called` (`customer` ; `scheduled` for a call Graine rang; `agent` for your own server's ring) and `call_reason` , and says when to offer a call back
- [ ] Webhooks: the events you need ticked, including `call_declined` , `call_missed` , `call_expired` if you track rings

Apps

- [ ] The SDK initialised at launch; `identify()` after sign-in and on account switch; `identify(null)` on sign-out
- [ ] Microphone declared on every platform (`NSMicrophoneUsageDescription` ; `RECORD_AUDIO` , `MODIFY_AUDIO_SETTINGS` where the package does not merge them)
- [ ] iOS: Push Notifications; Background Modes Audio and Voice over IP; `enableVoIPPushes()` at every launch, not behind sign-in
- [ ] Android: Firebase in the app; `GraineCalls.onNewToken` (or `registerPushToken` in React Native and Flutter) and `handlePush` first in your message handler; notifications asked for on Android 13+; the full-screen setting offered on Android 14+
- [ ] A privacy screen with `memories()` , `setMemory()` and `forgetMemory()`
- [ ] Store submissions: App Store review notes and privacy details; Google Play foreground-service (microphone) and full-screen-intent declarations and Data safety (section 5.8)
- [ ] `debug` off in release builds

Website

- [ ] The snippet on every page the agent belongs on; `identify()` on sign-in; CSP allows `https://www.graine.ai`

Tested

- [ ] The QA script (section 15) on an iPhone, two Android phones and the website, signed in as known test customers

16.2 Troubleshooting

What you see	Why, and what to do
Nothing appears in the app; the log says 403 / `no_origin`	`app://` is missing from the agent's Allowed domains
Nothing appears on the website	The exact host (with or without `www.`) is not in Allowed domains, or the key was rotated — copy the new snippet

What you see	Why, and what to do
The launcher appears but the agent never answers	The agent has no web-call variant yet: save the agent once
The bar says it needs the microphone, and no dialog appeared	The permission is not declared (`RECORD_AUDIO` , or `NSMicrophoneUsageDescription` on iOS), or the customer refused it before; Flutter: `requestMicrophone` is not set
iOS: the app closes as a call starts	`NSMicrophoneUsageDescription` is missing
Android: a call stays on "Connecting…"	Your activity does not forward `onRequestPermissionsResult` to the SDK
The agent does not know the screen	`setScreen` is not called on that screen or on every change; React Native: `navigationRef` is not the same ref as the `NavigationContainer`
"Sorry, I can't do that here"	The action is not declared on the agent, or the names differ
No component	Not ticked under *On screen*, not named in the prompt (or the step), or not synced to the agent
The agent does not remember someone	The app passes a different trait from the one *Who is the user?* names; the value is a placeholder; memory is off for them; or this is one device too many
The same person shows up twice	The identifier differs between devices (an ID on one, an email on the other) or its case changed
`device_limit` for a real customer	They use more devices than you allow: raise *Devices per user*, or **Reset devices**
A booked call never rings	Credentials not *Verified* for this bundle ID or package; a simulator; iOS: `enableVoIPPushes()` not called at launch; Android: `onNewToken` not forwarded, no `google-services.json` , or notifications not allowed; or the customer has not opened the app signed in for 30 days (`no_device`)
iOS: rings on development builds but not TestFlight	The environment is *Sandbox*: choose *Automatic* or *Production*
iOS: VoIP pushes stopped arriving altogether	An app that fails to report a VoIP push to CallKit loses them; call `enableVoIPPushes()` at every launch and send VoIP pushes only for calls
Android: a heads-up notification instead of a full-screen ring	Android 14+ without the full-screen permission. It still rings; offer the setting
Android: rings only while the app is open	`handlePush` is not the first thing in `onMessageReceived` , or notifications are not allowed
A call stops soon after the screen locks	A phone's battery saver: ask the customer to allow your app in battery settings
A call rang later than booked	It fell outside the Calling Window or inside quiet hours in the customer's time zone and moved to the next opening
The design did not change	Apps read the design at start: restart the app. Websites read it at page load
Every booking refused with `unavailable`	The agent is switched off, or the workspace is out of credit

16.3 Support

Write to **support@graine.ai** or use **https://www.graine.ai/support**. Include the platform and SDK version, the agent's ID (never the key's secret parts — the publishable key is public), the time of the event with its time zone, the call's ID (`call_…` or the booking's ID) and the SDK's debug log. Do not send customer identifiers or push tokens by email. The online documentation is at **https://www.graine.ai/docs/in-app**.